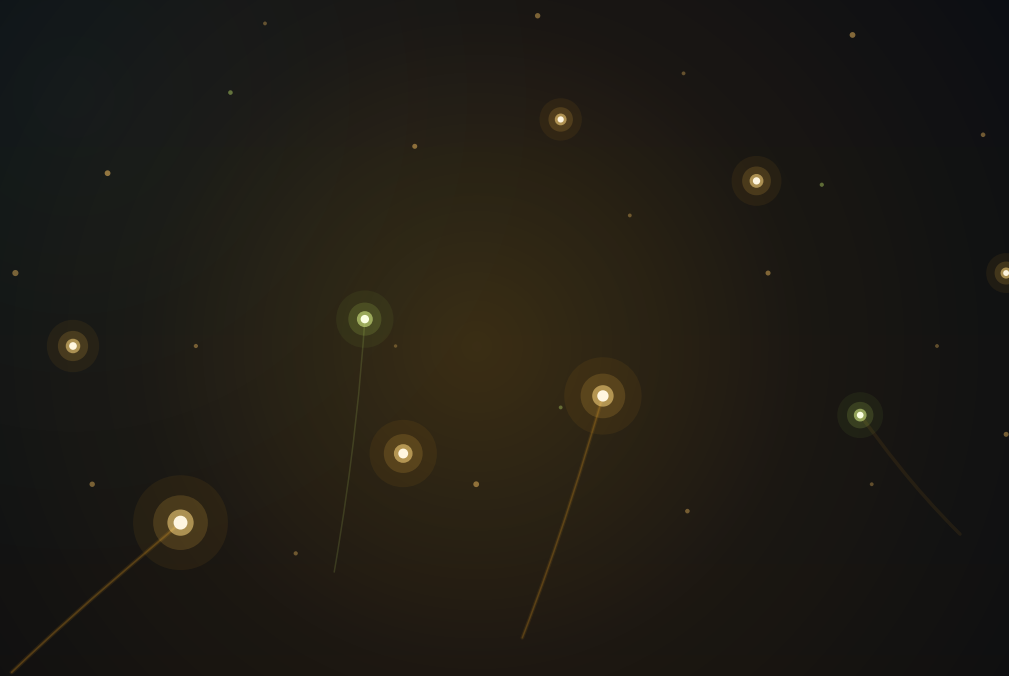


FIREFLY SOFTWARE FOUNDATION



firefly

for Rust by Example

Event-Driven Rust Microservices
with the Firefly Framework

 **FIREFLY** · tokio + axum · async-native Rust

Firefly for Rust by Example



Event-Driven Rust Microservices with the Firefly Framework

Firefly Software Foundation

Copyright © 2026 Firefly Software Foundation.

Licensed under the Apache License, Version 2.0 (the "License"); you may not use this material except in compliance with the License. You may obtain a copy of the License at <https://www.apache.org/licenses/LICENSE-2.0>.

Unless required by applicable law or agreed to in writing, software and documentation distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

First edition, 2026.

All code listings in this book were written and verified against the Firefly framework for Rust, version 26.6.x, targeting Rust 1.88+ on the `tokio` + `axum` stack.

Published by the Firefly Software Foundation.

Contents

Front Matter

Preface	6
Conventions	11
A Rust Primer	16

PART I Foundations

1. Why Firefly for Rust?	37
2. Quickstart: Zero to a Running Service	46
3. Configuration, Profiles & Secrets	58
4. Dependency Injection & the Application Context	74
5. Dependency Wiring & Composition	99
6. The Reactive Model — Mono & Flux	113

PART II Modeling & Persisting the Domain

7. Your First HTTP API	134
8. Persistence & Reactive Repositories	153
9. Domain-Driven Design	183
10. CQRS: Commands & Queries	200

PART III Event-Driven Architecture

11. Event-Driven Architecture & Messaging	224
12. Event Sourcing the Ledger	247

PART IV Into Microservices

13. HTTP Clients & Calling Other Services	270
14. The Experience Tier: Composing a BFF	283

15. Distributed Transactions: Sagas, Workflows & TCC	294
--	-----

PART V Secure, Observe & Ship

16. Security, Sessions & Identity	325
17. Observability & Health	344
18. Caching & Resilience	359
19. Scheduling, Notifications & Webhooks	371
20. Declarative Services with Macros	383
21. Testing Firefly Applications	407
22. The firefly CLI	420
23. Extending Firefly & Going to Production	430

Appendices

A. Crate & Module Index	444
Glossary	454

Preface

Rust gives you fearless concurrency, zero-cost abstractions, and a compiler that refuses to ship a data race. What it does not give you is *cohesion*. Every new back-office service forces the same cascade of decisions before a single line of business logic is written: which HTTP layer, which database story, how to wire dependencies, how to handle configuration, errors, correlation IDs, metrics, and graceful shutdown. **Firefly** changes that. It is an opinionated, convention-over-configuration framework that makes those cross-cutting decisions once, so every service shares one idiom — built from the ground up for Rust 1.88+ on `tokio` and `axum`.

This book teaches Firefly **by example**. You build one real application from an empty crate to a secured, observable, event-sourced service — making every concept concrete before moving to the next. The code in these pages is not illustrative pseudocode: every listing is a slice of a **real project that compiles, boots, and passes its tests** against the framework at version 26.6.x. Each snippet was lifted from the running sample and checked against the crate APIs, so what you read is what actually works. When a listing drifts from the source, the sample's build breaks and a test fails — that is the guarantee behind every listing in this book.

Who This Book Is For

This book is for intermediate Rust developers comfortable with `async` / `await`, traits, and the basics of HTTP services. You need no prior framework expertise — if you have built anything with `axum`, `actix`, or `sqlx`, you are well prepared.

If you have used an opinionated, batteries-included framework or a reactive-streams library before, Firefly's concepts — beans and stereotypes, declarative messaging, application events, `Mono` / `Flux` — will land quickly. A **Design note** callout appears wherever an idea will feel familiar, so you can lean on what you already know; each one is framed as Firefly's own design choice, not a translation from another framework.

What You Will Build: Lumen

Every chapter advances **Lumen**, a digital-wallet and ledger service — the worked example this book is built around. Lumen lets a customer open a wallet, deposit and withdraw money, transfer funds between wallets, and read a live balance. Behind that small surface sits the full spread of patterns a real back-office service needs: a value object that does exact money arithmetic, an aggregate that enforces invariants, CQRS with a read-side cache, domain events, an event-sourced ledger, a compensating transfer saga, JWT-secured endpoints, an actuator surface, a scheduled task, and an end-to-end test suite.

The single most important property of Lumen is its dependency list:

TOML

[dependencies]

```
firefly = { version = "26.6.5" } # the whole framework — and every macro
axum    = { version = "0.7" }   # you author the handler functions
serde   = { version = "1", features = ["derive"] }
```

One Firefly dependency. The entire framework — CQRS, dependency injection, the reactive web stack, event-driven messaging, event sourcing, saga orchestration, scheduling, resilience, security, observability — and every `#[derive(...)]` / `#[...]` macro arrives through `use firefly::prelude::*`. The chapters make a deliberate point of this: even Lumen's typed error enums hand-write `Display` + `std::error::Error` instead of pulling in `thiserror`, so the one-dependency promise holds end to end.

The journey follows a deliberate arc, one slice of Lumen at a time:

- **Part I — Foundations.** You scaffold the first Lumen service, bind typed configuration and profiles, learn how the composition root wires collaborators,

master the **Mono** / **Flux** reactive surface, and expose your first validated REST endpoints.

- **Part II — Modeling & Persisting.** You stand up a read model behind a repository, model the domain with a **Money** value object and a **Wallet** aggregate, and split reads from writes with CQRS command and query handlers dispatched through a bus.
- **Part III — Event-Driven.** The aggregate raises domain events; a **#[event_listener]** projection keeps the read model current; and an **event-sourced ledger** rebuilds every balance by folding its event stream — with the same events ready to flow out to Kafka or RabbitMQ.
- **Part IV — Into Microservices.** Lumen reaches beyond its own process: a typed HTTP client sketch shows how a wallet would call an external payments provider, and an orchestrated **transfer saga** moves money across wallets and *compensates* when the credit leg fails.
- **Part V — Secure · Observe · Ship.** You secure the endpoints with JWT bearer auth and path-based RBAC, make the service observable with metrics, tracing, and an actuator admin surface, add a read-side cache and a scheduled housekeeping task, test the whole stack in-process, and finally ship it behind the **firefly** CLI with graceful shutdown and a reactive streaming endpoint.

By the last page you have a working, tested, observable, secured, event-sourced service — and the mental model to extend it.

How to Use This Book

Read sequentially. Each chapter builds on the one before, and the Lumen codebase grows incrementally; skipping ahead leaves gaps. The **Reactive Model** chapter is the keystone — the whole reactive surface builds on **Mono** and **Flux**, so read it before the service-building chapters. The early chapters (1–5) introduce the framework with

small standalone snippets; **Lumen proper begins in Chapter 6** and grows from there. Each chapter is *additive* — it never rewrites what an earlier chapter shipped, only extends it — so the final state is exactly the companion crate.

Type every listing yourself. Reading and typing code at the same time is how the patterns stick. Resist copy-pasting until you have written each listing at least once.

Run it. Lumen really runs. From the workspace root:

Shell

```
cargo run -p firefly-sample-lumen      # boot the service (API + admin)
cargo test -p firefly-sample-lumen     # run the unit + HTTP test suite
```

Whenever a chapter adds a feature, start the app or the tests and watch it work. Seeing real JSON come back from a real endpoint — and watching a saga *compensate* a failed transfer — is worth a hundred diagrams.

Each chapter closes with a **Recap** of what changed in the Lumen codebase and a set of **Exercises** that push one step further. The exercises are optional but recommended for anything you intend to apply immediately.

Conventions in Brief

Typographic and structural conventions — code-listing captions, callout types, and design notes — are demonstrated, with live examples, in the **Conventions** section that follows.

The Companion Code

The complete, runnable Lumen project lives in the framework's `samples/lumen` directory. It is a single, clean Firefly crate — one module per concern (`money`, `domain`, `ledger`, `commands`, `transfer`, `security`, `web`, `housekeeping`) — that you grow chapter by chapter; the finished source there is the destination this book walks you to.

Build it once with `cargo build -p firefly-sample-lumen`, and use it to compare your work, catch up if you fall behind, or simply run the parts you are reading about. The chapter-by-chapter map of *what code lands where* lives alongside it in `docs/book/LUMEN-ARC.md`.

Conventions

This page explains the typographic and structural conventions used throughout the book — and demonstrates each one with a live example, so the first time you meet a callout or a code caption in a chapter it already looks familiar.

Code Listings

Every multi-line code example is real, compiling Rust drawn from the Lumen companion crate. Where it helps, a listing is introduced with the **file it lives in** so you can find it in `samples/lumen`, as in "`samples/lumen/src/money.rs`". Inline code references within prose use `monospace`, as in "the `#[rest_controller]` attribute generates the wallet router."

Here is a representative listing — the constructor and exact-arithmetic core of Lumen's `Money` value object, lifted verbatim from `samples/lumen/src/money.rs`:

Rust

```

/// An exact monetary amount, expressed in integer minor units (cents).
#[derive(Debug, Clone, Copy, Default, PartialEq, Eq, PartialOrd, Ord, Serialize,
Deserialize)]
#[serde(transparent)]
pub struct Money {
    cents: i64,
}

impl Money {
    /// A zero amount – the opening balance of a brand-new wallet.
    pub const ZERO: Money = Money { cents: 0 };

    /// Returns a new `Money` that is `self + other` (immutable addition).
    #[must_use]
    pub const fn add(self, other: Money) -> Money {
        Money { cents: self.cents + other.cents }
    }
}

```

A snippet annotated `rust,ignore` or `rust,no_run` elides surrounding setup for focus, but the API names, types, and method signatures are exactly what the crates expose. A listing fenced as plain `text` is shell output, a banner, or an HTTP exchange rather than Rust source:

Text

```

$ cargo run -p firefly-sample-lumen
:: lumen :: digital-wallet & ledger (v26.6.5)

```

The One-Dependency Reminder

Because Lumen's defining property is its single Firefly dependency, every framework type you see is reached through the facade — `firefly::cqrs::Bus`, `firefly::eventsourcing::EventStore`, `firefly::reactive::Flux` — or, for the high-frequency surface and every macro, through one glob:

Rust

```
use firefly::prelude::*;
```

When a chapter introduces a new framework type, the prose names the facade path it lives behind, so you always know it came in through that one dependency.

Callouts

Four callout styles appear throughout the body. Each is a blockquote that opens with a bold label, and the design theme styles them distinctly:

NOTE

Notes provide supplementary context or clarify a subtlety in the main text. Worth reading, but not blocking.

TIP

Tips share a shortcut, idiom, or best practice that will save you time in real projects — for example, keeping money in integer cents so floating-point drift can never corrupt a balance.

⚠ WARNING

Warnings flag a common mistake or a sharp edge that causes hard-to-debug problems if ignored — for example, that Lumen's free-function CQRS handlers publish their collaborators through a process-global `OnceLock`, so a second `build_app()` in the same test binary keeps the *first* wiring.

***Design note.** Design-note callouts explain why Firefly does something a particular way, and point out where an idea will feel familiar if you have used an opinionated, batteries-included framework or a reactive-streams library before. They are orientation, framed as Firefly's own design choices — not a translation table for another framework. You will meet these in nearly every chapter.*

Reference Tables

When a chapter introduces a family of related APIs, a reference table collects them in one place so you can take in the whole surface at a glance:

Declarative attribute	What it generates
<code>#[rest_controller]</code>	an axum router from the annotated handler methods
<code>#[event_listener]</code>	a broker subscription bound to an event type
<code>#[scheduled]</code>	a task registered on the scheduler
<code>#[saga]</code> / <code>Step</code>	an orchestrated, compensating distributed transaction

Recap & Exercises

Each chapter closes with two fixed sections:

- A **Recap** — **what changed in Lumen** that lists the files added or extended and the one-sentence "by the end of this chapter, Lumen can ..." payoff.

- A set of **Exercises** that push one step further — usually a small, self-contained extension to the code the chapter just shipped. They are optional but recommended for anything you intend to apply immediately.

Turn the page to [Why Firefly for Rust](#), where the Lumen journey begins.

A Rust Primer

This book builds reactive, asynchronous, event-sourced microservices in Rust. The chapters assume you can *read* Rust comfortably; this primer makes sure you can — even if your day job is Java/Spring or Python. It is not a complete Rust course. It teaches exactly the slice of the language the later chapters lean on, in the order they lean on it, so that when Chapter 6 hands you a `Mono<WalletEvent>` or Chapter 10 shows you `#[derive(Command)]`, none of the syntax is a surprise.

If you have already shipped Rust with `async`/`await`, traits, and ownership, skip ahead — Chapter 1 is where Lumen begins. If you are coming from a garbage-collected language, read this once, type the snippets, and keep it bookmarked.

NOTE

every code block in this primer was compiled and run before it was printed. The complete-program snippets build and execute as shown; the few that elide surrounding setup are marked and use real, current Firefly API names.

Cargo, Crates, Modules, and `use`

A Rust project is built by **Cargo**, the package manager and build tool — the rough equivalent of Maven/Gradle in the Java world or pip/Poetry in Python. A unit of compilation is a **crate**: either a binary (an application) or a library. The crates your project depends on are listed in a manifest file called `Cargo.toml`:

TOML

[package]

```
name = "lumen"
version = "0.1.0"
edition = "2021"
```

[dependencies]

```
firefly = { version = "26.6.5" }           # the whole framework
tokio   = { version = "1", features = ["full"] } # the async runtime
serde   = { version = "1", features = ["derive"] } # (de)serialization
```

Each entry under **[dependencies]** is a crate pulled from crates.io (Rust's package registry, like Maven Central or PyPI). The **features** list turns on optional parts of a crate — a compile-time switch, not a runtime flag.

The everyday commands:

Shell

```
cargo build    # compile the project
cargo run      # compile, then run the binary
cargo test     # compile and run the tests
cargo check    # type-check without producing a binary (fast)
```

Within a crate, code is organized into **modules** — namespaces that group related items. You reach into a module's contents with a path of **::**-separated segments, and the **use** keyword brings a path into scope so you can refer to it by its short name:

Rust

```

use std::sync::Arc;           // bring `Arc` into scope from the standard library
use std::collections::HashMap;

fn main() {
    let map: HashMap<String, i64> = HashMap::new(); // now just `HashMap`
    let _shared = Arc::new(map);
}

```

 SPRING PARITY

`use std::sync::Arc;` is Rust's `import java.util.concurrent.atomic.*;`. A crate is a Maven artifact; a module is a Java package; `Cargo.toml` is your `pom.xml`. The big difference is that there is no classpath at runtime — everything is resolved and linked at compile time into one binary.

Throughout the book, the entire framework arrives through a single glob import you will see at the top of nearly every listing:

Rust

```
use firefly::prelude::*;
```

That one line pulls in `Mono`, `Flux`, `Bus`, the macros, and the rest of the high-frequency surface. When a chapter introduces a new framework type, the prose names the facade path it lives behind — `firefly::cqrs::Bus`, `firefly::eventsourcing::EventStore` — so you always know it came in through that one dependency.

Variables and Basic Types

Variables are declared with `let`. They are **immutable by default**; add `mut` to make a binding reassignable. Types are usually inferred, but you can annotate them after a colon:

Rust

```
fn main() {
    let count = 3;           // immutable; inferred as i32
    let mut total = 0_i64;   // `mut` makes it reassignable; i64
    total += count as i64;   // `as` is an explicit numeric cast
    println!("total = {total}");

    let active: bool = true;
    let ratio: f64 = 0.5;    // 64-bit float
    let initial: char = 'L';
}
```

The numeric types spell out their size and signedness: `i32` / `i64` (signed), `u32` / `u64` (unsigned), `usize` (pointer-sized, used for lengths and indices), `f32` / `f64` (floats). Lumen keeps money in `i64` cents, never `f64`, so rounding can never corrupt a balance.

 SPRING PARITY

`let` declares a local like Java's `var`, but immutability is the default — the opposite of Java, where you must write `final` to opt *in*. In Rust you write `mut` to opt *out* of immutability. This default is load-bearing: the compiler uses it to reason about who is allowed to change what.

String vs &str

This distinction trips up every newcomer, so meet it early. There are two string types:

- **String** — an owned, heap-allocated, growable string. You *own* the buffer; when the `String` goes out of scope, its memory is freed.
- **&str** — a *borrowed view* into string data you do not own (a "string slice"). String literals like `"wallet"` are `&str`.

The idiom: take `&str` as a parameter, return `String` when you produce a new owned value. A `&str` can borrow from a `String`, a literal, or anywhere — so accepting `&str` makes a function maximally flexible.

Rust

```
fn greet(name: &str) -> String {    // borrow input, return a fresh owned
String
    format!("hello, {name}")
}

fn main() {
    let owned: String = String::from("Lumen");
    let literal: &str = "wallet";
    println!("{}", greet(&owned));  // pass a &str borrowed from the String
    println!("{}", greet(literal)); // or a literal directly
}
```

Ownership, Borrowing, and References

This is the concept that makes Rust *Rust*. It is also the one with no equivalent in Java or Python, so read this section slowly.

Every value has exactly one **owner** — the variable that is responsible for freeing it. When the owner goes out of scope, the value is dropped (its memory and resources released). There is no garbage collector deciding *when* that happens; it happens deterministically, at the end of the owner's scope.

When you assign a value to another variable or pass it to a function, ownership **moves**. The original binding can no longer be used:

Rust

```
let a = String::from("Lumen");
let b = a;           // ownership MOVES from `a` to `b`
println!("{}", a);   // COMPILE ERROR: value borrowed here after move
```

If you want to *use* a value without taking ownership of it, you **borrow** it with a reference. A reference is written `&T` (a shared/immutable borrow) or `&mut T` (an exclusive/mutable borrow). Borrowing lets a function read or modify a value and then hand it back:

Rust

```
fn len_of(s: &String) -> usize {    // shared borrow: may read, not modify
    s.len()
}

fn push_bang(s: &mut String) {      // exclusive borrow: may modify
    s.push('!');
}

fn main() {
    let mut name = String::from("Lumen");
    let n = len_of(&name);    // &name - a shared borrow
    push_bang(&mut name);    // &mut name - an exclusive borrow
    println!("{name} now has length {}", n + 1);
}
```

The compiler enforces one rule, the **borrow checker's** core invariant: at any moment you may have **either** any number of shared (`&T`) references **or** exactly one exclusive (`&mut T`) reference to a value — never both. That single rule is what makes data races impossible: you can never have one thread reading while another writes, because the type system will not let two such references coexist.

 SPRING PARITY

In Java or Python, objects live on the heap, every variable is a reference to them, and a garbage collector reclaims them at some unpredictable later time. Aliasing is free and unchecked — two threads can happily hold the same `ArrayList` and corrupt it. Rust trades that freedom for guarantees: ownership is tracked at compile time, memory is freed the instant the owner's scope ends (no GC pauses, no `finalize`), and the borrow rules make data races a *compile error* rather than a 2 a.m. production incident. The phrase you will hear is "fearless concurrency" — that is what it means.

 TIP

when a snippet does `Arc::clone(&repo)` or `&self` instead of moving a value, it is borrowing to avoid giving up ownership. Reading "is this a move or a borrow?" off the `&` is most of what it takes to follow the rest of the book.

Arc<T> – Shared Ownership Across Threads

Sometimes one owner is not enough: a service, a repository, a config object may need to be shared by many tasks running concurrently. `Arc<T>` — *Atomically Reference-Counted* — is the answer, and you will see it on nearly every page of this book. An `Arc<T>` is a thread-safe shared handle to a `T`; cloning it is cheap (it bumps a reference count, it does not copy the `T`), and the `T` is dropped only when the last `Arc` handle goes away.

Rust

```
use std::sync::Arc;

fn main() {
    let config = Arc::new(String::from("lumen-config"));
    let handle2 = Arc::clone(&config); // a second handle to the SAME value
    println!("{config} / {handle2}"); // both point at one allocation
}
```

The framework leans on this in a particular shape: `Arc<dyn Trait>` — a shared handle to *some* type that implements a trait, where the concrete type is hidden behind the interface. This is how Firefly passes around services and hexagonal "ports" so a wallet handler can depend on a `WalletRepository` without knowing whether it is backed by Postgres, MongoDB, or an in-memory map:

Rust

```
use std::sync::Arc;

trait WalletRepository: Send + Sync {           // a "port"
    fn balance(&self, id: u64) -> i64;
}

struct InMemoryRepo;                           // one "adapter"
impl WalletRepository for InMemoryRepo {
    fn balance(&self, _id: u64) -> i64 { 0 }
}

fn main() {
    // The caller holds the interface, not the concrete type:
    let repo: Arc<dyn WalletRepository> = Arc::new(InMemoryRepo);
    let also_repo = Arc::clone(&repo);        // share it with another task
    println!("{}", repo.balance(1));
    println!("{}", also_repo.balance(2));
}
```

`Arc` shares *read* access. To share *mutable* state, wrap the inner value in a lock — `Mutex<T>` (one accessor at a time) or `RwLock<T>` (many readers or one writer) — giving the familiar `Arc<Mutex<T>>` / `Arc<RwLock<T>>`. The lock enforces the "one writer" rule at run time for data the borrow checker cannot track across threads:

Rust

```
use std::sync::{Arc, Mutex};

fn main() {
    let counter = Arc::new(Mutex::new(0_u64));
    *counter.lock().unwrap() += 1;    // lock, mutate, unlock at end of statement
    println!("{}", *counter.lock().unwrap());
}
```

Structs, Enums, and `match`

A **struct** groups named fields — your data records and value objects. Methods are defined in an `impl` block; `&self` borrows the receiver (read), `&mut self` borrows it exclusively (mutate), and a method without `self` is an associated function (Rust's "static method"), commonly used for constructors:

Rust

```

struct Wallet {
    id: u64,
    balance: i64,
}

impl Wallet {
    fn new(id: u64) -> Self {           // associated fn (constructor by
convention)
        Wallet { id, balance: 0 }
    }
    fn deposit(&mut self, cents: i64) { // mutating method
        self.balance += cents;
    }
}

fn main() {
    let mut w = Wallet::new(1);
    w.deposit(500);
    println!("wallet {} has {} cents", w.id, w.balance);
}

```

An **enum** is far more powerful than a Java enum: each variant can carry its own data. Enums model "one of several shapes," which is exactly how the book represents commands, events, and states:

Rust

```

enum Command {
    Open { id: u64 },           // a struct-like variant
    Deposit { id: u64, cents: i64 }, // with named fields
    Close,                     // a unit variant, no data
}

```

You take an enum apart with `match` — an exhaustive, expression-valued switch.

Exhaustive means the compiler refuses to compile unless you handle every variant, so adding a new command later forces you to update every place that matches on it:

Rust

```
fn describe(cmd: &Command) -> String {
    match cmd {
        Command::Open { id }           => format!("open wallet {id}"),
        Command::Deposit { id, cents } => format!("deposit {cents} into {id}"),
        Command::Close                 => "close".to_string(),
    }
}
```

`match` is one form of **pattern matching**, which also appears in `let` destructuring, `if let`, and function parameters. You will read patterns constantly; recognizing that the left side of a `=>` *binds names by destructuring the value* is the key.

`Option<T>`, `Result<T, E>`, and the `?` Operator

Rust has no `null` and no exceptions. Absence and failure are ordinary values, expressed by two standard enums.

`Option<T>` says "maybe a `T`": it is either `Some(value)` or `None`. This replaces `null`/`None`/`nil` — and because it is a distinct type, you cannot accidentally use a missing value as if it were present.

Rust

```
fn first_char(s: &str) -> Option<char> {
    s.chars().next()    // None if the string is empty
}

fn main() {
    match first_char("Lumen") {
        Some(c) => println!("first char is {c}"),
        None    => println!("string was empty"),
    }
}
```

Result<T, E> says "a **T** or an error **E**": it is either **Ok(value)** or **Err(error)**. This is how every fallible operation reports failure — there are no thrown exceptions to catch:

Rust

```
#[derive(Debug)]
struct ParseError;

fn parse_amount(s: &str) -> Result<i64, ParseError> {
    s.parse::<i64>().map_err(|_| ParseError)    // convert the std error into
    ours
}

fn main() {
    println!("{:?}", parse_amount("500"));    // Ok(500)
    println!("{:?}", parse_amount("oops"));    // Err(ParseError)
}
```

Handling every **Result** by hand would be tedious, so Rust gives you the **?** operator. Applied to a **Result**, **?** unwraps the **Ok** value or, on **Err**, returns that error from the current function immediately. It threads failures up the call stack without exceptions and without boilerplate:

Rust

```

#[derive(Debug)]
struct ParseError;

fn parse_amount(s: &str) -> Result<i64, ParseError> {
    s.parse::<i64>().map_err(|_| ParseError)
}

fn double_amount(s: &str) -> Result<i64, ParseError> {
    let n = parse_amount(s)?;    // on Err, return early; on Ok, bind n
    Ok(n * 2)
}

fn main() {
    println!("{:?}", double_amount("21"));    // Ok(42)
    println!("{:?}", double_amount("nope"));  // Err(ParseError)
}

```

 SPRING PARITY

Where Java throws and catches, and Python raises and excepts, Rust *returns* errors as values and propagates them with `?`. A function's signature tells you up front whether it can fail (`-> Result<T, E>`) and with what — failure is part of the type, not a hidden control-flow path. Firefly fixes the error type for its reactive and web layers to one `FireflyError`, so the `?` operator threads failures straight through a pipeline and out as an RFC 9457 problem response.

Traits and `impl` – Rust's Interfaces

A **trait** defines shared behavior — a set of method signatures a type can promise to provide. It is the closest thing Rust has to a Java interface or a Python protocol/ABC. You implement a trait for a type with `impl Trait for Type`, and a trait may supply default method bodies:

Rust

```

trait Greeter {
    fn greet(&self) -> String;           // required method
    fn shout(&self) -> String {         // default method, built on the required
one
        self.greet().to_uppercase()
    }
}

struct English;
impl Greeter for English {
    fn greet(&self) -> String {
        "hello".to_string()
    }
}

fn main() {
    let e = English;
    println!("{} / {}", e.greet(), e.shout()); // hello / HELLO
}

```

When you want a value whose concrete type is decided at run time — "any `Greeter`, I do not care which" — you use a **trait object**, written `dyn Trait` and always behind a pointer like `Box<dyn Trait>` or the `Arc<dyn Trait>` you met earlier. This is dynamic dispatch, the same mechanism as a Java interface reference:

Rust

```

trait Greeter {
    fn greet(&self) -> String;
}
struct English;
impl Greeter for English {
    fn greet(&self) -> String { "hello".to_string() }
}

fn main() {
    let greeters: Vec<Box<dyn Greeter>> = vec![Box::new(English)];
    for g in &greeters {
        println!("{}", g.greet());
    }
}

```

`#[derive(...)]` – Macros That Write `impl` Blocks for You

A `#[derive(...)]` attribute is a **macro** that auto-generates a trait implementation at compile time, saving you the boilerplate. `#[derive(Debug)]` writes a `Debug` impl (so `{:?}` can print the value); `#[derive(Clone)]` writes `Clone`; `#[derive(PartialEq)]` writes `==`:

Rust

```

#[derive(Debug, Clone, PartialEq)]
struct Money {
    cents: i64,
}

fn main() {
    let a = Money { cents: 500 };
    let b = a.clone();           // Clone derived
    assert_eq!(a, b);           // PartialEq derived
    println!("{:?}", a);         // Debug derived -> Money { cents: 500 }
}

```

This is exactly the machinery behind Firefly's own macros. When a later chapter writes `#[derive(Command)]` on a struct, the framework's `Command` derive macro generates the wiring that lets that struct be dispatched through the CQRS `Bus` — you write the data, the macro writes the plumbing. The `#[rest_controller]`, `#[event_listener]`, and `#[saga]` attributes you will meet are the same idea: code that writes code at compile time.

Generics and Lifetimes – Just Enough

Generics let one piece of code work over many types, written with `<T>` type parameters and bounded by the traits the code needs. You have already used generic types — `Vec<T>`, `Option<T>`, `Mono<T>`. Here is a generic *function*; the bound `T: PartialOrd + Copy` means "any `T` you can compare and copy":

Rust

```
fn largest<T: PartialOrd + Copy>(items: &[T]) -> T {
    let mut max = items[0];
    for &x in items {
        if x > max { max = x; }
    }
    max
}

fn main() {
    println!("{}", largest(&[3, 7, 2])); // works on i32
    println!("{}", largest(&[1.0, 9.5])); // and on f64
}
```

Lifetimes are the part of the type system that tracks *how long a reference is valid*, so the compiler can prove a borrow never outlives the data it points to. They are written with a leading apostrophe, like `'a`, and most of the time the compiler infers them and you never write one. When you do see one — for instance on a function that returns a reference borrowed from its inputs — read `'a` as "lives at least as long as":

Rust

```
fn longest<'a>(a: &'a str, b: &'a str) -> &'a str {
    if a.len() >= b.len() { a } else { b }
}

fn main() {
    println!("{}", longest("wallet", "ledger")); // -> "wallet"
}
```

 NOTE

you do not need to *write* generics or lifetimes to read this book; you need to not be alarmed by `<T>` and `'a` when they appear in a signature. `'static` is the one named lifetime worth remembering: it means "valid for the entire run of the program," and you will see it as a bound (`T: Send + 'static`) on values handed to `async` tasks.

Closures

A **closure** is an anonymous function written with `|params| body`. It can capture variables from the surrounding scope. Closures are everywhere in reactive pipelines — every `.map(...)`, `.filter(...)`, and step callback takes one:

Rust

```
fn main() {
    let add_one = |x: i32| x + 1;           // a closure bound to a name
    println!("{}", add_one(41));           // 42

    let xs = vec![1, 2, 3, 4];
    let evens: Vec<i32> = xs.iter()
        .copied()
        .filter(|x| x % 2 == 0)             // a closure passed to filter
        .collect();
    println!("{}", evens);                 // [2, 4]
}
```


By default a closure *borrow*s what it captures. Prefix it with `move` to make it *take ownership* of its captures instead — essential when the closure outlives the current scope, as when it is shipped to another thread or an async task:

Rust

```
fn main() {
    let label = String::from("amount");
    let print_it = move || println!("{}", label); // `move`: closure now owns
    `label`
    print_it();
}
```

🌱 SPRING PARITY

A closure is Rust's lambda — `|x| x + 1` is Java's `x -> x + 1` or Python's `lambda x: x + 1`. The wrinkle is capture: `move` is how you say "capture by value," which you reach for constantly when handing work to `tokio` tasks, because the task may run after the spawning function has returned.

Async, `await`, and `tokio`

Everything in Firefly is **asynchronous**. An `async fn` does not run when you call it; instead it returns a **Future** — a lazy description of work that produces a value later. Nothing happens until that future is *driven*, which you do with `.await`. Awaiting a future suspends the current task until the value is ready, freeing the thread to do other work in the meantime — that is how a handful of threads can serve thousands of concurrent connections.

Rust

```

async fn fetch_balance(id: u64) -> i64 {    // returns a Future<Output = i64>
    // ... awaits a database, an HTTP call, etc. ...
    id as i64 * 100
}

async fn report(id: u64) {
    let bal = fetch_balance(id).await;        // suspend until the value is ready
    println!("balance: {bal}");
}

```

Futures need a **runtime** to drive them. Firefly uses **tokio**, the de-facto async runtime for Rust. You rarely touch it directly, but you will see its attribute macros mark the entry points: **#[tokio::main]** turns an **async fn main** into a real program, and **#[tokio::test]** does the same for an async test:

Rust

```

async fn fetch_balance(id: u64) -> i64 {
    id as i64 * 100
}

#[tokio::main]
async fn main() {
    let bal = fetch_balance(7).await;
    println!("balance: {bal}");    // balance: 700
}

```

🌿 SPRING PARITY

If `Mono` / `Flux` and reactive streams are familiar to you, you already have the right intuition. Rust's `Future` is the lower-level primitive that an `async fn` produces, and `tokio` is the scheduler that runs them — think of it as the event loop. Firefly layers its own reactive types, `Mono<T>` (0-or-1 value) and `Flux<T>` (0..N values), *on top of* this `async` / `await` foundation: a `Mono` is the reactive analog of "an async function that returns a `T`," and a `Flux` of "an async stream of `T`." They are lazy, composable, and backpressure-aware, and they are the keystone of the framework. The Reactive Model chapter is where you meet them in full.

Reading the Rest of the Book

That is the whole toolkit. With ownership and borrowing, `Option` / `Result` and `?`, traits and `#[derive]`, closures, and `async` / `await` in hand, every later listing is readable.

A few reminders that hold for the entire book:

- **The listings are real.** Every snippet from Chapter 1 onward is lifted from the runnable Lumen companion crate in `samples/lumen`; the build breaks if a listing drifts from the source. What you read is what compiles and runs.
- **One import does most of the work.** Listings begin with `use firefly::prelude::*;`, which brings in the whole high-frequency surface — `Mono`, `Flux`, `Bus`, and the macros — through Lumen's single `firefly` dependency.
- **A snippet marked `ignore` elides setup for focus.** When a listing omits surrounding code to keep your eye on one idea, it is flagged as such; the API names, types, and signatures in it are exactly what the crates expose.

Turn to [Conventions](#) for the typographic details, then begin the Lumen journey in [Why Firefly for Rust](#).



PART I

Foundations





CHAPTER 1

Why Firefly for Rust?

By the end of this chapter you will understand the problem Firefly solves, how its tiers fit together behind a single facade crate, and why Lumen — the digital-wallet service you grow over the rest of the book — depends on exactly **one** Firefly crate to get all of it. No code lands in Lumen yet; this chapter sets the stage and the philosophy. The next one boots the scaffold.

The cohesion problem

Picture your first day on a new Rust microservice. Before you write one line of business logic, you face a cascade of choices. Which HTTP layer — axum, actix, warp, poem? Which database story — sqlx, SeaORM, diesel, raw `tokio-postgres`? How do you wire dependencies — a hand-rolled `AppState`, a DI crate, lazy statics? How do you handle configuration, errors, correlation IDs, metrics, graceful shutdown? Every team invents its own answer.

You assemble a bespoke stack, glue it together with good intentions, and ship. Six months later a second team starts a second service and makes entirely different choices. Now you have two codebases with incompatible conventions, different error shapes, different observability stories, and no shared understanding of how anything works.

Rust gives you infinite choice. What it does not give you is cohesion.

The stack-assembly problem is not a skills failure — it is a tooling gap. Mature ecosystems closed it with a single opinionated, batteries-included framework that makes sensible choices, lets you override what matters, and enforces a consistent idiom across every service. Firefly is that framework for Rust: it makes the cross-cutting decisions once, so every service shares one idiom.

What is Firefly?

Firefly is a **cohesive, reactive, async-native framework** for building production-grade Rust services. It makes the cross-cutting decisions for you — HTTP middleware, configuration, caching, CQRS, messaging, security, observability — all integrated, all consistent, with production-ready defaults from the very first `cargo run`.

Under the hood Firefly delegates to battle-tested libraries — `tokio` for the runtime, `axum` / `tower` for HTTP, `serde` for serialization, `tracing` for logging, RustCrypto for crypto — but you depend on **Firefly's ports** (object-safe `async_trait` traits), and you select concrete adapters at wiring time. Swap an in-memory event store for PostgreSQL, or the in-process broker for Kafka, without touching a single line of business logic — exactly the swap Lumen is structured to make.

Firefly's defining principles:

- **Composed, not constructed.** One call wires the whole infrastructure tier — middleware chain, cache, CQRS bus, event broker, health composite, metrics,

scheduler, lifecycle. You write commands, queries, handlers, and routes; nothing more. Lumen builds its core with `WebStack::new(CoreConfig { .. })`.

- **Contract-first and interoperable.** The wire contract — the `application/problem+json` shape, the `Idempotency-Key` semantics, the saga step definitions, the event envelopes — is a stable, versioned, language-neutral specification. Any service that honors it interoperates with a Firefly service byte-for-byte, so Firefly slots into a polyglot fleet without bespoke glue.
- **Pluggable at the adapter layer.** Each integration point (cache, broker, IDP, ECM, notification channel) is an object-safe port with multiple adapter implementations selected at wiring time as an `Arc<dyn Port>`.
- **Observable by default.** `tracing` structured logging with correlation-id enrichment, actuator health/metrics endpoints, RFC 9457 error envelopes, and a startup banner are all on out of the box.
- **Reactive to the core.** A first-class `Mono` / `Flux` reactive surface runs from reactive endpoints through reactive repositories, the reactive HTTP client, and reactive EDA/CQRS — a lazy, composable, backpressure-aware streaming model built natively on tokio.

***Design note.** One `WebStack::new` call stands up the middleware, the bus, the broker, health, and metrics — Firefly's composition root. Configuration layers defaults → profile → environment, and any handler can return a `Mono<T>` / `Flux<T>`. If you have used a batteries-included framework before, this will feel familiar.*

The one-dependency facade

Here is the part that surprises people. Lumen — a service with CQRS, event sourcing, a saga, JWT security, scheduling, and an actuator surface — declares exactly one Firefly dependency. This is its real `Cargo.toml`:

TOML

[dependencies]

```
# The whole framework AND every `#[derive(...)]` / `#[...]` macro.
firefly = { version = "26.6.5" }

# The two ecosystem crates a Firefly service still writes against directly:
# axum (you author the controller handlers) and serde (your messages and
# event payloads are Serialize/Deserialize).
axum = { version = "0.7" }
serde = { version = "1", features = ["derive"] }
serde_json = "1"

# Async runtime, plus the id/clock crates the wallet domain uses.
tokio = { version = "1" }
uuid = { version = "1", features = ["v4"] }
chrono = { version = "0.4", features = ["serde"] }

# Backs the `async fn` trait methods the domain ports implement.
async-trait = { version = "0.1" }
```

The `firefly` crate is a **facade**: it re-exports every `firefly-*` crate behind a clean path (`firefly::cqrs`, `firefly::eventsourcing`, `firefly::reactive`, `firefly::security`, ...) and re-exports every macro at the crate root. The high-frequency surface — plus all the macros — comes in through a single glob:

Rust

```
use firefly::prelude::*;
```

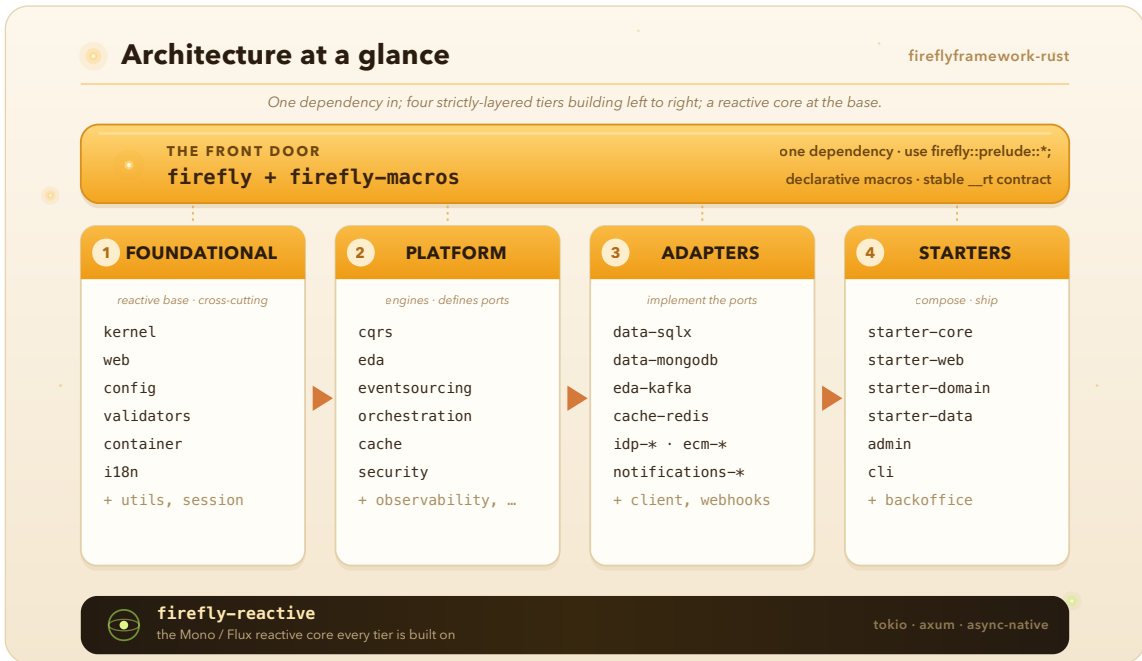
That one line gives Lumen the CQRS `Bus`, the dependency-injection `Container`, the `Scheduler`, the saga `Saga` / `Step`, the lifecycle `Application`, the reactive `Mono` / `Flux`, the `WebResult` / `WebError` web types, the `FireflyError` kernel error, and every `#[derive(...)]` / `#[...]` macro the service uses.

Lumen takes the discipline one step further: even its typed error enums — `MoneyError`, `DomainError`, `CqrsError` mapping — hand-write `Display` and `std::error::Error` instead of reaching for `thiserror`. The one-dependency promise holds end to end, and the chapters point it out where it matters.

***Design note.** The `firefly` facade is a single front-door crate: one coordinate on your dependency list pulls in a curated, calendar-version-aligned stack, and `use firefly::prelude::*` brings the whole high-frequency surface and every macro into scope at once. One dependency, no version skew to manage.*

The tiers behind the facade

Behind that single crate the framework is organized into strictly-layered tiers, with a left-to-right dependency direction. Each tier may depend on the tiers to its left, never to its right; the Cargo crate graph enforces the layering. You rarely name these crates directly — the facade re-exports them — but knowing the shape tells you where each capability lives.



Firefly at a glance: a service depends only on the firefly facade (the front door); the four tiers build left to right, each depending only on the tiers to its left, all on the firefly-reactive Mono / Flux core.

- **Foundational** crates are the vocabulary: `firefly-kernel` (errors, clock, correlation scopes, DDD kit), `firefly-reactive` (Mono / Flux), `firefly-web` (middleware), `firefly-config`, `firefly-validators`, `firefly-i18n`, and `firefly-container` — a full dependency-injection engine with component scanning and stereotype derives, covered in depth in Chapter 4.
- **Platform** crates are the capabilities: caching, CQRS, EDA, event sourcing, orchestration, scheduling, resilience, security, observability. Lumen reaches for `firefly::cQRS`, `firefly::eventsourcing`, `firefly::orchestration`, `firefly::scheduling`, and `firefly::security` here.
- **Adapters** are the concrete integrations: the REST/reactive HTTP client, the IDP vendors, ECM, notifications, the event transports (Kafka, RabbitMQ, Postgres outbox, Redis Streams), and the persistence adapters (`firefly-data-sqlx` for relational stores, `firefly-data-mongodb` for documents) — a pluggable multi-

database story that Chapter 7 builds on. Lumen ships on the in-memory adapters and points at the production swaps in callouts.

- **Starters** bundle a sensible default stack so a service depends on one crate. Lumen's web tier is `firefly::starter_web::WebStack`, which wires the core (`firefly::starter_core`) plus the web middleware in one constructor.

For the full per-crate catalogue see the [Module Index](#).

Choosing your adapters

Lumen runs with **zero external infrastructure** — that is what makes it a good teaching baseline and a fast test target. It boots on the in-process `MemoryEventStore` and the in-process broker, so `cargo run` and `cargo test` need nothing but the crate. When you are ready for production, you change the *wiring*, not the handlers:

- **Event store.** Swap `MemoryEventStore` for a durable adapter where the `Arc<dyn EventStore>` is constructed; the `Ledger`, the projection, and every command handler are untouched.
- **Event transport.** The in-process broker that carries Lumen's domain events implements the same `Broker` port as `firefly-eda-kafka`, `-rabbitmq`, `-postgres`, and `-redis`. Swap the constructor, keep your `#[event_listener]`.
- **Cache, identity, notifications.** Code against the parent-port trait (`cache::Adapter`, `security::Verifier`, `notifications::Channel`) and pull in the concrete adapter crate at wiring time, so heavy SDKs stay out of services that do not use them.

This is the thread that runs through the whole book: Lumen is written so the in-memory baseline and the production deployment differ only at the composition root.

The road ahead: Lumen, chapter by chapter

The rest of the book is Lumen's growth, additive and in order. The early chapters introduce the framework with small standalone snippets; **Lumen proper begins in Chapter 6:**

- **Foundations** — scaffold and boot Lumen, bind its configuration and profiles, understand the composition root, master `Mono` / `Flux`, and expose the first validated REST endpoints.
- **Modeling & persisting** — a read model behind a repository, the `Money` value object and the `Wallet` aggregate, and the CQRS command/query split on a bus.
- **Event-driven** — domain events, a projection that keeps the read model current, and the event-sourced ledger that folds its stream.
- **Into microservices** — an HTTP-client sketch and the compensating transfer saga.
- **Secure · observe · ship** — JWT bearer auth and RBAC, the actuator surface, caching, a scheduled task, the test suite, and the production entry point with graceful shutdown and a reactive streaming endpoint.

By the last page, Lumen is the complete `samples/lumen` crate — and you have written every line of it.

Recap – what changed in Lumen

Nothing in code yet. This chapter framed the journey:

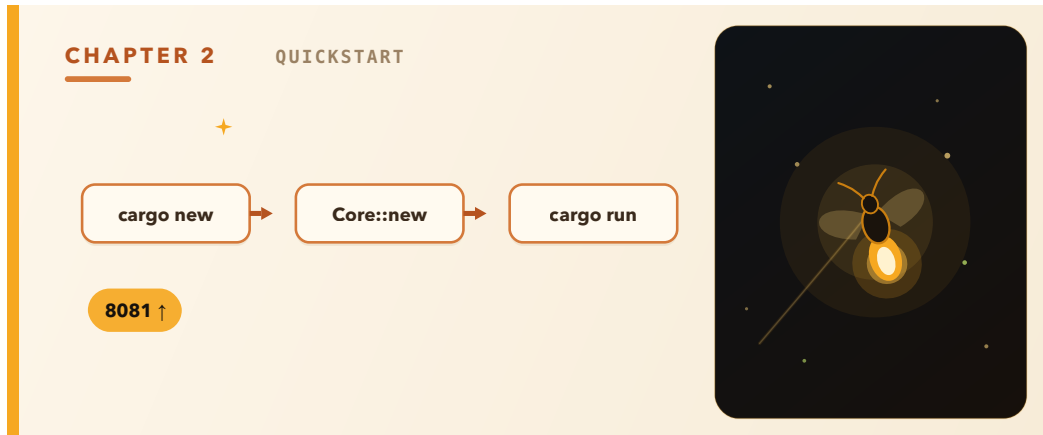
- The **cohesion problem** Firefly exists to solve, and the opinionated, one-framework answer it brings to Rust.
- The **one-dependency facade** — Lumen depends on a single `firefly` crate, and `use firefly::prelude::*;` brings in the whole high-frequency surface and every macro. Even the typed errors avoid `thiserror`, so the promise holds end to end.

- The tiers behind that facade (foundational → platform → adapters → starters) and the in-memory-to-production **adapter swap** that Lumen is built to make at the composition root.

Exercises

1. Open `samples/lumen/Cargo.toml` and confirm the dependency list: one `firefly`, plus `axum` / `serde` / `serde_json` / `tokio` / `uuid` / `chrono` / `async-trait`. Note that no `firefly-*` sub-crate is listed directly.
2. Skim `samples/lumen/src/lib.rs`. List the ten modules it declares (`money`, `domain`, `ledger`, `commands`, `transfer`, `tcc_transfer`, `security`, `compliance`, `web`, `housekeeping`) and predict which book part introduces each.
3. Run `cargo doc -p firefly-sample-lumen --open` and read the crate-level documentation. It contains the same "building block → module → Firefly surface" table the book is organized around.
4. For each of these production swaps, find the port trait it would implement in the facade: a Postgres event store, a Kafka broker, a Redis cache. (Hint: `firefly::eventsourcing`, `firefly::eda`, `firefly::cache`.)

The next chapter gets Lumen running. Turn to the [Quickstart](#).



CHAPTER 2

Quickstart: Zero to a Running Service

*By the end of this chapter **Lumen** — the digital-wallet and ledger service you will grow across the rest of the book — exists as a real crate: it compiles, prints a banner, serves a live actuator, and shuts down gracefully. It does almost nothing yet. That is the point. Everything from here on is additive: every later chapter slices a little more out of the finished `samples/lumen` crate and folds it back into the story, and nothing you write now gets thrown away.*

This chapter takes you from an empty directory to a running Lumen process in a few minutes. Two paths get you there: the `firefly` CLI (fastest) or plain `cargo`. Either way, the destination is the same — a binary crate whose *only* Firefly dependency is the `firefly` facade.

Prerequisites

Shell

```
rustc --version    # 1.88 or later
cargo --version
```

That is all you need. Lumen's default stack requires **no external infrastructure** — the event store, the event broker, and the read model are all pure Rust, in-process. You will swap each of them for real infrastructure (Postgres, Kafka) in [Production & Deployment](#), but never before you are ready.

Path A – scaffold with the **firefly** CLI

Install the developer CLI once, then scaffold the project:

Shell

```
cargo install --path crates/cli    # from a checkout of the framework
# or: cargo install firefly-cli

firefly new lumen --archetype web-api --features web,cqrs --git
cd lumen
cargo run
```

firefly new generates a Cargo crate with a **src/** tree, a **firefly.yaml**, a **.gitignore**, a **README.md**, a **Dockerfile**, and a **tests/** directory. The **web-api** archetype is the right starting shape for Lumen: a web service with the CQRS bus already wired. See [The CLI](#) for every archetype and generator.

 TIP

Run `firefly new --list` to see every archetype (`core` , `web-api` , `web` , `hexagonal` , `library` , `cli`) and feature flag, or `firefly new lumen --dry-run` to preview the plan without writing files.

Path B – start from cargo

If you would rather see every line yourself, create the crate by hand. This is exactly the shape `samples/lumen` has, so the rest of the book lines up with it listing for listing.

Shell

```
cargo new lumen
cd lumen
```

Lumen's `Cargo.toml` makes the one-dependency story concrete. The whole framework — CQRS, dependency injection, the reactive web stack, event sourcing, saga orchestration, scheduling, security, observability — and *every* `#[derive(...)]` / `#[...]` macro arrive through a single crate:

TOML

```

# Cargo.toml
[dependencies]
# The one-dependency front door: the `firefly` facade re-exports the whole
# framework AND every macro. Generated code resolves runtime types through the
# facade, so Lumen never lists the underlying `firefly-*` crates.
firefly = "26.6.5"

# The two ecosystem crates a Firefly service still writes against directly:
# axum (you author the controller handlers) and serde (your messages and event
# payloads are Serialize/Deserialize). serde_json encodes the event payloads.
axum = "0.7"
serde = { version = "1", features = ["derive"] }
serde_json = "1"

# The async runtime, and the id/clock crates the domain uses later.
tokio = { version = "1", features = ["rt-multi-thread", "macros", "net",
"signal"] }
uuid = { version = "1", features = ["v4"] }
chrono = "0.4"
async-trait = "0.1"

[features]
# The reactive streaming endpoint is feature-gated so the teaching baseline
# stays lean; chapter 20 turns it on. It needs nothing beyond `firefly`.
default = []
streaming = []

```

Design note. Many frameworks make you assemble a constellation of starter or plugin artifacts and keep their versions aligned by hand. Firefly collapses all of that into one `firefly` line: there is no starter to forget and no version skew between subsystems like `firefly-web` and `firefly-cqrs` — every `firefly-*` crate ships as one calendar-versioned release, and you depend on the facade.

Lumen's two entry points

A Firefly service has a *composition root* (where the application is assembled) and a *process entry point* (`main`, where it is run). Keeping them apart is what lets the tests drive the fully-wired app in-process without binding a socket — you will lean on that hard in [Testing](#).

Lumen names the assembled application `LumenApp` and builds it in `src/web.rs::build_app`. For now the body is almost empty; later chapters give it a controller, a CQRS bus, an event-sourced ledger, and a security chain. What matters in this chapter is the shape and the two constants that name the service:

Rust

```
// src/web.rs
use firefly::prelude::*;
use firefly::starter_web::WebStack;

/// Lumen's application name (banner + `/actuator/info`).
pub const APP_NAME: &str = "lumen";

/// The released framework version, surfaced in the banner.
pub const VERSION: &str = firefly::VERSION;

/// The fully-assembled Lumen application. Right now it carries only the
/// web-tier stack; the bus, ledger, and read model arrive in later chapters.
pub struct LumenApp {
    /// The web-tier starter (CORS / security-headers / correlation / metrics
    /// on by default), which `Deref`s to the infrastructure `Core`.
    pub web: WebStack,
}

/// Assembles a `LumenApp` over in-memory infrastructure – the default for
/// tests and a no-infra `cargo run`.
pub async fn build_app() -> LumenApp {
    let web = WebStack::new(firefly::starter_web::CoreConfig {
        app_name: APP_NAME.into(),
        app_version: VERSION.into(),
        ..Default::default()
    });
    LumenApp { web }
}
```

`WebStack::new` is the one call that wires the whole web tier: the RFC 9457 problem renderer, correlation-id propagation, idempotency replay, the in-process cache, the CQRS bus, the event broker, the health and metrics registries, the scheduler, plus the web batteries (CORS, security headers, request metrics, the access log) — all from

those defaulted `CoreConfig` fields. `WebStack` derefs to the infrastructure `Core`, so `app.web.bus`, `app.web.broker`, and friends are right there when later chapters need them.

Design note. `WebStack::new(CoreConfig { .. })` stands up the entire managed context in one declaration — and nothing is reflective or hidden. `CoreConfig` is a plain struct whose fields are the knobs, so "what is wired" is exactly "what the struct says." Firefly chose explicit, inspectable configuration over runtime magic on purpose.

The process entry point lives in `src/main.rs`. It builds the app, turns on logging, prints the banner, serves the public API and the actuator on separate ports, and runs under the lifecycle `Application` with graceful shutdown:

Rust

```
// src/main.rs
use firefly_sample_lumen::web::{build_app, APP_NAME, VERSION};

/// Default bind address of the public API server.
const DEFAULT_ADDR: &str = "127.0.0.1:8080";
/// Default bind address of the admin (actuator) server.
const DEFAULT_ADMIN_ADDR: &str = "127.0.0.1:8081";

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let app = build_app().await;
    // Best-effort: a test harness may already own the global subscriber.
    let _ = app.web.init_logging();

    let api = app.router();
    let admin = app.web.actuator_router(Vec::new());

    app.web.print_banner();
    println!("{}", APP_NAME) :: digital-wallet & ledger (v{VERSION});

    let api_addr = std::env::var("LUMEN_ADDR").unwrap_or_else(|_|
DEFAULT_ADDR.to_owned());
    let admin_addr =
        std::env::var("LUMEN_ADMIN_ADDR").unwrap_or_else(|_|
DEFAULT_ADMIN_ADDR.to_owned());

    let application = app
        .web
        .new_application()
        .on_server("api", move |shutdown| async move {
            let listener = tokio::net::TcpListener::bind(&api_addr).await?;
            axum::serve(listener, api)
                .with_graceful_shutdown(shutdown.wait())
                .await?;
            Ok(())
        })
        .on_server("admin", move |shutdown| async move {
            let listener = tokio::net::TcpListener::bind(&admin_addr).await?;
```

```

        axum::serve(listener, admin)
            .with_graceful_shutdown(shutdown.wait())
            .await?;
        Ok(())
    });

    if let Err(err) = application.run().await {
        if !err.is_cancelled() {
            eprintln!("application failed: {err}");
            std::process::exit(1);
        }
    }
    Ok(())
}

```

A few things to notice, because they recur in every chapter:

- `app.router()` is the public surface. In this chapter it is empty; in [Your First HTTP API](#) it gains the wallet routes, and in [Security](#) it gains the JWT layer. `main` never changes when that happens — the composition root absorbs the growth.
- `app.web.actuator_router(...)` is the management surface, served on a *separate* port (`8081` by default) so it never leaks onto the public network.
- `LUMEN_ADDR` / `LUMEN_ADMIN_ADDR` override the bind addresses from the environment — your first taste of the typed configuration story in [Configuration](#).
- `application.run()` traps SIGINT/SIGTERM and drains in-flight requests before exiting. A cancelled run is a clean shutdown, not an error.

Run it

Shell

```
cargo run
```

You will see the Firefly banner, then Lumen's own line:

Text

```
:: lumen :: digital-wallet & ledger (v26.6.5)
```

Even with no business routes yet, the actuator is live on the admin port:

Shell

```
# Liveness / readiness – on the admin port, never the public one.
curl localhost:8081/actuator/health
# {"status":"UP", ...}

# Build metadata – note app_name and app_version flow straight from CoreConfig.
curl localhost:8081/actuator/info
# {"app":{"name":"lumen","version":"26.6.5"}, ...}
```

What you got for free

Without writing any of it yourself, Lumen already has:

- **RFC 9457 problem responses.** Any handler error renders as `application/problem+json`, and a panic is caught and rendered as a 500 problem. (You will use this from the very first endpoint in chapter 6.)
- **Correlation IDs.** Every response echoes an `X-Correlation-Id`; an incoming one is honored and scoped through the whole request.
- **Idempotency.** Every `POST` / `PUT` / `PATCH` carrying an `Idempotency-Key` header is recorded; repeating the request replays the stored response, and reusing the key with a different body is a `409`.
- **A management surface.** `/actuator/{health,info,metrics,...}` on a separate listener.

- **Graceful shutdown.** `application.run()` traps SIGINT/SIGTERM and drains.

***Design note.** Health, info, and metrics on a dedicated management port, plus production-grade request middleware, all stood up by a single `WebStack::new` — no config file to author first and no annotations to remember. This is Firefly's actuator surface, on by default.*

Recap – what changed in Lumen

Before	After this chapter
empty directory	a compiling <code>firefly-sample-lumen</code> crate with one Firefly dependency
no entry point	<code>build_app()</code> composition root + a <code>main</code> that runs under the lifecycle <code>Application</code>
nothing to run	a live actuator on <code>:8081</code> , a (still-empty) public API on <code>:8080</code> , graceful shutdown
—	<code>APP_NAME</code> / <code>VERSION</code> constants that name the service and feed <code>/actuator/info</code>

Lumen is now a real, runnable service that happens to have no business logic. Every subsequent chapter fills that emptiness in — never by rewriting what you have, only by extending the composition root.

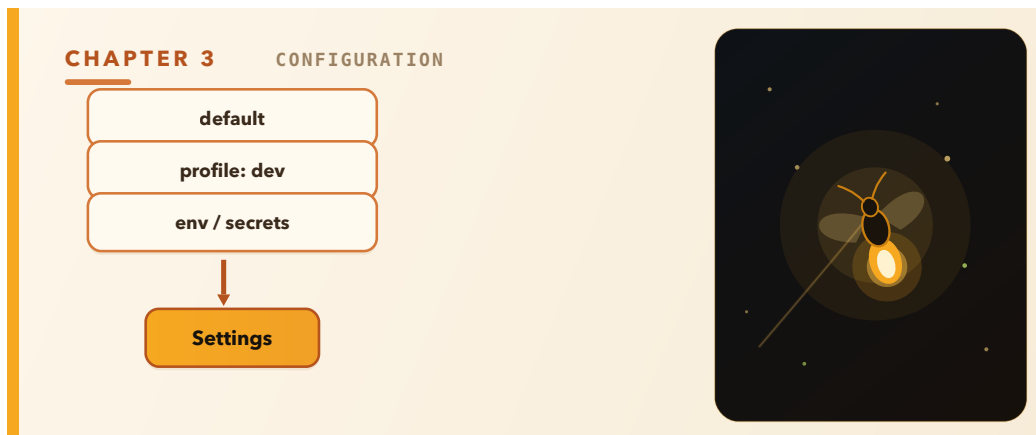
Exercises

1. **Move the ports.** Start Lumen with `LUMEN_ADDR=127.0.0.1:9090` `LUMEN_ADMIN_ADDR=127.0.0.1:9091` `cargo run`, then `curl localhost:9091/actuator/health`. Confirm the public and admin surfaces moved independently — this is the seam [Configuration](#) builds on.

2. **Read your own metadata.** `curl localhost:8081/actuator/info` and find the `app.name` / `app.version` values. Change `APP_NAME` in `web.rs`, re-run, and watch the banner and `/actuator/info` both follow — one constant, two surfaces.
3. **Provoke graceful shutdown.** Run Lumen, then press `Ctrl-C`. Notice the process exits cleanly (no stack trace): `application.run()` treated the signal as a shutdown, not a failure.
4. **Preview the scaffold.** Even if you took Path B, run `firefly new lumen2 --archetype web-api --dry-run` and compare the generated plan to the files you wrote by hand.

Where to go next

- Add typed, layered, profile-aware configuration in [Configuration](#) — and replace those raw `std::env::var` calls.
- Learn how the composition root resolves collaborators in [Dependency Wiring](#).
- Give Lumen its first real endpoints in [Your First HTTP API](#).



CHAPTER 3

Configuration, Profiles & Secrets

By the end of this chapter Lumen reads its identity and its bind addresses from configuration instead of hard-coding them: the `app_name` and `app_version` that flow into the banner and `/actuator/info`, and the `LUMEN_ADDR` / `LUMEN_ADMIN_ADDR` overrides `main` already honors. You will also see the typed, layered, profile-aware machinery Lumen grows into as it moves toward production.

In the last chapter Lumen named itself with two `pub const` strings and pulled its ports straight off the environment with `std::env::var`. That is the right starting point — but a real wallet service runs in dev, in CI, and in prod, and each environment wants different ports, log levels, and (eventually) database URLs. `firefly-config` provides **typed, layered configuration binding**: you declare a `serde`-deserializable struct, call `load` / `load_from_profile`, and the loader merges sources in precedence order, resolves the active profile, resolves `${...}` placeholders, and binds the flat dot-keyed map onto your struct. If you've used a batteries-included framework before, the shape will feel familiar.

Design note. Firefly binds a profile-aware `application.yaml` → profile → environment hierarchy onto typed structs. The flattening and binding rules are specified precisely (see below) so the same `application.yaml` produces the same keys deterministically — Firefly treats this determinism as a guarantee, not an accident.

Where Lumen is today: app identity

Recall the composition root from the Quickstart:

Rust

```
// src/web.rs
let web = WebStack::new(firefly::starter_web::CoreConfig {
    app_name: APP_NAME.into(),           // "lumen"
    app_version: VERSION.into(),         // firefly::VERSION
    ..Default::default()
});
```

`CoreConfig` is itself plain configuration: every field is a knob, and the two Lumen sets — `app_name` and `app_version` — are exactly the values `/actuator/info` reports and the banner prints. The remaining fields default (in-memory cache, in-process broker, a fresh CQRS bus), which is why a bare `cargo run` needs no infrastructure. Promoting any of those to real infrastructure is a one-field change you will make in [Production & Deployment](#); the config story in this chapter is how those values stop being literals and start coming from files and the environment.

Defining configuration

A configuration struct is plain `serde`. Nested structs become nested dot-keyed sections (`web.port`, `cache.adapter`). Here is the shape Lumen would adopt as it outgrows the two constants:

Rust

```

use serde::Deserialize;

#[derive(Debug, Deserialize)]
struct Web {
    /// Public API bind address – the typed home of LUMEN_ADDR.
    addr: String,
    /// Admin/actuator bind address – the typed home of LUMEN_ADMIN_ADDR.
    admin_addr: String,
}

#[derive(Debug, Deserialize)]
struct LumenConfig {
    name: String,
    web: Web,
    tags: Vec<String>,
}

```

The binder is **type-driven**: `"9090"` binds onto a `u16`, `"alpha,beta"` splits onto a `Vec<String>`, `"true"` parses onto a `bool`, and missing keys produce zero values — so plain `#[derive(Deserialize)]` structs bind without `#[serde(default)]`.

Loading with profiles

The canonical helper reads `application.yaml`, then the profile-specific `application-{profile}.yaml`, then `LUMEN_*` environment variables:

Rust

```
use firefly_config::{load_from_profile, ConfigError};

fn main() -> Result<(), ConfigError> {
    // dir, app basename, fallback profile (FIREFLY_PROFILE overrides).
    let cfg: LumenConfig = load_from_profile("/etc/lumen", "application",
"dev");
    println!("public API on {}", cfg.web.addr);
    Ok(())
}
```

`FIREFLY_PROFILE` selects the profile file at runtime — `FIREFLY_PROFILE=prod` reads `application-prod.yaml`. A comma-separated value (`FIREFLY_PROFILE=dev,cloud`) overlays one file per profile in order. This is how Lumen would carry an in-memory event store in `dev` and a Postgres event store in `prod` without a single `if` in the wiring code.

Source precedence

`Layered::new(vec![s1, s2, ...])` merges from left to right — last write wins. The canonical chain is:

Order	Source	Beats
1	Defaults — <code>StaticSource::new(name, entries)</code>	nothing
2	Base YAML — <code>from_optional_yaml("application.yaml")</code>	defaults
3	Profile YAML — <code>from_optional_yaml("application-prod.yaml")</code>	base
4	Environment — <code>from_env("LUMEN")</code>	YAML files
5	CLI flags — <code>FlagSource::new().set("web.addr", "0.0.0.0:80")</code>	everything

So an environment override (`LUMEN_WEB_ADDR=0.0.0.0:80`) always beats a YAML file, and a CLI override always beats both. That precedence is precisely why `main` can read `LUMEN_ADDR` and have it win over any baked-in default. Build the chain explicitly when you need full control:

Rust

```
use firefly_config::{from_env, from_optional_yaml, load, Source, StaticSource};

let sources: Vec<Box<dyn Source>> = vec![
    Box::new(StaticSource::new("defaults", [("web.addr".into(),
"127.0.0.1:8080".into())])),
    Box::new(from_optional_yaml("application.yaml")),
    Box::new(from_env("LUMEN")),
];
let cfg: LumenConfig = load(&sources)?;
```

YAML subset and value rules

Files are parsed by a line-by-line YAML-subset scanner (no general-purpose YAML dependency), so the flattened output is deterministic and stable for any given

`application.yaml`:

YAML

```
name: lumen
web:
  addr: 127.0.0.1:8080
  admin-addr: 127.0.0.1:8081
tags: wallet, ledger, demo  # sequences of scalars are comma-joined
```

- nested mappings become dot-joined, lower-cased keys;
- scalar lexemes are preserved verbatim until the binder parses them against the target field type;

- duplicate keys follow last-write-wins;
- aliases, anchors, multi-doc, tags, and flow sequences are deliberately not interpreted.

Supported leaf kinds: `String`, `bool` (accepts `1/0`, `t/f`, and `true/false` in any case), every integer width, `f32/f64`, `char`, unit enums (by variant name), `Option<T>`, sequences of scalars, and `HashMap<String, _>` subtrees. For durations, use an `i64` field plus a conversion: `Duration::from_millis(cfg.cache.ttl as u64)`.

NOTE

Keys are normalized kebab ↔ snake, so `admin-addr:` in YAML binds an `admin_addr` serde field.

Placeholders

`load` / `bind` run a post-merge pass resolving `${...}` placeholders in values (also exposed standalone as `resolve_placeholders(&flat)`):

YAML

```
name: lumen
datasource:
  url: ${DATABASE_URL:postgres://localhost/lumen}  # env, else default
  pool: ${name}-pool                               # config reference
```

- `${ENV_VAR}` — a literal environment variable;
- `${name}` — a config reference, resolved recursively with a depth-10 guard against cycles;
- `${key:default}` — a fallback when neither environment nor config resolves `key`;
- **environment beats config**: `${name}` honors `FIREFLY_NAME` before the merged map.

An unresolvable placeholder without a default raises `ConfigError::Placeholder`.

Binding config straight into a bean –

`#[derive(ConfigProperties)]`

Loading a struct by hand is fine for `main`. But Lumen's services want their configuration *injected*, not threaded through every constructor. The `#[derive(ConfigProperties)]` macro turns a `serde` struct into a container-managed, prefix-bound bean — the exact pattern the dependency-injection chapter builds on:

Rust

```
use firefly::prelude::*;
use serde::Deserialize;

/// Binds the `lumen.web.*` config subtree into an injectable bean.
#[derive(Deserialize, ConfigProperties, Default)]
#[firefly(prefix = "lumen.web")]
pub struct WebProperties {
    pub addr: String,
    #[serde(default)]
    pub admin_addr: String,
}
```

Any `#[derive(Service)]` bean can then `#[autowired] props: Arc<WebProperties>` and receive the bound values — no manual `load`, no global. You will wire one in [Dependency Wiring](#). For one-off scalars there is an even lighter touch: a `#[firefly(value = "${lumen.web.addr:127.0.0.1:8080}")]` field injects a single resolved value with a default.

Design note. Firefly offers two binding styles. A prefix-bound bean

(`#[derive(ConfigProperties)] + #[firefly(prefix = "...")]`) pulls a whole config subtree into one injectable struct, while single-value injection (`#[firefly(value = "${...}")]`) wires one resolved scalar onto a field. Both bind against the same merged, profile-resolved, placeholder-expanded map described above.

Config-driven auto-configuration – datasource and security from `application.yaml`

The properties-binding machinery so far hands you a typed struct. Firefly's infrastructure crates take the next step: a handful of subsystems are **config-driven and DI-free** — you bind a plain `serde` struct from `application.yaml`/env, then `await` a single auto-configure call at boot, and the subsystem stands itself up. No container, no manual builder chains, no `if scheme == "postgres"` branching in your wiring. Two subsystems Lumen leans on this way are its datasource and its security layer.

Both feed off one YAML tree. `firefly.datasource.*` binds onto `DataSourceProperties` and `firefly.security.*` onto `SecurityProperties`:

YAML

```

firefly:
  datasource:
    url: ${DATABASE_URL:postgres://localhost/lumen} # scheme picks the backend
    max-connections: 16
    min-connections: 2
    acquire-timeout-ms: 5000
    idle-timeout-ms: 600000
    max-lifetime-ms: 1800000
  security:
    jwt:
      jwk-set-uri: https://idp.example.com/.well-known/jwks.json
      issuer-uri: https://idp.example.com/
      audience: lumen-api
    bearer:
      header-name: Authorization
      allow-anonymous: false

```

Datasource – `DataSourceProperties` → pool → transaction manager

`DataSourceProperties` is a plain `serde` struct — `{ url, max_connections, min_connections, acquire_timeout_ms, idle_timeout_ms, max_lifetime_ms }`. The URL scheme selects the backend, each behind its own cargo feature: `postgres://` / `postgresql://` → PostgreSQL, `mysql://` → MySQL, `sqlite:` → SQLite. A `0` for any pool setting leaves the `sqlx` default in place.

`firefly_data_sqlx::auto_configure(&props)` does the one thing you want at boot: it builds the connection pool **and** registers a `SqlxTransactionManager` over it, so `#[transactional]` resolves with no manual wiring. The returned `Db` is the same pool, ready to build typed repositories. (For finer control, `Db::connect(url)` and `Db::connect_with(&props)` build just the pool.)

Rust

```
use firefly_data_sqlx::{auto_configure, DataSourceProperties};

// `Db` carries the pool; auto_configure also registers the tx manager.
let db = auto_configure(&props).await?;    // Result<Db, FireflyError>
```

Security – `SecurityProperties` → verifier → bearer layer

`SecurityProperties` nests `{ jwt: JwtProperties, bearer: BearerProperties }`. `JwtProperties` holds `{ jwk_set_uri, issuer_uri, audience, secret, algorithm, expiration_seconds }`; `BearerProperties` holds `{ header_name, allow_anonymous }`. Two functions turn that into running middleware:

- `verifier_from_config(&JwtProperties)` returns `Result<Option<Arc<dyn Verifier>>, SecurityError>`. A non-empty `jwk_set_uri` builds a JWKS (RS256) resource-server verifier; otherwise a non-empty `secret` builds an HMAC verifier (`HS256` / `HS384` / `HS512`); otherwise `None`.
- `bearer_layer_from_config(&SecurityProperties)` returns `Result<Option<BearerLayer>, SecurityError>` — the ready-to-mount layer with the configured header name and anonymous policy already applied, or `None` when no verifier is configured.

The one-call startup wiring

Bind one config struct, then drive both subsystems from it. The whole wiring is a load plus two awaited calls:

Rust

```

use firefly_config::{load_from_profile, ConfigError};
use firefly_data_sqlx::{auto_configure, DataSourceProperties};
use firefly_security::{bearer_layer_from_config, SecurityProperties};
use serde::Deserialize;

#[derive(Debug, Default, Deserialize)]
#[serde(default)]
struct Firefly {
    datasource: DataSourceProperties,
    security: SecurityProperties,
}

#[derive(Debug, Default, Deserialize)]
#[serde(default)]
struct LumenConfig {
    firefly: Firefly, // binds the `firefly.datasource.*` /
    `firefly.security.*` subtree
}

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    // 1. Load + merge + profile-resolve + placeholder-expand, then bind.
    let cfg: LumenConfig = load_from_profile("/etc/lumen", "application",
    "dev")?;

    // 2. Build the pool AND register the transaction manager in one await.
    let db = auto_configure(&cfg.firefly.datasource).await?;

    // 3. Build the ready-to-mount bearer layer (None if no JWT settings).
    let bearer = bearer_layer_from_config(&cfg.firefly.security)?;

    // `db` builds typed repositories; mount `bearer` on the web stack.
    // ...
    Ok(())
}

```

This is the same precedence chain from earlier in the chapter doing real work:

`DATABASE_URL` in the environment overrides the YAML default for the pool, and the JWKS endpoint can be re-pointed per profile without touching code. The `#[transactional]` machinery and the bearer middleware both pick up what `auto_configure` and `bearer_layer_from_config` registered — no globals threaded through your constructors.

***Design note.** Firefly deliberately keeps this path DI-free: the config structs are ordinary `serde` types and the auto-configure calls are ordinary `async fn`s you `await` at boot. You can adopt the full `#[derive(ConfigProperties)]` container later without rewriting any of it — the same bound values flow either way.*

Profile expressions

`accepts_profiles(&active, &exprs)` evaluates a profile-expression grammar — AND (`&`), OR (`|`), negation (`!`), and grouping with parentheses — against an active-profile list, useful for gating a bean that should exist only in some environments:

Rust

```
use firefly_config::{accepts_profiles, active_profiles};

let active = active_profiles("dev");           // e.g. ["prod", "cloud"]
accepts_profiles(&active, &["prod & cloud"]); // AND
accepts_profiles(&active, &["prod | qa"]);     // OR
accepts_profiles(&active, &["!test"]);         // negation
accepts_profiles(&active, &["(prod & cloud) | qa"]); // grouping
```

It returns `true` when any expression matches; a malformed expression evaluates to `false` (it never panics). The dependency-injection chapter shows how a bean declares `#[firefly(profile = "prod")]` so the container applies exactly this rule at scan time.

Runtime reload – the `/actuator/refresh` contract

`ReloadableConfig<T>` replays the full merge → placeholder-resolution → bind pipeline and atomically swaps the snapshot; a failed reload keeps the previous snapshot. This is the hook a `POST /actuator/refresh` endpoint wires up — so an operator could re-point Lumen's datasource without a restart.

Rust

```
use firefly_config::ReloadableConfig;

let cfg: ReloadableConfig<LumenConfig> = ReloadableConfig::load(sources)?;
let snapshot = cfg.get();                // Arc<LumenConfig>
let mut rx = cfg.subscribe();             // tokio watch receiver
let changed: Vec<String> = cfg.reload()?; // sorted, changed top-level keys
```

`Arc<ReloadableConfig<T>>` coerces to `Arc<dyn Refresher>` — the object-safe trait the actuator refresh endpoint depends on.

Property-source introspection and masking

`Layered::property_sources()` returns ordered, origin-attributed `PropertySourceView`s (highest precedence first) — the data Firefly's `/actuator/env` view renders, with secrets masked: keys naming secrets (`password`, `secret`, `token`, `credential`, `*key`) mask as `*****`, and URI userinfo passwords are redacted (`postgresql://user:*****@host`). The `mask` module exposes `mask_value`, `is_sensitive_key`, and `sanitize_uri` directly. This matters for Lumen the moment it has a JWT signing key (chapter 14) and a datasource URL — neither should ever appear in plaintext on `/actuator/env`.

In-process application events

`ApplicationEventBus` is an in-process, `TypeId`-dispatched, order-sorted, synchronous pub/sub for lifecycle and local notification events. This is distinct from the asynchronous `firefly-eda` broker Lumen uses for domain events (no transport, no topics; listeners run on the publishing thread):

Rust

```
use firefly_config::{ApplicationEventBus, ApplicationReadyEvent};

let bus = ApplicationEventBus::new();
bus.subscribe::<ApplicationReadyEvent, _>(|_e| { /* on ready */ });
bus.publish(&ApplicationReadyEvent);
```

Lifecycle events ship: `ContextRefreshedEvent`, `ApplicationReadyEvent`, `ContextClosedEvent`. Any `'static` type can be published as a domain event.

NOTE

Do not confuse this with [Event-Driven Architecture](#): the `ApplicationEventBus` is a *local* lifecycle/notification channel; Lumen's wallet domain events ride the `firefly-eda` `Broker` over a topic, with a real Kafka/RabbitMQ adapter waiting behind the in-memory default.

Pulling config from a config server

`ConfigClient` fetches a remote configuration document (compatible with the Spring Cloud Config server wire format) and flattens it into a `StaticSource` you slot into your chain above the defaults:

Rust

```

use firefly_config::ConfigClient;

let remote = ConfigClient::new("http://config:8888", "lumen")
    .with_profile("prod")
    .with_label("main")
    .with_basic_auth("user", "pass")
    .fetch_source()           // fail-fast; .fetch_source_or_empty() = soft
                              fallback
    .await?;
sources.insert(1, Box::new(remote)); // above defaults, below env/flags

```

A non-2xx response logs a warning and yields an empty map (soft miss); transport or decode failures raise `ConfigError::Remote`. The standalone config server lives in `firefly-config-server`.

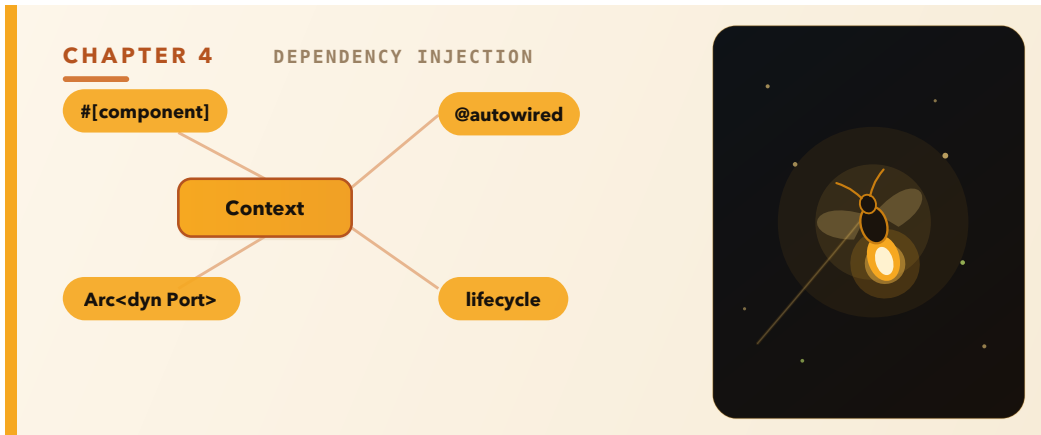
Recap – what changed in Lumen

Before	After this chapter
identity hard-coded in two <code>pub const</code> strings	the same values understood as <code>CoreConfig</code> knobs that feed the banner and <code>/actuator/info</code>
ports read with bare <code>std::env::var</code>	the typed home of <code>LUMEN_ADDR</code> / <code>LUMEN_ADMIN_ADDR</code> , sitting at the top of a documented precedence chain
no path to per-environment settings	profiles, placeholders, and <code>#[derive(ConfigProperties)]</code> ready for injection in the next chapter
secrets unconsidered	masking + <code>/actuator/env</code> redaction in place before Lumen ever holds a signing key

Exercises

1. **Promote the ports to YAML.** Write an `application.yaml` with `web.addr` / `web.admin-addr`, load it with `load_from_profile`, and confirm a `LUMEN_WEB_ADDR` environment variable still wins (precedence row 4 beats row 2).
2. **Add a profile.** Create `application-prod.yaml` that overrides `web.addr` to `0.0.0.0:80`, run with `FIREFLY_PROFILE=prod`, and verify the prod value takes effect while `dev` keeps the localhost binding.
3. **Bind a `ConfigProperties` bean.** Define the `WebProperties` struct above, set its keys via a `ConditionContext`, and resolve it from a `Container` (you will recognize this pattern in the next chapter's DI tests).
4. **Mask a secret.** Add a `datasource.password` key and call `Layered::property_sources()`; confirm the value renders as `*****` rather than in plaintext.

Next, see how Lumen's composition root resolves its collaborators — explicitly today, with the best-in-class DI container as the scan-driven alternative — in [Dependency Wiring](#).



CHAPTER 4

Dependency Injection & the Application Context

The [previous chapter](#) wired Lumen the idiomatic-Rust way: an explicit composition root (`build_app`) that constructs each collaborator and hands it to the controller. That style is the default, and Lumen keeps it for teachability — the dependency graph is visible in code and checked by the compiler. But Firefly also ships the *other* style: a **dependency-injection container** where beans declare what they need with `#[autowired]` , and the container resolves the graph for you.

By the end of this chapter you will know how Lumen's exact collaborators — `ReadModel` , `Ledger` , `WalletApi` — would be expressed as **beans**: declared with stereotype derives, wired by constructor injection, disambiguated with `#[firefly(primary)]` , gated by profiles and conditions, bracketed by lifecycle hooks, and discovered by component scanning. This is the full Firefly DI surface, told against Lumen's own collaborators.

Design note. Firefly's container offers a declarative DI model — `stereotype` derives, `#[autowired]`, `#[bean]` factories, `primary`, profiles, the `condition_on_*` family, and lifecycle hooks. Because Rust has no runtime reflection, discovery is **link-time** (via `inventory`) and autowiring is generated by a `derive` macro rather than inferred at runtime; a bean is discoverable exactly when its crate is linked. The model is: declare intent, let the container provide the instances. The surface will feel familiar if you've used a batteries-included framework, but the link-time mechanism is Firefly's own.

Two styles, one mental model

Lumen's `build_app` (chapter 4) is constructor injection written by hand:

Rust

```
let read_model = Arc::new(ReadModel::default());
let ledger = Ledger::new(Arc::clone(&store), Arc::clone(&broker));
// ... hand the collaborators to the controller state.
```

The container inverts the *who calls whom*: instead of `build_app` calling each constructor, every bean declares its dependencies and the container calls the constructors in dependency order. The collaborators are the same; only the wiring moves from a function into declarations next to each type.

TIP

Prefer explicit construction (chapter 4) for a small service like Lumen — it compiles faster and the graph is right there in `build_app`. Reach for the container when the graph grows large, when you want a central registry the admin dashboard can introspect (`/beans`), or when you prefer declarative, scan-driven wiring over a hand-written root.

Stereotypes: declaring your beans

A **bean** is any value the container builds, wires, and owns. You make a type a bean with a **stereotype derive** — a thin annotation that generates a `firefly_register(&Container)` method *and* submits a link-time scan thunk. Five stereotypes ship, all functionally equivalent, differing only in the architectural intent they document and the label they carry into `/beans`:

Derive	Documents
<code>#[derive(Component)]</code>	a generic managed bean
<code>#[derive(Service)]</code>	business-logic / use-case
<code>#[derive(Repository)]</code>	data access / a port
<code>#[derive(Configuration)]</code>	a holder of <code>#[bean]</code> factories
<code>#[derive(Controller)]</code>	a web controller bean

Lumen's read model is a data-access bean, so it would carry `#[derive(Repository)]`:

Rust

```
use firefly::prelude::*;
use std::sync::Mutex;
use std::collections::HashMap;

#[derive(Repository, Default)]
pub struct ReadModel {
    rows: Mutex<HashMap<String, WalletView>>,
}

// generated: ReadModel::firefly_register(&container) + a component-scan thunk
```

 TIP

Choosing `Repository` over `Component` costs nothing technically but tells every reader exactly what `ReadModel` is for.

Constructor injection with `#[autowired]`

A bean's dependencies are its `#[autowired]` fields. The container resolves each one by type and injects it before the bean is built. The field type controls the *shape* of the injection:

Field type	Resolves via
<code>Arc<T></code>	<code>resolve::<T>()</code> (required)
<code>Option<Arc<T>></code>	<code>resolve::<T>().ok()</code> (optional)
<code>Vec<Arc<T>></code>	<code>resolve_all::<T>()</code> (all implementations)
<code>Provider<T></code>	a deferred handle (<code>container.provider::<T>()</code>)

Lumen's `Ledger` depends on an event store and a broker; as a bean it autowires them, and a `WalletApi` controller bean autowires the `Bus` and the `Ledger`:

Rust

```

use firefly::prelude::*;
use std::sync::Arc;

#[derive(Service)]
pub struct Ledger {
    #[autowired] store: Arc<dyn firefly::eventsourcing::EventStore>,
    #[autowired] broker: Arc<dyn firefly::eda::Broker>,
}

#[derive(Controller)]
pub struct WalletApi {
    #[autowired] bus: Arc<Bus>,
    #[autowired] ledger: Arc<Ledger>,
}

```

When the container builds `WalletApi`, it builds `Ledger` first (recursively resolving its store and broker), exactly as `build_app` does by hand — but the order is derived from the field types, not written out. A field with no attribute is built with `Default::default()`; a `#[firefly(value = "${...}")]` field is bound from config (see *Config-driven injection* below).

 NOTE

`#[autowired]` is constructor injection: a missing required dependency is a loud `ContainerError::NoSuchBean` at resolve time, with fuzzy "did you mean..." suggestions, rather than a `None` panic three frames deep.

*Note — the **Default** rule.* The generated factory constructs the struct as a literal, filling `#[autowired]` / `#[firefly(value = ...)]` fields from the container and **every other field** from `Default::default()`. So a stereotype struct needs `#[derive(Default)]` if — and

only if — it has at least one field that is neither `#[autowired]` nor `#[firefly(value = ...)]` (like `ReadModel`'s `rows` above). An all-autowired struct, or a unit/field-less holder like a `#[derive(Configuration)]`, compiles without it.

Note — `Provider<T>` in a hand-wired container. Autowiring a `Provider<T>` field needs the container to hold a handle to itself, which only a shared container has. Build one with `Container::shared()` (or call `install_shared_handle()` on an `Arc<Container>`); an `ApplicationContext` already does this for you. Resolving a `Provider<T>` field against a bare `Container::new()` panics with a message telling you to use `shared()`.

Resolution rules, `#[firefly(primary)]`, and ports

Lumen depends on *ports* — `EventStore`, `Broker` — and selects an adapter at wiring time. The container expresses "depend on the port, get the adapter" with a trait-object **binding**. Register the concrete type, bind the trait to it, then resolve the trait:

Rust

```
let c = Container::new();
firefly::register_all!(&c, [MemoryEventStore]);
c.bind::<dyn firefly::eventsourcing::EventStore, MemoryEventStore>(|a| a);
let store: Arc<dyn firefly::eventsourcing::EventStore> = c.resolve().unwrap();
```

When a single implementation is bound, it resolves immediately. When **several** adapters back one port — the classic in-memory-vs-Redis split — exactly one must be marked primary, or resolution fails loudly:

Rust

```
#[derive(Repository)]
#[firefly(primary)]           // the default adapter
pub struct MemoryEventStore { /* ... */ }

#[derive(Repository)]
pub struct PostgresEventStore { /* ... */ } // activated by profile/condition
```

The resolution rules, in strict priority order:

1. **Direct registration** — `T` registered directly resolves to it.
2. **Single binding** — one implementation bound to a trait resolves to it.
3. `#[firefly(primary)]` — among several bindings, the primary one wins.
4. **Error** — `NoSuchBean` when nothing matches; `NoUniqueBean` (naming every competing candidate) when several match with no primary.

NOTE

`#[firefly(primary)]` marks the default adapter among several bound to one port; with no primary, resolution fails loudly with a `NoUniqueBean` error that names every competing candidate. Moving `primary` from one adapter to the other is the *only* change needed to swap Lumen's backing store — nothing in `Ledger` changes.

`#[firefly(order)]` and named beans

`#[firefly(order = N)]` controls initialization sequence and the order `resolve_all::<T>()` returns implementations (lower runs first); the constants `HIGHEST_PRECEDENCE` / `LOWEST_PRECEDENCE` mark the extremes. When two beans share a type — say a primary and a read-replica store — give one a name and select it with a **qualifier**:

Rust

```
#[derive(Repository)]
#[firefly(name = "replica")]
pub struct ReplicaStore { /* ... */ }

#[derive(Service)]
pub struct ReportService {
    #[firefly(qualifier = "replica")] store: Arc<ReplicaStore>,
}
```

NOTE

A mistyped qualifier name pointing at the wrong type is a clear `NoSuchBean`, not a silently-wrong injection.

Bean factories: `#[derive(Configuration)]` + `#[bean]`

Not every dependency is a type you own — a third-party client needs constructor arguments, an interface needs a hand-built adapter. For these, a `#[derive(Configuration)]` holder exposes `#[bean]` factory methods. Each method is keyed by its **return type**; its `Arc<Dep>` arguments are resolved from the container (so a factory can depend on other beans). This is how Lumen would produce its `InMemoryBroker` behind the `Broker` port:

Rust

```

use firefly::prelude::*;
use std::sync::Arc;

#[derive(Configuration)]
pub struct LumenInfraConfig;

#[firefly::bean]
impl LumenInfraConfig {
    #[bean(primary)]
    fn broker(&self) -> firefly::eda::InMemoryBroker {
        firefly::eda::InMemoryBroker::new()
    }

    #[bean]
    fn ledger(&self, store: Arc<dyn firefly::eventsourcing::EventStore>) ->
    Ledger {
        Ledger::new(store, /* ... */)
    }
}

```

You **do not** call anything to wire these: `Container::scan()` discovers the holder *and* its `#[bean]` methods (each submits its own `inventory` thunk) and registers them automatically. (`LumenInfraConfig::firefly_register_beans(&container)` is still generated for the explicit `register_all!` path and for generic holders that can't be inventoried.)

A `#[bean]` method returns a **concrete (sized) type** — that is the bean's key. To expose it behind a port, add `#[firefly(provides = "dyn Broker")]` to the holder or call `Container::bind` after registration. Per-method options mirror the struct-level conditionals:

```

#[bean(name = "...", scope = "...", primary, order = N, profile = "...",
condition_on_property = "k=v", condition_on_class = "...", condition_on_bean =
"T", condition_on_missing_bean = "T", condition_on_single_candidate = "T")].

```

NOTE

A `#[bean]` method is keyed by its return type: the container registers the produced value *under that type*, so a downstream `#[autowired] ledger: Arc<Ledger>` finds it. Swapping `InMemoryBroker` for a Kafka adapter is one line in this holder; the rest of Lumen is untouched.

Scopes

Every bean has a **scope** controlling how long its instance lives. Pass it as

```
#[firefly(scope = "...")]:
```

Scope	Behaviour	Use for
<code>singleton</code> (default)	one instance, cached after first resolve	stateless services, pools, caches
<code>transient</code>	a fresh instance on every resolve	per-operation state (a saga's scratch context)
<code>request</code>	one per HTTP request (needs a request <code>ScopeHandler</code>)	the authenticated user, a request trace id
<code>session</code>	one per session (needs a session <code>ScopeHandler</code>)	per-session state

Rust

```
#[derive(Component)]
#[firefly(scope = "transient")]
pub struct TransferContext {           // a fresh scratch pad per transfer
    pub steps: Vec<String>,
}
```

NOTE

Firefly's scopes are `singleton` (default), `transient` (a fresh instance per resolve), `request` (one per HTTP request), and `session` (one per session). Almost every Lumen bean is a singleton; the `request` and `session` scopes resolve through a `ScopeHandler` (see below). The scope name is the lower-case variant: `Scope::{Singleton, Transient, Request, Session}` in `crates/container/src/scope.rs`.

Request and session scopes: the `ScopeHandler` SPI

Rust has no ambient request thread-local the way a reflective JVM container does, so the `request` and `session` scopes are **driven explicitly** by an implementation of the `ScopeHandler` SPI — the direct analog of Spring's `org.springframework...config.Scope`. A handler caches an instance per key (per request, per session) and evicts it when that key ends:

Rust

```
use firefly::prelude::*;
use std::sync::Arc;

// A host installs the handlers once at startup. Until one is installed,
// resolving a request/session-scoped bean is a NoSuchBean ("no active
// request context"), matching the Spring/pyfly behaviour.
container.register_request_scope(Arc::new(my_request_handler));
container.register_session_scope(Arc::new(my_session_handler));
```

`register_request_scope` / `register_session_scope` back the two built-in scopes; arbitrary custom scopes use `register_scope("name", handler)` (rejecting empty names and the four built-ins). All three live on `Container` (`crates/container/src/lib.rs`), and a `ScopeHandler` is just a `get(name, factory)` plus `remove(name)` (`crates/container/src/scope.rs`).

Design note. The request/session lifecycle is installed, not implicit: a host registers a `ScopeHandler` via `register_request_scope` / `register_session_scope`, and request/session-scoped beans resolve through it. A bare `Container` with no handler installed reports `NoSuchBean` for those scopes rather than silently leaking a singleton — the deferral is explicit, the same trade described under *What Rust's model changes below*.

`RefreshScope` – Spring Cloud `@RefreshScope` parity

A fourth, ready-made handler ships for config-reload: `RefreshScope` (`crates/container/src/scope.rs`). A refresh-scoped bean is cached like a singleton, but a `refresh()` call evicts every refresh-scoped instance so the next resolution rebuilds it against the new config — the hook a future `/actuator/refresh` calls on a config change. Register it under the conventional name `REFRESH_SCOPE_NAME` (`"refresh"`):

Rust

```
use firefly::prelude::*;
use std::sync::Arc;

let refresh = Arc::new(firefly::container::RefreshScope::new());
container.register_scope(firefly::container::REFRESH_SCOPE_NAME,
refresh.clone());

// On a config-change event:
let evicted: Vec<String> = refresh.refresh(); // rebuild on next resolve
```

For a single singleton there is a lighter hook: `container.reset_instance:::<T>()` drops just `T`'s cached instance so it is rebuilt on next resolve (returns whether an instance was actually evicted). It is the per-bean form of the same refresh-on-config-change idea.

 SPRING PARITY

`RefreshScope` / `REFRESH_SCOPE_NAME` mirror Spring Cloud's `@RefreshScope`, and `reset_instance::<T>()` is the per-bean refresh hook. Both exist so a config reload can rebuild affected beans without restarting the process.

`#[firefly(lazy)]` – opting out of the eager warm pass

By default singletons are built **eagerly** when an `ApplicationContext` starts (see *Startup and eager initialization* below). `#[firefly(lazy)]` (Spring `@Lazy`) opts a singleton **out of that eager warm pass** — it is then built on first resolve instead:

Rust

```
#[derive(Service)]
#[firefly(lazy)]                                // skipped at startup; built on first
use
pub struct ExpensiveReportEngine { /* ... */ }
```

 SPRING PARITY

`#[firefly(lazy)]` is `@Lazy`: it removes the bean from the startup warm pass so an expensive or rarely-used singleton is constructed only when something resolves it. (The macro carries the flag into the scan registry so `warm_singletons` can skip it — `crates/macros/src/container.rs`, `crates/firefly/src/context.rs`.) Unlike Spring it does **not** create a proxy and so does not break a true construction-time dependency cycle; reach for `Provider<T>` to defer a dependency (see *What Rust's model changes*).

Lifecycle: post-construct and pre-destroy

A bean often needs one-time setup after its dependencies are injected — Lumen's projection, for instance, **subscribes to the broker** once at startup — and a clean teardown on shutdown. Name a method per hook on the struct attribute:

Rust

```
#[derive(Service)]
#[firefly(post_construct = "on_start", pre_destroy = "on_stop")]
pub struct ProjectionListener { /* ... */ }

impl ProjectionListener {
    fn on_start(&self) { /* subscribe to wallets.events */ }
    fn on_stop(&self) { /* unsubscribe, flush */ }
}
```

`on_start` runs after construction (all `#[autowired]` fields set), so it can safely query collaborators. `Container::destroy()` (the `ApplicationContext`'s `close()`) runs every `pre_destroy` hook in **reverse** construction order, so a listener that started after the read model is stopped before it.

NOTE

`post_construct` runs after construction and injection, so it can safely query collaborators; `pre_destroy` runs on close in **reverse** construction order, so a listener started after the read model is stopped before it. (Note these are *attribute keys on the struct*, not standalone macros.)

Startup and eager initialization

A bare `Container` builds singletons **lazily** — the first `resolve::<T>()` constructs `T` and caches it. The `ApplicationContext`, however, is **eager** by default, matching Spring's fail-fast startup: `ApplicationContext::build()` scans the crate graph, registers the survivors, then immediately **warms** every non-lazy singleton by resolving it once (`crates/firefly/src/context.rs`). Two things follow from that warm pass:

- **Fail-fast.** A construction error — a missing required dependency, a panic in a `#[post_construct]` — surfaces at *startup*, not deep into the first request.

- **`#[post_construct]` runs at startup.** Because warming a bean constructs it, each non-lazy singleton's `post_construct` hook fires during `build()`, before the context hands you the container. (Lumen's projection subscribes to the broker here, not on first request.)

That puts the whole bean lifecycle in one line:

`scan` → `register` → `warm non-lazy singletons` → `#[post_construct]` → `(serve)` → `close()` → `#[pre_destroy]` in reverse order

You can opt the warm pass out entirely with `.eager(false)`, or opt a single bean out with `#[firefly(lazy)]` (above):

Rust

```
use firefly::prelude::*;

let ctx = ApplicationContext::builder()
    .profiles(["prod"])
    .eager(false)           // skip the warm pass; everything builds lazily
    .build();
```

Design note. Eager warm-up at `build()` is where Firefly matches Spring's "validate the wiring at startup" guarantee. It is a context-level policy: a bare `Container` stays lazy. `.eager(false)` turns the whole warm pass off; `#[firefly(lazy)]` excludes one bean from it. `close()` is the symmetric shutdown — it runs every `#[pre_destroy]` in reverse construction order and evicts the cached singletons (`crates/firefly/src/context.rs`).

The `ApplicationContext` builder surface

`ApplicationContext::builder()` accepts more than `.profiles()` and `.property()`; the full surface (`crates/firefly/src/context.rs`):

Method	Purpose
<code>.profiles([...])</code>	active profiles (defaults to <code>FIREFLY_PROFILE</code> , then <code>"default"</code>)
<code>.property(k, v)</code> / <code>.properties(map)</code>	add config properties for placeholders and conditions
<code>.config_sources(vec![...])</code>	merge <code>firefly_config::Source</code> layers (env, YAML) into the property map
<code>.class(label)</code>	mark a feature "label" present for <code>condition_on_class</code> checks
<code>.eager(bool)</code>	warm non-lazy singletons at <code>build()</code> — default <code>true</code>
<code>.build()</code>	construct the shared container, scan, then (by default) warm singletons

NOTE

`.class(label)` is Firefly's stand-in for `@ConditionalOnClass`: Rust has no classpath to probe, so a host declares which optional features are "present" by label, and `condition_on_class = "label"` beans gate on it.

Config-driven injection

Two stereotype attributes pull configuration straight into a bean, so a service never threads config through its constructor. They are covered in full in [Configuration](#) §"Binding config straight into a bean"; the DI-relevant summary:

Single value — `#[firefly(value = "${key:default}")]`. Binds one resolved, placeholder-expanded scalar onto a field (parsed via `FromStr`); the `:default` tail supplies a fallback when the key is absent:

Rust

```
#[derive(Service)]
pub struct WalletApi {
    #[autowired] ledger: Arc<Ledger>,
    #[firefly(value = "${lumen.web.addr:127.0.0.1:8080}")] addr: String,
}
```

Whole subtree — `#[derive(ConfigProperties)]`. Binds a `serde` struct under a prefix and registers it as an injectable singleton (the `@EnableConfigurationProperties` equivalent); any bean can then autowire it:

Rust

```
use serde::Deserialize;

#[derive(Deserialize, ConfigProperties, Default)]
#[firefly(prefix = "lumen.web")]
pub struct WebProperties {
    pub addr: String,
    #[serde(default)] pub admin_addr: String,
}

#[derive(Service)]
pub struct ReportService {
    #[autowired] props: Arc<WebProperties>,    // the bound subtree, injected
}
```

`ConfigProperties` is the sixth container-aware derive alongside the five stereotypes: it generates a `firefly_register` that binds and registers the struct, and carries a `config_properties` stereotype label into `/beans`. Single values use `${...}` placeholders only — SpEL `#{...}` is out of scope (see *What Rust's model changes*).

 SPRING PARITY

`#[firefly(value = "${...}")]` is `@Value` (placeholder form), and `#[derive(ConfigProperties)]` + `#[firefly(prefix = "...")]` is `@ConfigurationProperties`. Both bind against the same merged, profile-resolved, placeholder-expanded config map.

Conditional beans and profiles

Conditions answer "should this bean exist *at all*, given the environment?" — the mechanism that lets one Lumen codebase run on in-memory infrastructure in tests and real infrastructure in production with no `if` in the service code. They are evaluated by `Container::scan` in two passes:

Pass 1 (config/profile facts, knowable before any bean is built):

Rust

```
#[derive(Repository)]
#[firefly(profile = "prod", condition_on_property =
  "lumen.store.postgres=true")]
pub struct PostgresEventStore { /* ... */ }
```

Pass 2 (registry-dependent, knowable only after pass 1 settles) — the **default-with-override** pattern: ship a fallback that yields to any user-provided implementation:

Rust

```
#[derive(Repository)]
#[firefly(condition_on_property = "lumen.store.postgres=true")]
pub struct PostgresEventStore { /* ... */ } // real store when configured

#[derive(Repository)]
#[firefly(condition_on_missing_bean = "EventStore")]
pub struct MemoryEventStore { /* ... */ } // fallback whenever none is wired
```

The full conditional family: `condition_on_property = "key=value"`, `condition_on_class = "label"`, `condition_on_bean = "Type"`, `condition_on_missing_bean = "Type"`, `condition_on_single_candidate = "Type"`, plus `profile = "expr"` (which supports a boolean grammar — `prod & cloud`, `dev | test`, `!staging`, parentheses).

***Design note.** The conditional family is `condition_on_property`, `condition_on_class`, `condition_on_bean`, `condition_on_missing_bean`, `condition_on_single_candidate`, and `profile`. The scan evaluates them in two passes: pass-1 facts (config, feature labels) settle first, then pass-2 conditions evaluate against the resulting bean registry — which is how all of Firefly's own auto-configuration backs off when you provide your own.*

Auto-configuration

An **auto-configuration** is how a starter contributes sensible defaults that get out of your way the moment you declare your own. Derive `#[derive(AutoConfiguration)]` on a holder and guard each `#[bean]` with `condition_on_missing_bean`:

Rust

```

use firefly::prelude::*;

#[derive(AutoConfiguration, Default)]
pub struct CacheAutoConfiguration;

#[firefly::bean]
impl CacheAutoConfiguration {
    #[bean(condition_on_missing_bean = "CacheClient", condition_on_property =
"cache.type=memory")]
    fn cache_client(&self) -> CacheClient {
        CacheClient::in_memory()           // the default – only if you didn't
wire one
    }
}

```

`#[derive(AutoConfiguration)]` is a `#[derive(Configuration)]` whose beans are ordered to register **last** during the scan. Because its `#[bean]`s carry `condition_on_missing_bean`, the two-pass scan registers your unconditional bean first, then *skips* the auto-configuration default — so your bean always wins, and you never write an `if`. Drop the starter from your dependencies and the auto-configuration code isn't linked, so it contributes nothing: discovery is link-time, not reflective.

***Design note.** An auto-configuration is a holder whose `#[bean]`s carry `condition_on_missing_bean` and register **last** in the scan: your own bean always wins, and you never write an `if`. Discovery is link-time, so an auto-configuration is "present" — and contributes its defaults — exactly when its crate is compiled in, and contributes nothing once you drop the starter.*

Component scanning vs. `register_all!`

`Container::scan()` collects every non-generic stereotype derive's `inventory` thunk across the whole crate graph, applies conditions and profiles, and registers the survivors. Drive it through the `ApplicationContext`, which builds the condition context from the active profiles and config:

Rust

```
use firefly::prelude::*;

let ctx = ApplicationContext::builder()
    .profiles(["prod"])
    .property("lumen.store.postgres", "true")
    .build(); // scans, then (by default) warms
             singletons
let store: Arc<dyn firefly::eventsourcing::EventStore> =
    ctx.resolve().unwrap(); // -> PostgresEventStore (prod)
println!("{}", beans registered", ctx.bean_count());
```

To scan only part of the crate graph, pass the base module paths:

Rust

```
let c = Container::new();
c.scan_packages(&["lumen::domain", "lumen::adapters"]); // these modules only
```

A registration matches when its defining module path equals a base package or is a descendant of it; conditions and profiles are applied exactly as in `scan()`.

Generic beans cannot be inventoried (the monomorphization is chosen at the use site), so register those with the explicit-list fallback:

Rust

```
let c = Container::new();
firefly::register_all!(&c, [ReadModel, Ledger, WalletApi]); // calls each
firefly_register
let api = c.resolve::<WalletApi>().unwrap();
```

***Design note.** Discovery happens at link time via `inventory`, not at runtime via reflection — so a bean is only discoverable if its crate is actually linked. `register_all!` is the explicit fallback for generics, which can't be inventoried.*

Introspection – the `/beans` view

The container is observable. `container.beans()` returns a `BeanDescriptor` per registration (name, type, scope, stereotype, primary, initialized, resolution count) and `bean_stats()` aggregates counts per stereotype — exactly what the admin dashboard's `/beans` page renders, and what `firefly beans --url ...` (the [CLI chapter](#)) prints. Errors carry diagnostics too: `fuzzy_suggestions(name)` powers the "did you mean..." hints, and `CircularDependency` is caught with a per-thread resolution stack.

What Rust's model changes

Firefly's container is deliberately *not* a line-for-line clone of Spring's. The single structural fact behind every difference is that **Rust has no runtime reflection**: a bean is built by a generated factory closure that resolves its own dependencies and constructs the struct in one shot, rather than Spring's "instantiate → populate → call init" phases. There is no post-instantiation seam to weave behaviour into, and there is

no way to hand a bean a reflective handle to the container. The following Spring features are therefore replaced by Rust idioms rather than ported — each is a deliberate choice, not a missing feature:

- No `BeanPostProcessor` / `BeanFactoryPostProcessor`. There is no post-instantiation interception phase (the factory closure builds the bean in one shot), and no definition-rewriting pass. Cross-cutting behaviour is instead composed explicitly — wrap a collaborator, or use a `#[bean]` factory to build the wired-up instance you want.
- No `*Aware` interfaces (`ApplicationContextAware`, `EnvironmentAware`, ...). A bean is not handed the context by a callback. Autowire what you actually need — `Arc<WebProperties>` for config, `Provider<T>` for a deferred dependency — rather than reaching back into the container.
- No `FactoryBean`. A `#[bean]` factory method (above) produces any type and is keyed by its return type; that covers the "a bean whose job is to build another bean" case without a distinct `FactoryBean` abstraction.
- No `@Scope(proxyMode)` scoped proxies. A `request` / `session`-scoped bean is resolved through its `ScopeHandler`, not injected into a singleton via a transparent proxy. To pull a shorter-scoped (or deferred) dependency into a longer-lived bean, hold a `Provider<T>` and call `.get()` at the point of use — that is the idiomatic substitute for both `@Lazy` proxies and scoped proxies.
- No per-bean `SmartLifecycle` phases. The container has `#[post_construct]` / `#[pre_destroy]` (the `InitializingBean` / `DisposableBean` substitute), and application-level start/stop ordering lives in the separate `firefly-lifecycle` crate — there is no per-bean `start` / `stop` / `isRunning` / phase negotiation.
- No SpEL `#{...}`. `#[firefly(value = "${...}")]` does placeholder injection only. Expressions, method/bean references, and arithmetic in config are intentionally out of scope for the typed-Rust idiom.

Design note. Read this list as substitutions, not gaps: `Provider<T>` stands in for `@Lazy` / `scoped proxies` / `ObjectFactory` when you need to defer or reach a shorter-scoped dependency; `#[post_construct]` / `#[pre_destroy]` stand in for `InitializingBean` / `DisposableBean`; a `#[bean]` factory stands in for `FactoryBean`; and explicit composition stands in for `BeanPostProcessor` weaving. The reflective machinery is gone, but every job it did has a typed, compile-checked counterpart.

What changed in Lumen

Nothing in `samples/lumen` — Lumen deliberately keeps the explicit composition root from chapter 4 for teachability. What this chapter showed is the *other valid wiring*, told against Lumen's real collaborators: `ReadModel` as a `#[derive(Repository)]`, `Ledger` / `WalletApi` autowiring their dependencies, `MemoryEventStore` vs. a hypothetical `PostgresEventStore` disambiguated by `#[firefly(primary)]` and gated by `profile` / `condition_on_missing_bean`, a `#[derive(Configuration)]` + `#[bean]` factory producing the broker, lifecycle hooks bracketing the projection, eager warm-up at `ApplicationContext::build()`, and `Container::scan()` / `register_all!` discovering it all. Every attribute shown — `autowired`, `primary`, `order`, `qualifier`, `scope`, `lazy`, `profile`, the `condition_on_*` family, `post_construct`, `pre_destroy`, `provides`, `value`, and `prefix` (on `#[derive(ConfigProperties)]`) — is a real option on the stereotype derives.

Exercises

1. Express Lumen as beans. Take `ReadModel`, `Ledger`, and `WalletApi` and add the stereotype derives + `#[autowired]` fields shown above. Wire them with `register_all!(&c, [ReadModel, Ledger, WalletApi])` and `resolve::<WalletApi>()`. Confirm the container builds the graph in dependency order.

2. Default-with-override. Give `MemoryEventStore`

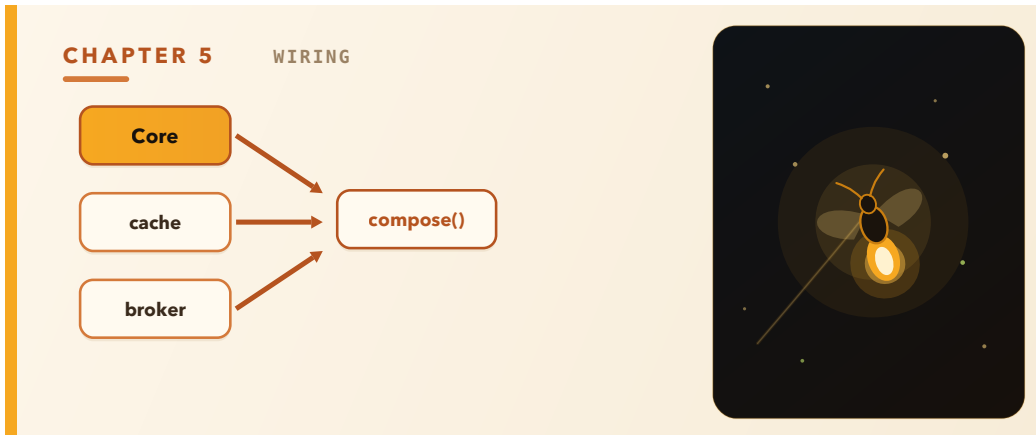
`#[firefly(condition_on_missing_bean = "EventStore")]` and

`PostgresEventStore` `#[firefly(condition_on_property =`

`"lumen.store.postgres=true")]`. Scan with and without the property set; confirm which store resolves each time.

3. Primary swap. Bind two adapters to one port without a primary and observe the `NoUniqueBean` error naming both candidates. Add `#[firefly(primary)]` to one and watch resolution succeed — with no change to the consuming bean.**4. Lifecycle order.** Add `post_construct` / `pre_destroy` hooks to two beans where one depends on the other; call `ApplicationContext::close()` and confirm the dependent is torn down first.

You now have both wiring styles in hand: explicit construction (the default Lumen uses) and the full Firefly DI container. The reactive model underpins everything that follows — continue to [The Reactive Model](#).

**CHAPTER 5**

Dependency Wiring & Composition

By the end of this chapter you will understand Lumen's **composition root** — the `build_app()` function that resolves every collaborator and hands the wallet controller its state — and you will know both ways Firefly lets you wire a service: explicit construction (Lumen's choice, for teachability) and the best-in-class DI container with component scanning (`#[derive(Service)]`, `#[autowired]`, `firefly::scan`). Lumen keeps an explicit root and shows the scan alternative, so you can pick the style that fits your team.

Every service has a moment where the pieces come together: the cache, the bus, the repositories, the services that depend on them. Firefly calls that the *composition root*, and it gives you two ways to write it. The explicit style builds collaborators with plain constructors and passes them where they are needed — idiomatic Rust, checked entirely by the compiler. The container style is a full dependency-injection engine: declare beans with stereotype derives, mark dependencies `#[autowired]`, and let `firefly::scan` discover and wire the whole graph. This chapter covers both, using Lumen as the worked example, and is honest about when to reach for each.

Lumen's explicit composition root

Lumen wires its application explicitly, in one function, on purpose: a reader can see the entire dependency graph in a single screen, and the borrow checker proves it is correct before the program runs. Here is the shape `build_app` grows into (later chapters add the bus middleware, the ledger, and the security chain — the *structure* is what matters now):

Rust

```
// src/web.rs
use std::sync::Arc;
use firefly::cQRS::QueryCache;
use firefly::eventsourcing::MemoryEventStore;
use firefly::prelude::*;
use firefly::starter_web::WebStack;

/// The fully-assembled Lumen application: the web-tier stack, the CQRS bus, the
/// ledger application service, and the read model.
pub struct LumenApp {
    pub web: WebStack,
    pub bus: Arc<Bus>,
    pub ledger: Ledger,
    pub read_model: Arc<ReadModel>,
    pub query_cache: QueryCache,
}

/// Assembles a `LumenApp` over **in-memory** infrastructure – the default for
/// tests and a no-infra run.
pub async fn build_app() -> LumenApp {
    let web = WebStack::new(firefly::starter_web::CoreConfig {
        app_name: APP_NAME.into(),
        app_version: VERSION.into(),
        ..Default::default()
    });

    /// Reuse the WebStack's already-wired collaborators – no duplication,
/// no globals. `web` Derefs to the Core, so `web.bus` / `web.broker`
/// are the same instances the middleware and actuator see.
    let bus = Arc::clone(&web.bus);
    let store: Arc<dyn firefly::eventsourcing::EventStore> =
Arc::new(MemoryEventStore::new());
    let broker = Arc::clone(&web.broker);
    let read_model = Arc::new(ReadModel::default());

    let ledger = Ledger::new(Arc::clone(&store), Arc::clone(&broker));
    let query_cache = QueryCache::new();
}
```

```
LumenApp { web, bus, ledger, read_model, query_cache }
}
```

Three ideas carry the whole design:

- **The root reuses, it does not re-create.** `web.bus` and `web.broker` are the instances `WebStack::new` already wired; `build_app` clones the `Arc` s rather than standing up a second bus. There is exactly one of each, and everything downstream shares it.
- **Ports are `Arc<dyn Trait>` fields.** `store: Arc<dyn EventStore>` is "depend on the interface, inject the implementation" with no machinery. Swap `MemoryEventStore` for a Postgres-backed store and *only this line* changes — the ledger, the handlers, and the controller never notice.
- **The controller gets its state from the root.** When the wallet routes arrive in [Your First HTTP API](#), `LumenApp::router()` constructs the `WalletApi` controller from these resolved collaborators and hands it to the macro-generated router as axum `State`.

TIP

Explicit wiring keeps the dependency graph visible in code and checked by the compiler. There is no reflection and no startup-time magic; if it compiles, it is wired. For an application as focused as Lumen, this is the clearest choice — and it is why the book uses it.

***Design note.** This is constructor injection done by hand: the `LumenApp { .. }` literal is the place where dependencies are "injected." Lumen spells the graph out explicitly rather than inferring it — the trade-off is verbosity for transparency, and for many Rust services transparency wins.*

The container as a composition root

`WebStack` / `Core` is itself a wired bundle of the components a typical service needs. You read them straight off the struct (Lumen does), or pass overrides in `CoreConfig`:

Field / accessor	Type
<code>web.bus</code>	<code>Arc<cqrs::Bus></code> (validation pre-installed)
<code>web.cache</code>	<code>Arc<dyn cache::Adapter></code> (Memory by default)
<code>web.broker</code>	<code>Arc<dyn eda::Broker></code> (InMemory by default)
<code>web.scheduler</code>	<code>Arc<scheduling::Scheduler></code>
<code>web.metrics</code>	<code>Arc<actuator::MetricRegistry></code>
<code>web.health</code>	<code>Arc<actuator::HealthComposite></code>

Rust

```
let web = WebStack::new(firefly::starter_web::CoreConfig {
    app_name: "lumen".into(),
    cache: Some(Arc::new(my_redis_adapter)), // Arc<dyn cache::Adapter>
    broker: Some(Arc::new(my_kafka_broker)), // Arc<dyn eda::Broker>
    ..Default::default()
});
```

Override any field and everything downstream uses your choice — the same "swap the adapter, keep the code" move you will make per subsystem throughout the book.

The best-in-class DI container – `firefly-container`

When you would rather *declare* beans and let the framework discover and wire them, Firefly ships a full dependency-injection container with **component scanning**, stereotype derives, constructor-style `#[autowired]` injection, qualifier/primary/order

disambiguation, `Vec` and `Provider` injection, `#[bean]` factories, lifecycle hooks, and conditional/profile gating. It is `TypeId`-keyed, `Send + Sync`, and shareable as `Arc<Container>`. Beans default to singleton lifetime; the container also supports transient, request, and session scopes, which [Dependency Injection](#) covers in full.

Stereotypes – declaring your beans

You make a type visible to the container by deriving a **stereotype**. All five register the type as a managed bean; the difference is the architectural role each name communicates (and that the web layer uses to find controllers):

Derive	Role
<code>#[derive(Service)]</code>	Business-logic layer: use-case orchestration.
<code>#[derive(Component)]</code>	Generic managed bean with no specific role.
<code>#[derive(Repository)]</code>	Data-access layer: databases, external storage, ports.
<code>#[derive(Configuration)]</code>	A factory holder that can carry <code>#[bean]</code> methods.
<code>#[derive(Controller)]</code>	HTTP layer (<code>#[rest_controller]</code> builds on this).

Design note. The five stereotype derives document an architectural role — business logic (`Service`), generic managed bean (`Component`), data access (`Repository`), factory configuration (`Configuration`), or HTTP layer (`Controller`). The role is recorded on each bean, so the admin dashboard's `/beans` view can group beans by layer.

`#[autowired]` – constructor injection without a constructor

Mark a field `#[autowired]` and the container fills it in by type. This is the Rust spelling of constructor injection — you declare *what* a bean needs; the container supplies it. Here is how Lumen's ledger-and-read-model pair would look in container style:

Rust

```

use std::sync::Arc;
use firefly::prelude::*;

#[derive(Repository, Default)]
struct WalletReadModel { /* in-memory rows */ }

#[derive(Service)]
struct WalletService {
    #[autowired]
    read_model: Arc<WalletReadModel>, // resolved by type, recursively
}

```

When the container constructs `WalletService` it first constructs `WalletReadModel`, then injects it. A dependency that does not exist surfaces as a clear resolution error at startup — not a panic three frames deep at runtime.

`#[autowired]` injects more than a single `Arc<T>`:

- `#[autowired] widgets: Vec<Arc<Widget>>` injects **every** registered `Widget`, ordered by each bean's `order` (collection injection).
- `#[autowired] maybe: Option<Arc<Thing>>` injects `Some` when a `Thing` is registered and `None` when it is not — an optional dependency.
- `#[autowired] tickets: Provider<Ticket>` injects a **deferred** handle: `tickets.get()` resolves a fresh instance on each call, the way you would reach for a transient bean inside a singleton.

Component scanning – `firefly::scan`

Rust has no runtime package introspection, so discovery is **link-time**: every stereotype derive emits an `inventory` thunk, and `firefly::scan(&container)` (equivalently `container.scan()`) collects every submitted thunk across the whole crate graph and registers them — honoring conditionals and profiles as it goes. The usual entry point is the `ApplicationContext`, which wraps the container with the full startup sequence:

Rust

```
use firefly::prelude::*;

let ctx = ApplicationContext::builder()
    .profiles(["test"])
    .property("feature.audit", "on")
    .build();
let c = ctx.container();

// Every stereotype-derived bean in the crate graph is discovered and wired.
let svc = c.resolve:::<WalletService>().expect("scan registered the service");
```

Design note. `firefly::scan` / `ApplicationContext::builder()` discover every stereotype-derived type in the linked crate graph and register it, subject to its conditions and the active profiles. Because Rust has no runtime reflection, discovery is link-time (via `inventory`): a bean is discoverable exactly when its crate is compiled into the binary. The one Rust-specific note: generic types can't be inventoried (the monomorphization is chosen at the use site), so you register those with the explicit `register_all!` fallback below.

Interface auto-binding, `primary`, `order`, and qualifiers

Bind a trait object to an implementation right on the derive, and resolve the trait afterward — "depend on the port, get the adapter":

Rust

```

trait Clock: Send + Sync { fn now(&self) -> u64; }

#[derive(Component, Default)]
#[firefly(provides = "dyn Clock", primary)]
struct SystemClock;
impl Clock for SystemClock { fn now(&self) -> u64 { 42 } }

// elsewhere: c.resolve::<dyn Clock>() yields the SystemClock instance.

```

When several beans satisfy the same interface, `#[firefly(... primary)]` picks the default for `resolve`, and `resolve_all::<dyn Trait>()` returns *all* of them ordered by `order`. For the rare case where you need a *specific* named instance rather than any satisfying one, the container supports qualifier-by-name resolution.

Declaring `provides =` on the derive is the scan-friendly way to bind a trait to an implementation. When you are wiring a container by hand instead, the equivalent move is an explicit `Container::bind::<dyn Trait, Concrete>()` call; both register the same trait-to-adapter mapping. [Dependency Injection](#) covers `bind`, named beans, and the full disambiguation surface in depth.

`#[bean]` factories – wiring things you do not own

Not every dependency is a type you can annotate. Third-party clients need constructor arguments; some beans are clearest as a factory. A `#[derive(Configuration)]` holder with `#[firefly::bean]` methods produces beans keyed by their **return type** — the way you wire a third-party client or a hand-built adapter the container does not construct directly:

Rust

```

use firefly::prelude::*;

#[derive(Configuration, Default)]
struct LumenInfraConfig;

#[firefly::bean]
impl LumenInfraConfig {
    // Registered as `dyn EventStore` – swap MemoryEventStore for a Postgres
    // store here and nothing else in Lumen changes.
    #[bean(primary)]
    fn event_store(&self) -> Arc<dyn firefly::eventsourcing::EventStore> {
        Arc::new(firefly::eventsourcing::MemoryEventStore::new())
    }
}

```

`#[bean]` methods may declare parameters; the container resolves each by type before the method runs. A `#[bean(profile = "prod")]` method registers only when the `prod` profile is active — the factory-level twin of the conditional gating below.

Lifecycle hooks – `#[post_construct]` / `#[pre_destroy]`

Real infrastructure beans need to *act* once wired (open a pool, subscribe to a topic) and undo it on shutdown. Name the methods on the derive:

Rust

```

#[derive(Service, Default)]
#[firefly(post_construct = "started", pre_destroy = "stopped")]
struct ProjectionSubscriber { /* ... */ }

impl ProjectionSubscriber {
    fn started(&mut self) { /* subscribe the read-model projection */ }
    fn stopped(&self)     { /* drain and unsubscribe */ }
}

```

`post_construct` runs after construction and injection complete; `pre_destroy` runs on `container.destroy()` in reverse construction order, so a subscriber started after the store is torn down before it. This is precisely the lifecycle Lumen drives by hand in `main` today (subscribe the projection, then run); the container would manage it for you.

Design note. `#[firefly(post_construct = "...", pre_destroy = "...")]` name a method to run after a bean's dependencies are injected and a method to run on shutdown, with a "destroy in reverse construction order" guarantee.

Conditional and profile gating – the same codebase in every environment

Conditions answer "should this bean exist at all, given the environment?" — how one codebase runs with cheap in-memory adapters in dev and real infrastructure in prod, without an `if` in your service code:

Rust

```
// Registered only when the property is present and not false.
#[derive(Service, Default)]
#[firefly(condition_on_property = "feature.audit=on")]
struct AuditService;

// Registered only under the named profile.
#[derive(Service, Default)]
#[firefly(profile = "prod")]
struct PostgresHealthCheck;
```

`firefly::scan` evaluates these as it collects each thunk, so the resolved container holds exactly the beans the environment calls for. This is the mechanism behind the "swap the adapter" callouts throughout the book — and the reason Lumen can stay in-memory for teaching while a production deployment flips to real infrastructure through configuration alone.

register_all! – the explicit fallback

When you want an explicit list (for generics that can't be scanned, or simply to keep wiring local to one test), `register_all!` registers a known set on a container:

Rust

```
let c = Container::new();
firefly::register_all!(&c, [WalletReadModel, WalletService]);
let svc = c.resolve::<WalletService>().expect("service resolves");
```

Errors and introspection

The error taxonomy is precise: a missing bean, a non-unique bean with no `primary`, and a detected circular dependency each surface as a distinct, named error at resolution time. For diagnostics, the container can list its registered beans and report per-bean resolution stats — the data the admin dashboard's `/beans` view renders.

Choosing a style

Both styles are first-class; neither is required by the core or the starters.

- **Explicit construction** (Lumen's choice) keeps the graph visible and compiler-checked, compiles faster, and reads top-to-bottom. Prefer it for a focused service whose wiring fits on a screen.
- **The container** shines as a service grows many beans, many adapters, and many environment-specific variations — when "declare the bean, let scan find it" removes more boilerplate than the indirection costs. Reach for it when you want declarative, scan-driven wiring or the `/beans` introspection.

You can even mix them: keep an explicit root and resolve a scanned sub-graph from a `Container` field on `LumenApp`.

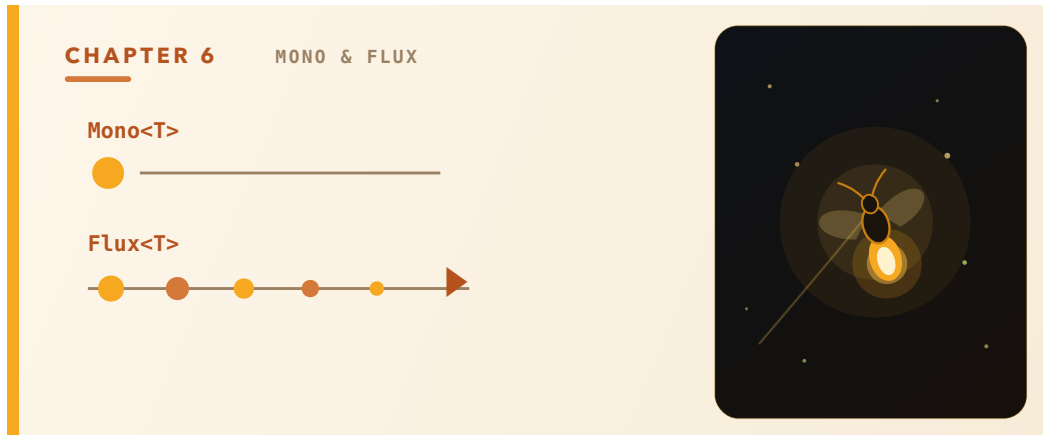
Recap – what changed in Lumen

Before	After this chapter
<code>build_app</code> understood only as "the thing <code>main</code> calls"	named as the composition root : it reuses the <code>WebStack</code> 's wired collaborators and hands them down
ports felt abstract	seen concretely as <code>Arc<dyn EventStore></code> / <code>Arc<dyn Broker></code> fields — one line to swap an adapter
only one wiring style implied	both styles in hand: explicit (Lumen's) and the full DI container with scan, stereotypes, <code>#[autowired]</code> , <code>#[bean]</code> , lifecycle, and conditions

Exercises

1. **Trace the single bus.** In `build_app`, print `Arc::strong_count(&bus)` before and after constructing `LumenApp`. Confirm there is exactly one `Bus` shared by the middleware, the controller, and the handlers — never a second one.
2. **Scan a two-bean graph.** Recreate the `WalletReadModel` / `WalletService` pair above, build an `ApplicationContext`, and `resolve::<WalletService>()`. Then add `#[firefly(condition_on_property = "wallet.enabled=true")]` to the service and watch it disappear from the container until you set the property.
3. **Auto-bind a port.** Define a `Clock` trait, give `SystemClock` `#[firefly(provides = "dyn Clock", primary)]`, and resolve `dyn Clock`. Add a second implementation without `primary` and observe the non-unique-bean error; move `primary` to fix it.
4. **Produce a bean from a factory.** Move Lumen's `MemoryEventStore` behind a `#[derive(Configuration)]` holder with a `#[bean]` method returning `Arc<dyn EventStore>`, and resolve the trait object. Note that swapping in a real store is now a one-method change — the same seam the explicit root has.

With the wiring understood, the reactive primitives underpin everything that follows — read [The Reactive Model](#) next, then give Lumen its first endpoints in [Your First HTTP API](#).



CHAPTER 6

The Reactive Model – Mono & Flux

This is the keystone chapter. `firefly-reactive` is Firefly's production-grade reactive core: two lazy, composable, backpressure-aware publishers — `Mono` (0-or-1 value) and `Flux` (0..N values) — built natively on `tokio`. Every reactive surface in the framework — reactive HTTP endpoints, reactive repositories, the reactive `WebClient`, reactive EDA and CQRS — is built on the two types you will learn here. Read this before the service-building chapters.

If you've used a reactive-streams library before, the `Mono` / `Flux` shapes will feel familiar; everything below is Firefly's own API.

*By the end of this chapter, Lumen will have the vocabulary it leans on twice over: the `Flux<WalletEvent>` that backs its NDJSON / SSE stream-a-wallet's-events endpoint (turned on in *Production & Deployment*), and the lazy `Mono<R>` that `Bus::send_mono` / `Bus::query_mono` return so a wallet command can be composed into a reactive pipeline. No Lumen file lands yet — but every reactive shape in the chapters ahead is one of the two publishers below.*

Mono and Flux

Two publishers:

- `Mono<T>` — a producer of *at most one* value (0-or-1, plus a terminal error). The reactive analog of "an async function that returns a `T`."
- `Flux<T>` — a producer of *0..N* values plus a terminal completion-or-error. The reactive analog of "an async stream of `T`."

Both are **lazy**: building a pipeline does nothing; work runs only when you subscribe, block, or await. Both are `Send + 'static`, so a `Mono` or `Flux` drops directly into an axum handler.

The error type is fixed to `firefly_kernel::FireflyError`. Fixing the error keeps the operator surface ergonomic (no error type parameter) and wires straight into the framework's RFC 9457 problem responses.

Rust

```

use firefly_reactive::{Flux, Mono};

# async fn ex() {
    // Mono: one value, lazily transformed, then awaited.
    let n = Mono::just(20)
        .map(|x| x + 1)
        .filter(|x| *x > 10)
        .default_if_empty(0)
        .block()
        .await
        .unwrap();
    assert_eq!(n, Some(21));

    // Flux: a stream of values, filtered + mapped, collected to a Vec.
    let xs = Flux::range(1, 5)
        .filter(|x| x % 2 == 1)
        .map(|x| x * 10)
        .collect_list()
        .block()
        .await
        .unwrap() // Result -> Option
        .unwrap(); // Option -> Vec (collect_list always yields a list)
    assert_eq!(xs, vec![10, 30, 50]);
# }

```

⚠ WARNING

`Mono::block()` is `async`: it never parks a Tokio worker. It resolves the publisher in place and returns `Result<Option<T>, FireflyError>`, so `.block().await` is the idiomatic way to run a pipeline to completion. Despite the name, it does *not* park a thread the way a blocking call would.

Core concepts at a glance

The terms you will meet throughout the chapter, in one place:

firefly-reactive	What it is
<code>Mono<T></code>	a producer of 0-or-1 value plus a terminal error
<code>Flux<T></code>	a producer of 0..N values plus a terminal signal
<code>firefly_kernel::FireflyError</code> (fixed)	the single, fixed error signal
<code>Ok(None)</code> from a <code>Mono</code>	the empty / complete-with-no-value outcome
<code>Err(FireflyError)</code> (terminal)	an error signal — short-circuits the pipeline
<code>Mono::block</code> (async — never parks a thread)	resolve a publisher in place, then <code>.await</code> it
<code>Mono::subscribe</code> / <code>Flux::subscribe</code>	drive a pipeline with explicit callbacks
<code>Scheduler::Immediate</code>	run work inline on the current task
<code>Scheduler::Parallel</code>	run work on the Tokio worker pool (CPU-bound)
<code>Scheduler::BoundedElastic</code>	run blocking work without starving the worker pool
<code>subscribe_on</code> / <code>publish_on</code>	choose <i>where</i> the source / downstream runs
<code>FluxSink</code> / <code>Flux::create</code>	push values imperatively into a <code>Flux</code>
<code>Backoff</code> + <code>*::retry_backoff</code>	re-subscribe on error with a backoff schedule
<code>Mono::into_future</code> / <code>.await</code>	bridge a <code>Mono</code> out to a plain <code>Future</code>
<code>Flux::to_stream</code> / <code>Flux::into_stream</code>	bridge a <code>Flux</code> out to a plain <code>Stream</code>

Creating publishers

`Mono` constructors:

Constructor	Produces
<code>Mono::just(v)</code>	exactly <code>v</code>
<code>Mono::just_or_empty(opt)</code>	<code>v</code> if <code>Some</code> , empty if <code>None</code>
<code>Mono::empty()</code>	completes with no value (<code>Ok(None)</code>)
<code>Mono::error(e)</code>	terminal error
<code>Mono::from_future(fut)</code>	awaits a <code>Future<Output = T></code>
<code>Mono::from_result_future(fut)</code>	awaits a <code>Future<Output = Result<T, _>></code>
<code>Mono::from_callable(f)</code>	runs a <code>FnOnce() -> Result<Option<T>, _></code> on subscribe
<code>Mono::defer(factory)</code>	builds the <code>Mono</code> fresh per subscription

`Flux` constructors:

Constructor	Produces
<code>Flux::just(vec)</code>	each element of the <code>Vec</code>
<code>Flux::from_iter(iter)</code>	each element of an iterator
<code>Flux::range(start, count)</code>	<code>start, start+1, ...</code> (count items)
<code>Flux::empty()</code> / <code>Flux::never()</code>	completes immediately / never emits
<code>Flux::error(e)</code>	terminal error
<code>Flux::from_stream(s)</code>	a <code>Stream<Item = Result<T, FireflyError>></code>
<code>Flux::from_value_stream(s)</code>	a <code>Stream<Item = T></code>
<code>Flux::create(producer)</code>	imperative push via a <code>FluxSink</code> (see below)
<code>Flux::interval(period)</code>	<code>0, 1, 2, ...</code> on a timer
<code>Flux::generate(seed, step)</code>	stateful generation

Operator quick reference

`Mono` and `Flux` share most operator names; the differences reflect cardinality.

Category	Mono	Flux
transform	<code>map</code> <code>map_async</code> <code>flat_map</code> <code>flat_map_many</code> <code>filter</code>	<code>map</code> <code>map_async</code> <code>flat_map(n)</code> <code>concat_map</code> <code>filter</code> <code>scan</code> <code>index</code> <code>flat_map_iterable</code>
reduce/term	<code>then</code> <code>then_return</code> <code>zip_with</code>	<code>reduce</code> <code>collect_list</code> <code>collect_map</code> <code>count</code> <code>all</code> <code>any</code> <code>then</code> <code>last</code> <code>next</code> <code>single</code> <code>element_at</code>
limit/slice	—	<code>take</code> <code>take_while</code> <code>take_last</code> <code>skip</code> <code>skip_while</code> <code>distinct</code> <code>distinct_until_changed</code>
combine	<code>when</code> <code>zip</code>	<code>merge_with</code> <code>concat_with</code> <code>zip_with</code> <code>combine_latest</code> <code>start_with</code> <code>switch_if_empty</code> <code>default_if_empty</code>
error	<code>on_error_return</code> <code>on_error_resume</code> <code>on_error_map</code> <code>retry</code> <code>retry_backoff</code>	<code>on_error_resume</code> <code>on_error_continue</code> <code>retry</code> <code>retry_backoff</code>
time	<code>timeout</code> <code>delay_element</code>	<code>timeout</code> <code>delay_elements</code> <code>sample</code> <code>debounce</code> <code>interval</code>
backpressure	—	<code>on_backpressure_buffer</code> <code>on_backpressure_drop</code> <code>on_backpressure_latest</code> <code>limit_rate</code>
window	—	<code>buffer</code> <code>window</code> <code>group_by</code>
side-effect	<code>do_on_next</code> <code>do_on_success</code> <code>do_on_error</code> <code>do_on_finally</code>	<code>do_on_next</code> <code>do_on_complete</code> <code>do_on_error</code> <code>do_on_finally</code>
schedule	<code>subscribe_on</code> <code>publish_on</code>	<code>subscribe_on</code> <code>publish_on</code>
cache/view	<code>cache</code> <code>as_flux</code>	—

Transforming and chaining

Rust

```
use firefly_reactive::{Flux, Mono};

# async fn ex() {
// flat_map: chain a Mono onto the result of another (sequential dependency).
let total = Mono::just(3)
    .flat_map(|seed| Mono::just(seed * 10))
    .map(|x| x + 1)
    .block()
    .await
    .unwrap();
assert_eq!(total, Some(31));

// flat_map on a Flux runs up to N inner publishers concurrently.
let doubled = Flux::range(1, 3)
    .flat_map(2, |n| Mono::just(n * 2).as_flux())
    .collect_list()
    .block()
    .await
    .unwrap()
    .unwrap();
assert_eq!(doubled.len(), 3);
# }
```


Combining

Rust

```
use firefly_reactive::{zip, Mono};

# async fn ex() {
  // zip two Monos into a tuple – both run, then combine.
  let pair = zip(Mono::just("alice"), Mono::just(42))
    .block()
    .await
    .unwrap();
  assert_eq!(pair, Some(("alice", 42)));
# }
```

Error semantics

An **Err** item is **terminal** in a **Flux**: every operator short-circuits on the first error and propagates it downstream — there is no per-element error channel. To recover:

- **Mono::on_error_return(fallback)** — substitute a value;
- **Mono::on_error_resume(f)** / **Flux::on_error_resume(f)** — switch to a fallback publisher, keeping items emitted before the error;
- **Flux::on_error_continue(handler)** — drop the failing element and keep the rest (for operators that re-signal per item);
- **Mono::on_error_map(f)** — translate the error.

retry and **retry_backoff** re-subscribe to a **factory closure**, since Rust streams and futures are single-use:

Rust

```

use std::time::Duration;
use std::sync::Arc;
use std::sync::atomic::{AtomicUsize, Ordering};
use firefly_reactive::{Backoff, Mono};
use firefly_kernel::FireflyError;

# async fn ex() {
let calls = Arc::new(AtomicUsize::new(0));
let c = calls.clone();
let value = Mono::retry_backoff(
    move || {
        let c = c.clone();
        Mono::from_callable(move || {
            let n = c.fetch_add(1, Ordering::SeqCst);
            if n < 2 { Err(FireflyError::internal("flaky")) } else {
Ok(Some(n)) }
            })
        },
        Backoff::new(5, Duration::from_millis(10)),
    )
    .block()
    .await
    .unwrap();
assert_eq!(value, Some(2));
# }

```

`Mono::timeout` / `Flux::timeout` map a missed deadline to a 504 `FireflyError` (code `REACTIVE_TIMEOUT`), which renders as an RFC 9457 problem response.

Imperative emission with `FluxSink`

When values arrive from a callback or an imperative loop, push them into a `Flux` with `Flux::create`:

Rust

```
use firefly_reactive::Flux;

# async fn ex() {
let flux = Flux::create(|sink| {
    for i in 1..=3 {
        sink.next(i);
    }
    sink.complete();
});
let out = flux.collect_list().block().await.unwrap();
assert_eq!(out, Some(vec![1, 2, 3]));
# }
```

Schedulers

A `Scheduler` decides *where* work runs. `subscribe_on` hops the source onto a scheduler; `publish_on` switches the thread for everything downstream.

Rust

```
use firefly_reactive::{Flux, Scheduler};

# async fn ex() {
let out = Flux::range(1, 3)
    .subscribe_on(Scheduler::Parallel) // run the source on the Tokio worker
    pool
    .map(|x| x * 2)
    .collect_list()
    .block()
    .await
    .unwrap();
assert_eq!(out, Some(vec![2, 4, 6]));
# }
```

Where `subscribe_on` chooses where the *source* runs, `publish_on` switches the scheduler for everything *downstream* of it — the offload point can sit anywhere in the chain, so a cheap source can hop onto a worker thread right before an expensive map:

Rust

```
use firefly_reactive::{Flux, Scheduler};

# async fn ex() {
let out = Flux::range(1, 3)
    .map(|x| x + 1) // runs wherever the subscribe happens
    .publish_on(Scheduler::Parallel) // everything below hops to the worker
    pool
    .map(|x| x * 10) // runs on the Tokio worker pool
    .collect_list()
    .block()
    .await
    .unwrap();
assert_eq!(out, Some(vec![20, 30, 40]));
# }
```

`Scheduler::Immediate` runs inline, `Scheduler::Parallel` uses the Tokio worker pool for CPU-bound work, and `Scheduler::BoundedElastic` is for blocking calls.

Reactive HTTP endpoints

`firefly-web` ships responders that turn a `Mono` / `Flux` into an axum response. A reactive handler simply returns one of them; the responder drives the publisher and writes the response. They use the stable `firefly-sse` wire format, so any client that speaks NDJSON or SSE consumes them directly.

Responder	Behaviour
<code>MonoJson(Mono<T>)</code>	<code>Ok(Some)</code> → 200 JSON; <code>Ok(None)</code> → 404 problem+json; <code>Err</code> → that error's RFC 9457 response
<code>NdJson(Flux<T>)</code>	<code>application/x-ndjson</code> , one element per line, backpressured
<code>Sse(Flux<T>)</code>	<code>text/event-stream</code> , one <code>data:</code> frame per element
<code>SseEvents(Flux<Event>)</code>	<code>text/event-stream</code> with full <code>id</code> / <code>event</code> / <code>retry</code> control

Rust

```

use axum::{routing::get, response::IntoResponse, Router};
use firefly_reactive::{Flux, Mono};
use firefly_web::{MonoJson, NdJson, Sse};

async fn one_order() -> impl IntoResponse {
    // Ok(Some) -> 200 application/json; Ok(None) -> 404 problem+json;
    // Err -> that error's problem response.
    MonoJson(Mono::just(serde_json::json!({ "id": "o1" })))
}

async fn stream_orders() -> impl IntoResponse {
    // application/x-ndjson, one line per element, backpressured.
    NdJson(Flux::just(vec![1, 2, 3]))
}

async fn live_orders() -> impl IntoResponse {
    // text/event-stream, one `data:` frame per element.
    Sse(Flux::just(vec![1, 2, 3]))
}

let app: Router = Router::new()
    .route("/orders/one", get(one_order))
    .route("/orders", get(stream_orders))
    .route("/orders/live", get(live_orders));

```

The responders, precisely:

- **MonoJson(Mono<T>)** resolves the `Mono : Ok(Some) → 200 application/json ; Ok(None) → 404 application/problem+json ; Err` → that error's problem response.
- **NdJson(Flux<T>)** streams `application/x-ndjson` — one compact JSON doc + `'\n'` per element, flushed incrementally with real backpressure. The `Flux`'s `Stream` is bridged straight into an axum streaming `Body`; the whole stream is **never** buffered. An `Err` item mid-stream terminates the body cleanly.
- **Sse(Flux<T>)** streams `text/event-stream` — each element serialized to a bare `data: <json>\n\n` frame, byte-identical to the `firefly-sse` writer.
- **SseEvents(Flux<Event>)** streams pre-built `firefly_sse::Event` values — use it when you need `id` / `event` / `retry` fields.

⚠ WARNING

Backpressure is real, not cosmetic. A slow client throttles the producer; nothing is buffered up front. This is what lets a `NdJson` endpoint stream a million rows without the response landing fully in memory.

How Lumen will use this

Lumen's optional `GET /api/v1/wallets/:id/events` endpoint is exactly this shape. It replays a wallet's persisted event stream as a `Flux<WalletEvent>` and hands it to `NdJson` (or `Sse` with `?format=sse`). The whole handler — drawn verbatim from `samples/lumen/src/web.rs`, feature-gated behind `streaming` — is the responders above applied to the wallet domain:

Rust

```
// samples/lumen/src/web.rs – the reactive streaming handler (feature
`streaming`).
use firefly::reactive::Flux;
use firefly::web::{NdJson, Sse};

async fn stream_events(
    State(api): State<WalletApi>,
    Path(id): Path<String>,
    axum::extract::Query(params): axum::extract::Query<StreamParams>,
) -> Response {
    // A missing wallet is decided before the streaming head is committed.
    let events = match api.ledger.load_events(&id).await {
        Ok(events) => events,
        Err(e) => return WebError::from(domain_to_web(e)).into_response(),
    };
    let items: Vec<WalletEvent> =
events.iter().map(WalletEvent::from_domain).collect();
    let flux = Flux::just(items);
    if params.format.as_deref() == Some("sse") {
        Sse(flux).into_response()
    } else {
        NdJson(flux).into_response()
    }
}
```

Two details worth carrying forward. First, the *not-found* decision happens **before** the `Flux` is built, so a 404 still renders as a clean problem response rather than a half-open stream. Second, Lumen reaches the reactive types through the one-dependency facade — `firefly::reactive::Flux` and `firefly::web::{NdJson, Sse}` — never the underlying `firefly-reactive` / `firefly-web` crates. The full endpoint, including the route wiring, returns in [Production & Deployment](#).

The reactive `WebClient`

Firefly's reactive HTTP client hands its terminal operators back as `Mono` / `Flux`, so an outbound call drops straight into a reactive pipeline and composes end-to-end with the `NdJson` / `Sse` responders above. Full treatment in [HTTP Clients](#); the shape:

Rust

```
use firefly_client::WebClientBuilder;
use http::Method;
use serde::Deserialize;

#[derive(Deserialize)]
struct Order { id: String }
#[derive(Deserialize)]
struct Tick { seq: u64 }

# async fn ex() {
let client = WebClientBuilder::new("https://api.example.com").build();

// body_to_mono – the whole body decoded as one T.
let _order: firefly_reactive::Mono<Order> =
    client.get().uri("/orders/o1").retrieve().body_to_mono::<Order>();

// body_to_flux – a streamed NDJSON/SSE body decoded element-by-element,
// lazily and with backpressure.
let _ticks: firefly_reactive::Flux<Tick> = client
    .get()
    .uri("/ticks")
    .header("Accept", "application/x-ndjson")
    .retrieve()
    .body_to_flux::<Tick>();
# }
```


NOTE

The client has **no baked-in retry**. Compose `Mono::retry` / `Mono::retry_backoff` on the returned publisher, so retry policy lives at the call site where it belongs rather than hidden inside the client.

Reactive repositories, EDA, and CQRS

The same two types thread through the rest of the framework — and through Lumen:

- **Repositories** — `ReactiveCrudRepository<T, ID>` returns `Mono` / `Flux`; the SQL adapters stream rows out of `find_all()` as a `Flux` so a huge table never lands fully in memory. See [Persistence](#).
- **EDA** — `InMemoryBroker::subscribe_reactive(topic)` yields a `Flux<Event>`, and `publish_mono(event)` is a cold reactive publish. Lumen's ledger publishes every wallet event to a `Broker`; see [EDA](#).
- **CQRS** — `Bus::send_mono` / `Bus::query_mono` wrap the dispatch in a lazy `Mono<R>`, running the same handler lookup and middleware chain. Lumen's wallet commands ride this bus; see [CQRS](#).

A taste of the reactive bus, the shape a Lumen `OpenWallet` could take when composed reactively (it dispatches the same handler the controller's `bus.send(..)` does, only lazily):

Rust

```

use std::sync::Arc;
use firefly::cQRS::Bus;

// `send_mono` / `query_mono` take `&Arc<Bus>` so the lazy Mono can own the bus.
let bus: Arc<Bus> = /* the WebStack's bus */;
let balance = bus
    .query_mono::<_, WalletView>(GetWallet { id: wallet_id })
    .map(|view| view.balance)
    .block()
    .await?;           // Ok(Some(<cents>))

```

NOTE

Because `firefly-reactive` fixes its error channel to `FireflyError`, a failed dispatch is mapped from the bus's `CQRSError` into a status-faithful `FireflyError` (validation → 422, authorization → 403, missing handler → 500) with the original error preserved as `source()` — so a reactive command flows straight into the RFC 9457 problem stack.

Interop with raw `Stream` / `Future`

The reactive types are not a walled garden. Convert in and out at the edges:

- **In:** `Flux::from_stream` (a `Stream<Item = Result<T, FireflyError>>`), `Flux::from_value_stream` (a `Stream<Item = T>`), `Mono::from_future`, `Mono::from_result_future`.
- **Out:** `Flux::to_stream` / `Flux::into_stream`, `Mono::into_future` (or just `.await` the `Mono`).

What changed in Lumen

No Lumen source file lands in this chapter — but Lumen now has the two publishers every reactive surface it touches is built from:

- `Flux<T>` is the engine behind the `GET /api/v1/wallets/:id/events` streaming endpoint: a wallet's events replay through `Flux::just(items)` and stream out as `NdJson` (or `Sse`), with the not-found check resolved *before* the stream head is committed. Lumen reaches it as `firefly::reactive::Flux` + `firefly::web::{NdJson, Sse}` — one dependency, no crate sprawl.
- `Mono<R>` is the lazy result of `Bus::send_mono` / `Bus::query_mono`, the reactive face of the wallet command/query bus you wire in `CQRS`. Its `FireflyError` channel is the same one the RFC 9457 problem responses render from.

Exercises

1. **Map a balance.** Build `Mono::just(1_250_i64)` (a balance in cents), `map` it to a major-unit `f64` (`cents as f64 / 100.0`), and `block().await` it. Confirm you get `Some(12.5)`.
2. **Stream wallet events as a `Flux`.** Make a `Vec<i64>` of signed balance deltas (`[1000, 50, -25]`), wrap it with `Flux::just`, `scan` a running balance, and `collect_list` it. Verify the running balances are `[1000, 1050, 1025]` — a hand-rolled version of what the Lumen streaming endpoint emits per event.
3. **Recover from a flaky source.** Write a `Mono::from_callable` that returns `Err(FireflyError::internal("flaky"))` the first two times and `Ok(Some(n))` afterward, then wrap it in `Mono::retry_backoff(factory, Backoff::new(5, Duration::from_millis(10)))`. Assert it resolves to a value — the retry factory pattern Lumen's HTTP client would use against an external FX provider.

4. **Pick a responder.** Given a `Flux<WalletEvent>`, decide which responder a real-time dashboard wants (`Sse`) versus a bulk export (`NdJson`), and explain in one sentence why backpressure matters for the export case.

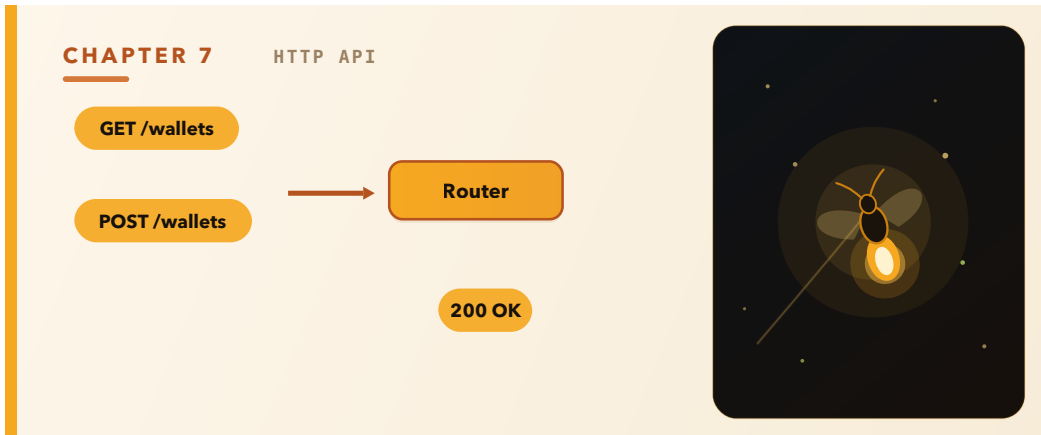
You now have the vocabulary the rest of the book builds on. Next, put it to work in [Your First HTTP API](#).



PART II

Modeling & Persisting the Domain





CHAPTER 7

Your First HTTP API

By the end of this chapter Lumen has a real HTTP surface: a `WalletApi` controller declared with `#[rest_controller]`, a `POST /api/v1/wallets` endpoint that opens a wallet and answers `201 Created`, a `GET /api/v1/wallets/:id` that returns a `WalletView`, and typed errors that render as RFC 9457 problem documents — with an in-process test that drives the whole router without binding a socket. This is the chapter where Lumen stops being a banner and starts being a service.

So far Lumen compiles, runs, and serves an actuator, and you know how its composition root resolves collaborators. Now you give it endpoints. The HTTP layer is axum; Firefly contributes the controller macro, the problem rendering, and the correlation/idempotency middleware you met in the Quickstart, all woven in by the `WebStack`. You write handlers; the framework supplies the wiring.

The `#[rest_controller]` macro

Lumen's wallet endpoints live on one type, `WalletApi`, whose `impl` block carries `#[rest_controller]`. The macro reads each verb attribute and generates a `WalletApi::routes(state) -> axum::Router` function — so a controller is just an `impl` with annotated methods, and the routing table is derived from the code rather than maintained beside it.

Rust

```
// src/web.rs
use std::sync::Arc;
use axum::extract::{Path, State};
use axum::Json;
use firefly::prelude::*;
use firefly::web::{WebError, WebResult};

use crate::commands::{GetWallet, OpenWallet};
use crate::domain::WalletView;

/// The wallet HTTP surface. It carries the CQRS `Bus` it dispatches through;
/// the controller stays thin and delegates every decision to a handler.
#[derive(Clone)]
pub struct WalletApi {
    pub bus: Arc<Bus>,
    // (the ledger and query cache fields arrive in later chapters)
}

/// `#[rest_controller(path = "...")]` generates `WalletApi::routes(state) ->`
/// `axum::Router`. Each method carries one verb mapping and returns
/// `WebResult<T>`, so a handler error renders as RFC 9457
/// `application/problem+json`.
#[rest_controller(path = "/api/v1")]
impl WalletApi {
    /// `POST /api/v1/wallets` - open a wallet. Validation failures surface as
    /// 422 problems; success answers `201 Created` with the view.
    #[post("/wallets")]
    async fn open(
        State(api): State<WalletApi>,
        Json(body): Json<OpenWallet>,
    ) -> WebResult<(axum::http::StatusCode, Json<WalletView>)> {
        let view: WalletView = api.bus.send(body).await.map_err(cqrs_to_web)?;
        Ok((axum::http::StatusCode::CREATED, Json(view)))
    }

    /// `GET /api/v1/wallets/:id` - fetch the read-model view. An unknown id
    /// renders as a 404 problem.
    #[get("/wallets/:id")]

```



```

async fn get(
  State(api): State<WalletApi>,
  Path(id): Path<String>,
) -> WebResult<Json<WalletView>> {
  let view: WalletView = api.bus.query(GetWallet {
    id }).await.map_err(cqrs_to_web)?;
  Ok(Json(view))
}
}

```

Three things to read here:

- **The path is composed.** `#[rest_controller(path = "/api/v1")]` is the prefix; `#[post("/wallets")]` and `#[get("/wallets/:id")]` are the suffixes. The macro joins them into `/api/v1/wallets` and `/api/v1/wallets/:id`.
- **Each handler is a plain axum handler.** `State`, `Path`, and `Json` are axum's own extractors — Firefly does not replace them. You write the function; the macro only registers it on the router.
- **The controller is thin.** `open` and `get` translate HTTP into a message and hand it to the CQRS `Bus`, then translate the result (or error) back into an HTTP response. The wallet *logic* lives behind the bus, where `CQRS` puts it. Treat `api.bus.send(...)` / `api.bus.query(...)` here as "dispatch to the handler that knows how"; the bus, the commands, and the read model are the subjects of chapters 7 through 11.

Design note. `#[rest_controller(path = "/api/v1")]` declares a controller and its path prefix; `#[get]` / `#[post]` declare the verb mappings. Beyond generating the router, the macro emits a route descriptor per endpoint that feeds the actuator `/mappings` view and the OpenAPI generator — so the routing table is derived from your code rather than maintained beside it, and the documentation surfaces stay in sync with the handlers automatically. If you've used a batteries-included framework before, this declarative-controller style will feel familiar.

The wire shape – `WalletView`

The view a handler returns is a plain `serde` struct. It is the *read model* projection of a wallet — flat, query-optimized, and decoupled from the internal aggregate. The balance travels as an integer count of minor units (cents), so `€10.00` is the JSON number

`1000`:

Rust

```
// src/domain.rs
#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize)]
pub struct WalletView {
    pub id: String,
    pub owner: String,
    /// The current balance, in minor units (cents).
    pub balance: i64,
    /// The aggregate version (number of events applied).
    pub version: i64,
}
```

The request body Lumen accepts on `POST /api/v1/wallets` is just as ordinary — the `OpenWallet` message, with a `#[serde(rename)]` so the JSON field is `openingBalance` while the Rust field stays `snake_case`:

JSON

```
{ "owner": "alice", "openingBalance": 1000 }
```

Content negotiation – one DTO, JSON or XML

Lumen's handlers answer `application/json` because they return `Json<WalletView>` — a deliberate, format-pinned contract. But a controller can also hand the framework a DTO and let the *client* pick the wire format. Wrap the return value in `Negotiate(dto)` and the response is rendered with the converter the request's `Accept` header selects —

`JsonMessageConverter` for `application/json`, `XmlMessageConverter` for `application/xml` / `text/xml` — while the request body is read by its `Content-Type` the same way:

Rust

```
// a format-agnostic variant of the wallet GET
use firefly::web::Negotiate;

#[get("/wallets/:id")]
async fn get(
    State(api): State<WalletApi>,
    Path(id): Path<String>,
) -> WebResult<Negotiate<WalletView>> {
    let view: WalletView = api.bus.query(GetWallet {
        id }).await.map_err(cqrs_to_web)?;
    Ok(Negotiate(view))
}
```

The same handler now serves both wire shapes from the one `WalletView`:

Text

```
GET /api/v1/wallets/wlt_1 Accept: application/json
→ { "id": "wlt_1", "owner": "alice", "balance": 1000, "version": 1 }

GET /api/v1/wallets/wlt_1 Accept: application/xml
→ <response><id>wlt_1</id><owner>alice</owner><balance>1000</balance>...</response>
```

You wire none of this. The `ContentNegotiationLayer` is installed by default in `WebStack::apply_middleware` — it sits closest to your routes, so a `Negotiate` response is re-rendered to the client's `Accept` before the outer middleware edge runs,

and a plain `Json<T>` (or any other) response passes through untouched. An absent or empty `Accept` defaults to JSON, and an unmatched type falls back to the first registered converter (JSON), so the negotiation never fails the request.

Design note. `Negotiate(dto)` hands the framework a DTO and lets the request's `Accept` header pick the wire format — `JsonMessageConverter` for `application/json`, `XmlMessageConverter` for `application/xml` / `text/xml` — with no controller code. The `JsonMessageConverter` / `XmlMessageConverter` pair ships in the registry, and `apply_middleware` installs the `ContentNegotiationLayer` by default, so negotiation is on out of the box. Add a converter — say CBOR — by implementing `MessageConverter` and adding it; user converters take priority over the built-ins.

Firefly's field-level format contract is `serde`: the rules live in derives on the DTO and apply identically whichever converter renders the value — `#[serde(rename_all = "camelCase")]` to fix the on-the-wire naming, `#[serde(tag = "type")]` for polymorphic enums, `chrono` types for date/time, and `#[serde(deny_unknown_fields)]` to reject unexpected keys on read:

Rust

```
#[derive(Serialize, Deserialize)]
#[serde(rename_all = "camelCase", deny_unknown_fields)]
pub struct WalletView {
    pub id: String,
    pub owner: String,
    pub balance: i64,
    pub version: i64,
}
```

JSON serialization & content negotiation

The `serde` derives above pin the wire shape *per type* — practical when a handful of DTOs each want their own contract. But a service usually wants **one** house style: every response in `camelCase`, nulls dropped, the same inclusion rules everywhere. Firefly gives you a single object to express that policy and a way to install it across the whole service: `firefly_web`'s `ObjectMapper`.

`ObjectMapper` is a builder. You set a property-naming convention, an inclusion rule, and whether to pretty-print, and the resulting mapper translates between your Rust structs and JSON on the wire — applying the policy on the way out and its inverse on the way in:

Rust

```
use firefly::web::{ObjectMapper, PropertyNaming, Inclusion};

// camelCase on the wire, drop nulls, compact output.
let mapper = ObjectMapper::new()
    .naming(PropertyNaming::CamelCase)
    .inclusion(Inclusion::NonNull)
    .pretty(false);
```

The naming and inclusion options are:

Option	Effect
<code>PropertyNaming::AsIs</code> (default)	leave field names untouched
<code>PropertyNaming::CamelCase</code>	<code>opening_balance</code> → <code>openingBalance</code>
<code>PropertyNaming::SnakeCase</code>	<code>openingBalance</code> → <code>opening_balance</code>
<code>PropertyNaming::KebabCase</code>	<code>opening_balance</code> → <code>opening-balance</code>
<code>PropertyNaming::PascalCase</code>	<code>opening_balance</code> → <code>OpeningBalance</code>
<code>PropertyNaming::ScreamingSnakeCase</code>	<code>opening_balance</code> → <code>OPENING_BALANCE</code>
<code>Inclusion::Always</code> (default)	serialize every field
<code>Inclusion::NonNull</code>	omit <code>null</code> fields
<code>Inclusion::NonEmpty</code>	omit <code>null</code> , empty strings, and empty collections

The mapper serializes and deserializes through the same methods you'd reach for on `serde_json`, but with the policy applied:

Rust

```
// Serialize: a snake_case Rust struct speaks camelCase on the wire.
let json: String = mapper.to_string(&wallet)?;           // -> Result<String,
FireflyError>
let value: serde_json::Value = mapper.to_value(&wallet)?;

// Deserialize: read camelCase back into the snake_case struct.
let wallet: WalletView = mapper.from_str(&json)?;        // -> Result<T,
FireflyError>
let wallet: WalletView = mapper.from_value(value)?;
```

The naming transform is **reversible**: a `snake_case` Rust struct speaks `camelCase` on the wire and reads it back the same way, so the same `ObjectMapper` sits on both ends of a request/response without you tracking which direction a field is travelling. If you

need the raw transform — for instance to post-process a `serde_json::Value` you built by hand — `apply_write(value) -> Value` renames toward the wire and `apply_read(value) -> Value` renames back toward your structs.

NOTE

A renaming mapper rewrites *every* object key in the document — it works on the JSON tree, so it cannot tell a struct field from a key inside a free-form `HashMap` you carry as data. Use a global naming policy on **DTO-shaped** payloads; for a type whose body holds arbitrary string-keyed data, leave the global policy at `AsIs` and name that one type with `#[serde(rename_all = "camelCase")]` — that is type-aware and never touches data keys. (Field names with a trailing number round-trip cleanly when written glued, `opening_balance2`, rather than with a separating underscore, `opening_balance_2`, which normalises to `opening_balance2`.)

Installing a global policy with `MappingJsonConverter`

A free-standing `ObjectMapper` is useful, but the point is to make the *whole service* observe one policy without decorating every DTO. `MappingJsonConverter` wraps a mapper and implements `firefly_web::MessageConverter` for `application/json`. Add it to a `MessageConverterRegistry` and every negotiated JSON request and response — the `Negotiate(dto)` responses from the previous section, and any JSON body the framework reads — flows through your naming and inclusion policy:

Rust

```

use firefly::web::{ObjectMapper, PropertyNaming, Inclusion,
MappingJsonConverter};

// One mapper expresses the service-wide JSON contract.
let mapper = ObjectMapper::new()
    .naming(PropertyNaming::CamelCase)
    .inclusion(Inclusion::NonNull);

// Wrap it as the JSON converter and register it so every negotiated
// application/json exchange observes the policy.
registry.add(std::sync::Arc::new(MappingJsonConverter::new(mapper)));

```

Because it registers as a user converter, `MappingJsonConverter` takes priority over the built-in `JsonMessageConverter` for `application/json` — so installing it once at the composition root is all it takes to apply a global JSON naming and inclusion policy to the entire HTTP surface, instead of repeating `#[serde(rename_all = ...)]` on every DTO. Per-type `serde` attributes still work and compose on top: reach for them when a specific type needs to deviate from the house style, and let `MappingJsonConverter` carry the default everywhere else.

Typed errors → RFC 9457 problems

A handler that returns `WebResult<T>` turns any error into the right `application/problem+json` response via `?.` `WebResult<T>` is an alias whose error arm is a `WebError`, and the framework knows how to render it. Lumen's controller maps the bus's error channel onto a precise HTTP status with one helper:

Rust

```
// src/web.rs
use crate::domain::DomainError;

/// Maps a bus `CqrsError` onto the precise HTTP problem the domain implies:
/// a validation failure → 422, a not-found detail → 404, an
/// insufficient-funds / non-positive detail → 422, otherwise 500.
fn cqrs_to_web(err: CqrsError) -> WebError {
    match err {
        CqrsError::Validation(detail) =>
            WebError::from(FireflyError::validation(detail)),
        CqrsError::Handler(detail) => {
            if detail.ends_with("not found") {
                WebError::from(FireflyError::not_found(detail))
            } else if detail == DomainError::InsufficientFunds.to_string()
                || detail == DomainError::NonPositiveAmount.to_string()
                || detail == DomainError::OwnerRequired.to_string()
            {
                WebError::from(FireflyError::validation(detail))
            } else {
                WebError::from(FireflyError::not_found(detail))
            }
        }
        other => WebError::from(FireflyError::internal(other.to_string())),
    }
}
```

The `FireflyError` constructors map straight to HTTP status — pick the one that matches the failure and the renderer does the rest:

Constructor	Status	Use
<code>FireflyError::bad_request(detail)</code>	400	malformed input
<code>FireflyError::unauthorized(detail)</code>	401	missing/invalid credentials
<code>FireflyError::forbidden(detail)</code>	403	authenticated but not allowed
<code>FireflyError::not_found(detail)</code>	404	absent resource
<code>FireflyError::conflict(detail)</code>	409	state conflict
<code>FireflyError::validation(detail)</code>	422	semantic validation failure
<code>FireflyError::internal(detail)</code>	500	server fault

A rendered problem for an unknown wallet looks like this — note the dedicated `application/problem+json` content type, which the tests assert on:

JSON

```
{
  "type": "https://fireflyframework.org/problems/not-found",
  "title": "Not Found",
  "status": 404,
  "detail": "wallet wlt_does_not_exist not found"
}
```

Design note. Returning `WebResult<T>` turns any `FireflyError` into the right `application/problem+json` response via `?`, with the problem rendering built in — you never write an error-to-status mapping for the framework's own errors. The RFC 9457 contract is stable and language-neutral, so a Firefly 404 presents identically to every client regardless of which service produced it.

Mounting the routes

The controller's `routes(state)` function returns a plain `axum::Router`, which the composition root wraps in the `WebStack` middleware chain. `LumenApp::router` is that one place — construct the controller from the resolved collaborators, call `WalletApi::routes`, and hand it to `apply_middleware`:

Rust

```
// src/web.rs
impl LumenApp {
    /// Builds the public router: the macro-generated wallet routes wrapped in
    /// the web middleware chain.
    pub fn router(&self) -> axum::Router {
        let state = WalletApi { bus: Arc::clone(&self.bus) };
        let routes = WalletApi::routes(state);
        self.web.apply_middleware(routes)
    }
}
```

Every request to a wallet route now passes through the canonical chain you got for free in the Quickstart — the RFC 9457 problem layer, correlation-id propagation, and idempotency replay — before it reaches your handler. You wrote the two handlers; the rest of the request lifecycle is the framework's.

NOTE

`LumenApp::router` is the *only* function that changes as Lumen grows. The streaming endpoint merges into it in [Production](#); the JWT security layer wraps it in [Security](#). `main` keeps calling `app.router()` and never learns the difference — the composition root absorbs every addition.

Proving it works – an in-process round-trip

Because `router()` is a self-contained `axum::Router`, Lumen's tests drive it **in-process** with `tower::ServiceExt::oneshot` — no socket bound, no port to race on. This is the first end-to-end test, the open-then-get round-trip:

Rust

```

// tests/http.rs
use axum::body::Body;
use axum::http::{Request, StatusCode};
use firefly_sample_lumen::build_router;
use firefly_sample_lumen::domain::WalletView;
use http_body_util::BodyExt;
use tower::ServiceExt;

#[tokio::test]
async fn open_then_get_round_trips_through_cqrs() {
    // POST /api/v1/wallets → 201 Created with the opened view.
    let res = build_router()
        .await
        .oneshot(
            Request::post("/api/v1/wallets")
                .header("content-type", "application/json")
                .body(Body::from(
                    serde_json::to_vec(&serde_json::json!({
                        "owner": "alice", "openingBalance": 1_000
                    })))
                .unwrap(),
        ))
        .unwrap(),
    )
    .await
    .unwrap();
    assert_eq!(res.status(), StatusCode::CREATED, "open should 201");

    let bytes = res.into_body().collect().await.unwrap().to_bytes();
    let opened: WalletView = serde_json::from_slice(&bytes).unwrap();
    assert_eq!(opened.owner, "alice");
    assert_eq!(opened.balance, 1_000);
    assert_eq!(opened.version, 1);

    // GET /api/v1/wallets/:id → 200 OK with the same view.
    let res = build_router()
        .await
        .oneshot(

```

```

        Request::get(&format!("/api/v1/wallets/{}", opened.id))
            .body(Body::empty())
            .unwrap(),
    )
    .await
    .unwrap();
    assert_eq!(res.status(), StatusCode::OK);
}

```

`build_router()` is the testable composition root: it is `build_app().await` followed by `.router()`, returning exactly the `axum::Router` `main` serves. The test exercises the macro-generated routes, the JSON contract, and the status-code mapping — the same code path a real client hits, minus the network.

The error paths are tested the same way. An empty `owner` is a `422` problem; an id that was never opened is a `404` problem — and both assert the `application/problem+json` content type, so the RFC 9457 contract is part of the test suite, not just the prose:

Rust

```

#[tokio::test]
async fn unknown_wallet_is_404_problem() {
    let res = build_router()
        .await
        .oneshot(
            Request::get("/api/v1/wallets/wlt_does_not_exist")
                .body(Body::empty())
                .unwrap(),
        )
        .await
        .unwrap();
    assert_eq!(res.status(), StatusCode::NOT_FOUND);
    let ct = res.headers().get("content-type").unwrap().to_str().unwrap();
    assert!(ct.contains("application/problem+json"));
}

```

Design note. `oneshot` against `build_router()` runs the whole controller stack in the test process with no live server and no socket bound, so the test exercises the real request path at full speed and without port contention. `Testing` builds this into a full strategy.

Recap – what changed in Lumen

Before	After this chapter
an empty public router	a <code>WalletApi</code> controller declared with <code>#[rest_controller]</code> and two real endpoints
no client contract	<code>POST /api/v1/wallets → 201 + WalletView</code> , <code>GET /api/v1/wallets/:id → 200 / 404</code> , all JSON
errors unconsidered	typed <code>FireflyError</code> → RFC 9457 <code>application/problem+json</code> with the right status
nothing to test	a <code>tower::oneshot</code> round-trip that drives the full router in-process, content-type assertions included

The controller is deliberately thin: it speaks HTTP and delegates the wallet logic to the bus. That seam is what the next several chapters fill in — the read model the `GET` serves, the domain that enforces the rules, and the CQRS handlers the `POST` dispatches to.

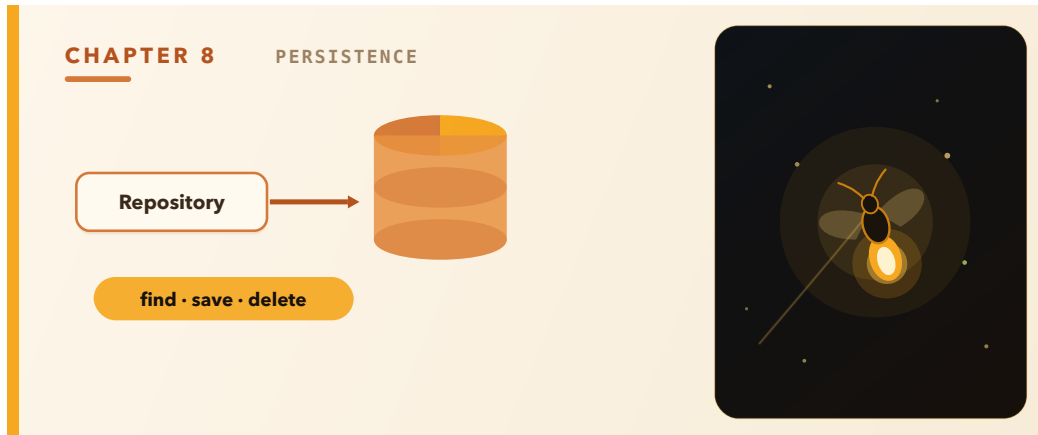
Exercises

1. **Add a route.** Give `WalletApi` a `#[get("/wallets")] list` method that returns `WebResult<Json<Vec<WalletView>>>`. Watch `WalletApi::routes` pick it up automatically — you never touch a routing table.
2. **Shape an error.** Make `cqrs_to_web` (or a small handler of your own) return `FireflyError::conflict("wallet already closed")` and confirm the response is

a `409` with `application/problem+json`. Try `bad_request` and `forbidden` too, and read the rendered `type / title / status` for each.

3. **Honor idempotency.** `POST /api/v1/wallets` twice with the same `Idempotency-Key` header and identical body; confirm the second response carries `Idempotent-Replay: true`. Then change the body under the same key and observe the `409`. You wrote none of this — it came with `apply_middleware`.
4. **Write the round-trip yourself.** Copy the `open_then_get` test, change the owner and opening balance, and assert the returned `balance` matches. Run `cargo test -p firefly-sample-lumen` and watch it pass against the real router.

Next, give the `GET` endpoint a real backing store with [Persistence & Reactive Repositories](#), then put the rules behind the bus in [Domain-Driven Design](#) and [CQRS](#).



CHAPTER 8

Persistence & Reactive Repositories

Lumen has a wallet API that returns a `WalletView` — but where does that view come from? At the end of [Your First HTTP API](#) the answer was "an in-memory map." This chapter gives Lumen a real persistence vocabulary: the **repository pattern** as the seam between the query side and storage, the in-memory baseline that keeps the teaching build infrastructure-free, and the pluggable reactive-repository / SQL upgrade path that swaps that baseline for Postgres, MySQL, SQLite, or MongoDB without touching a call site.

By the end of this chapter, Lumen will have its query-side read store — the `ReadModel` — framed as a repository: a map of wallet id → `WalletView` that the `GetWallet` query reads from and the event projection (built in [EDA](#)) writes to. You will see the in-memory baseline that ships in `samples/lumen`, and exactly which `firefly-data` adapter you would drop in to make it durable — the book's recurring swap the adapter move.

`firefly-data` provides the framework's persistence vocabulary: a composable `Filter` query DSL, a `Page<T>` paged-result envelope, a blocking `Repository<T, K>` contract, and — built on `firefly-reactive` — a reactive **CRUD** surface that returns `Mono` / `Flux` and streams rows lazily. This chapter covers both, with a real streaming SQL repository.

***Design note.** Firefly's data layer is the Repository pattern, expressed as idiomatic Rust: you depend on a trait — `Repository<T, K>` (blocking) or `ReactiveCrudRepository<T, ID>` (reactive) — and an adapter supplies the SQL. Depend on the port, swap the backend. If you have used a reactive-streams library before, the `Mono` / `Flux` surface will feel familiar.*

Lumen's read store, as a repository

Lumen splits its write model from its read model (the CQRS shape you build out in `CQRS`). The write side is the event-sourced `Ledger`; the read side is a flat, query-optimized `ReadModel` that `GET /api/v1/wallets/:id` serves. In `samples/lumen` the read model is an in-memory map — small, exact, and dependency-free, the right baseline for teaching:

Rust

```
// samples/lumen/src/ledger.rs – the CQRS query side.
use std::collections::HashMap;
use std::sync::Mutex;

use crate::domain::WalletView;

/// The in-memory read model: a map of wallet id → WalletView, upserted by the
/// projection and served by the GetWallet query. A real service would back this
/// with firefly's reactive repository over Postgres; an in-memory map keeps the
/// teaching baseline dependency-free.
#[derive(Debug, Default)]
pub struct ReadModel {
    rows: Mutex<HashMap<String, WalletView>>,
}

impl ReadModel {
    /// Upserts a projected view, replacing any previous row for the id.
    pub fn upsert(&self, view: WalletView) {
        self.rows
            .lock()
            .expect("read model lock")
            .insert(view.id.clone(), view);
    }

    /// Looks a projected view up by id.
    pub fn find(&self, id: &str) -> Option<WalletView> {
        self.rows.lock().expect("read model lock").get(id).cloned()
    }
}
```

Two things are deliberate. First, the surface is a **repository in miniature**:

`upsert(view)` and `find(id)` are the only operations the query side needs, so those are the only operations it exposes. Second, the keys and values are the plain domain

types — a `WalletView` keyed by its `id` — so when you swap the map for a database adapter, the *shape* the rest of Lumen sees does not move: a `find` still returns `Option<WalletView>`, an `upsert` still takes one.

That is the whole point of treating the read store as a repository: the `GetWallet` handler depends on "give me the view for this id," not on a `HashMap`. Below is the production surface you would lower this onto — same contract, a real database behind it.

The `Page<T>` envelope

`Page<T>` is the canonical paged-result envelope with a stable, versioned JSON shape — any client that honors that contract deserializes it uniformly, so generated SDK clients consume it without per-service handling:

Rust

```
pub struct Page<T> {
    pub content: Vec<T>,
    pub number: usize,      // zero-based page index
    pub size: usize,
    pub total_elements: u64,
    pub total_pages: usize, // derived
}
```

A *list wallets* endpoint — a natural Lumen extension — returns a `Page<WalletView>` so a client can page through accounts without ever loading the whole table.

The `Filter` DSL

A `Filter` composes predicates, sorts, and a page window, and renders to a parameterized `WHERE` clause via `to_sql()`:

Rust

```

use firefly_data::{Direction, Filter, Op, Predicate};
use serde_json::json;

let filter = Filter::default()
    .where_eq("owner", json!("alice"))
    .add(Predicate { field: "balance".into(), op: Op::Gte, value: json!(
100_000) })
    .order_by("version", Direction::Desc)
    .paged(0, 20);

let (where_clause, args) = filter.to_sql();
// where_clause: a parameter-indexed " WHERE ..." fragment
// args:          the bound values, in order
assert!(where_clause.contains("WHERE"));
assert_eq!(args.len(), 2);

```

The operators (`Op`) cover `Eq`, `Ne`, `Lt`, `Lte`, `Gt`, `Gte`, `Like`, `ILike`, `In`, and `IsNil` — the last skips an argument slot, so a predicate list and its argument list stay aligned. A "rich wallets" query (`balance >= 100_000`, newest first) is exactly the filter above.

The blocking `Repository` contract

`Repository<T, K>` is the object-safe `async_trait` port; `MemoryRepository` implements it for tests, and you back it with your driver in production:

Rust

```

#[async_trait]
pub trait Repository<T, K>: Send + Sync {
    async fn find_by_id(&self, id: &K) -> Result<T, DataError>;
    async fn find(&self, filter: &Filter) -> Result<Page<T>, DataError>;
    // save / delete / count ...
}

```

This is the contract Lumen's `ReadModel` is a hand-rolled, two-method subset of. Lower it onto `Repository<WalletView, String>` and `find` / `find_by_id` / `save` come from the framework.

The reactive CRUD surface

On top of the blocking contract, `firefly-data` adds a **reactive** CRUD surface built on `Mono` / `Flux` (the publishers from [The Reactive Model](#)). It is purely additive: nothing about the existing `Repository` API changes.

Method	Returns
<code>find_all()</code>	<code>Flux<T></code>
<code>find_all_by_id(ids)</code>	<code>Flux<T></code>
<code>find_by_id(id)</code>	<code>Mono<T></code>
<code>exists_by_id(id)</code>	<code>Mono<bool></code>
<code>save(e)</code>	<code>Mono<T></code>
<code>save_all(es)</code>	<code>Flux<T></code>
<code>delete_by_id(id)</code>	<code>Mono<()></code>
<code>delete_all()</code>	<code>Mono<()></code>
<code>count()</code>	<code>Mono<u64></code>
<code>Specification</code> + <code>Pageable</code>	<code>ReactiveSpecificationRepository</code>

A "no row" `find_by_id` resolves to an **empty** `Mono` — the reactive equivalent of Lumen's `ReadModel::find` returning `None`.

In-memory, for tests

`ReactiveMemoryRepository` is the reactive twin of `MemoryRepository`. Drive the publishers with `block()` / `collect_list()`. Here it is holding wallet views — the reactive version of Lumen's read store:

Rust

```
use firefly_data::{ReactiveCrudRepository, ReactiveMemoryRepository};

#[derive(Clone, PartialEq, Debug)]
struct WalletView { id: String, owner: String, balance: i64, version: i64 }

#[tokio::main]
async fn main() {
    let repo = ReactiveMemoryRepository::new(|w: &WalletView| w.id.clone());

    // save -> Mono<T>
    repo.save(WalletView { id: "wlt_1".into(), owner: "alice".into(), balance:
1000, version: 1 })
        .block().await.unwrap();

    // find_all -> Flux<T>, collected to a Vec
    let all = repo.find_all().collect_list().block().await.unwrap().unwrap();
    assert_eq!(all.len(), 1);

    // find_by_id miss -> empty Mono (Lumen's `ReadModel::find` returning None)
    assert_eq!(repo.find_by_id("ghost".into()).block().await.unwrap(), None);
    assert_eq!(repo.count().block().await.unwrap(), Some(1));
}
```

Swapping `ReadModel` for this repository is mechanical: `upsert` becomes `save`, `find` becomes `find_by_id(...).block().await`, and the query handler keeps its `Option<WalletView>` shape.

Real SQL, streaming rows as a **Flux**

`PostgresReactiveRepository` is a production repository over `tokio-postgres`. Reads drive the driver's `query_raw` row stream, so each row is decoded by a `RowMapper` and emitted the moment it arrives over the wire — a million-row table never lands fully in memory. Writes use a per-entity `inserter` closure that renders a `T` to an upsert whose `RETURNING` projects the configured columns. Backing Lumen's read model with it looks like this:

Rust

```

use std::sync::Arc;
use firefly_data::{PostgresReactiveRepository, ReactiveCrudRepository,
TableConfig};
use firefly_kernel::FireflyError;
use tokio_postgres::{Row, types::ToSql, NoTls};

#[derive(Clone, PartialEq, Debug)]
struct WalletView { id: String, owner: String, balance: i64, version: i64 }

# async fn ex() -> Result<(), Box<dyn std::error::Error>> {
let (client, conn) =
    tokio_postgres::connect("postgres://localhost/lumen", NoTls).await?;
tokio::spawn(async move { let _ = conn.await; });
let client = Arc::new(client);

let repo: PostgresReactiveRepository<WalletView, String> =
PostgresReactiveRepository::new(
    Arc::clone(&client),
    TableConfig::new("wallet_views", "id", ["id", "owner", "balance",
"version"]),
    // RowMapper: decode a WalletView from each streamed row.
    |row: &Row| Ok(WalletView {
        id: row.try_get("id").map_err(|e|
FireflyError::internal(e.to_string()))?,
        owner: row.try_get("owner").map_err(|e|
FireflyError::internal(e.to_string()))?,
        balance: row.try_get("balance").map_err(|e|
FireflyError::internal(e.to_string()))?,
        version: row.try_get("version").map_err(|e|
FireflyError::internal(e.to_string()))?,
    }),
    // inserter: upsert RETURNING the projected columns (the projection's
upsert).
    |w: &WalletView| (
        "INSERT INTO \"wallet_views\" (\"id\", \"owner\", \"balance\",
\"version\") \
        VALUES ($1, $2, $3, $4) \
        ON CONFLICT (\"id\") DO UPDATE SET \"owner\" = EXCLUDED.\"owner\", \

```

```

    \"balance\" = EXCLUDED.\"balance\", \"version\" = EXCLUDED.\"version\"
  \
  RETURNING \"id\", \"owner\", \"balance\", \"version\".to_string(),
  vec![
    Box::new(w.id.clone()) as Box<dyn ToSql + Sync + Send>,
    Box::new(w.owner.clone()) as Box<dyn ToSql + Sync + Send>,
    Box::new(w.balance) as Box<dyn ToSql + Sync + Send>,
    Box::new(w.version) as Box<dyn ToSql + Sync + Send>,
  ],
),
);

// Rows stream lazily out of find_all() as a Flux – a *list wallets* endpoint.
let all = repo.find_all().collect_list().block().await?.unwrap();
# Ok(())
# }

```

Use `stream_query(sql, params)` for custom derived queries: any `SELECT` projecting the configured columns is streamed row-by-row through the same `RowMapper`. This `Flux` plugs directly into a `NdJson` / `Sse` endpoint, so a database read streams to the client end-to-end with backpressure — no collect-then-emit step anywhere in the path. (That is the same `Flux → NdJson` seam Lumen's events endpoint uses, only sourced from rows instead of an event store.)

Reactive specification / paging

`ReactiveSpecificationRepository` runs a composable `Specification` predicate with an optional `Pageable` window and **streams** the matches as a `Flux`, with no intermediate `Page<T>` envelope — so it plugs straight into an NDJSON / SSE endpoint with backpressure.

Sorting & paging – the `ReactiveSortingRepository` (free)

firefly-data's `ReactiveSortingRepository<T, ID>` adds whole-collection sorting and paging — `find_all_sorted(RequestSort) -> Flux<T>` and `find_all_paged(Pageable) -> Flux<T>` — and you write **no** code for it: it is a blanket

`impl` over any repository that is both a `ReactiveCrudRepository` and a `ReactiveSpecificationRepository`. The sort/page is run as a match-all `Specification`, so every `SqlxReactiveRepository` and `ReactiveMemoryRepository` acquires it automatically.

Rust

```
use firefly_data::{
    Pageable, ReactiveCrudRepository, ReactiveMemoryRepository,
    ReactiveSortingRepository,
    RequestSort,
};

#[derive(Clone, PartialEq, Debug, serde::Serialize)]
struct WalletView { id: String, owner: String, balance: i64 }

#[tokio::main]
async fn main() {
    let repo = ReactiveMemoryRepository::new(|w: &WalletView| w.id.clone());
    for (id, owner) in [("w1", "carol"), ("w2", "alice"), ("w3", "bob")] {
        repo.save(WalletView { id: id.into(), owner: owner.into(), balance: 0 })
            .block().await.unwrap();
    }

    // find_all(Sort) – ordered by owner ascending, streamed as a Flux.
    let sorted = repo
        .find_all_sorted(RequestSort::by(["owner"]))
        .collect_list().block().await.unwrap().unwrap();
    assert_eq!(sorted[0].owner, "alice");

    // find_all(Pageable) – page 1 (1-based), size 2, sorted; a Flux window.
    let page = repo
        .find_all_paged(Pageable::of(1, 2, RequestSort::by(["owner"])).unwrap())
        .collect_list().block().await.unwrap().unwrap();
    assert_eq!(page.len(), 2);
}
```

NOTE

`find_all_paged` streams the page as a `Flux` window rather than buffering a `Page<T>` envelope — reach for `Page<T>` + a count query when you actually need totals.

Pluggable databases – a new DB is a new adapter

`firefly-data` is the `ports` crate: it owns no driver and implies no SQL engine. The `Filter` DSL, the composable `Specification`, the repository traits, and the auditing / soft-delete policies are all storage-agnostic. The lowering surface is what makes this hexagonal:

- a `SqlDialect` trait with three shipped impls — `PostgresDialect`, `MySqlDialect`, `SqliteDialect` — so `Filter::to_sql_with(&dialect)` and `Specification::to_sql_with(&dialect)` render the *same* query tree per backend, getting placeholder style (`$1` vs `?`), identifier quoting (`"id"` vs ``id``), `IN`-list shape, and case-insensitive `LIKE` right for each. (`Filter::to_sql` / `Specification::to_sql` stay the PostgreSQL default.)
- a `Specification::to_mongo()` / `Filter::to_mongo()` that lowers the same tree to a MongoDB `$`-operator filter document.

Two adapter crates — `firefly-data-sqlx` (covering the three relational backends) and `firefly-data-mongodb` — implement those ports so you code once and swap backends.

***Design note.** This is hexagonal architecture — ports & adapters. Your service depends on the repository port; the adapter (the relational crate, the Mongo crate) is chosen at wiring time. Swapping Postgres for MySQL is a pool change, not a code change — the call sites never move.*

Relational – `firefly-data-sqlx` (Postgres / MySQL / SQLite)

`SqlxReactiveRepository` (and the blocking `SqlxRepository`) serve all three relational backends from one codebase. A `Db` enum tags a `PgPool` / `MySqlPool` / `SqlitePool` with its `Backend`, and the repository picks the matching `SqlDialect` at runtime — so "new relational DB = new pool", not "new adapter". `UPSERT` is dialect-aware (`ON CONFLICT ... DO UPDATE` for Postgres/SQLite, `ON DUPLICATE KEY UPDATE` for MySQL), reads stream off sqlx's row stream into a `Flux`, and an optional `Auditor` / `SoftDeletePolicy` auto-stamps and hides rows on every write/read.

SQLite-in-memory is the **no-infrastructure default** — the same role the in-memory map plays in `samples/lumen`, but exercising the real adapter:

Rust

```

use firefly_data::{ReactiveCrudRepository, TableConfig};
use firefly_data_sqlx::{AnyRow, ColumnValue, Db, SqlxReactiveRepository};
use firefly_kernel::FireflyError;

#[derive(Debug, Clone, PartialEq)]
struct WalletView { id: String, owner: String, balance: i64 }

#
tokio::runtime::Builder::new_current_thread().enable_all().build().unwrap().block_on(async
{
    let pool = sqlx::SqlitePool::connect("sqlite::memory:").await.unwrap();
    sqlx::query(r#"CREATE TABLE "wallet_views" ("id" TEXT PRIMARY KEY, "owner" TEXT
NOT NULL, "balance" BIGINT NOT NULL)"#)
        .execute(&pool).await.unwrap();

    let repo: SqlxReactiveRepository<WalletView, String> =
SqlxReactiveRepository::new(
    Db::Sqlite(pool),
    TableConfig::new("wallet_views", "id", ["id", "owner", "balance"]),
    // RowMapper: decode by column name – backend-agnostic via AnyRow.
    |row: &AnyRow| Ok:::<_, FireflyError>(WalletView {
        id: row.get_str("id")?,
        owner: row.get_str("owner")?,
        balance: row.get_i64("balance")?,
    }),
    // RowWriter: the entity's (column, value) pairs.
    |w: &WalletView| vec![
        ColumnValue::new("id", w.id.clone()),
        ColumnValue::new("owner", w.owner.clone()),
        ColumnValue::new("balance", w.balance),
    ],
);
repo.save(WalletView { id: "wlt_1".into(), owner: "alice".into(), balance:
1000 })
    .block().await.unwrap();
# });

```

Switching to Postgres or MySQL is `Db::Postgres(pg_pool)` / `Db::MySql(my_pool)` — the repository call sites do not change. That is the upgrade path the `samples/lumen` comment promises: the in-memory `ReadModel` becomes a `SqlxReactiveRepository<WalletView, String>`, and the `GetWallet` handler is none the wiser.

The constructor takes a `Db`, a `TableConfig`, a `RowMapper` (reads), and a `RowWriter` (writes); three chainable builders add cross-cutting behaviour, each returning a fresh repository:

- `.with_auditor(Auditor)` — stamps `created_at` / `updated_at` / `created_by` / `updated_by` on every write (insert vs update is decided by whether the row already exists): automatic auditing.
- `.with_soft_delete(SoftDeletePolicy)` — hides soft-deleted rows from every read and turns `delete_by_id` into a `deleted_at` stamp instead of a physical `DELETE`: logical (soft) delete.
- `.with_version_column("version")` — turns on optimistic locking (next section).

Building the pool from config — `auto_configure`

In the examples above the pool is constructed by hand. In a real service the connection settings live in configuration, and Firefly turns them into a live pool — plus a registered transaction manager — in a single awaited call at startup. There is no dependency-injection container in the loop: you load the config, bind a plain `serde` struct, and `await` one function.

`DataSourceProperties` is that struct, bound from the `firefly.datasource.*` config tree:

Rust

```

use firefly_data_sqlx::DataSourceProperties;

// Bound from `firefly.datasource.*` (e.g. an application.yaml / env overrides).
pub struct DataSourceProperties {
    pub url: String,                // scheme picks the backend (see below)
    pub max_connections: u32,       // `0` leaves the driver default
    pub min_connections: u32,       // `0` leaves the driver default
    pub acquire_timeout_ms: u64,    // `0` leaves the driver default
    pub idle_timeout_ms: u64,       // `0` leaves the driver default
    pub max_lifetime_ms: u64,       // `0` leaves the driver default
}

```

The URL scheme selects the backend (each behind its cargo feature):

`postgres://` / `postgresql://` → PostgreSQL, `mysql://` → MySQL, `sqlite:` → SQLite. So a config change from `postgres://...` to `mysql://...` moves the whole service to MySQL with no code edit — the pluggable-database promise, driven from configuration.

Three entry points build the `Db`:

- `Db::connect(url).await -> Result<Db, FireflyError>` — a pool from a URL, using driver defaults.
- `Db::connect_with(&props).await` — a pool honouring the full `DataSourceProperties` (sizes, timeouts, lifetimes).
- `data_sqlx::auto_configure(&props).await` — the **one-call startup path**: it builds the pool *and* registers a `SqlxTransactionManager`, so `#[firefly::transactional]` resolves its manager with no manual wiring. The returned `Db` then builds your typed repositories.

The shape of a boot sequence is: load config → bind `DataSourceProperties` → `await auto_configure` once → build repositories from the returned `Db`.

Rust

```

use firefly_data::TableConfig;
use firefly_data_sqlx::{auto_configure, AnyRow, ColumnValue,
DataSourceProperties, SqlxReactiveRepository};
use firefly_kernel::FireflyError;

#[derive(Debug, Clone, PartialEq)]
struct WalletView { id: String, owner: String, balance: i64 }

# async fn boot(props: DataSourceProperties) -> Result<(), Box<dyn
std::error::Error>> {
// One awaited call: builds the pool AND registers the SqlxTransactionManager.
let db = auto_configure(&props).await?;

// The returned Db builds typed repositories – no DI container involved.
let wallets: SqlxReactiveRepository<WalletView, String> =
SqlxReactiveRepository::new(
    db.clone(),
    TableConfig::new("wallet_views", "id", ["id", "owner", "balance"]),
    |row: &AnyRow| Ok:<:_, FireflyError>(WalletView {
        id: row.get_str("id")?,
        owner: row.get_str("owner")?,
        balance: row.get_i64("balance")?,
    }),
    |w: &WalletView| vec![
        ColumnValue::new("id", w.id.clone()),
        ColumnValue::new("owner", w.owner.clone()),
        ColumnValue::new("balance", w.balance),
    ],
);
// Because auto_configure registered the manager, a `#[firefly::transactional]`
// fn that writes through `wallets` is now atomic with no further wiring.
# Ok(())
# }

```

Design note. Configuration drives the runtime, not a container. A plain `serde` struct is bound from `firefly.datasource.*`, and a single awaited `auto_configure` builds the pool and registers the transaction manager — the wiring is explicit, compiler-checked, and visible in one place rather than assembled by reflection at runtime.

Optimistic locking – `with_version_column`

`with_version_column("version")` makes a `save` a **version-guarded** conditional upsert: every write bumps the version column and guards the conflict-update on the version the entity was loaded with (`WHERE version = <loaded>`). If a concurrent writer moved the stored version on, the guarded update matches zero rows and the save fails with `DataError::OptimisticLock` rather than silently overwriting the other change. The entity's `RowWriter` must emit the version column carrying the loaded value. (Conflict detection is enforced on Postgres and SQLite; on MySQL the version is bumped but the guard is not applied.)

Rust

```

use firefly_data::{DataError, Repository, TableConfig};
use firefly_data_sqlx::{AnyRow, ColumnValue, Db, SqlxRepository};
use firefly_kernel::FireflyError;

#[derive(Debug, Clone)]
struct Account { id: String, balance: i64, version: i64 }

# async fn ex(pool: sqlx::PgPool) -> Result<(), Box<dyn std::error::Error>> {
let repo: SqlxRepository<Account, String> = SqlxRepository::new(
    Db::Postgres(pool),
    TableConfig::new("accounts", "id", ["id", "balance", "version"]),
    |row: &AnyRow| Ok:::<_, FireflyError>(Account {
        id: row.get_str("id")?,
        balance: row.get_i64("balance")?,
        version: row.get_i64("version")?,
    }),
    |a: &Account| vec![
        ColumnValue::new("id", a.id.clone()),
        ColumnValue::new("balance", a.balance),
        // The loaded version – the conditional upsert guards on it.
        ColumnValue::new("version", a.version),
    ],
)
.with_version_column("version");

// Two callers loaded the same Account at version 1. The first save wins
// (the row is now version 2); the second save's guard (WHERE version = 1)
// matches nothing, so it fails with OptimisticLock – the caller reloads +
// retries.
let stale = repo.save(Account { id: "acc_1".into(), balance: 50, version:
1 }).await;
assert!(matches!(stale, Err(DataError::OptimisticLock)));
# Ok(())
# }

```

NOTE

A stale write fails with `DataError::OptimisticLock` rather than silently overwriting a concurrent change; the caller reloads and retries. Lumen's *write* side reaches the same "lost-update prevention" guarantee differently — through the event store's optimistic-concurrency `append` (see [Event Sourcing](#)) — but a relational `Account` / `Order` repository uses a version column.

Declarative derived queries – `#[firefly::repository]`

Beyond CRUD, the `#[firefly::repository]` macro derives a query straight from a *method name*: `find_by_status(&str)` becomes `WHERE status = ?`. Apply it to an `impl` block of **typed stub methods** named with the framework's grammar (`find_by_...`, `count_by_...`, `exists_by_...`, `delete_by_...`); the macro discards the placeholder body and generates a real one that marshals the arguments and delegates to the tested runtime engine. The return type selects the operation:

Return shape	Generated call
<code>Result<Vec<T>, DataError></code>	<code>find_by_derived</code>
<code>Result<Option<T>, DataError></code>	<code>find_by_derived (first)</code>
<code>Result<i64, DataError></code>	<code>count_by_derived</code>
<code>Result<bool, DataError></code>	<code>exists_by_derived</code>
<code>Result<u64, DataError></code>	<code>delete_by_derived</code>

Each `impl`-block type exposes the backing repository via `self.repository()` (override with `#[repository(repo = "...")]`), returning a `SqlxReactiveRepository<Entity, Id>`:

Rust

```

use firefly_data::DataError;
use firefly_data_sqlx::SqlxReactiveRepository;

struct Account { /* ... */ }

struct AccountRepo {
    repo: SqlxReactiveRepository<Account, String>,
}

impl AccountRepo {
    // The accessor the macro calls (default name `repository`).
    fn repository(&self) -> &SqlxReactiveRepository<Account, String> {
        &self.repo
    }
}

#[firefly::repository]
impl AccountRepo {
    async fn find_by_status(&self, status: &str) -> Result<Vec<Account>,
DataError> { unimplemented!() }
    async fn find_by_owner_and_status(&self, owner: &str, status: &str) ->
Result<Vec<Account>, DataError> { unimplemented!() }
    async fn count_by_owner(&self, owner: &str) -> Result<i64, DataError> {
unimplemented!() }
    async fn exists_by_email(&self, email: &str) -> Result<bool, DataError> {
unimplemented!() }
    async fn delete_by_status(&self, status: &str) -> Result<u64, DataError> {
unimplemented!() }
}

```

Paged derived queries – a trailing `Pageable`

A `find_by_...` method whose last argument is a `Pageable` and which returns `Result<Vec<T>, DataError>` is a *paged* derived query: the pageable's sort and window are appended to the generated `WHERE`, and the runtime backs it with

`SqlxReactiveRepository::find_by_derived_paged(method_name, &args, &Pageable)`. Build the page with `Pageable::of(page, size, sort)` — `page` is 1-based — and `RequestSort::of([Order::desc("id")])` for the ordering:

Rust

```
use firefly_data::{DataError, Order, Pageable, RequestSort};

#[firefly::repository]
impl AccountRepo {
    // The trailing Pageable makes this a paged query: WHERE owner = ?,
    // then the pageable's ORDER BY + LIMIT/OFFSET window.
    async fn find_by_owner(&self, owner: &str, page: Pageable) ->
    Result<Vec<Account>, DataError> {
        unimplemented!()
    }
}

# async fn ex(accounts: &AccountRepo) -> Result<(), DataError> {
    let page = Pageable::of(1, 20, RequestSort::of([Order::desc("id")])).unwrap();
    let rows = accounts.find_by_owner("alice", page).await?;
    # Ok(())
    # }
```

Custom queries – `#[query(...)]`

When the method-name grammar can't express the query you need, annotate the stub with `#[query(...)]` and write the SQL yourself. A `:name` placeholder binds to the argument named `name`, and the **return type** selects the **operation** exactly as for derived methods — `Vec<T>` / `Option<T>` for a list, `i64` for a count, `bool` for an exists, and `u64` for a *modifying* statement (`INSERT` / `UPDATE` / `DELETE`, returning the affected-row count):

Rust

```

use firefly_data::DataError;

#[firefly::repository]
impl AccountRepo {
    // Native SQL; :status binds to the `status` argument.
    #[query("SELECT id, owner FROM accounts WHERE status = :status ORDER BY id
DESC")]
    async fn list_by_status(&self, status: &str) -> Result<Vec<Account>,
DataError> {
        unimplemented!()
    }

    // i64 return -> a count query.
    #[query("SELECT COUNT(*) FROM accounts WHERE owner = :owner")]
    async fn tally(&self, owner: &str) -> Result<i64, DataError> {
        unimplemented!() }

    // u64 return -> a modifying statement; the value is the affected-row count.
    #[query("UPDATE accounts SET status = :to WHERE status = :from")]
    async fn retire(&self, from: &str, to: &str) -> Result<u64, DataError> {
        unimplemented!() }
}

```

`#[query("...")]` is shorthand for `#[query(sql = "...")]` (native SQL). For a portable, entity-oriented query use the JPQL-like form `#[query(jpql = "...", entity = "Account")]`, whose `FROM <Entity>` is transpiled to the configured table so the same string runs on Postgres, MySQL, or SQLite.

Under the hood the runtime engine does two things. `QueryMethodParser` parses the method name — prefix (`find` / `count` / `exists` / `delete`), `By`, then a chain of `And` / `Or` property conditions — into a query the active `SqlDialect` lowers to a parameterized statement, so the same `find_by_owner_and_status` runs on Postgres, MySQL, or SQLite; a trailing `Pageable` routes to `find_by_derived_paged`. The `#[query(...)]` attribute, in turn, lowers to the repository's `query_list` /

`query_count` / `query_exists` / `query_execute` helpers — list, count, exists, and modifying ops respectively — binding the `:name` placeholders and (for the JPQL form) transpiling `FROM <Entity>` to the table.

 TIP

Reach for the method-name grammar for simple predicates and a trailing `Pageable` for paged reads; use `#[query(...)]` for anything the name grammar can't express. A relational `Account` / `Order` service writes these; Lumen's event-sourced read side stays a hand-rolled in-memory `ReadModel` instead.

Document – `firefly-data-mongodb` (MongoDB)

`MongoRepository<T, ID>` puts a MongoDB collection behind the same `ReactiveCrudRepository` + `ReactiveSpecificationRepository` traits, lowering a `Specification` via `Specification::to_mongo()` exactly as the relational adapters lower it via `to_sql`. A `BaseDocument` mixin (embedded with `#[serde(flatten)]`) carries the audit stamps and soft-delete column, and `with_soft_delete(policy)` makes every read inject a `{"<column>": null}` guard while `delete_by_id` becomes a logical delete. Reads stream lazily off the driver cursor as a `Flux`.

Rust

```

use firefly_data::{ReactiveCrudRepository, Specification};
use firefly_data_mongodb::{BaseDocument, MongoRepository};
use mongodb::bson::{Bson, Document};
use serde::{Deserialize, Serialize};

#[derive(Serialize, Deserialize)]
struct WalletDocument {
    #[serde(rename = "_id")] id: String,
    owner: String,
    balance: i64,
    #[serde(flatten)] base: BaseDocument,
}

# async fn run() -> Result<(), Box<dyn std::error::Error>> {
let client = mongodb::Client::with_uri_str("mongodb://localhost:27017").await?;
let collection =
client.database("lumen").collection::<Document>("wallet_views");
let repo: MongoRepository<WalletDocument, String> =
    MongoRepository::new(collection, |w: &WalletDocument|
Bson::String(w.id.clone()));

repo.save(WalletDocument {
    id: "wlt_1".into(), owner: "alice".into(), balance: 1000, base:
BaseDocument::new(),
}).block().await?;
# Ok(())
# }

```

Because all four backends sit behind the same ports, a service that codes against `Repository` / `ReactiveCrudRepository` / `Specification` can move from Postgres to MySQL, SQLite, or MongoDB by swapping the adapter constructor — and adding a *new* database is "write a `firefly-data-<tech>` crate that implements the ports", not "rewrite the data layer." Both adapters are tested against real Postgres, MySQL, SQLite, and MongoDB.

One-dependency note. *Lumen pulls none of these adapters in its default build — they are opt-in cargo features on the `firefly` facade (`firefly = { version = "26.6", features = ["data-sqlx"] }`), re-exported as `firefly::data_sqlx` / `firefly::data_mongodb`. The teaching build stays lean; the production build adds exactly the driver it needs.*

Schema migrations

`firefly-migrations` is a forward-only SQL migration runner. Migration files are named `V{version}_{description}.sql` (e.g. `V001_init.sql`); each runs once, in version order, inside a transaction. The runner works against any store behind the small synchronous `Database` port:

Rust

```
use firefly_migrations::{run, DirSource};

let src = DirSource { dir: "migrations".into() };
run(&mut db, &src)?; // applies pending migrations in order
let status = firefly_migrations::inspect(&mut db, &src)?; // applied + pending
```

The [CLI](#) wraps this: `firefly db init`, `firefly db migrate -m "create wallet_views"`, `firefly db upgrade --url sqlite://lumen.db`, and `firefly db status`. A `V001_wallet_views.sql` creating the read-model table is all the schema Lumen's durable read side needs.

Transactions – `#[firefly::transactional]`

`firefly-transactional` brings declarative transactions to Rust: annotate an `async fn` that returns `Result<_, E>` (where `E: From<firefly::transactional::TxError>`) with `#[firefly::transactional]` and the body runs inside a transaction — `commit` on `Ok`, `rollback` on `Err`.

Rust

```

use firefly::transactional::TxError;
use firefly_data_sqlx::SqlxReactiveRepository;

#[derive(Debug)]
enum TransferError { /* ... */ Tx(TxError) }
impl From<TxError> for TransferError { fn from(e: TxError) -> Self {
TransferError::Tx(e) } }

struct Accounts { repo: SqlxReactiveRepository<Account, String> }
struct Ledger { repo: SqlxReactiveRepository<Entry, String> }

#[derive(Debug, Clone)] struct Account { id: String, balance: i64 }
#[derive(Debug, Clone)] struct Entry { id: String, account: String, delta:
i64 }

#[firefly::transactional(
    propagation = "requires_new",
    isolation = "serializable",
    read_only,
    timeout_ms = 5000,
)]
async fn record(accounts: &Accounts, ledger: &Ledger) -> Result<(),
TransferError> {
    // ... repository writes here join this transaction automatically ...
    Ok(())
}

```

The attributes are `propagation` (`required` / `requires_new` / `nested` / `supports` / `not_supported` / `mandatory` / `never`), `isolation` (`read_committed` / `repeatable_read` / `serializable` / ...), `read_only`, and `timeout_ms`.

Ambient enlistment is what makes this seamless. The manager opens a sqlx transaction and stows it in a task-local stack; while that scope is active, every `SqlxReactiveRepository` / `SqlxRepository` write *inside the fn* routes onto the active

transaction instead of a fresh pool connection — so a plain sequence of `repo.save(...).await?` calls is atomic with **no change to the repository code**. An atomic two-repository money transfer:

Rust

```
use firefly::transactional::TxError;

#[firefly::transactional] // defaults: REQUIRED, datasource isolation, read-
write
async fn transfer(
    accounts: &SqlxReactiveRepository<Account, String>,
    ledger: &SqlxReactiveRepository<Entry, String>,
    from: Account,
    to: Account,
    amount: i64,
) -> Result<(), TxError> {
    // All four writes enlist in the same ambient transaction. If any await
    // returns Err, the whole unit of work rolls back; otherwise it commits.
    accounts.save(Account { balance: from.balance - amount, ..from.clone() })
        .into_future().await?;
    accounts.save(Account { balance: to.balance + amount, ..to.clone() })
        .into_future().await?;
    ledger.save(Entry { id: "e1".into(), account: from.id, delta: -amount })
        .into_future().await?;
    ledger.save(Entry { id: "e2".into(), account: to.id, delta: amount })
        .into_future().await?;
    Ok(())
}
```

For programmatic control there is `firefly::transactional::transactional(opts, f)` and `transactional_on(&manager, opts, f)` for an explicit manager, with `TxOptions`, `Propagation`, and `Isolation` builders. The sqlx adapter — `SqlxTransactionManager`, registered once at startup (the `auto_configure` path below

does this for you) — supplies the real behaviour: full propagation (`REQUIRED` / `REQUIRES_NEW` / `NESTED` / `SUPPORTS` / `NOT_SUPPORTED` / `MANDATORY` / `NEVER`), isolation, read-only, a statement timeout, and `SAVEPOINT`-based `NESTED` nesting.

NOTE

The killer feature is **ambient enlistment**: while a transactional scope is active, every repository write inside the fn joins the active transaction automatically — a plain sequence of `repo.save(...).await?` calls is atomic with no change to the repository code, carried on a task-local rather than a thread-bound connection. (Lumen's write side reaches durability differently — through the event store's optimistic-concurrency `append`, covered in [Event Sourcing](#) — but a relational `Account` / `Order` service spanning two repositories is exactly what `#[transactional]` is for.)

What changed in Lumen

Lumen now has a clear persistence story, even though its teaching build stays infrastructure-free:

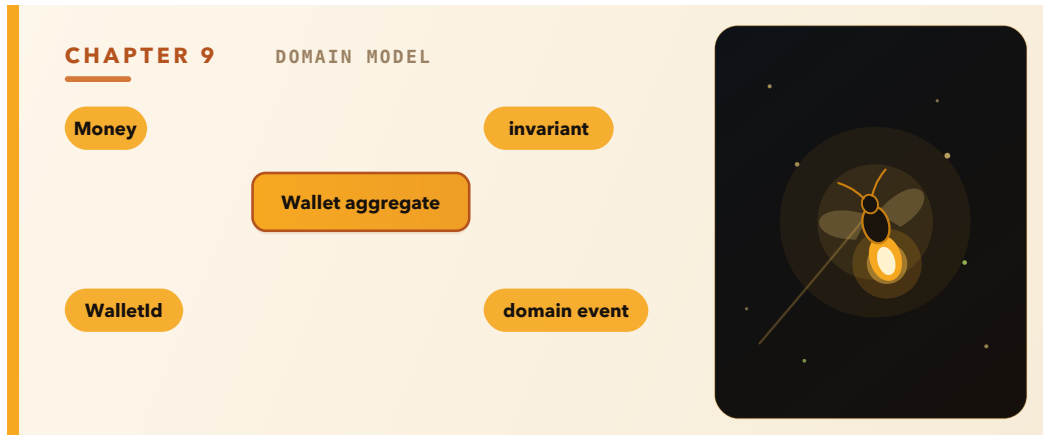
- **The read store is a repository.** `ReadModel` (in `samples/lumen/src/ledger.rs`) is a `Mutex<HashMap<String, WalletView>>` exposing exactly `upsert(view)` and `find(id)` — the two operations the query side needs, keyed by the `WalletView`'s own id. The `GetWallet` handler depends on the *contract*, not the map.
- **The baseline is in-memory by choice.** The map keeps the dependency footprint at one Firefly crate; the comment in the source names the upgrade explicitly — "a real service would back this with firefly's reactive repository over Postgres."
- **The upgrade is an adapter swap.** Lowering `ReadModel` onto `ReactiveCrudRepository<WalletView, String>` — `SqlxReactiveRepository` for Postgres/MySQL/SQLite, `MongoRepository` for Mongo — turns `upsert` into `save` and `find` into `find_by_id`, with the query handler's `Option<WalletView>` shape

unchanged. A new database is a new pool (relational) or a new adapter crate, never a rewrite.

Exercises

1. **Re-skin `ReadModel` as a trait.** Define `trait WalletViews { fn upsert(&self, v: WalletView); fn find(&self, id: &str) -> Option<WalletView>; }`, implement it for the in-memory `ReadModel`, and have the `GetWallet` handler take `&dyn WalletViews`. Confirm the rest of Lumen still compiles — proof the query side depends on the contract, not the map.
2. **Back the read model with SQLite.** Using the `SqlxReactiveRepository` listing above, create a `wallet_views` table in `sqlite::memory:`, `save` two views, and `find_all().collect_list()` them. Assert both come back. This is the production read store, exercised end to end against the real adapter.
3. **Page the wallets.** Build a `Filter` that selects wallets with `balance >= 100_000` ordered by `version` descending, page `(0, 20)`, and print its `to_sql()`. Then describe (one sentence each) how the same filter would render under `MySqlDialect` and `SqliteDialect`.
4. **Trace the swap.** List the exact lines in `samples/lumen/src/ledger.rs` that would change if `ReadModel` became a `SqlxReactiveRepository<WalletView, String>`, and which lines in `samples/lumen/src/commands.rs` (the `GetWallet` handler) would *not* — confirming the adapter boundary holds.

With persistence framed, the next chapter models the business itself — the `Money` value object and the `Wallet` aggregate. See [Domain-Driven Design](#).



CHAPTER 9

Domain-Driven Design

Lumen can open wallets and read them back, and it has a place to put the read model. But look closely and something is missing: nothing yet *owns* the rules. Where is "you cannot withdraw more than you hold"? Where is "an amount must be positive"? Where is "a wallet must have an owner"? Right now those would live as scattered `if`-statements in a handler — exactly the kind of rule a future developer can bypass by writing to the store directly.

Domain-Driven Design fixes that by making the model responsible for its own invariants. This chapter builds Lumen's domain core: the `Money` value object (immutable, integer cents, exact arithmetic) and the `Wallet` aggregate that guards the overdraft, positive-amount, and owner-required rules — each command raising the domain event that records what happened. Both files are drawn verbatim from `samples/lumen`.

By the end of this chapter, Lumen will have `src/money.rs` and the heart of `src/domain.rs`: a `Money` value object closed under `add` / `subtract`, a `Wallet` aggregate with `open` / `deposit` / `withdraw` that enforce the invariants and `raise` events, and a typed `DomainError` family whose `Display` strings surface verbatim as RFC 9457 problem details. The aggregate carries `#[derive(AggregateRoot)]`; nothing carries `thiserror`.

Design note. This chapter builds Lumen's domain core with the classic DDD building blocks — value objects, aggregates, aggregate roots, domain events — as idiomatic Rust.

`#[derive(AggregateRoot)]` generates the `AGGREGATE_TYPE` constant and the `aggregate()` / `aggregate_mut()` accessors over the embedded `AggregateRoot` (which itself carries the uncommitted-event buffer and version), so your code holds only the rules.

The `Money` value object

A value object is defined entirely by its attributes — it has no identity — and it is **immutable**: every operation returns a *new* value rather than mutating in place. Money is the textbook example, and getting it right matters more here than almost anywhere: amounts are stored as integer **minor units** (cents), so the arithmetic is exact. No binary floating-point drift, the classic correctness bug a money type exists to prevent.

Here is Lumen's `Money`, the whole core of `src/money.rs`:

Rust

```
// samples/lumen/src/money.rs
use std::fmt;
use serde::{Deserialize, Serialize};

/// An exact monetary amount, expressed in integer minor units (cents).
#[derive(Debug, Clone, Copy, Default, PartialEq, Eq, PartialOrd, Ord, Serialize,
Deserialize)]
#[serde(transparent)]
pub struct Money {
    cents: i64,
}

impl Money {
    /// A zero amount – the opening balance of a brand-new wallet.
    pub const ZERO: Money = Money { cents: 0 };

    /// Builds a `Money` from a raw minor-unit (cent) count.
    pub const fn cents(cents: i64) -> Self {
        Money { cents }
    }

    /// Builds a `Money` from a whole-currency unit count (`from_units(10)` is €10.00).
    pub const fn from_units(units: i64) -> Self {
        Money { cents: units * 100 }
    }

    /// The amount in minor units (cents) – the wire representation.
    pub const fn cents_value(self) -> i64 {
        self.cents
    }

    /// Whether this amount is strictly positive (`> 0`).
    pub const fn is_positive(self) -> bool {
        self.cents > 0
    }

    /// Returns a new `Money` that is `self + other` (immutable addition).

```

```

#[must_use]
pub const fn add(self, other: Money) -> Money {
    Money { cents: self.cents + other.cents }
}

/// Returns `self - other`, or `MoneyError::Overdraw` if that would go below
zero.
pub fn subtract(self, other: Money) -> Result<Money, MoneyError> {
    if other.cents > self.cents {
        return Err(MoneyError::Overdraw);
    }
    Ok(Money { cents: self.cents - other.cents })
}

/// Validates that this amount is strictly positive, returning it unchanged
on success.
pub fn require_positive(self) -> Result<Money, MoneyError> {
    if self.is_positive() {
        Ok(self)
    } else {
        Err(MoneyError::NonPositive)
    }
}
}

```

Several design choices are load-bearing:

- **Integer cents, never a float.** The single field is `cents: i64`, kept private so the only way to a `Money` is through a constructor. €10.00 is `Money::cents(1_000)`; €12.50 after an addition is `Money::cents(1_250)`. The math is exact by construction.
- **Immutable.** `add` is `#[must_use]` and `const`; it returns a fresh `Money` and leaves the operands untouched. So does `subtract`. There is no `add_assign` — a value object is replaced, not edited.

- **Closed under the wallet's operations.** `add` for credits, `subtract` (fallible, guarding against overdraw) for debits, and `require_positive` for the guard every mutating command runs before raising an event.
- `#[serde(transparent)]`. `Money` serializes as the bare cent integer — a balance of €10.00 is the JSON number `1000`, the contract the read model and the event payloads share. No `{ "cents": 1000 }` wrapper.

`Display` renders the human form (`1250` cents → `"12.50"`) for logs and the banner, and the error type is hand-written:

Rust

```
/// The typed error a `Money` operation can fail with.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum MoneyError {
    /// An amount was expected to be strictly positive (> 0) but was not.
    NonPositive,
    /// A subtraction would drop the balance below zero.
    Overdraw,
}

impl fmt::Display for MoneyError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            MoneyError::NonPositive => f.write_str("amount must be positive"),
            MoneyError::Overdraw => f.write_str("amount exceeds balance"),
        }
    }
}

impl std::error::Error for MoneyError {}
```

One-dependency note. `MoneyError` hand-writes `Display` and `std::error::Error` rather than deriving them with `thiserror`. That is on purpose: Lumen depends on exactly one Firefly crate plus `axum` and `serde`, and the book keeps that promise all the way down — even the error enums. Two trait impls per error type is a small price for an honest dependency list.

Design note. `Money` is a frozen, value-equal type — Rust makes the immutability a compiler guarantee. The "integer minor units, never a float" discipline is what any correct money type needs: floating-point drift is the classic correctness bug a money type exists to prevent.

The `Wallet` aggregate

`Money` solves *representation*. The `Wallet` aggregate owns *behavior*: it is the consistency boundary, the single entry point through which every change to a wallet must flow, so the invariants cannot be bypassed.

Lumen's `Wallet` is event-sourced (the full event-store machinery arrives in [Event Sourcing](#)), but the DDD shape is visible already. The aggregate embeds the framework's `AggregateRoot` — the uncommitted-event buffer plus a version — and `#[derive(AggregateRoot)]` generates the `AGGREGATE_TYPE` constant and the `aggregate()` / `aggregate_mut()` accessors:

Rust

```
// samples/lumen/src/domain.rs
use firefly::eventsourcing::{AggregateRoot, DomainEvent};
use firefly::prelude::*;

use crate::money::{Money, MoneyError};

/// The aggregate-type discriminator stamped onto every event a Wallet raises.
pub const AGGREGATE_TYPE: &str = "Wallet";

/// The event-sourced wallet aggregate.
#[derive(Debug, Clone, AggregateRoot)]
#[firefly(aggregate_type = "Wallet")]
pub struct Wallet {
    /// The framework aggregate root – uncommitted-event buffer + version.
    pub root: AggregateRoot,
    /// The owner's display name.
    pub owner: String,
    /// The current balance as a `Money` value object.
    pub balance: Money,
    /// Whether the wallet has been opened (an empty stream is "absent").
    pub opened: bool,
}
```

The aggregate's projected state — `owner`, `balance`, `opened` — is the *result* of applying its events. The behavior methods enforce the rules.

The factory: `open`

`open` is the sole way to bring a wallet into existence. It validates inputs, constructs the aggregate, and `raise` s the opening event:

Rust

```

impl Wallet {
    /// Opens a fresh wallet, raising a `WalletOpened` event.
    pub fn open(
        id: impl Into<String>,
        owner: impl Into<String>,
        opening_balance: Money,
    ) -> Result<Self, DomainError> {
        let id = id.into();
        let owner = owner.into();
        if owner.trim().is_empty() {
            return Err(DomainError::OwnerRequired);
        }
        if opening_balance.cents_value() < 0 {
            return Err(DomainError::NonPositiveAmount);
        }
        let mut wallet = Wallet {
            root: AggregateRoot::new(&id, AGGREGATE_TYPE),
            owner: owner.clone(),
            balance: Money::ZERO,
            opened: false,
        };
        wallet.raise(
            WalletOpened::EVENT_TYPE,
            &WalletOpened { wallet_id: id, owner, opening_balance:
opening_balance.cents_value() },
        );
        wallet.balance = opening_balance;
        wallet.opened = true;
        Ok(wallet)
    }
}

```

Two invariants are enforced before any event is raised: the owner must be non-blank (`OwnerRequired`), and the opening balance must not be negative (a *zero* opening balance is allowed). Using a factory rather than a public constructor guarantees the `WalletOpened` event is never forgotten — there is no back-channel that produces a wallet without recording its birth.

The behavior methods: `deposit` and `withdraw`

The two mutating commands follow one shape: check the wallet exists, validate the amount, apply the `Money` operation, raise the event, update state. The overdraft rule is enforced exactly once — by `Money::subtract`:

Rust

```

impl Wallet {
    /// Credits `amount` to the wallet, raising a `MoneyDeposited` event.
    pub fn deposit(&mut self, amount: Money) -> Result<(), DomainError> {
        self.require_opened()?;
        let amount = amount.require_positive()?;
        self.raise(
            MoneyDeposited::EVENT_TYPE,
            &MoneyDeposited { wallet_id: self.root.id.clone(), amount:
amount.cents_value() },
        );
        self.balance = self.balance.add(amount);
        Ok(())
    }

    /// Debits `amount` from the wallet, raising a `MoneyWithdrawn` event.
    pub fn withdraw(&mut self, amount: Money) -> Result<(), DomainError> {
        self.require_opened()?;
        let amount = amount.require_positive()?;
        let remaining = self.balance.subtract(amount)?; // Overdraw ->
InsufficientFunds
        self.raise(
            MoneyWithdrawn::EVENT_TYPE,
            &MoneyWithdrawn { wallet_id: self.root.id.clone(), amount:
amount.cents_value() },
        );
        self.balance = remaining;
        Ok(())
    }

    fn require_opened(&self) -> Result<(), DomainError> {
        if self.opened {
            Ok(())
        } else {
            Err(DomainError::NotFound(self.root.id.clone()))
        }
    }
}

```


Read `withdraw` carefully — it is where the consistency boundary earns its keep. The order is *validate, then mutate*: `require_opened`, `require_positive`, and the `subtract` overdraft check all run **before** the event is raised. If the withdrawal would overdraw, `Money::subtract` returns `MoneyError::Overdraw`, the `?` converts it to `DomainError::InsufficientFunds` (via the `From` impl below), and the method returns *without raising anything*. The aggregate's invariant — "balance never goes below zero" — is physically unreachable from outside, because the only path to a withdrawal runs this gauntlet first.

The typed `DomainError` family

The errors are a closed enum with stable `Display` strings — stable because tests assert on them and they surface verbatim as the RFC 9457 problem `detail` once mapped at the HTTP boundary (see [CQRS](#) and [First HTTP API](#)):

Rust

```

/// The typed domain-error family.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum DomainError {
    /// A command referenced an amount that was not strictly positive.
    NonPositiveAmount,
    /// A withdrawal (or transfer debit) exceeded the available balance.
    InsufficientFunds,
    /// A command targeted a wallet that was never opened.
    NotFound(String),
    /// The owner name was empty when opening a wallet.
    OwnerRequired,
}

impl std::fmt::Display for DomainError {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        match self {
            DomainError::NonPositiveAmount => f.write_str("amount must be
positive"),
            DomainError::InsufficientFunds => f.write_str("insufficient funds"),
            DomainError::NotFound(id) => write!(f, "wallet {id} not found"),
            DomainError::OwnerRequired => f.write_str("owner is required"),
        }
    }
}

impl std::error::Error for DomainError {}

impl From<MoneyError> for DomainError {
    fn from(e: MoneyError) -> Self {
        match e {
            MoneyError::NonPositive => DomainError::NonPositiveAmount,
            MoneyError::Overdraw => DomainError::InsufficientFunds,
        }
    }
}

```

The `From<MoneyError> for DomainError` impl is what lets `withdraw` write `self.balance.subtract(amount)?` and have an overdraw surface as `InsufficientFunds` — the value object reports the arithmetic fact, the aggregate translates it into domain language. Like `MoneyError`, `DomainError` hand-writes its `Display` and `Error` impls: no `thiserror`, one dependency.

NOTE

Lumen returns a typed `DomainError` rather than throwing: `InsufficientFunds` / `NonPositiveAmount` / `OwnerRequired` become 422 problems and `NotFound` becomes a 404, decided by a `match` at the web boundary (a returned value, checked by the compiler — no exception-to-status table to keep in sync).

The read-model view

The aggregate is the *write* model. What `GET /api/v1/wallets/:id` returns is a flat, query-optimized `WalletView` — the read model the previous chapter framed as a repository. The aggregate produces it on demand:

Rust

```

impl Wallet {
    /// The current read-model view of this aggregate.
    pub fn view(&self) -> WalletView {
        WalletView {
            id: self.root.id.clone(),
            owner: self.owner.clone(),
            balance: self.balance.cents_value(),
            version: self.root.version,
        }
    }
}

/// The read-model projection of a wallet – the wire shape served by
/// `GET /api/v1/wallets/:id` and stored in the read-model repository.
#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize)]
pub struct WalletView {
    pub id: String,
    pub owner: String,
    /// The current balance, in minor units (cents).
    pub balance: i64,
    /// The aggregate version (number of events applied).
    pub version: i64,
}

```

Keeping `Wallet` (rich, rule-enforcing, event-raising) and `WalletView` (flat, serializable, no behavior) as separate types is the same domain/persistence separation the previous chapter drew around the repository: the aggregate never serializes itself, and the wire shape never carries an invariant. The `version` field lets a client detect staleness under the eventual consistency CQRS introduces.

Proving the invariants

Because the aggregate is a plain struct with plain methods, you can exercise every rule with no database and no HTTP — the unit tests that ship in `samples/lumen/src/domain.rs`:

Rust

```
#[test]
fn open_validates_owner_and_balance() {
    assert_eq!(Wallet::open("w1", " ", Money::cents(100)).unwrap_err(),
DomainError::OwnerRequired);
    assert_eq!(Wallet::open("w1", "alice", Money::cents(-1)).unwrap_err(),
DomainError::NonPositiveAmount);
    let w = Wallet::open("w1", "alice", Money::ZERO).unwrap();
    assert!(w.opened);
    assert_eq!(w.balance, Money::ZERO);
}

#[test]
fn withdraw_rejects_overdraft() {
    let mut w = Wallet::open("w1", "alice", Money::cents(100)).unwrap();
    assert_eq!(w.withdraw(Money::cents(101)).unwrap_err(),
DomainError::InsufficientFunds);
    // The failed command raised no event beyond the open.
    assert_eq!(w.root.uncommitted().len(), 1);
}

#[test]
fn deposit_and_withdraw_update_balance_and_raise_events() {
    let mut w = Wallet::open("w1", "alice", Money::cents(100)).unwrap();
    w.deposit(Money::cents(50)).unwrap();
    assert_eq!(w.balance, Money::cents(150));
    w.withdraw(Money::cents(30)).unwrap();
    assert_eq!(w.balance, Money::cents(120));
}
```

The `withdraw_rejects_overdraft` test makes the consistency boundary concrete: after a rejected withdrawal, the aggregate's uncommitted-event buffer still holds exactly one event — the `WalletOpened` from the factory. The overdraft never produced a `MoneyWithdrawn`, so nothing partial can ever be persisted. That is the difference between a service-level guard (a convention) and an aggregate invariant (a physical constraint): the model exposes no mechanism to violate it.

What changed in Lumen

Lumen now has a domain core that owns its rules:

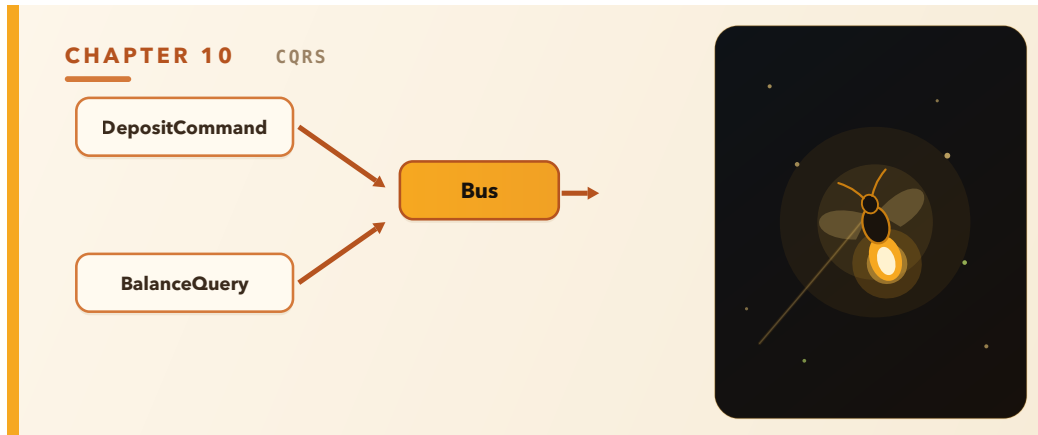
- `src/money.rs` — a `Money` value object: immutable, integer cents, `#[serde(transparent)]` so it rides the wire as a bare number, closed under `add` / `subtract` / `require_positive`, with a hand-written `MoneyError` (no `thiserror`).
- `src/domain.rs` — the `Wallet` aggregate carrying `#[derive(AggregateRoot)]` (which generates `AGGREGATE_TYPE` and the `aggregate()` accessors). `open` / `deposit` / `withdraw` enforce the three invariants — owner required, amounts positive, no overdraft — *before* raising an event, so a rejected command leaves the event buffer untouched.
- The typed `DomainError` family with stable `Display` strings that surface as RFC 9457 problem details, plus the `From<MoneyError>` bridge that turns an arithmetic overdraft into `InsufficientFunds`.
- `WalletView` — the flat read-model projection the aggregate hands out via `view()`, kept a separate type from the rule-enforcing aggregate.

The event payloads (`WalletOpened` / `MoneyDeposited` / `MoneyWithdrawn`) and the `rehydrate` / `apply` fold that reconstruct a wallet from its stream get their full treatment in [Event Sourcing](#). Here, the shape that matters is the DDD one: a value object you cannot corrupt and an aggregate you cannot bypass.

Exercises

1. **Break an invariant, watch it hold.** In a `#[cfg(test)]` block, open a wallet with `Money::cents(100)` and call `withdraw(Money::cents(200))`. Assert the error is `DomainError::InsufficientFunds` and that `w.root.uncommitted().len()` is still `1` — proving the rejected command raised no event.
2. **Add a `transfer_within` rule (domain only).** Write a free function `fn transfer(from: &mut Wallet, to: &mut Wallet, amount: Money) -> Result<(), DomainError>` that calls `from.withdraw(amount)?` then `to.deposit(amount)?`. Test that a transfer exceeding the source balance fails on the withdraw leg and leaves the *target* balance unchanged. (The real, persisted version becomes the saga in [Sagas](#).)
3. **Confirm the wire shape.** Serialize a freshly opened wallet's `view()` with `serde_json` and assert it equals `{"id":"wlt_1","owner":"alice","balance":250,"version":1}` — verifying that `Money`'s `#[serde(transparent)]` and `WalletView`'s field order produce the contract the read model and clients share.
4. **Justify the hand-written error.** In two sentences, explain why `MoneyError` and `DomainError` implement `Display` / `Error` by hand instead of deriving with `thiserror`, and what it would cost the book's one-dependency promise to add the crate.

Next, separate the write path from the read path with the command/query bus. See [CQRS](#).



CHAPTER 10

CQRS: Commands & Queries

Lumen's `Wallet` aggregate enforces its own rules, and the read model has a home. But the controller still needs a way to *deliver* an instruction to the write side and a *question* to the read side — and to do it without the two paths sharing a code path, so reads can be cached and writes can be validated independently.

CQRS — Command Query Responsibility Segregation — draws that bright line. Writes become **commands** (`OpenWallet`, `Deposit`, `Withdraw`); reads become a **query** (`GetWallet`). Each travels a typed `Bus`, matched to its handler by `std::any::TypeId`, through a middleware chain that validates commands and caches queries. This chapter wires Lumen's bus end to end, exactly as `samples/lumen` does.

By the end of this chapter, Lumen will have `src/commands.rs`: the `OpenWallet` / `Deposit` / `Withdraw` commands and the `GetWallet` query as `#[derive(Command)]` / `#[derive(Query)]` structs, free-`fn` handlers marked `#[command_handler]` / `#[query_handler]`, the bus wired with validation and query-cache middleware, and the read-after-write cache invalidation that keeps a balance from going stale after a deposit.

Design note. The `Bus` is Firefly's command/query dispatcher: it matches each message to its handler by `TypeId` and runs it through a middleware chain that validates commands and caches queries. Handlers are free `async fn`s a macro registers — `bus.send` / `bus.query` dispatch to them. The whole path is ordinary Rust: no proxies, no reflection, just a typed registry and a function call.

Commands, queries, and the `Message` trait

Every command and query implements `Message`. Hand-writing it is one line, but the trait's optional methods — `validate`, `cache_ttl` — are overridable defaults that the matching middleware picks up automatically:

Rust

```
pub trait Message: Clone + Serialize + Send + Sync + 'static {
    fn validate(&self) -> Result<(), CqrsError> { Ok(()) } //
    ValidationMiddleware
    fn cache_ttl(&self) -> Option<Duration> { None } // QueryCache
}
```

`Clone` stands in for pass-by-value handler invocation; `Serialize` seeds the query cache key. Lumen never writes that impl by hand — it derives it.

Lumen's commands and query

The four messages are plain structs carrying `#[derive(Command)]` / `#[derive(Query)]`, which generate the `Message` impl. The `#[firefly(validate)]` field attribute makes a field required (the generated `validate()` rejects an empty `String` or a non-positive number), and `#[firefly(cache_ttl = "...")]` is reflected on the query's generated `cache_ttl`:

Rust

```
// samples/lumen/src/commands.rs
use firefly::prelude::*;
use serde::{Deserialize, Serialize};

/// `POST /api/v1/wallets` command – open a new wallet.
#[derive(Debug, Clone, Default, Serialize, Deserialize, Command)]
#[serde(default)]
pub struct OpenWallet {
    /// The wallet owner's display name – required.
    #[firefly(validate)]
    pub owner: String,
    /// The opening balance, in minor units (cents); must be `>= 0`.
    #[serde(rename = "openingBalance")]
    pub opening_balance: i64,
}

/// `POST /api/v1/wallets/:id/deposit` command – credit a wallet.
#[derive(Debug, Clone, Default, Serialize, Deserialize, Command)]
#[serde(default)]
pub struct Deposit {
    /// The wallet to credit – required.
    #[firefly(validate)]
    #[serde(rename = "walletId")]
    pub wallet_id: String,
    /// The amount to credit, in minor units (cents); must be `> 0`.
    #[firefly(validate)]
    pub amount: i64,
}

/// `POST /api/v1/wallets/:id/withdraw` command – debit a wallet.
#[derive(Debug, Clone, Default, Serialize, Deserialize, Command)]
#[serde(default)]
pub struct Withdraw {
    #[firefly(validate)]
    #[serde(rename = "walletId")]
    pub wallet_id: String,
    #[firefly(validate)]
    pub amount: i64,
}
```

```

}

/// `GET /api/v1/wallets/:id` query – cached for 30 seconds.
#[derive(Debug, Clone, Default, Serialize, Deserialize, Query)]
#[firefly(cache_ttl = "30s")]
pub struct GetWallet {
    /// The wallet id to fetch.
    pub id: String,
}

```

A few choices echo the domain chapter. The commands carry `i64` cents, not a `Money` — the handler constructs the value object, keeping the wire contract a bare number and the validation simple. `#[firefly(validate)]` on `amount` rejects a zero or negative amount *before* the handler runs, so the aggregate is never even called with structurally wrong data. And `#[serde(rename = ...)]` keeps the JSON camelCase (`openingBalance`, `walletId`) while the Rust fields stay `snake_case`.

NOTE

`#[firefly(validate)]` makes a field required — the generated `validate()` rejects an empty `String` or a non-positive number before the handler runs — and the check is generated by the derive macro at compile time, not reflected at runtime.

`#[firefly(cache_ttl = "30s")]` sets the query's cache TTL, which the `QueryCache` middleware picks up off the message.

The free-`fn` handlers

A handler is a free `async fn` marked `#[command_handler]` or `#[query_handler]`. The macro reads the argument type as the dispatch key and generates a `register_<fn>(bus)` helper that installs it. The handler turns the bus's `CqrsError` channel into the precise outcome; the web layer (next) restores the HTTP status:

Rust

```

use crate::domain::{DomainError, Wallet, WalletView};
use crate::ledger::{Ledger, ReadModel};
use crate::money::Money;

/// Handles `OpenWallet`. `#[command_handler]` generates
`register_open_wallet(bus)`.
#[command_handler]
pub async fn open_wallet(cmd: OpenWallet) -> Result<WalletView, CqrsError> {
    if cmd.opening_balance < 0 {
        return Err(CqrsError::validation("openingBalance must be >= 0"));
    }
    state()
        .ledger
        .open(&cmd.owner, Money::cents(cmd.opening_balance))
        .await
        .map_err(to_cqrs)
}

/// Handles `Deposit`. Generates `register_deposit(bus)`.
#[command_handler]
pub async fn deposit(cmd: Deposit) -> Result<WalletView, CqrsError> {
    state()
        .ledger
        .deposit(&cmd.wallet_id, Money::cents(cmd.amount))
        .await
        .map_err(to_cqrs)
}

/// Handles `Withdraw`. Generates `register_withdraw(bus)`.
#[command_handler]
pub async fn withdraw(cmd: Withdraw) -> Result<WalletView, CqrsError> {
    state()
        .ledger
        .withdraw(&cmd.wallet_id, Money::cents(cmd.amount))
        .await
        .map_err(to_cqrs)
}

```

```

/// Handles `GetWallet`. Generates `register_get_wallet(bus)`.
#[query_handler]
pub async fn get_wallet(q: GetWallet) -> Result<WalletView, CqrsError> {
    if let Some(view) = state().read_model.find(&q.id) {
        return Ok(view);
    }
    let events = state().ledger.load_events(&q.id).await.map_err(to_cqrs)?;
    Ok(Wallet::rehydrate(&q.id, &events).view())
}

```

Each command handler constructs the `Money` value object from the command's `i64`, delegates to the `Ledger` application service (which rehydrates the aggregate, runs the domain command, and persists — see [Event Sourcing](#)), and maps a `DomainError` onto the bus's `CqrsError` channel:

Rust

```

fn to_cqrs(e: DomainError) -> CqrsError {
    CqrsError::handler(e.to_string())
}

```

The `get_wallet` query is the read-after-write pattern in miniature: it serves from the projected `ReadModel` first, and *only* if the projection has not yet caught up does it fall back to folding the event stream. That fallback is what keeps a read immediately after a write from returning a stale balance under the eventual consistency the projection introduces.

Why free functions, not handler structs? Rust free functions cannot capture state, so the resolved collaborators live in a module-local static and the handler reads them through `state()`. That is the next section — and it is the one piece of CQRS wiring Lumen does by hand.

Publishing collaborators: the `OnceLock` pattern

Because a `#[command_handler]` free function can't own a `Ledger` or a `ReadModel`, Lumen publishes the resolved collaborators once at startup into a process-global `OnceLock`, and the handlers reach them through `state()`:

Rust

```
use std::sync::{Arc, OnceLock};

/// The collaborators the CQRS handlers need.
struct HandlerState {
    ledger: Ledger,
    read_model: Arc<ReadModel>,
}

static STATE: OnceLock<HandlerState> = OnceLock::new();

/// Publishes the handlers' collaborators and returns the effective state —
/// the one actually held by the process-global OnceLock.
pub fn bind(ledger: Ledger, read_model: Arc<ReadModel>) -> (Ledger,
Arc<ReadModel>) {
    let effective = STATE.get_or_init(|| HandlerState { ledger, read_model });
    (effective.ledger.clone(), Arc::clone(&effective.read_model))
}

fn state() -> &'static HandlerState {
    STATE.get().expect("commands::bind must run before a handler dispatches")
}
```

The subtle part is the return value. The global binds only **once**, so a second `build_app()` in the same test binary keeps the *first* ledger and read model. `bind` returns the *effective* pair — whichever the `OnceLock` actually holds — so the composition root can wire the controller's saga, the event projection, and the free-fn

handlers all against the **same** collaborators, no matter how many times the app is built. (This is why the HTTP test suite can call `build_router()` per test and still share one ledger.)

NOTE

Because a `#[command_handler]` free function can't capture state, Lumen publishes its resolved collaborators once at startup with a one-line `bind` and the handlers reach them through `state()` — the standard pattern for free-fn handlers in Firefly. If you prefer constructor injection, declare the handler as a `#[derive(Component)]` bean instead (see [Dependency Wiring](#)); Lumen keeps the explicit `bind` for teachability.

Wiring the bus

The composition root in `src/web.rs` builds the bus, layers the middleware, binds the collaborators, and installs the handlers via the generated `register_*` helpers (gathered behind one `register(&bus)` fn):

Rust

```
// samples/lumen/src/commands.rs
/// Installs every generated handler-registration helper on `bus`.
pub fn register(bus: &Bus) {
    register_open_wallet(bus);
    register_deposit(bus);
    register_withdraw(bus);
    register_get_wallet(bus);
}
```


Rust

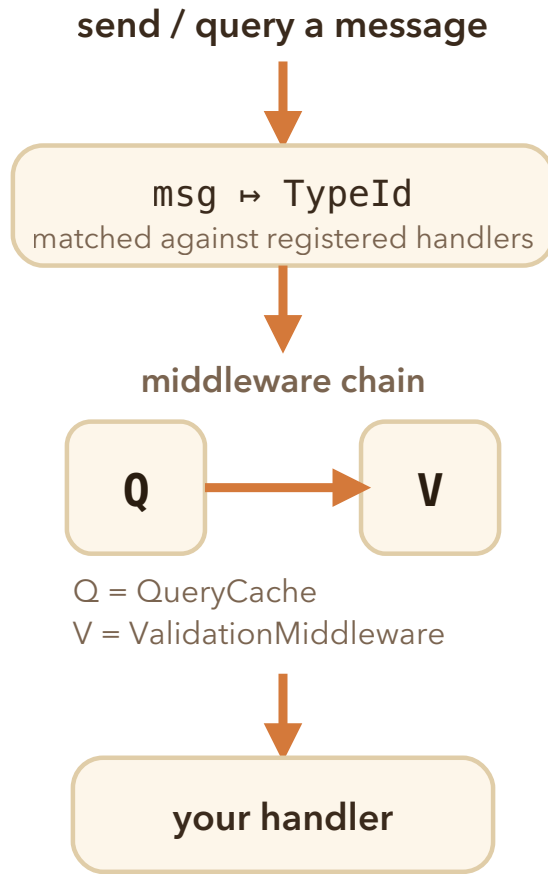
```
// samples/lumen/src/web.rs - build_app(), the relevant slice.
let bus = Arc::clone(&web.bus);
let read_model = Arc::new(ReadModel::default());

// Read-side caching on the bus (honours GetWallet's 30s cache_ttl).
let query_cache = QueryCache::new();
bus.use_middleware(query_cache.middleware());
// Validation middleware enforces the `#[firefly(validate)]` checks.
bus.use_middleware(firefly::cqrs::ValidationMiddleware::new());

// Publish the handler collaborators, then install the generated registrations.
let (ledger, read_model) = crate::commands::bind(
    Ledger::new(Arc::clone(&store), Arc::clone(&broker)),
    read_model,
);
crate::commands::register(&bus);
```

Middleware runs first-registered = outermost. Two ship in this chain (a third, authorization, arrives with [Security](#)):

Middleware	Behaviour
<code>QueryCache::middleware()</code>	memoises results for messages whose <code>cache_ttl</code> is <code>Some</code>
<code>ValidationMiddleware</code>	calls <code>Message::validate</code> before dispatch, short-circuits on error



A message is matched to its handler by `TypeId`, then runs the registered middleware chain (here `QueryCache` then `ValidationMiddleware`) before the handler executes.

Command/query segregation

The bus dispatches commands and queries through one registry keyed by `TypeId`, but it does not treat them as interchangeable: each registered handler carries the **kind** of the message it serves. That kind is a property of the message type, exposed as

```
Message::kind() -> MessageKind:
```

Rust

```
pub enum MessageKind { Command, Query }
```

The default is `MessageKind::Command`. `#[derive(Command)]` keeps that default; `#[derive(Query)]` overrides `kind()` to return `MessageKind::Query`. Nothing in Lumen's `src/commands.rs` changes — `OpenWallet` / `Deposit` / `Withdraw` are already commands and `GetWallet` is already a query, so the segregation falls out of the derives the chapter introduced. The bus records each message's kind at registration time and lets you ask about the two halves separately:

Rust

```
use firefly::cqrs::{Bus, MessageKind};

let bus = Bus::new();
crate::commands::register(&bus); // installs the three commands + one query

// Inspect the registry, split by CQRS kind.
let commands = bus.command_handler_names(); // ["...:Deposit",
"...:OpenWallet", "...:Withdraw"]
let queries = bus.query_handler_names(); // ["...:GetWallet"]
assert_eq!(bus.handler_count(), 4);

// The general form both of the above delegate to:
assert_eq!(bus.handler_names_by_kind(MessageKind::Query), queries);

// Type-level membership and removal.
assert!(bus.has_handler::<GetWallet>());
assert!(bus.unregister::<GetWallet>()); // true — one was present
assert!(!bus.has_handler::<GetWallet>());
```

`command_handler_names()` and `query_handler_names()` are thin wrappers over `handler_names_by_kind(MessageKind)`, each returning the fully-qualified type names sorted alphabetically — the same list `handler_names()` returns, but filtered to one

kind. `handler_count()` is the total registry size; `has_handler::<C>()` tests membership for a message type; and `unregister::<C>()` removes a handler, returning whether one was present (useful when a test wants to swap a handler without rebuilding the bus).

This is exactly what the admin `/cqrs` view consumes: because the bus now knows each handler's kind, the dashboard tags every registration with a badge (commands blue, queries green) and shows separate command/query counts, rather than one undifferentiated handler list.

NOTE

Firefly keeps a single `Bus` and recovers the command/query split from each message's `kind()` (set by the `Command` / `Query` derive), rather than from two distinct buses. `command_handler_names()` / `query_handler_names()` are the filtered views the admin `/cqrs` dashboard renders; `has_handler::<C>()` / `unregister::<C>()` test membership and remove a handler by type.

Correlation propagation

A command rarely acts alone. `bus.send(Deposit { .. })` runs a handler that may start the transfer saga (`Sagas`) or `tokio::spawn` a follow-up task — and each of those leaves the original request task. For the logs and traces to read as *one* operation, they must all share a single correlation id.

`firefly::cqrs::CorrelationMiddleware` enforces that at the dispatch boundary. Add it to the bus like any other middleware:

Rust

```
use firefly::cQRS::{Bus, CorrelationMiddleware};

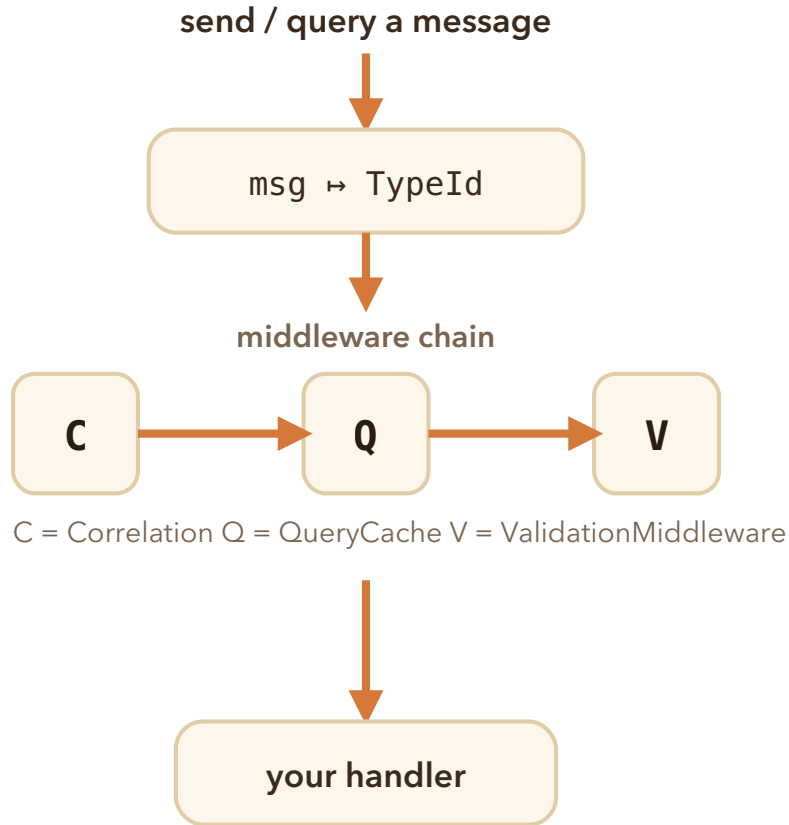
let bus = Bus::new();
bus.use_middleware(CorrelationMiddleware::new()); // outermost – runs first
```

On each dispatch the middleware **ensures-or-generates** a correlation id: if the request is already running under one — the **firefly-web** correlation layer sets a task-local id per HTTP request — it reuses that id, so the command and the saga/spawned task it triggers all trace to the same value. If no ambient id is present (a background job, a test, an internal dispatch), it generates a fresh one for the span of that dispatch and restores the prior scope on the way out, so sibling operations never leak ids into one another.

Rust

```
// Inside a handler (or anything it calls), the id is observable:
let trace = firefly_kernel::correlation_id(); // Some(<id>) under the
middleware
```

Because Lumen wires the bus in **build_app()**, adding **CorrelationMiddleware** there means the same id that the HTTP layer stamped on **POST /wallets/:id/deposit** flows into the **Deposit** handler, into the transfer saga it may start, and into the events the saga publishes — without any handler touching the id explicitly. Register it first so it sits outermost in the chain shown earlier — the correlation scope must already be open before **QueryCache** and **ValidationMiddleware** run, so anything they log carries the id too:



Registering `CorrelationMiddleware` first puts it outermost: the correlation scope opens before `QueryCache` and `ValidationMiddleware` run, so everything they log carries the id.

Design note. `CorrelationMiddleware` ensures one logical request keeps one correlation id across the command boundary and any saga or `tokio::spawn` ed continuation it triggers: it reuses an ambient id when present (the web layer sets one per HTTP request) and generates one otherwise, restoring the prior scope on the way out. Firefly threads the id through a task-local that this middleware scopes per dispatch, so a handler never has to pass it by hand.

Dispatching from the controller

The `#[rest_controller]` (built in [First HTTP API](#)) holds the `Bus` and dispatches through `send` / `query`. `Bus::query` is a readability synonym for `send`. A failed dispatch is a `CqrsError`, which the web layer maps to the right RFC 9457 status:

Rust

```
// samples/lumen/src/web.rs - WalletApi handlers.
#[post("/wallets")]
async fn open(
    State(api): State<WalletApi>,
    Json(body): Json<OpenWallet>,
) -> WebResult<(axum::http::StatusCode, Json<WalletView>)> {
    let view: WalletView = api.bus.send(body).await.map_err(cqrs_to_web)?;
    Ok((axum::http::StatusCode::CREATED, Json(view)))
}

#[get("/wallets/:id")]
async fn get(
    State(api): State<WalletApi>,
    Path(id): Path<String>,
) -> WebResult<Json<WalletView>> {
    let view: WalletView = api.bus.query(GetWallet {
        id },).await.map_err(cqrs_to_web)?;
    Ok(Json(view))
}
```

`cqrs_to_web` is the seam where a domain failure becomes an HTTP status. It reads the `CqrsError` and its detail string — which, recall, is the `DomainError`'s stable `Display` text from the previous chapter — and chooses the status:

Rust

```

fn cQRS_to_web(err: CQRSError) -> WebError {
    match err {
        CQRSError::Validation(detail) =>
WebError::from(FireflyError::validation(detail)),
        CQRSError::Handler(detail) => {
            if detail.ends_with("not found") {
                WebError::from(FireflyError::not_found(detail)) //
404
            } else if detail == DomainError::InsufficientFunds.to_string()
                || detail == DomainError::NonPositiveAmount.to_string()
                || detail == DomainError::OwnerRequired.to_string()
            {
                WebError::from(FireflyError::validation(detail)) //
422
            } else {
                WebError::from(FireflyError::not_found(detail))
            }
        }
        other => WebError::from(FireflyError::internal(other.to_string())), //
500
    }
}

```

This is why the domain chapter insisted the `Display` strings be *stable*: they are the contract `cQRS_to_web` matches on to recover the precise status.

Read-after-write: invalidating the cache

`GetWallet` is cached for 30 seconds. Without care, a deposit would update the balance while a cached `GetWallet` kept serving the old one for up to 30 seconds. Lumen closes that gap by invalidating the cached query family after every mutation:

Rust

```
// samples/lumen/src/web.rs - deposit handler.
#[post("/wallets/:id/deposit")]
async fn deposit(
    State(api): State<WalletApi>,
    Path(id): Path<String>,
    Json(body): Json<AmountBody>,
) -> WebResult<Json<WalletView>> {
    let cmd = Deposit { wallet_id: id, amount: body.amount };
    let view: WalletView = api.bus.send(cmd).await.map_err(cqrs_to_web)?;
    api.query_cache.invalidate_type::<GetWallet>(); // read-after-write
    Ok(Json(view))
}
```

This is where `WalletApi` finally grows the second field [First HTTP API](#) deferred: alongside `bus`, the controller now holds the `QueryCache` handle built when the bus was wired, so `api.query_cache` has a receiver. The same `QueryCache` instance is registered as middleware *and* stored on the controller — one cache, read by the bus and invalidated by the handler.

`QueryCache::invalidate_type::<GetWallet>()` evicts every cached result for exactly that query type. The withdraw handler does the same, and the transfer saga ([Sagas](#)) — which touches two wallets — invalidates the whole `GetWallet` family. The query cache's backend swap (Redis / Postgres) and event-driven invalidation get their own treatment in [Caching](#); here, the point is that the *bus* is where read-after-write consistency lives, not the handler.

The reactive bus

The bus also exposes a reactive surface that wraps the eventual result in a lazy `Mono<R>` — the same handler lookup, the same middleware chain, run only when the `Mono` is subscribed, blocked, or awaited. The methods take `&Arc<Bus>` so the `Mono` can own the bus:

Method	Returns
<code>Bus::send_mono(cmd)</code>	<code>Mono<R></code>
<code>Bus::query_mono(q)</code>	<code>Mono<R></code>
<code>Bus::send_mono_with_context</code>	<code>Mono<R></code>
<code>Bus::query_mono_with_context</code>	<code>Mono<R></code>

The `*_with_context` variants carry an explicit `ExecutionContext` into the dispatch — the correlation id, tenant, and authenticated principal — for when a `Mono` is composed outside the task-local scope the HTTP layer establishes (a background job or a reactive pipeline assembled before the request context is in play). The plain `send_mono` / `query_mono` inherit whatever context is ambient at subscribe time.

A reactive `GetWallet`, composing on the `Mono` from [The Reactive Model](#):

Rust

```
use std::sync::Arc;
use firefly::cQRS::Bus;

let balance = bus
    .query_mono::<_, WalletView>(GetWallet { id: wallet_id })
    .map(|view| view.balance)
    .block()
    .await?;           // Ok(Some(<cents>))
```

Because `firefly-reactive` fixes its error channel to `FireflyError`, a failed dispatch is mapped from `CqrsError` into a status-faithful `FireflyError` (validation → 422, missing handler → 500), with the original `CqrsError` preserved as `source()`. So a reactive command flows straight into the RFC 9457 problem stack while staying inspectable.

Proving the bus

Lumen's `src/commands.rs` exercises the full dispatch path with no HTTP — the test that ships in the crate:

Rust

```
#[tokio::test]
async fn handlers_dispatch_through_the_bus() {
    let ledger = Ledger::new(Arc::new(MemoryEventStore::new()),
Arc::new(InMemoryBroker::new()));
    bind(ledger, Arc::new(ReadModel::default()));
    let bus = Bus::new();
    bus.use_middleware(firefly::cqrs::ValidationMiddleware::new());
    register(&bus);

    let opened: WalletView = bus.send(OpenWallet { owner: "alice".into(),
opening_balance: 100 }).await.unwrap();
    assert_eq!(opened.balance, 100);

    let after: WalletView = bus.send(Deposit { wallet_id: opened.id.clone(),
amount: 50 }).await.unwrap();
    assert_eq!(after.balance, 150);

    let fetched: WalletView = bus.query(GetWallet { id:
opened.id.clone() }).await.unwrap();
    assert_eq!(fetched.id, opened.id);
}
```

And the validation derive is testable on its own — no bus needed, because

`#[derive(Command)]` generates `validate()` directly on the type:

Rust

```
#[test]
fn deposit_validates_required_fields() {
    assert!(Deposit::default().validate().is_err());
    assert!(
        Deposit { wallet_id: "wlt_1".into(), amount: 0 }.validate().is_err(),
        "zero amount fails the #[firefly(validate)] check"
    );
    assert!(Deposit { wallet_id: "wlt_1".into(), amount:
10 }.validate().is_ok());
}

#[test]
fn get_wallet_carries_cache_ttl() {
    assert!(GetWallet::default().cache_ttl().is_some());
}
```

What changed in Lumen

Lumen's read and write paths are now separate, typed, and bus-dispatched:

- `src/commands.rs` — `OpenWallet` / `Deposit` / `Withdraw` carry `#[derive(Command)]` with `#[firefly(validate)]` on required fields; `GetWallet` carries `#[derive(Query)]` with `#[firefly(cache_ttl = "30s")]`. The derives generate the `Message` impl, the `validate()` checks, and the query's `cache_ttl`.
- Free-`fn` handlers marked `#[command_handler]` / `#[query_handler]` generate `register_open_wallet` / `register_deposit` / `register_withdraw` / `register_get_wallet`. Command handlers build the `Money` value object and delegate to the `Ledger`; the query serves the `ReadModel` and falls back to folding the stream for read-after-write freshness.

- The `OnceLock` bind pattern publishes the resolved `Ledger` + `ReadModel` once and returns the *effective* pair, so every collaborator across repeated builds shares one ledger and one read model.
- The bus is wired with `QueryCache::middleware()` and `ValidationMiddleware`, dispatched from the `#[rest_controller]` via `bus.send` / `bus.query`, with `cqrs_to_web` mapping a `CqrsError` (carrying the domain `Display` string) to the right RFC 9457 status — 422 for business rules, 404 for not-found.
- Read-after-write is enforced at the bus boundary:
`query_cache.invalidate_type::<GetWallet>()` runs after every mutation.

Exercises

1. Watch validation short-circuit. In a test, build a `Bus`, add `ValidationMiddleware::new()`, `register` the handlers, and `bus.send` a `Deposit { wallet_id: "wlt_1".into(), amount: 0 }`. Assert the result is a `CqrsError::Validation` and that the ledger was never touched (open a wallet first, then deposit zero, and confirm its balance is unchanged).
2. Prove the cache, then bust it. With the full `build_app()` bus, `query(GetWallet { id })` twice and confirm the second is served from cache (instrument the `ReadModel::find` or trace a counter). Deposit into the wallet, then `query` again — assert the new balance comes back, proving `invalidate_type::<GetWallet>()` did its job.
3. Add a `CloseWallet` command. Define `CloseWallet { #[firefly(validate)] wallet_id: String }` with `#[derive(Command)]`, write a `#[command_handler]` `close_wallet` that returns a `WalletView`, add `register_close_wallet(bus)` to `register`, and dispatch it. (You do not need a domain `close` yet — returning the current view is enough to exercise the wiring.)

4. **Reactive compose.** Rewrite the `get` controller handler to use `bus.query_mono::<_, WalletView>(GetWallet { id }).map(|v| v.balance)` and return just the balance as JSON. Note where the `FireflyError` channel takes over from `CqrsError`.

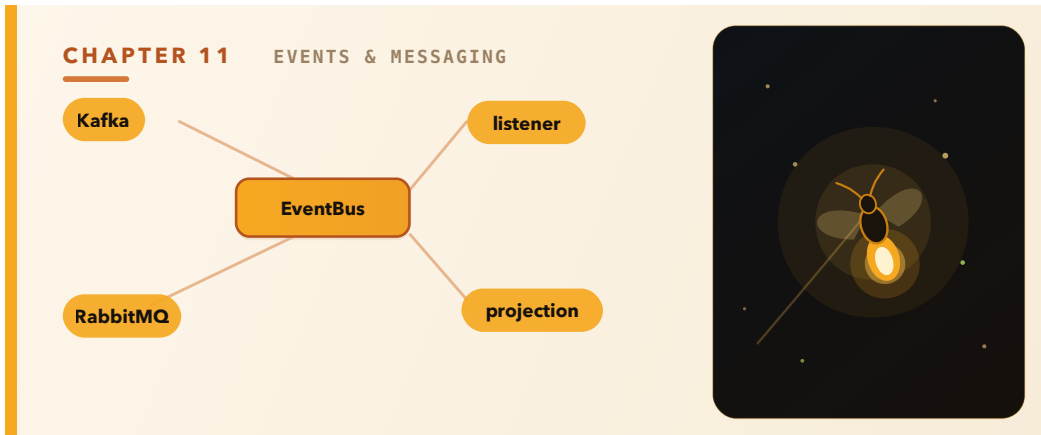
The bus dispatches *within* the service. To propagate what happened *between* collaborators — the read-model projection, external subscribers — fan out domain events. Continue to [Event-Driven Architecture & Messaging](#).



PART III

Event-Driven Architecture



**CHAPTER 11**

Event-Driven Architecture & Messaging

By the end of the [CQRS chapter](#), Lumen could open a wallet, deposit, withdraw, and read a balance — but the command side and the query side were quietly cheating. The `Wallet` aggregate raised crisp domain events (`WalletOpened` , `MoneyDeposited` , `MoneyWithdrawn`), the `Ledger` persisted them, and then nothing carried them anywhere. The read model the `GetWallet` query serves had to be repaired on the fly by re-folding the event stream.

By the end of *this* chapter, Lumen closes the loop. Every event the ledger persists is also **published** to a `Broker` , and a read-model **projection** — declared with one `#[event_listener]` attribute — consumes those events and keeps the query side current without the write side knowing it exists. That is event-driven architecture: a fact is published once, and any number of independent reactions subscribe to it. The audit trail, the welcome notification, the balance read model — each becomes a subscriber you can add months later without touching a single command handler.

`firefly-eda` is the framework's **event-driven architecture port**. It defines the **Event** envelope every Firefly event flows through, the **Publisher** / **Subscriber** / **Broker** ports, an in-process **InMemoryBroker**, and the messaging machinery — glob topics, consumer groups, retry/DLQ, event filters, and a reactive **Flux** subscription surface. The production transports (Kafka, RabbitMQ, Postgres outbox, Redis Streams) implement the same ports and slot in at wiring time, so Lumen's projection never changes when the broker does.

***Design note.** The **Broker** is Firefly's transport-agnostic messaging port: you publish an **Event** to it and subscribe handlers with `#[event_listener(topic = ...)]`. `wrap_listener` adds retry and dead-lettering; subscriptions accept glob topic patterns. Production transports (Kafka, RabbitMQ, Postgres outbox, Redis Streams) implement the same port and slot in at wiring time, so a handler never changes when the broker does.*

Two kinds of "event" in one wallet

Before wiring anything, it is worth being precise about the word *event*, because Lumen ends up using it for two different things and confusing them leads to the wrong port.

A **domain event** in the event-sourcing sense — `firefly::eventsourcing`'s **DomainEvent** — is the durable, versioned fact the **Wallet** aggregate raises and the [next chapter](#) makes the source of truth. It lives in the event *store*.

A **messaging event** — `firefly::eda`'s **Event** — is the wire envelope that carries a fact *to subscribers*. It lives on the *broker*. Lumen bridges the two with one function (`to_envelope`, below): the ledger persists a **DomainEvent**, then maps it onto an **Event** and publishes it. This chapter is about the second kind — getting the fact onto the wire and reacting to it. The first kind is the next chapter's subject.

The **Event** envelope

Event is Firefly's canonical wire envelope, with a stable JSON shape (fixed field names and omission rules) so producers and consumers agree on the bytes regardless of broker or service — any system that honours the contract interoperates. Construct one with **Event::new**, which also stamps **correlation_id** from the kernel's task-local correlation scope:

Rust

```
use firefly_eda::Event;

let ev = Event::new(
    "orders.created", // topic
    "OrderCreated",   // event type
    "orders-svc",      // source
    Some(br#"{"id":"o1"}"#.to_vec()), // payload (base64 on the wire)
)
.with_header("x-tenant", "acme")
.with_key(b"customer-42".to_vec()); // partition / routing key
```

The **key** carries the intended partition/routing key per the **Event** contract; it is omitted from the wire when absent, so older events stay byte-for-byte identical. Note that the current adapters do not key off it yet: the Kafka adapter derives its record key from **correlation_id** (falling back to the event id), and the RabbitMQ adapter routes on the topic. Treating **key** as the partition/routing key is design intent rather than a guarantee of today's adapters.

Lumen's domain-event-to-envelope bridge

Lumen never builds an `Event` by hand in a handler. The ledger owns one mapping function that turns a persisted `DomainEvent` into the canonical envelope, carrying the JSON-encoded domain event as the payload and the wallet id as the intended partition key — the `Event` contract's design for keeping per-wallet events ordered, once the adapters key off it:

Rust

```
use firefly::eda::Event;
use firefly::eventsourcing::DomainEvent;

/// The EDA topic every wallet domain event is published to.
pub const EVENTS_TOPIC: &str = "wallets.events";
/// The logical EDA source stamped on published events.
pub const EVENT_SOURCE: &str = "lumen";

/// Maps a persisted `DomainEvent` onto the canonical EDA `Event` envelope.
pub fn to_envelope(event: &DomainEvent) -> Event {
    let payload = serde_json::to_vec(event).expect("domain event serialises");
    Event::new(
        EVENTS_TOPIC,
        event.event_type.clone(),
        EVENT_SOURCE,
        Some(payload),
    )
    .with_key(event.aggregate_id.clone().into_bytes())
    .with_header("aggregateType", "Wallet")
    .with_header("aggregateId", event.aggregate_id.clone())
    .with_header("version", event.version.to_string())
}
```

Three design choices repay attention. The `topic` (`wallets.events`) is a shared constant — the publisher and the projection key off the same value, so the channel name can never drift. The `key` is the wallet id — the intended partition key so that, once a broker keys off it, every event for one wallet lands on the same partition and

stays in order. (Today's Kafka adapter keys records on `correlation_id` and the RabbitMQ adapter routes on the topic, so this is the `Event` contract's design intent rather than a current guarantee.) The **headers** (`aggregateId` , `version`) carry just enough routing metadata for a subscriber to find and re-fold the affected aggregate without decoding the payload — which is exactly what Lumen's projection does below.

Publishing from the ledger

The `Ledger` is the single write path every command and the transfer saga call. After it appends an aggregate's uncommitted events to the store with optimistic concurrency, it publishes each one — `to_envelope` then `broker.publish` — so the projection downstream can react:

Rust

```

use std::sync::Arc;
use firefly::eda::Broker;
use firefly::eventsourcing::{EventSourcingError, EventStore};

use crate::domain::{DomainError, Wallet};

/// Appends the aggregate's uncommitted events at `expected_version`
/// (optimistic concurrency) then publishes each to the EDA broker.
async fn commit(&self, wallet: &mut Wallet, expected: i64) -> Result<(),
DomainError> {
    let events = wallet.take_uncommitted();
    if events.is_empty() {
        return Ok(());
    }
    self.store
        .append(&wallet.root.id, expected, events.clone())
        .await
        .map_err(|e| match e {
            EventSourcingError::Concurrency => {
                DomainError::NotFound(format!("{}: concurrent modification",
wallet.root.id))
            }
            other => DomainError::NotFound(format!("{}: {other}",
wallet.root.id)),
        })?;
    for event in &events {
        self.broker
            .publish(to_envelope(event))
            .await
            .map_err(|e| DomainError::NotFound(format!("publish failed:
{e}")))?;
    }
    Ok(())
}

```

Notice the ordering: **append before publish**. A subscriber must never see a fact that did not persist. If the append fails (including the optimistic-concurrency race) the loop is never reached, so no event is broadcast. The store backing this `Ledger` is the in-memory `MemoryEventStore`; the [next chapter](#) is where that store earns the name *event-sourced*.

NOTE

Append before publish: a subscriber must never see a fact that did not persist. The gap between append and publish — where a crash could persist a fact but drop the broadcast — is what the transactional outbox in the [next chapter](#) eliminates.

The in-process broker

`InMemoryBroker` is the default — fan-out delivery, glob topic matching, and per-`(topic, group)` round-robin, with no external dependency. It is the broker Lumen's `WebStack` exposes as `web.broker`, and it is everything the teaching build (and the test suite) needs. Subscribe a handler, publish an event:

Rust

```

use firefly_edo::{handler, Event, InMemoryBroker};

#[tokio::main]
async fn main() {
    let broker = InMemoryBroker::new();

    broker
        .subscribe(
            "wallets.events",
            handler(|ev: Event| async move {
                println!("observed {} for {}", ev.event_type,
                    ev.headers.get("aggregateId").map(String::as_str).unwrap_or("?"));
                Ok(())
            }),
        )
        .unwrap();

    let ev = Event::new("wallets.events", "WalletOpened", "lumen",
        Some(br#"{"wallet_id":"wlt_1"}"#.to_vec()));
    broker.publish(ev).await.unwrap();
    broker.close().unwrap();
}

```

`InMemoryBroker::publish` awaits each subscribed handler sequentially on the publisher's task; the first handler error short-circuits and is returned to the publisher. After `close()`, publish and subscribe fail with `EdaError::Closed`.

The read-model projection – `#[event_listener]`

Here is where Lumen closes the CQRS loop. The **projection** is a free `async fn` carrying one attribute. The `#[event_listener(topic = "wallets.events")]` macro generates a `subscribe_project_wallet_event(broker)` helper that subscribes the function to the topic; for each delivered event it reloads the affected wallet's stream, folds it into a `WalletView`, and upserts it into the read model:

Rust

```
use firefly::eda::Event;
use firefly::prelude::*;

use crate::domain::Wallet;

/// The read-model projection. `#[event_listener]` generates
/// `subscribe_project_wallet_event(broker)`, which subscribes this fn to
/// `EVENTS_TOPIC`. For each event it reloads the wallet's stream, folds it into
/// a `WalletView`, and upserts it – the idempotent rebuild-from-stream pattern.
#[event_listener(topic = "wallets.events")]
pub async fn project_wallet_event(ev: Event) -> FireflyResult<()> {
    let Some(state) = projection_state() else {
        return Ok(());
    };
    let Some(wallet_id) = ev.headers.get("aggregateId") else {
        return Ok(());
    };
    // A transient store miss is swallowed so one poison message never stalls
    // the projection – the EDA at-least-once contract.
    if let Ok(events) = state.store.load(wallet_id).await {
        let view = Wallet::rehydrate(wallet_id, &events).view();
        state.read_model.upsert(view);
    }
    Ok(())
}
```

Two properties make this a *good* projection rather than just a working one.

It is **idempotent**. Rather than mutating the read-model row from the single delivered event (`balance += amount`), it reloads the wallet's full stream and rebuilds the view from scratch. Under EDA's at-least-once delivery a redelivered `MoneyDeposited` would double-count if you applied the delta — but re-folding the same stream converges on the same `WalletView` no matter how many times the event arrives. The header carries the `aggregateId` ; that is all the projection needs to find the stream.

It is **decoupled**. `project_wallet_event` imports no command, calls no handler, and has no idea a deposit was processed. It reacts purely to the published fact. You can add a `FraudDetector` or a `WelcomeNotifier` subscriber next to it without touching a line of the command path.

NOTE

`#[event_listener(topic = "wallets.events")]` generates `subscribe_project_wallet_event(broker)` : the framework wires the subscription for you; you write only the reaction.

Wiring state into a free-fn listener

A Rust free fn cannot capture wiring state, so Lumen uses the same publish-collaborators-once pattern its CQRS handlers use: the resolved collaborators are placed in a process-global `OnceLock` at startup, and the listener reads them back through a small accessor. (When you would rather have the collaborators injected, the [dependency-injection chapter](#)'s `#[derive(Component)]` listener is the constructor-injection alternative.)

Rust

```

use std::sync::{Arc, OnceLock};
use firefly::eventsourcing::EventStore;

use crate::ledger::ReadModel;

/// The collaborators the free-fn projection needs.
struct ProjectionState {
    store: Arc<dyn EventStore>,
    read_model: Arc<ReadModel>,
}

static PROJECTION: OnceLock<ProjectionState> = OnceLock::new();

/// Publishes the projection's collaborators and returns the *effective* state
/// (the first call wins, so repeated builds in one test binary share one
state).
pub fn bind_projection(
    store: Arc<dyn EventStore>,
    read_model: Arc<ReadModel>,
) -> (Arc<dyn EventStore>, Arc<ReadModel>) {
    let effective = PROJECTION.get_or_init(|| ProjectionState { store,
read_model });
    (
        Arc::clone(&effective.store),
        Arc::clone(&effective.read_model),
    )
}

fn projection_state() -> Option<&'static ProjectionState> {
    PROJECTION.get()
}

```

The composition root ties it all together: it binds the projection's collaborators to the *same* store and read model the command handlers use, then awaits the generated subscription so the events the handlers publish are exactly the events the projection consumes:

Rust

```
// in build_app() - crate::web
ledger::bind_projection(Arc::clone(ledger.store()), Arc::clone(&read_model));
crate::commands::register(&bus);
// Subscribe the projection to the effective ledger's broker.
ledger::subscribe_project_wallet_event(ledger.broker().as_ref())
    .await
    .expect("projection subscription");
```

With that one `await`, every `POST /api/v1/wallets/:id/deposit` flows command → ledger → store → broker → projection → read model, and the next `GET /api/v1/wallets/:id` is served from the projected view. The HTTP test suite proves the loop converges: it deposits, withdraws, then reads back the balance and `version` the projection folded — no manual repair needed.

Glob topics and consumer groups

A subscription topic is a glob pattern (`*`, `?`, `[..]`, `{a,b}`); a published event is delivered to every subscription whose pattern matches its topic. Lumen subscribes to the exact `wallets.events`, but a multi-event service could fan a single listener across a family:

Rust

```
broker.subscribe("wallets.*", handler(|ev| async move { Ok(()) })).unwrap();
// matches wallets.events, wallets.audit, ...
```

Consumer groups give competing-consumer delivery: within a group each matching event goes to exactly **one** member (round-robin); distinct groups each get their own copy:

Rust

```
broker.subscribe_group("wallets.events", "projections", handler1).unwrap();  
broker.subscribe_group("wallets.events", "projections", handler2).unwrap();  
// each event reaches exactly one of handler1/handler2
```

This is how you would scale Lumen's projection horizontally: run several projector instances in one group and the broker shares the partitions among them, each instance owning a slice of the wallet space.

Retry and dead-letter

`wrap_listener(handler, publisher, policy)` is the adapter-agnostic retry/DLQ wrapper. A failing delivery is retried up to `retries` times with linear backoff (`retry_delay * attempt`); on exhaustion the event is republished to the dead-letter topic (when set), carrying the original payload/key/headers plus `x-original-topic` and `x-exception` diagnostic headers:

Rust

```

use std::sync::Arc;
use std::time::Duration;
use firefly_eda::{handler, wrap_listener, InMemoryBroker, ListenerPolicy};

#
tokio::runtime::Builder::new_current_thread().enable_all().build().unwrap().block_on(async
{
    let broker = Arc::new(InMemoryBroker::new());
    let inner = handler(|_ev| async {
        Err(firefly_kernel::FireflyError::internal("boom")) });
    let wrapped = wrap_listener(
        inner,
        broker.clone(),
        ListenerPolicy::with_retries(3)
            .retry_delay(Duration::from_millis(50))
            .dead_letter_topic("wallets.events.DLT"),
    );
    broker.subscribe("wallets.events", wrapped).unwrap();
# });

```

Lumen's projection takes a gentler path — it *swallows* a transient store miss and returns `Ok(())` rather than failing the delivery, so a single poison message never stalls the stream. That is the right call for a rebuild-from-stream projection (the next redelivery, or the next event for the wallet, converges anyway). A side-effecting listener — one that sends an email or calls an external API — is where `wrap_listener` and a dead-letter topic earn their keep.

For an inspectable record of failures (rather than a routing topic), wire an `EdaDeadLetterStore` via `ListenerPolicy::dead_letter_store`: an exhausted event is captured into the store (queryable with `list` / `get` / `remove`).

Event filters

`EventFilter` is a per-envelope delivery gate layered over topic matching. Where the broker decides *which* subscriptions a topic reaches, a filter decides whether a reached subscription actually *runs*. Two ship — a header regex filter and an arbitrary predicate filter. Lumen's envelopes carry an `aggregateType` header, so a header filter could restrict a subscriber to `Wallet` events:

Rust

```
use firefly_eda::{handler, with_filters, Event, HeaderEventFilter,
InMemoryBroker};

#
tokio::runtime::Builder::new_current_thread().enable_all().build().unwrap().block_on(async
{
    let broker = InMemoryBroker::new();
    let inner = handler(|_ev: Event| async { Ok(()) });
    let gated = with_filters(inner, [HeaderEventFilter::new("aggregateType",
r"^Wallet$").unwrap()]);
    broker.subscribe("wallets.events", gated).unwrap();
    # });
```

An event must pass *every* filter to be delivered; a non-matching event is dropped before the handler body runs.

The reactive subscription surface

`InMemoryBroker::subscribe_reactive(topic)` is the reactive twin of `subscribe` — a `Flux<Event>` that emits every event delivered to the topic, composing with Firefly's full reactive-streams operator set. `publish_mono(event)` is the cold reactive publish (nothing happens until the `Mono` is subscribed):

Rust

```

use std::sync::Arc;
use firefly_eda::{Event, InMemoryBroker};

#
tokio::runtime::Builder::new_current_thread().enable_all().build().unwrap().block_on(async
{
    let broker = Arc::new(InMemoryBroker::new());
    let flux = broker.subscribe_reactive("wallets.*").unwrap();

    broker
        .publish_mono(Event::new("wallets.events", "WalletOpened", "lumen", None))
        .block()
        .await
        .unwrap();
    broker.close().unwrap(); // terminates the Flux

    let events = flux.take(1).collect_list().block().await.unwrap().unwrap();
    assert_eq!(events[0].topic, "wallets.events");
# });

```

Deliveries are buffered through a bounded channel; when the downstream consumer falls behind, the newest events are dropped (`onBackpressureDrop`) rather than blocking or failing the publisher — extending "a slow consumer never fails publishers" to the reactive surface. This is the same `Flux` Lumen's optional streaming endpoint composes over (see [Production & Deployment](#)).

In-process events and after-commit externalization

The broker carries events *between* services. Inside one service you often want the same decoupling without a network hop: one component raises a fact, others react, and none of them knows the others exist. That is Spring's `ApplicationEventPublisher` / `@EventListener`, and Firefly ships it as a thread-safe, async, in-process bus alongside the broker.

Publish with `publish_event`, and listen with `#[application_event_listener]` on a free async function that takes the event by shared reference. Listeners are discovered across the crate graph (the same `inventory` scan that finds your components), so there is no manual registration:

Rust

```
use firefly::prelude::*;

struct WalletOpened { id: String }

#[firefly::application_event_listener]
async fn audit_opening(event: &WalletOpened) {
    tracing::info!(wallet = %event.id, "wallet opened");
}

// somewhere in a command handler:
publish_event(WalletOpened { id: wallet_id }).await;
```

Listening relative to a transaction

A plain listener runs the instant you publish. Often that is too early: you do not want to send a "wallet opened" notification until the database transaction that opened it has actually committed. `#[transactional_event_listener]` binds the listener to a phase of the surrounding `#[transactional]` boundary — `after_commit` (the default), `before_commit`, `after_rollback`, or `after_completion`:

Rust

```
#[firefly::transactional_event_listener] // after_commit
async fn notify_owner(event: &WalletOpened) {
    // Runs only once the opening transaction commits; never on a rollback.
    mailer.send_welcome(&event.id).await;
}
```


Events published inside a transaction are buffered and dispatched at the chosen phase; a rolled-back transaction fires the `after_rollback` listeners and never the `after_commit` ones, so a failed write can never leak a "success" side-effect. With no transaction active the listener falls back to running immediately (treating the work as already committed), so the same handler is useful in a unit test or a datasource-less path. If you want transactional event semantics without a SQL datasource at all, register the `LocalTransactionManager` (the Rust equivalent of Spring's `ResourcelessTransactionManager`).

Bridging in-process events to the broker

The two layers compose into the pattern you almost always want: do the in-process work, and once it commits, publish an integration event to the broker — never a "ghost" message for a transaction that rolled back. That is Spring Modulith's event externalization, and `externalize_after_commit` wires it in one line:

Rust

```
// at startup, once per externalized event type:
firefly::eda::externalize_after_commit::<WalletOpened>("wallet.events",
"wallet.opened");

// thereafter, an ordinary in-process publish inside a transaction...
publish_event(WalletOpened { id: wallet_id }).await;
// ...is serialized to JSON and published to the "wallet.events" topic on the
// registered broker the moment the transaction commits.
```

`externalize_after_commit` simply registers an `after_commit` listener that forwards through `publish_to_broker` (which serializes the payload and publishes via the `register_broker`-registered `Broker`). A committed transaction reaches Kafka, RabbitMQ, or whichever transport you wired; a rolled-back one publishes nothing.

Three distinct roles, easy to keep straight: `#[event_listener("topic")]` consumes from a broker topic (the `@KafkaListener` analog above); `#[application_event_listener]` / `#[transactional_event_listener]` handle *in-process* events; and `externalize_after_commit` is the bridge from the second to a broker producer.

Production transports

Each transport crate implements the same `Broker` port; swap the constructor and keep every handler. Code against `firefly_eda::Broker` and select the adapter at wiring time. For Lumen this is a one-line change in `build_app`: replace the in-memory broker with a Kafka one and the projection, the ledger, and every command keep compiling unchanged.

Crate	Backend	Constructor
<code>firefly-eda-kafka</code>	Apache Kafka	<code>new_kafka_broker(KafkaConfig)?</code>
<code>firefly-eda-rabbitmq</code>	RabbitMQ	<code>RabbitMqBroker::new(RabbitMqBrokerConfig)</code>
<code>firefly-eda-postgres</code>	Postgres outbox	<code>PostgresBroker::new(PostgresConfig::new(dsn))</code>
<code>firefly-eda-redis</code>	Redis Streams	<code>RedisStreamsBroker::connect(RedisConfig::new(url))?</code>

Kafka, for example — note the handler body is identical to Lumen's:

Rust

```

use firefly_eda::{handler, Event};
use firefly_eda_kafka::{new_kafka_broker, KafkaConfig};

# async fn ex() -> firefly_eda::EdaResult<()> {
let broker = new_kafka_broker(KafkaConfig {
    brokers: vec!["kafka:9092".into()],
    client_id: "lumen".into(),
    consumer_group: "lumen-projections".into(),
    ..Default::default()
})?;

broker
    .subscribe("wallets.events", handler(|ev: Event| async move {
        println!("observed {}", ev.event_type);
        Ok(())
    }))
    .await?;

let ev = Event::new("wallets.events", "WalletOpened", "lumen", None);
broker.publish(ev).await?;
# Ok(())
# }

```

Redis Streams uses a connect-then-start lifecycle:

Rust

```

use firefly_eda::{handler, Event};
use firefly_eda_redis::{RedisConfig, RedisStreamsBroker};

# async fn ex() -> firefly_eda::EdaResult<()> {
let broker = RedisStreamsBroker::connect(
    RedisConfig::new("redis://localhost:6379/0")
        .with_streams(["wallets.events"])
        .with_group("lumen-projections"),
)?;
broker.subscribe("wallets.*", handler(|ev: Event| async move {
    println!("got {}", ev.event_type);
    Ok(())
})).await?;
broker.start().await?;
# Ok(())
# }

```

NOTE

The Postgres broker is a **transactional outbox**: events are written in the same transaction as your state change and drained to consumers via **LISTEN / NOTIFY**, giving at-least-once delivery without a separate broker. That closes the append-then-publish gap discussed above; the [next chapter](#) covers the outbox primitive directly.

Broker health

EventPublisherHealthIndicator adapts any broker implementing the **BrokerHealth** ping probe to a **firefly_observability::Indicator**, surfacing broker liveness on **/actuator/health** under the **eventPublisher** id — so when Lumen graduates to a real broker, its readiness shows up alongside the rest of the service's health (see [Observability](#)).

Recap – what changed in Lumen

The CQRS loop is closed. Where Chapter 9's command handlers persisted events and left the read side to repair itself, Lumen now publishes every persisted event and projects it back automatically.

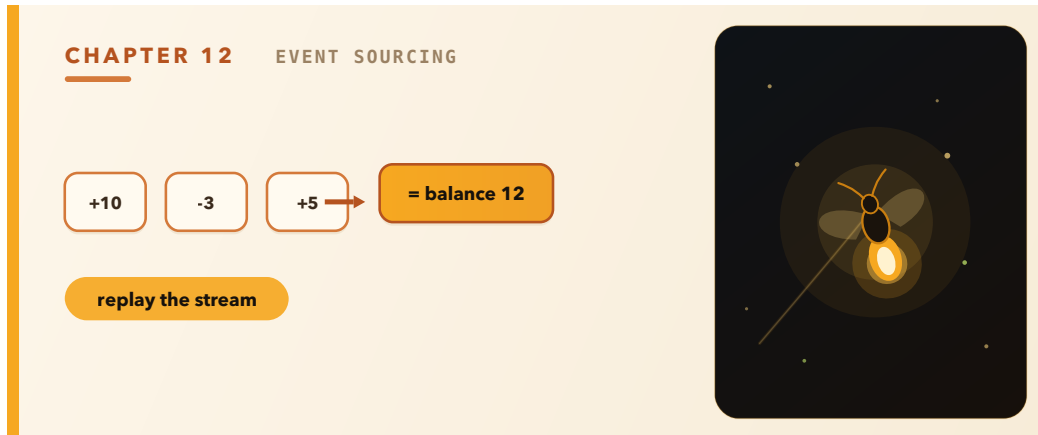
Piece	Role
<code>EVENTS_TOPIC / EVENT_SOURCE</code>	Shared constants the publisher and listener agree on
<code>to_envelope(&DomainEvent)</code>	Bridges a persisted domain event to the wire <code>Event</code> (key = wallet id, headers carry routing)
<code>Ledger::commit</code>	Appends, then publishes each event — save before you publish
<code>#[event_listener(topic = "wallets.events")]</code>	Generates <code>subscribe_project_wallet_event(broker)</code>
<code>project_wallet_event</code>	The idempotent rebuild-from-stream projection that feeds the read model
<code>bind_projection / projection_state</code>	Publish-once wiring so the free-fn listener reaches its collaborators
<code>web.broker (InMemoryBroker)</code>	The default transport — swap the constructor for Kafka/RabbitMQ/Redis, keep the listener

Three principles carry forward: **save before you publish** so a subscriber never sees an uncommitted fact; **make projections idempotent** so at-least-once redelivery is harmless (Lumen re-folds the stream rather than applying a delta); and **depend on the Broker port, not the adapter** so the in-memory broker becomes Kafka with a one-line change.

The events Lumen publishes here are still backed by a transient in-memory store. The [next chapter](#) makes those events the *source of truth* — durable, replayable, the canonical record from which every balance is recomputed.

Exercises

1. Add a `WelcomeNotifier` listener. Write a second `#[event_listener(topic = "wallets.events")]` free fn that reacts only to `WalletOpened` (check `ev.event_type`) and logs a welcome line carrying the `aggregateId` header. Subscribe it next to the projection in `build_app` and confirm — via an `InMemoryBroker` unit test that publishes a `WalletOpened` envelope — that it fires, while the existing command handlers stay untouched.
2. Prove idempotency. In a test, build a `Ledger` over a `MemoryEventStore` and an `InMemoryBroker`, subscribe the projection, open a wallet, and deposit twice. Then publish the *same* `MoneyDeposited` envelope a second time with `broker.publish(to_envelope(&event))` and assert the read-model `WalletView` balance is unchanged — the rebuild-from-stream fold absorbs the redelivery.
3. Gate by aggregate type. Wrap the projection's handler with `with_filters` and a `HeaderEventFilter::new("aggregateType", r"^Wallet$")`, then publish an envelope whose `aggregateType` header is `"Account"` and confirm the projection does not run for it. Explain why a header filter is a cheaper guard than checking inside the handler body.
4. Swap in a real broker (sketch). Add the `eda-kafka` feature to the crate and write the `build_app` variant that constructs `new_kafka_broker(...)` instead of using `web.broker`. You do not need a running Kafka — the point is to confirm the projection, the ledger, and the command handlers compile unchanged against the `Broker` port.



CHAPTER 12

Event Sourcing the Ledger

The [last chapter](#) left one question politely unasked. Lumen's **Ledger** persists wallet events and the projection rebuilds the read model by re-folding the stream — but *what stream*? So far the wallet's canonical state has been implied. By the end of this chapter it is explicit and load-bearing: the **Wallet** aggregate holds **no stored balance at all**. Its balance is a pure function of an append-only stream of **WalletOpened**, **MoneyDeposited**, and **MoneyWithdrawn** events, recomputed every time the aggregate is loaded.

That is **event sourcing**: instead of storing current state and discarding each change, you store the *sequence of changes* and derive state by replaying them. A financial ledger is the ideal domain for it — accountants have known for centuries that a ledger's authority comes from its entries, not from the running total at the foot of the column. The total is a *derived fact*; the entries are the *source of truth*. By the end of this chapter, an auditor asking "what was wallet `wlt_...`'s balance after the third movement?" gets an answer Lumen can *prove* from the stream, not merely report from a column.

`firefly-eventsourcing` provides the framework's event-sourced aggregate primitives: an `AggregateRoot` that tracks uncommitted events, an `EventStore` with optimistic concurrency, snapshots, projections, a global cross-aggregate stream, a transactional outbox, and multi-tenancy. Where the [EDA chapter](#)'s `Event` envelope was the *transport* for a fact, the `DomainEvent` here is the *record* of it — the durable truth from which state is rebuilt.

Design note. `firefly-eventsourcing` provides the event-sourced aggregate primitives used here: `AggregateRoot::raise` to record an event, an `EventStore` with optimistic concurrency, and projections that build read models. A command `raise`s an event; the same `apply` fold runs on both the write path and replay — that symmetry is the correctness guarantee of event sourcing. The `DomainEvent` serializes to a stable, versioned, language-neutral JSON contract, so any service that honors it interoperates regardless of the language it is written in.

State storage vs event storage

The clearest way to feel the shift is to compare what Lumen's storage *holds* in each model.

In the **state-storage model**, the store keeps only the wallet's current state:

id	owner	balance	version
wlt_a1	alice	120	3

Every deposit and withdrawal overwrites `balance`. The history is gone — you know the wallet holds 120 cents now; you cannot know how it got there.

In the **event-storage model**, the store keeps the stream:

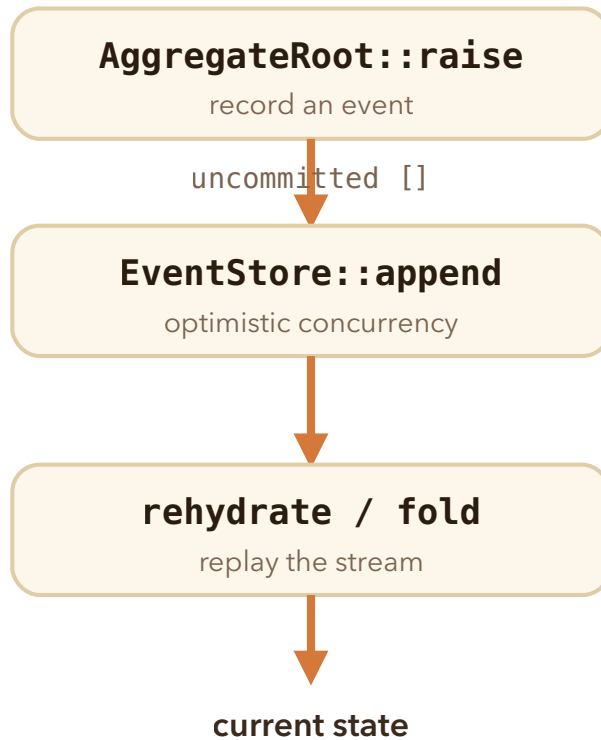
aggregate_id	version	event_type	payload
wlt_a1	1	WalletOpened	<code>{"wallet_id":"wlt_a1","owner":"alice","opening_bala</code>
wlt_a1	2	MoneyDeposited	<code>{"wallet_id":"wlt_a1","amount":50}</code>
wlt_a1	3	MoneyWithdrawn	<code>{"wallet_id":"wlt_a1","amount":30}</code>

The current balance is still 120 cents — but now you can read every decision that led to it, replay to any version, and audit the lot. The trade-off is real: reads cost a replay (mitigated by **snapshots**) and events are immutable (schema evolution handled by **upcasters**). Both have first-class support below.

 NOTE

Event sourcing is not the same as the [previous chapter](#)'s EDA. There, the aggregate stored its state and *published* events as a side effect. Here the events *are* the state: there is no **balance** column to keep in sync — the balance is computed by folding the stream every time the aggregate loads.

The mental model



The event-sourcing cycle. `raise` stages an event on the aggregate, `EventStore::append` persists the uncommitted events under optimistic concurrency, and a later load rehydrates the aggregate by folding its stream back into the current state.

The Wallet's domain events

In Lumen the three events are plain payload structs carrying `#[derive(DomainEvent)]`. The derive stamps each with a stable `EVENT_TYPE` discriminator (its struct name) and a `to_domain_event(...)` conversion onto the framework wire event — so the event type is never spelled as a bare string literal at the call sites:

Rust

```

use firefly::eventsourcing::DomainEvent;
use serde::{Deserialize, Serialize};

/// Payload of the event raised when a wallet is opened.
#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize, DomainEvent)]
pub struct WalletOpened {
    pub wallet_id: String,
    pub owner: String,
    /// The opening balance, in minor units (cents).
    pub opening_balance: i64,
}

/// Payload of the event raised when money is credited to a wallet.
#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize, DomainEvent)]
pub struct MoneyDeposited {
    pub wallet_id: String,
    pub amount: i64,
}

/// Payload of the event raised when money is debited from a wallet.
#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize, DomainEvent)]
pub struct MoneyWithdrawn {
    pub wallet_id: String,
    pub amount: i64,
}

```

`#[derive(DomainEvent)]` generates, for each struct, a `pub const EVENT_TYPE:` `&'static str` equal to the struct name (`"WalletOpened"`, ...), an `event_type()` accessor, and a `to_domain_event(aggregate_id, aggregate_type, version)` that JSON-encodes the payload into a framework `DomainEvent`. That generated `EVENT_TYPE` is the only thing the aggregate and its `apply` fold reference, so a rename of the struct flows through automatically.

NOTE

`#[derive(DomainEvent)]` generates, per struct, a `pub const EVENT_TYPE` equal to the struct name (the routing discriminator), an `event_type()` accessor, and a `to_domain_event(...)` that JSON-encodes the payload into a framework `DomainEvent` — so the event type is never a bare string literal at the call sites, and a struct rename flows through automatically. The generated JSON is a stable, versioned, language-neutral contract that any service honoring it can consume.

The Wallet aggregate – raise, then apply

The `Wallet` carries `#[derive(AggregateRoot)]`, which finds the embedded framework `AggregateRoot` field (`root`) and generates `Wallet::AGGREGATE_TYPE` plus `aggregate()` / `aggregate_mut()` accessors. The projected state (`owner`, `balance`, `opened`) is *not* stored — it is folded from the stream:

Rust

```
use firefly::eventsourcing::{AggregateRoot, DomainEvent};

use crate::money::Money;

#[derive(Debug, Clone, AggregateRoot)]
#[firefly(aggregate_type = "Wallet")]
pub struct Wallet {
    /// The framework aggregate root – uncommitted-event buffer + version.
    pub root: AggregateRoot,
    pub owner: String,
    /// Folded from the stream; never stored.
    pub balance: Money,
    /// Whether the wallet has been opened (an empty stream is "absent").
    pub opened: bool,
}
```

Every command follows the canonical event-sourcing shape: validate the invariant, **raise** the matching event onto the embedded root, then apply it to in-memory state. The write path and the replay path run the *same* **apply** code — that symmetry is the correctness guarantee of event sourcing.

Rust

```
/// Credits `amount` to the wallet, raising a `MoneyDeposited` event.
pub fn deposit(&mut self, amount: Money) -> Result<(), DomainError> {
    self.require_opened()?;
    let amount = amount.require_positive()?;
    self.raise(
        MoneyDeposited::EVENT_TYPE,
        &MoneyDeposited {
            wallet_id: self.root.id.clone(),
            amount: amount.cents_value(),
        },
    );
    self.balance = self.balance.add(amount);
    Ok(())
}

/// Serialises a `#[derive(DomainEvent)]` payload and raises it onto the embedded
/// root under `event_type` – the discriminator from the generated `EVENT_TYPE`.
fn raise<P: Serialize>(&mut self, event_type: &str, payload: &P) {
    let bytes = serde_json::to_vec(payload).expect("domain event payload serialises");
    self.root.raise(event_type, bytes);
}
```

AggregateRoot::raise buffers the event (so the ledger can persist it) and bumps the version. **withdraw** is the same shape, with one extra guard: it computes the remaining balance *first* and lets **Money::subtract** reject an overdraw — so a failed withdrawal raises **no** event at all, leaving the stream clean. That overdraft guard is the trigger the transfer saga relies on in [Sagas, Workflows & TCC](#).

Rehydration – folding the stream

Rehydration is the load path: rebuild a wallet by folding its full ordered stream through the same `apply` the commands use. An empty stream yields an unopened wallet — which is how the ledger distinguishes "absent" from "exists":

Rust

```

/// Rebuilds a wallet by folding `events` (its full ordered stream).
pub fn rehydrate(id: &str, events: &[DomainEvent]) -> Self {
    let mut wallet = Wallet {
        root: AggregateRoot::new(id, AGGREGATE_TYPE),
        owner: String::new(),
        balance: Money::ZERO,
        opened: false,
    };
    for event in events {
        wallet.apply(event);
        // Keep the root version in lock-step with the stream head so a
        // subsequent command appends at the right expected version.
        wallet.root.version = event.version;
    }
    wallet
}

/// Folds one persisted event into the projected state.
fn apply(&mut self, event: &DomainEvent) {
    match event.event_type.as_str() {
        WalletOpened::EVENT_TYPE => {
            if let Ok(p) =
serde_json::from_slice::<WalletOpened>(&event.payload) {
                self.owner = p.owner;
                self.balance = Money::cents(p.opening_balance);
                self.opened = true;
            }
        }
        MoneyDeposited::EVENT_TYPE => {
            if let Ok(p) =
serde_json::from_slice::<MoneyDeposited>(&event.payload) {
                self.balance = self.balance.add(Money::cents(p.amount));
            }
        }
        MoneyWithdrawn::EVENT_TYPE => {
            if let Ok(p) =
serde_json::from_slice::<MoneyWithdrawn>(&event.payload) {
                self.balance = Money::cents(self.balance.cents_value() -

```

```

    p.amount);
        }
    }
    _ => {}
}
}

```

Note that `apply` folds `MoneyWithdrawn` with a raw subtraction (`Money::cents(self.balance.cents_value() - p.amount)`) rather than the overdraft-guarded `Money::subtract` the `withdraw` command uses. That asymmetry is deliberate: replay never re-validates. The guard already ran at *write* time and a failed withdrawal raised no event, so every event in the stream is a fact that already passed its invariant — replay simply applies it.

The folding logic in `apply` is matched on the *same* `EVENT_TYPE` constant the commands raise under, so the two halves can never disagree about an event's name. Lumen's unit tests prove the replay law directly: open + deposit + withdraw on a *writer* wallet, take its uncommitted stream, then `Wallet::rehydrate` a fresh wallet from that stream and assert the rebuilt balance, owner, and version match — state recomputed from events, never stored.

Design note. The discipline: a command `raise` *s* the event, `apply` mutates the projected fields, and `load` replays the same `apply` to rebuild state. Lumen registers no handler table — it `match` es on the generated `EVENT_TYPE` const, the Rust-idiomatic way to keep the write fold and the replay fold from ever disagreeing about an event's name.

Raising and appending events

The framework `AggregateRoot` accumulates `DomainEvent`s as you `raise` them; you `take_uncommitted` and `append` them to the store. `append` enforces optimistic concurrency: pass the version you loaded, and a concurrent writer's append fails with `EventSourcingError::Concurrency`:

Rust

```
use firefly_event sourcing::{AggregateRoot, EventStore, MemoryEventStore};

#[tokio::main]
async fn main() {
    let store = MemoryEventStore::new();

    let mut user = AggregateRoot::new("u1", "User");
    user.raise("UserCreated", br#"{"name":"alice"}"#);
    user.raise("UserRenamed", br#"{"name":"bob"}"#);

    let events = user.take_uncommitted();
    // expected_version 0 -> this is a brand-new aggregate.
    if let Err(err) = store.append(&user.id, 0, events).await {
        eprintln!("append failed (raced): {err}");
    }

    assert_eq!(store.load("u1").await.unwrap().len(), 2);
}
```

The `EventStore` port:

Rust

```
#[async_trait]
pub trait EventStore: Send + Sync {
    async fn append(&self, aggregate_id: &str, expected_version: i64,
                    events: Vec<DomainEvent>) -> Result<(), EventSourcingError>;
    async fn load(&self, aggregate_id: &str) -> Result<Vec<DomainEvent>,
EventSourcingError>;
    async fn load_after(&self, aggregate_id: &str, since_version: i64)
        -> Result<Vec<DomainEvent>, EventSourcingError>;
    async fn stream_all(&self, after_event_id: Option<&str>, limit: usize,
tenant: Option<&str>)
        -> Result<Vec<StreamedEvent>, EventSourcingError>;
}
```

The default is `MemoryEventStore` — the in-process store Lumen runs on, ideal for development and tests. `SqlEventStore` backs it with a SQL store over the `firefly-transactional Database` port for production; swapping it is a one-line change in `build_app`, exactly like swapping the broker in the last chapter.

The Ledger ties it together

Lumen's `Ledger` is the application service that owns the store (and the broker from the [last chapter](#)). Every command rehydrates, runs the domain method, and commits with optimistic concurrency. Here is `deposit` and the load path — the version the wallet rehydrated to is the `expected_version` the append must match:

Rust

```

/// Credits `amount` to `wallet_id`, persisting + publishing `MoneyDeposited`.
pub async fn deposit(&self, wallet_id: &str, amount: Money) ->
Result<WalletView, DomainError> {
    let mut wallet = self.load(wallet_id).await?;
    let expected = wallet.root.version;
    wallet.deposit(amount)?;
    self.commit(&mut wallet, expected).await?;
    Ok(wallet.view())
}

/// Rehydrates the aggregate from its persisted stream.
async fn load(&self, wallet_id: &str) -> Result<Wallet, DomainError> {
    let events = self.load_events(wallet_id).await?;
    Ok(Wallet::rehydrate(wallet_id, &events))
}

/// Loads the full event stream, mapping an absent aggregate to a domain 404.
pub async fn load_events(&self, wallet_id: &str) -> Result<Vec<DomainEvent>,
DomainError> {
    match self.store.load(wallet_id).await {
        Ok(events) => Ok(events),
        Err(EventSourcingError::AggregateNotFound) => {
            Err(DomainError::NotFound(wallet_id.to_string()))
        }
        Err(e) => Err(DomainError::NotFound(format!("{wallet_id}: {e}"))),
    }
}

```

The `commit` method (shown in full in the [last chapter](#)) appends at `expected` then publishes each event. The two chapters meet here: this one supplies the durable, replayable store; that one carries each appended event onto the wire so the projection can react.

Optimistic concurrency in practice

Two concurrent requests — say a deposit from the app and a fee withdrawal from a job — can both load wallet `wlt_a1` at version 3, each apply a change, and each try to append at `expected_version = 3`. The first append wins and the stream advances to 4; the second now mismatches and the store returns

`EventSourcingError::Concurrency`. Lumen maps that to a `DomainError::NotFound` detail ("concurrent modification") so the caller retries from a fresh load. You never manage version numbers by hand — the version the wallet rehydrated to is the token, and the store enforces it.

NOTE

`append(id, expected_version, events)` enforces optimistic concurrency: the version the wallet rehydrated to is the token, and a stale append fails with `EventSourcingError::Concurrency`. Catch it and retry the load-mutate-save cycle (or surface a 409) — never swallow it.

Typed aggregates and the repository

Lumen folds the stream by hand in `Wallet::apply` because it teaches the mechanic clearly. For larger aggregates the framework offers a thinner path: implement `EventSourcedAggregate` — a typed `apply_event` plus optional snapshot serialization — and let `EventSourcedRepository` tie `load` (snapshot + replay) and `save` (append + snapshot policy) together:

Rust

```

use firefly_event sourcing::{
    AggregateRoot, DomainEvent, EventSourcedAggregate, EventSourcedRepository,
    EventSourcingError, MemoryEventStore,
};
use std::sync::Arc;

#[derive(Default)]
struct Wallet { root: AggregateRoot, balance: i64 }

impl EventSourcedAggregate for Wallet {
    const AGGREGATE_TYPE: &'static str = "Wallet";
    fn root(&self) -> &AggregateRoot { &self.root }
    fn root_mut(&mut self) -> &mut AggregateRoot { &mut self.root }
    fn apply_event(&mut self, event: &DomainEvent) -> Result<(),
EventSourcingError> {
        if event.event_type == "Credited" {
            let amount: i64 = serde_json::from_slice(&event.payload)
                .map_err(|e| EventSourcingError::Projection(e.to_string()))?;
            self.balance += amount;
        }
        Ok(())
    }
}

# async fn ex() -> Result<(), EventSourcingError> {
    let repo =
EventSourcedRepository::<Wallet>::new(Arc::new(MemoryEventStore::new()));

    let mut w = Wallet::default();
    w.root_mut().raise("Credited", b"500");
    repo.save(&mut w).await?; // append uncommitted

    let reloaded = repo.load(&w.root.id).await?; // snapshot + replay
    assert!(reloaded.is_some());
    # Ok(())
    # }

```

`EventSourcedRepository::with_snapshots(store, snapshots, interval)` enables periodic state captures so rehydration does not replay the entire history.

Snapshots – when streams get long

Event sourcing trades write simplicity for read cost: a wallet with 10,000 movements replays 10,000 events every load. **Snapshots** cut that down. A snapshot is a serialized checkpoint of the aggregate's state at a version; on load, the repository deserializes the latest snapshot and replays only the events after it. A snapshot at version 9,000 turns a 10,000-event replay into 1,000.

Lumen's wallets are short-lived enough that the in-memory store's full replay is fine, so the sample does not wire snapshots — but the seam is there: `with_snapshots(store, MemorySnapshotStore::new(), 100)` would checkpoint every time a wallet's stream crosses a 100-event boundary. Snapshots are an optimization, never a correctness requirement: remove them and the system is slower but still correct.

NOTE

`with_snapshots(store, MemorySnapshotStore::new(), interval)` checkpoints aggregate state every time a stream crosses an interval boundary; on load, the repository deserializes the latest snapshot and replays only the events after it. The interval-crossing trigger handles a batch that straddles the threshold (version 95 → 105 still snapshots). Snapshots are an optimization, never a correctness requirement.

Projections – building read models

A `Projection` is a read-side handler. Register projections on a `ProjectionRunner` and replay an aggregate's events through them. This is the event-store sibling of the [last chapter](#)'s event-bus listener: Lumen's `project_wallet_event` reacts to events as they are published, whereas a `ProjectionRunner` can replay history from the beginning to rebuild a read model from scratch:

Rust

```
use std::sync::Arc;
use firefly_eventsourcing::{FunctionProjection, ProjectionRunner};

let runner = ProjectionRunner::new();
runner.register(Arc::new(FunctionProjection::new("balances", |event| async move {
    // update a read-model row from the event ...
    Ok(())
})));

runner.replay(&store, "wlt_a1").await?; // replay one aggregate's stream
```

This rebuildability is unique to event sourcing. If Lumen's read model is ever lost or its schema changes, you stop the projector, clear the read model, and replay every stream — the history is right there in the store. A state-storage model cannot do this; it discarded the history at write time.

The global stream

`EventStore::stream_all` exposes the global, cross-aggregate, ordered event stream with a resumable cursor — the engine for read models that span many aggregates (think: "all movements across all wallets, in order"). The runner consumes it in batches, at-least-once and in-order:

Rust

```
// Drive one batch; returns the next cursor + any per-event error.
let (next_cursor, err) = runner
    .drive_once(&store, None, 100, None)
    .await?;

// Or replay the whole global stream from a start cursor.
let cursor = runner.replay_all(&store, None, 100, None).await?;
```

The transactional outbox

The [last chapter](#) noted a gap in `Ledger::commit`: it appends, then publishes, and a crash *between* the two persists the fact but drops the broadcast. `TransactionalOutbox` closes that gap. Instead of publishing directly, a writer `enqueue`s the `DomainEvent`; a background relay polls and forwards each pending record to an `OutboxSink`, retrying up to `max_attempts`. The default `EdaSink` bridges each `DomainEvent` to a `firefly_eda::Event` and publishes it — the same `to_envelope`-shaped bridge, but durable:

Rust

```

use std::sync::Arc;
use firefly_event sourcing::{EdaSink, TransactionalOutbox};

let outbox = TransactionalOutbox::new(Arc::new(EdaSink::new(
    broker,           // the Arc<dyn firefly_eda::Publisher>
    "wallet.events",  // destination topic
    "lumen",         // logical source stamped onto every Event::source
)))
.with_max_attempts(5);

outbox.enqueue(some_event).await;           // a writer enqueues
outbox.start().await;                       // background relay forwards + retries
// ... later
let dead = outbox.dead_letters().await; // exhausted records, for inspection
outbox.stop().await;

```

Exhausted records become dead letters — excluded from the publish loop and surfaced for inspection or manual retry. This is the upgrade path for a production Lumen: enqueue into the outbox inside the same store transaction as the append, and let the relay guarantee at-least-once delivery to the broker even across crashes — which is exactly why the projection was built to be **idempotent** in the last chapter.

Design note. `TransactionalOutbox` closes the append-then-publish gap: instead of publishing directly, a writer `enqueue`s the `DomainEvent` in the same store transaction as the append, and a background relay forwards each pending record to an `OutboxSink`, retrying up to `max_attempts` and dead-lettering on exhaustion. Recording the event durably before dispatching it is what gives at-least-once delivery across crashes — and why the projection was built to be idempotent.

Schema evolution – upcasters

`EventUpcaster` migrates events on the `read` paths only, so consumers always observe current-schema events while the stored history stays untouched. Suppose Lumen later needs a `reference` field on every deposit for reconciliation: new events carry it, old `MoneyDeposited` events do not, and an upcaster fills the gap on load:

Rust

```
use std::sync::Arc;
use firefly_eventsourcing::{EventUpcaster, MemoryEventStore};

let store = MemoryEventStore::with_upcasters(vec![Arc::new(MyUpcaster)]);
// every event returned by load / load_after passes through applicable upcasters
```

Old data becomes readable without a migration; new data is written in the current schema. The events themselves stay immutable — you never rewrite history.

Multi-tenancy

An optional `DomainEvent::tenant_id` (stamped from `AggregateRoot::with_tenant`, persisted and filterable, omitted from JSON when `None`) is threaded through `append` / `load` / `stream_all`, so one store serves many tenants with per-tenant isolation on the global stream — the route a multi-bank Lumen deployment would take to keep each tenant's wallet streams separate.

Recap – what changed in Lumen

The wallet's balance is no longer a stored value — it is a *computation* over an immutable stream, and the stream is the system of record.

Piece	Role
<code>#[derive(DomainEvent)]</code>	Generates <code>EVENT_TYPE</code> + <code>to_domain_event(...)</code> for each payload struct
<code>#[derive(AggregateRoot)]</code>	Generates <code>AGGREGATE_TYPE</code> + <code>aggregate()</code> / <code>aggregate_mut()</code> over the embedded <code>root</code>
<code>Wallet::raise</code> / <code>apply</code>	Command applies the event; the same fold runs on write and on replay
<code>Wallet::rehydrate</code>	Rebuilds a wallet by folding its full stream — empty stream = unopened
<code>EventStore</code> / <code>MemoryEventStore</code>	The append-only log; <code>SqlEventStore</code> for production
<code>append(id, expected_version, ...)</code>	Optimistic concurrency — the rehydrated version is the token
<code>ProjectionRunner</code>	Rebuilds read models from history (the store-side sibling of the EDA listener)
<code>TransactionalOutbox</code>	Closes the append-then-publish gap with at-least-once relay

Three ideas carry forward. **The events are the truth** — there is no balance column to drift. **Write and replay share one fold** — `apply` runs the same way whether a command just raised the event or a load is rebuilding from history, which is the correctness guarantee. **Depend on the `EventStore` port** — the in-memory store becomes SQL with a one-line swap, just as the broker became Kafka.

When a business process spans multiple aggregates and needs compensation — moving money from one wallet to another, atomically — folding a single stream is no longer enough. Continue to [Sagas, Workflows & TCC](#), where the transfer saga drives two wallets and rolls the debit back when the credit fails.

Exercises

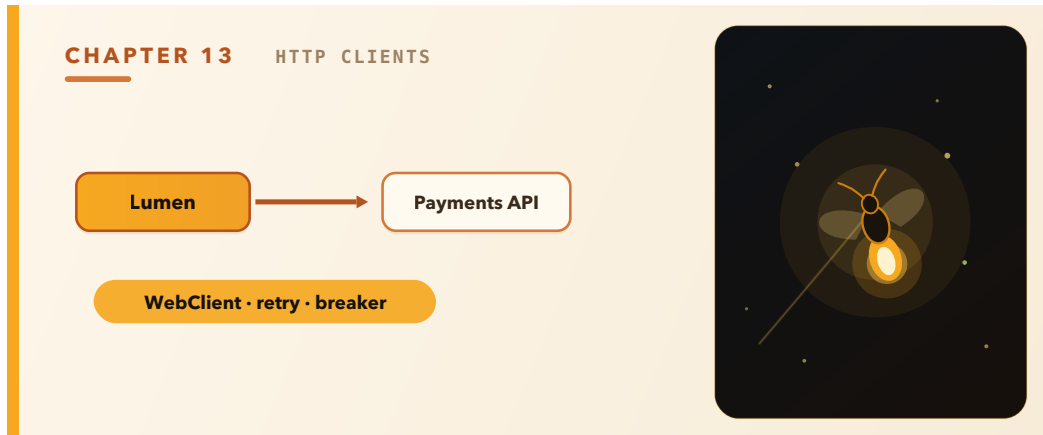
1. **Replay to a point in time.** Open a wallet and make three deposits. Load the raw stream with `ledger.load_events(&id)`, take only the events with `version <= 2`, and `Wallet::rehydrate` a fresh wallet from that slice. Assert the balance equals opening + first deposit only — the "time-travel query" a state-storage model cannot answer.
2. **Prove the overdraft guard raises no event.** Open a wallet with 100 cents, attempt to `withdraw` 101, and assert it errors with `DomainError::InsufficientFunds`. Then call `root.uncommitted()` and assert the buffer still holds exactly one event (the `WalletOpened`) — the failed command left the stream clean.
3. **Force an optimistic-concurrency conflict.** Append the open event for a wallet at `expected_version = 0`. Then, without reloading, raise a second event and append it *also* at `expected_version = 0`. Assert the second append returns `EventSourcingError::Concurrency`, and explain why a fresh load (which advances `expected` to 1) would have succeeded.
4. **Add a `ProjectionRunner` rebuild.** Register a `FunctionProjection` that tallies the count of `MoneyDeposited` events per wallet into an in-memory map, `replay` one wallet's stream through it, and assert the count. Then clear the map and replay again — confirming the read model is rebuildable from the store alone, with no live event traffic.



PART IV

Into Microservices





CHAPTER 13

HTTP Clients & Calling Other Services

By the end of this chapter, you will know how Lumen reaches *out* — how a wallet service settles a transfer through an external **Payments** processor, and how an **experience tier** sits in front of Lumen and composes it with its neighbors into one journey-shaped API. Lumen itself stays deliberately self-contained (every leg of its transfer drives the in-process **Ledger**), so this chapter is the "next adapter you would add": a typed outbound client wired into the transfer flow, plus the BFF that fans out across services.

When a transfer needs to settle against a real payment rail, the credit leg stops being a local **Ledger::deposit** and becomes a network call. That call can time out, fail halfway, or land on an overloaded service — failure modes a local method call never had. **firefly-client** gives you a typed client for that call instead of a hand-rolled **request** session threaded with retry and timeout logic.

The crate ships two HTTP clients that share the same automatics — default `Accept` / `Content-Type`, correlation-id and W3C trace-context propagation, and RFC 7807 problem decode into a typed `FireflyError`:

- the eager `RestClient` (built with `RestBuilder`) — an `async fn` that awaits a `Result`, with a built-in retry budget;
- the reactive `WebClient` — whose terminal operators hand back `Mono` / `Flux`, so an outbound call drops straight into a reactive pipeline.

The crate also ships scaffolds for SOAP, gRPC, GraphQL, and WebSocket clients. Everything is reachable through the one `firefly` facade as `firefly::client`.

Two clients, one contract. `RestClient` is the eager `async fn`-and-await client with a built-in retry budget; `WebClient` is the reactive one whose `body_to_mono` / `body_to_flux` / `exchange` terminals return publishers you compose. Both are values built with a fluent builder — no annotated interface to generate from, no reflection — and the resilience decorators (covered below) wrap the call.

The eager `RestClient`

Build a client with `RestBuilder`, then call `request` with a method, path, and optional body. The retry budget, timeout, and default headers are configured on the builder. Here Lumen builds the Payments client it would call from the credit leg of a transfer:

Rust

```

use std::time::Duration;
use firefly::client::RestBuilder;
use http::Method;
use serde::{Deserialize, Serialize};

#[derive(Serialize)]
struct SettleTransfer { wallet_id: String, amount: i64, reference: String }
#[derive(Deserialize)]
struct Payment { id: String, status: String }

#[tokio::main]
async fn main() {
    let payments = RestBuilder::new("https://payments.internal")
        .with_header("X-Tenant", "lumen")
        .with_timeout(Duration::from_secs(5))
        .with_retries(3)
        .build();

    let req = SettleTransfer {
        wallet_id: "wlt_alice".into(),
        amount: 300,
        reference: "transfer-42".into(),
    };
    match payments.request:::<_, Payment>(Method::POST, "/payments",
Some(&req)).await {
        Ok(payment) => println!("settled {} ({})", payment.id, payment.status),
        Err(err) => {
            // Upstream RFC 7807 problems are decoded into a typed FireflyError.
            if let Some(fe) = err.as_firefly() {
                eprintln!("payments upstream {}: {}", fe.status, fe.detail);
            }
        }
    }
}

```


A non-2xx `application/problem+json` response is decoded into a `FireflyError`, so an upstream failure carries the upstream's status and detail straight through Lumen's own error stack. `err.as_firefly()` is the typed accessor that recovers the upstream's typed problem, and `ClientError` also offers predicate helpers like `is_not_found()`, `is_unprocessable_entity()`, and `is_retryable()` so a caller can branch on the failure class without matching raw status codes.

Where this plugs into the saga. In `Sagas` the credit leg was `ledger.deposit(&to, amount)`. In a split deployment that becomes `payments.request::<_, Payment>(Method::POST, "/settle", ...)`. The saga shape does not change — it is still a `Step` with the debit's compensation — only the body of the credit step now does I/O across the network, which is exactly why the compensation (refund the debit) matters more than ever.

Idempotency on the wire

A transfer's settlement call must be idempotent: if `POST /payments` times out and the saga's retry fires it again, Payments must not create two payments. Carry a stable `Idempotency-Key` — typically the transfer id — so the upstream deduplicates a re-delivered request. Set it as a default header on a per-call builder, or thread it through the request:

Rust

```
let payments = RestBuilder::new("https://payments.internal")
    .with_header("Idempotency-Key", &transfer_id) // stable per business op
    .with_timeout(Duration::from_secs(2))
    .build();
```

The deduplication itself is the upstream's job; the client's job is to forward the key *consistently* across retries.

The reactive **WebClient**

The reactive client returns **Mono** / **Flux**, so an outbound call drops straight into a reactive pipeline and composes end-to-end with the **NdJson** / **Sse responders** Lumen's streaming endpoint uses. The fluent chain reads top to bottom — build, address, send, decode:

Rust

```

use firefly::client::WebClientBuilder;
use http::Method;
use serde::{Deserialize, Serialize};

#[derive(Serialize)]
struct SettleTransfer { wallet_id: String, amount: i64 }
#[derive(Deserialize)]
struct Payment { id: String }
#[derive(Deserialize)]
struct LedgerTick { seq: u64 }

#[tokio::main]
async fn main() {
    let client = WebClientBuilder::new("https://payments.internal")
        .with_header("X-Tenant", "lumen")
        .build();

    // body_to_mono - a single value -> Mono<Payment>.
    let _payment = client
        .method(Method::POST)
        .uri("/payments")
        .body(&SettleTransfer { wallet_id: "wlt_alice".into(), amount: 300 })
        .retrieve()
        .body_to_mono::<Payment>();

    // body_to_flux - a streamed NDJSON OR SSE body, decoded lazily
    // element-by-element with backpressure.
    let _ticks = client
        .get()
        .uri("/ledger/ticks")
        .header("Accept", "application/x-ndjson")
        .retrieve()
        .body_to_flux::<LedgerTick>();

    // exchange - raw status + headers without raising on a non-2xx.
    let _resp = client.get().uri("/health").retrieve().exchange();
}

```

The terminal operators:

Operator	Returns	Behavior
<code>body_to_mono::<T>()</code>	<code>Mono<T></code>	the whole body decoded as one <code>T</code>
<code>body_to_flux::<T>()</code>	<code>Flux<T></code>	a streamed NDJSON/SSE body, element-by-element
<code>exchange()</code>	<code>Mono<WebClientResponse></code>	the raw status + headers + body, no raise

Streaming semantics

`body_to_flux` consumes the byte stream chunk-by-chunk and decodes one element per frame, lazily and with backpressure — a slow downstream throttles the producer, and `.take(n)` stops pulling early. The decoder is chosen from the response `ContentType`:

- `application/x-ndjson` (and any non-SSE type) → one JSON document per newline-terminated line;
- `text/event-stream` → SSE frames separated by a blank line; the `data:` lines are concatenated and comment / `event:` / `id:` lines are ignored.

A malformed element terminates the stream with a decode `FireflyError` — the first error is terminal, the reactive-streams contract Firefly's `Flux` honors. This is the *consumer* side of the same wire format Lumen's own `GET /api/v1/wallets/:id/events` endpoint produces: one service streams the wallet's event log, another reads it back element by element.

Inspecting the raw response

`exchange()` hands back a `WebClientResponse` without raising on a non-2xx, so you can inspect it and decide:

Rust

```
let resp = client.get().uri("/health").retrieve().exchange().block().await?.unwrap();
if resp.is_success() {
    let body: serde_json::Value = resp.body_json()?;
} else if let Some(problem) = resp.problem() {
    // a decoded RFC 7807 FireflyError, if the body was a problem document
}
```

No baked-in retry

Retries are composed, not baked in. Unlike `RestBuilder::with_retries`, the `WebClient` has **no** retry budget. Compose retries on the returned publisher with `Mono::retry` / `Mono::retry_backoff`, so retry policy stays a property of the call site, not the client:

Rust

```
use firefly::reactive::{Backoff, Mono};
use std::time::Duration;

let payment = Mono::retry_backoff(
    || client.get().uri("/payments/p1").retrieve().body_to_mono:::<Payment>(),
    Backoff::new(3, Duration::from_millis(100)),
);
```

Composing with resilience

Both clients are deliberately small. For circuit breaking, rate limiting, or bulkheads, wrap calls in `firefly-resilience` decorators (covered in [Caching](#) and applied the same way to outbound calls). The circuit breaker is what keeps a sick Payments service from dragging Lumen down with it:

Rust

```

use firefly::resilience::{CircuitBreaker, CircuitConfig};

// CircuitBreaker::execute returns the operation's value (Result<T, _>), so the
// guarded call still yields the Payment. (Chain::execute is for guarded ops
// whose value you discard – it returns Result<(), _>.)
let breaker = CircuitBreaker::new(CircuitConfig::default());

let payment = breaker.execute(|| async {
    payments.request::<_, Payment>(Method::POST, "/payments", Some(&req)).await
}).await?;

```

When repeated calls fail, the breaker opens and rejects subsequent calls immediately with `ResilienceError::CircuitOpen` instead of waiting on a timeout — so one slow upstream cannot exhaust Lumen's task pool. Resilience belongs at the *client* layer, configured once, not scattered through every handler.

The experience tier: a Lumen BFF

A mobile or web frontend rarely wants a single domain service's raw shape — it wants a *journey*: "show me this wallet's balance **and** its pending payments, in one call." Calling Lumen for the balance and Payments for the pending list and merging in the client means two round trips, two failure domains, and the frontend leaking knowledge of both services' internals. The **Backend-for-Frontend (BFF)** pattern moves that composition server-side.

Firefly ships a dedicated starter for this tier: `firefly-starter-experience`. It builds on `WebStack` (so it inherits CORS, security headers, request metrics, correlation, and the actuator surface) and adds the BFF building blocks:

- `DomainClients` — a registry of named `RestClient`s for the downstream domain services;

- `SignalService` — gates a long-running, signal-driven workflow step parks on until a caller delivers a named signal (the experience-tier `Workflow` from `Sagas`);
- a Redis-capable `WorkflowState` keyed by correlation id, plus a `WorkflowQueryService` for journey-status reads.

A Lumen experience service registers its downstream clients up front, then composes them:

Rust

```
use firefly_starter_experience::{ExperienceStack, DomainClients};
use firefly::starter_web::CoreConfig;

let bff = ExperienceStack::new(CoreConfig {
    app_name: "lumen-mobile-bff".into(),
    ..Default::default()
});

// Register the domain SDKs this BFF composes. `register` returns an
// Arc<RestClient> already wired with correlation + trace propagation.
let wallets = bff.clients.register("wallets", "https://lumen.internal");
let payments = bff.clients.register("payments", "https://payments.internal");
```

The composition root then fans out across the registered clients — both calls go out concurrently, so the composite latency is bounded by the slower upstream rather than their sum — and degrades gracefully if one upstream is circuit-open (show the balance, leave the pending list empty) instead of failing the whole response.

*The tier boundary is strict. The dependency direction is `channel` → `experience` → `domain` → `core`. An experience service **never** owns a database, **never** calls a core service directly, and **never** calls a sibling experience service — it composes domain SDKs only. Lumen is a domain/core-style service built on the `firefly` facade; the BFF is a separate crate that depends on*

`firefly-starter-experience` and on Lumen's published client. That separation is why the experience starter is not bundled into the one-dependency facade — a domain service does not need it.

The BFF pattern. A BFF is a thin application that aggregates several domain services into one journey-shaped API, server-side. `Mono::zip_with` (and the concurrent fan-out above) launches the upstream calls together, so the composite latency is bounded by the slower upstream rather than their sum, and a circuit-open upstream degrades gracefully instead of failing the whole response. The team-ownership model follows from the tier boundary: domain teams own stable, fine-grained contracts; the frontend team owns the BFF that adapts them.

Other protocols

The crate ships builders/scaffolds for the protocols a back-office platform needs — SOAP (CXF-style envelope), gRPC, GraphQL, and WebSocket — selected by feature so heavy dependencies stay out of services that do not use them. The REST, GraphQL, and SOAP surfaces are fully wired; the streaming protocols (gRPC and WebSocket) are feature-gated.

Outbound calls inherit the caller's correlation id automatically, so a request that fans out to three upstreams stitches together in your traces.

What changed in Lumen

- We sketched the **Payments client** Lumen would build to settle a transfer's credit leg over the network — `RestBuilder::new(...).with_retries(...).build()` — and showed how an upstream RFC 7807 problem decodes into a typed `FireflyError` via `err.as_firefly()`.

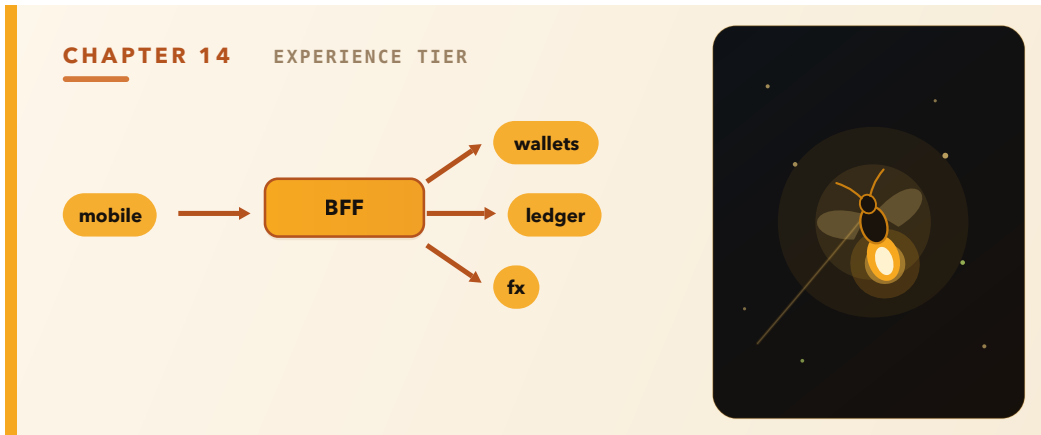
- We saw how the saga's credit step changes from a local `Ledger::deposit` to a resilient outbound call, why an `Idempotency-Key` is mandatory across retries, and why the debit's compensation matters more once I/O can fail.
- We introduced the reactive `WebClient` (`body_to_mono` / `body_to_flux` / `exchange`), its streaming decode (the consumer side of Lumen's own NDJSON/SSE endpoint), and the rule that retries are *composed* on the returned `Mono`, not baked into the client.
- We met the **experience tier** — `firefly-starter-experience` with `DomainClients`, `SignalService`, and `WorkflowState` — and saw how a Lumen BFF composes the wallet and payments services into one journey-shaped API, with the strict `experience → domain → core` dependency direction.

Exercises

1. **Decode an upstream problem.** Stand up a stub that answers `POST /payments` with a `422 application/problem+json` body. Call it through a `RestClient` and assert that `err.as_firefly()` returns `Some`, that `fe.status == 422`, and that `err.is_unprocessable_entity()` is `true` — so the saga can map the upstream rejection onto Lumen's own `422` instead of a `500`.
2. **Wrap the credit leg in a circuit breaker.** Take the transfer saga's credit step and replace `ledger.deposit(...)` with a `CircuitBreaker`-guarded `payments.request(...)`. Drive the stub to fail enough times to trip the breaker, and assert the next call returns `ResilienceError::CircuitOpen` *immediately* (no timeout) — and that the saga still compensates the debit.

3. **Compose a BFF summary.** Build a tiny `ExperienceStack`, register a `wallets` and a `payments` client against two local stubs, and write a handler that fetches the balance and the pending list concurrently. Make the payments stub return an error and assert the handler still returns the balance with an empty pending list — proving partial degradation rather than a `500`.

The next chapter secures the inbound side — JWT bearer auth and path-based RBAC on Lumen's mutating routes. Continue to [Security](#).



CHAPTER 14

The Experience Tier: Composing a BFF

Lumen, as the book has built it, is a self-contained service: it owns the wallet domain *and* the HTTP API that fronts it. Real Firefly systems split that responsibility across **three service tiers**, and Lumen sits at the bottom two. By the end of this chapter you will know where a frontend-facing API for Lumen belongs — the **experience tier** — and you will build a small Backend-for-Frontend that composes Lumen as a downstream domain SDK, drives a multi-step "fund a wallet and confirm" journey with a signal-gated workflow, and survives a client disconnect by persisting journey state.

This is the one Firefly tier Lumen itself never enters, so the chapter introduces it from first principles and then wires it against the service you already have.

***What the experience tier is.** The experience tier is Firefly's Backend-for-Frontend layer, built on the `firefly-starter-experience` crate: it composes downstream domain SDKs, drives multi-request journeys with signal-gated workflows, and persists journey state across requests. A signal gate parks a workflow step until a named signal arrives; a domain-SDK registry resolves each downstream client by logical name.*

The three service tiers

Firefly services are layered into three **service** tiers (distinct from the crate-graph tiers in the architecture docs). The dependency direction is strict and one-way: `channel` → `experience` → `domain` → `core`.

Service tier	Owns	Talks to	Rust starter
core	the database (R2DBC/sqlx, migrations, CRUD)	nothing below	<code>firefly-starter-core</code> / <code>firefly-starter-data</code>
domain	sagas, CQRS, event sourcing, third-party adapters	core SDKs	<code>firefly-starter-domain</code>
experience (BFF)	signal-driven workflows, stateless aggregation, atomic REST	domain SDKs <i>only</i>	<code>firefly-starter-experience</code>

Lumen, with its event-sourced ledger, CQRS bus, and transfer saga, is a **domain** service. An **experience** service is a Backend-for-Frontend: it aggregates several domain SDKs into APIs shaped for *one* frontend or channel. It **never** owns a database, **never** calls a core service directly, and **never** calls a sibling experience service — it composes domain SDKs (over `firefly-client`) and nothing else.

The tier boundary is a type. The strict `channel → experience → domain → core` direction is enforced by the starters themselves: an experience stack can register domain SDKs and nothing else. A BFF never owns a database and never calls a core service or a sibling BFF directly — the dependency direction is one-way by construction.

ExperienceStack – batteries for a BFF

`ExperienceStack::new(cfg)` builds a full `WebStack` (so it inherits every web battery from the [production chapter](#) — CORS, security headers, request metrics, access log, correlation, idempotency, the actuator surface) and adds the four experience-tier building blocks:

Rust

```
use firefly_starter_experience::{CoreConfig, ExperienceStack};

let bff = ExperienceStack::new(CoreConfig {
    app_name: "lumen-bff".into(),
    app_version: "1.0.0".into(),
    ..CoreConfig::default()
});
```

The stack adds five experience-tier public fields on top of the embedded `web: WebStack` (`Bff` is an alias for `ExperienceStack`):

Field	Type	Role
<code>clients</code>	<code>DomainClients</code>	the domain-SDK registry, resolved by logical name
<code>signals</code>	<code>Arc<SignalService></code>	the signal gates a workflow step parks on
<code>state</code>	<code>WorkflowState</code>	Redis-capable persisted journey state, keyed by correlation id
<code>query</code>	<code>Arc<WorkflowQueryService></code>	the journey-status query surface
<code>children</code>	<code>Arc<ChildWorkflowService></code>	child-workflow composition for nested journeys

`ExperienceStack` `Deref`s to its `WebStack` (which derefs to `Core`), so every web + core method you met building Lumen — `apply_middleware`, `actuator_router`, `new_application`, `with_security`, `cache`, `bus`, `scheduler` — is reachable directly on the BFF value. There is also a bootstrap pair, `register_experience_stack(cfg)` (`== ExperienceStack::new`) and `enable_experience_stack(cfg)` (stamps the tier defaults onto a `CoreConfig`), so you can reach the tier through whichever spelling reads more naturally at the call site.

Two ways to build the stack. `ExperienceStack::new(cfg)` and the equivalent `register_experience_stack(cfg)` both build the BFF; `enable_experience_stack(cfg)` stamps the tier defaults onto a `CoreConfig`. Because `ExperienceStack` derefs to `WebStack` (which derefs to `Core`), the BFF is a web service plus the journey machinery.

Composing domain SDKs – `DomainClients`

A BFF reaches its downstream domain services through named `RestClient` s, one per dependency. `DomainClients` is the registry; register Lumen under a logical name and resolve it by that name from any handler or workflow step:

Rust

```
// Experience -> Domain only. Register Lumen as a downstream domain SDK.
bff.clients.register("wallets", "https://lumen.internal");

// later, in a handler or workflow step:
let wallets = bff.clients.get("wallets").expect("wallets SDK");
// wallets is an Arc<RestClient> with correlation-id propagation, JSON codec,
// RFC 7807 error decoding, and retry/backoff all inherited from firefly-client.
```

`register(name, base_url)` builds a default client; `register_client(name, client)` takes a pre-tuned `RestClient` (custom timeout, headers, retry policy). Re-registering a name replaces it (last wins), and `names()` lists every registered SDK.

Resolve clients by name. `DomainClients` lets a step resolve the right downstream client by logical name instead of threading a `RestBuilder` through every call site. The clients carry the same correlation propagation and RFC 7807 decoding the [HTTP-clients chapter](#) covered.

Signal-driven journeys

A BFF journey is rarely one request. "Fund a wallet, wait for the customer to confirm the amount, then commit the transfer" is three interactions over time. A **workflow** with a **signal gate** models exactly that: steps that call domain SDKs, and a gate that parks until an atomic endpoint delivers a named signal.

The journey is a `Workflow` of `Node` s; `Node::wait_for_signal` parks on the stack's `SignalService` until the signal arrives:

Rust

```
use std::sync::Arc;
use firefly_starter_experience::{Node, Workflow};

let signals = Arc::clone(&bff.signals);
let journey_id = "j-1".to_string();

let workflow = Workflow::new("fund-and-confirm")
    // 1. reserve: call the Lumen "wallets" SDK to open/lock the funds.
    .node(Node::new("reserve", || async { Ok(()) }))
    // 2. await-confirm: park until POST /journeys/{id}/confirm delivers
    "confirmed".
    .node(
        Node::wait_for_signal("await-confirm", &signals, journey_id.clone(),
        "confirmed")
        .depends_on(["reserve"]),
    )
    // 3. commit: call the Lumen "wallets" SDK to run the transfer.
    .node(Node::new("commit", || async { Ok(()) }).depends_on(["await-
confirm"]));
```

`Workflow::run().await` executes the nodes in dependency order; when it reaches `await-confirm` it parks. An atomic endpoint later calls `bff.signals.deliver(&journey_id, "confirmed", payload)` and the parked node resumes. Delivery is **buffered** — if the signal arrives before the gate parks, there is no lost wakeup. `signals.list_active()` lists the journeys currently parked on a gate.

Signal gates. `Node::wait_for_signal(...)` parks a workflow node until `signals.deliver(...)` delivers the named signal — the external event that satisfies the gate. Delivery is buffered, so a signal that arrives before the gate parks is not lost. The workflow is the same DAG-with-compensation engine from the [sagas chapter](#) (`firefly-orchestration`), here driven by signals rather than run to completion in one call.

Persisting journey state – `WorkflowState`

Because a journey spans several requests, its state must outlive any one of them. `WorkflowState` round-trips a workflow run's `StepContext` snapshot through the stack's cache `Adapter`, keyed by correlation id. The in-memory adapter is the default; point it at `firefly-cache-redis`'s `RedisAdapter` for cross-restart durability — the convention the experience tier is built around.

Rust

```
use firefly_starter_experience::CoreConfig;
use firefly_orchestration::StepContext;

// Save when a journey parks:
let ctx = StepContext::new();
ctx.set_correlation_id("j-1");
ctx.set_variable("phase", serde_json::json!("AWAITING_CONFIRM"));
bff.state.save(&ctx).await?;

// Rehydrate from a later request to advance it:
if let Some(ctx) = bff.state.load("j-1").await? {
    // ... advance the journey ...
}

// Discard when the journey completes:
bff.state.delete("j-1").await?;
```

A miss on an unknown journey is `Ok(None)`, not an error — so a status check on a journey that never existed is a clean 404, not a 500.

Durable journey state. `WorkflowState` persists a workflow run's `StepContext` through the `cache Adapter`, keyed by correlation id — point it at Redis for cross-restart durability, the Firefly convention of "hold workflow state in Redis." A parked journey saves its `StepContext`; a later request loads it and resumes, surviving the client disconnect an in-memory waiter would not.

Querying journey status – `WorkflowQueryService`

The frontend polls "where is my journey?" while it waits. `WorkflowQueryService` holds the live `StepContext` per run and answers named queries against it — the main recovery mechanism when a client reconnects:

Rust

```
bff.query.register(&journey_id, ctx.clone());           // on start
bff.query.register_query(&journey_id, "phase", |ctx| { // a named query
    ctx.variable("phase").unwrap_or(serde_json::json!("UNKNOWN"))
});
let phase = bff.query.query(&journey_id, "phase"?);    // GET /journeys/{id}
bff.query.unregister(&journey_id);                     // on completion
```

The atomic-endpoint contract

Put together, an experience controller exposes one request per journey phase — the atomic REST shape:

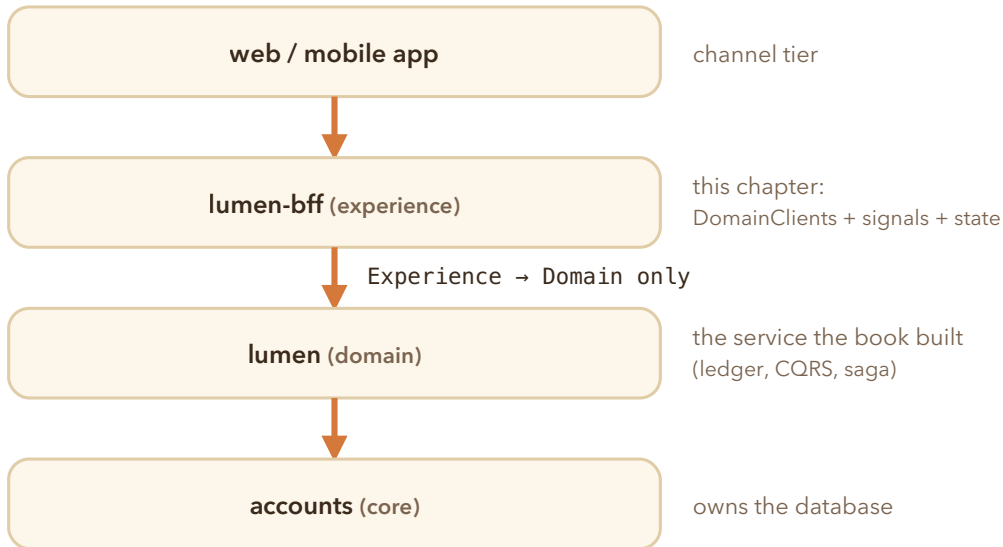
Method & path	Does
<code>POST /journeys</code>	start the workflow (calls the "wallets" SDK to reserve), persist <code>WorkflowState</code> , park on the gate, return the journey id
<code>POST /journeys/:id/confirm</code>	deliver the <code>confirmed</code> signal — the parked workflow resumes and commits the transfer via the "wallets" SDK
<code>GET /journeys/:id</code>	report the persisted phase (or 404 if unknown)

Each phase is one HTTP request; state lives in the cache (Redis-capable), so a client can resume a journey after a disconnect. Because the controller runs through `bff.apply_middleware(routes)`, every response inherits the web batteries — the start response carries `X-Frame-Options: DENY` and a correlation id just as Lumen's own responses do, and the same `ExperienceStack::with_security(chain)` filter chain from the [security chapter](#) guards the mutating routes.

This is exactly the contract the crate's own boot test proves end to end (its checkout journey uses an `AWAITING_PAYMENT` phase rather than Lumen's `AWAITING_CONFIRM`, but the shape is identical): two mock domain SDKs composed through a signal-gated workflow, driven with `tower::oneshot` — start parks on the gate, the status endpoint reports the persisted phase, delivering the signal advances the workflow off-task, and the final status flips to `COMPLETED`.

Where Lumen fits

Drawn as the full estate, Lumen is one of the domain services a BFF composes:



Lumen as one service in the estate. Calls flow strictly downward; the BFF on the experience tier composes Lumen on the domain tier, which owns its logic, while the core service owns the database.

The BFF never reaches into Lumen's event store or bus — it speaks to Lumen's public HTTP API through the registered `"wallets"` SDK, exactly as any external client would, and adds only the journey orchestration a frontend needs.

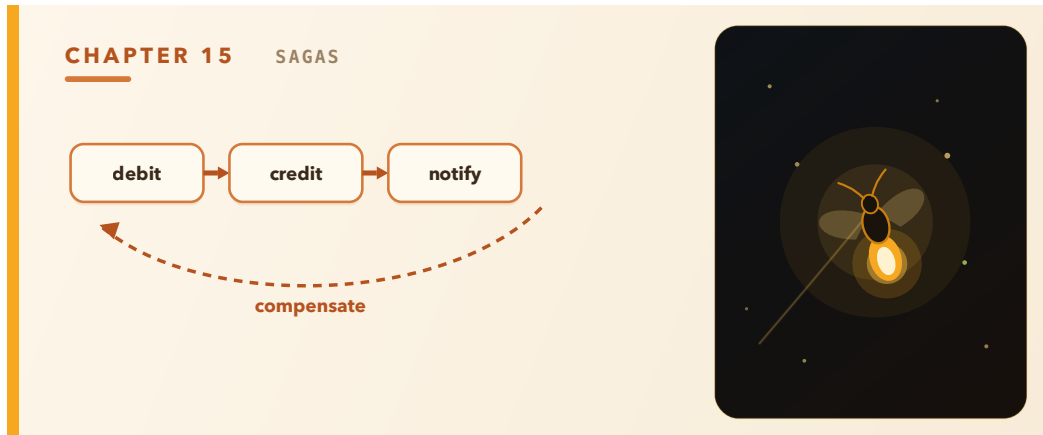
What changed in Lumen

Lumen itself is unchanged — it is a *domain* service, and this chapter is about the tier *above* it. What you built is the mental model and the wiring for a frontend-facing BFF: an `ExperienceStack` inheriting Lumen's web batteries, `DomainClients` registering Lumen as the `"wallets"` SDK (Experience → Domain only), a signal-gated `Workflow` modeling a multi-request "fund and confirm" journey, `WorkflowState` persisting that journey through a Redis-capable cache adapter, and `WorkflowQueryService` answering status polls — every API drawn from the real `firefly-starter-experience` surface.

Exercises

1. **Register Lumen as a domain SDK.** Build an `ExperienceStack`, call `bff.clients.register("wallets", "http://localhost:8080")`, and confirm `bff.clients.get("wallets")` returns a client and `bff.clients.names()` lists it.
2. **Park and resume.** Build a two-node workflow whose second node is a `Node::wait_for_signal` gate. Run it on a task, assert `bff.signals.list_active()` contains the journey id, then `deliver` the signal and confirm the workflow completes.
3. **Persist a journey.** Save a `StepContext` with a `phase` variable via `bff.state.save`, `load` it back from a fresh handle, and confirm the variable survives. Then `delete` it and confirm `load` returns `Ok(None)`.
4. **Atomic endpoints.** Wire the three-route controller above with `bff.apply_middleware(routes)` and drive it with `tower::oneshot`: start, poll status (`AWAITING_CONFIRM`), confirm, poll again (`COMPLETED`). Assert the start response carries the inherited `X-Frame-Options` header.

With Lumen in production (the previous chapter) and its place in the tiered estate clear, the capstone re-reads the whole service through the declarative-macro lens. Continue to [Declarative Services with Macros](#).



CHAPTER 15

Distributed Transactions: Sagas, Workflows & TCC

By the end of this chapter, Lumen can **move money between two wallets** — and do it *safely*. A transfer is not a single command: it debits one wallet, then credits another, and those are two independent writes to two independent event streams. If the credit leg fails after the debit already committed, the source owner is out of pocket with nothing on the other side. There is no **BEGIN ... COMMIT** that spans two aggregates, so Lumen reaches for the pattern that does the job: a **saga** that compensates the debit when the credit fails.

We build the **POST /api/v1/transfers** endpoint on top of the **Ledger** the CQRS handlers already use, so a transfer raises *real* **MoneyWithdrawn** / **MoneyDeposited** events on both streams — and a refund raises a real **MoneyDeposited** on the source stream. Everything stays observable on the event-sourced ledger you built in chapter 11.

`firefly-orchestration` ships the three classic **distributed-transaction engines** every Firefly platform agrees on. Each composes async steps, runs as a plain future on the caller's task, applies a per-step retry policy, threads a typed context blackboard, and respects cooperative cancellation. And — this is the change since the last edition — you no longer hand-build them as values. Lumen declares each engine with an attribute macro on an `impl` block, exactly as it declares CQRS handlers and controllers.

Engine	Topology	Compensation	Declared with
<code>Saga</code>	Dependency-ordered steps	Reverse-order, configurable policy	<code>#[saga] + #[saga_step]</code>
<code>Workflow</code>	DAG with parallel layers	Reverse-order, configurable policy	<code>#[workflow] + #[workflow_step]</code>
<code>Tcc</code>	Try-all then Confirm-all	Cancel-tried on a Try failure	<code>#[tcc] + #[participant]</code>

Design note. Firefly's orchestration model is declarative. You write an ordinary `impl` block of `async fn(&self, ...) -> Result<T, E>` methods and annotate them: `#[saga_step]` for a saga leg, `#[workflow_step]` for a DAG node, `#[participant]` for a TCC actor. The macro lowers those methods onto the same `firefly-orchestration` engines — `depends_on` orders them, `compensate` names the undo, a step's `Ok(T)` is published for later steps, and an `Err(E)` triggers compensation in reverse order. If you've used Java's `@Saga` or pyfly's saga decorators, this is the Rust spelling: the control flow lives in methods you can read top to bottom, and the wiring is generated for you.

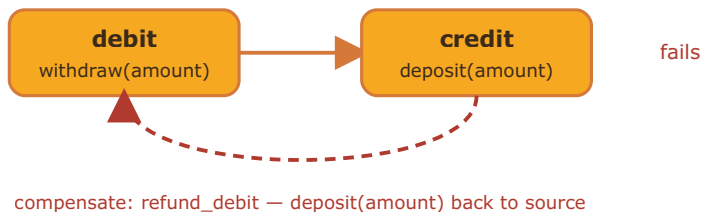
The problem with distributed writes

Make the failure modes concrete before writing a line of code. A Lumen transfer has two legs:

1. **Debit the source** — `withdraw(amount)`, which enforces `balance >= 0`.
2. **Credit the destination** — `deposit(amount)`.

Each leg is an independent `Ledger` call that appends to that wallet's own event stream. A missing destination wallet, or an overdraft on the source, fails one leg after the other may already have committed. Retrying the *whole* operation is unsafe — you might debit twice. Silently skipping the failed leg leaves the balances inconsistent.

The principled answer is **eventual consistency with explicit compensation**. Each leg commits to its own stream independently, and you design a recovery path — a *compensating transaction* — for every step that could succeed before a later one fails. A compensation is not a database rollback; it is a *semantic undo*: "re-credit the source" is a brand-new `deposit` that restores the balance, and it leaves an auditable refund event behind.



The transfer saga: `debit` then `credit`. A failed credit runs the debit's compensation in reverse, refunding the source.

Saga – dependency-ordered steps with compensation

A `Saga` runs steps in dependency order; on any step failure it compensates the completed steps in reverse order. You declare it with `#[firefly::saga]` on an `impl` block, marking each leg with `#[saga_step]`:

Rust

```
#[firefly::saga(name = "money-transfer", policy = "stop_on_error")]
impl TransferSaga {
    #[saga_step(id = "reserve", compensate = "refund")]
    async fn reserve(&self, #[input] req: TransferReq) -> Result<Reserved,
MyErr> { /* ... */ }

    async fn refund(&self, #[from_step("reserve")] r: Reserved) -> Result<(),
MyErr> { /* ... */ }

    #[saga_step(id = "credit", depends_on =
["reserve"], retry = 3, backoff_ms = 100)]
    async fn credit(&self, #[from_step("reserve")] r: Reserved) -> Result<(),
MyErr> { /* ... */ }
}
```

The macro generates two methods on the type:

- `TransferSaga::saga(self: Arc<Self>) -> Saga` — builds the engine from your steps, their `depends_on` order, compensations, and retry policies.
- `TransferSaga::run(self: Arc<Self>, input) -> Result<Outcome, SagaFailure>` — serialises `input` into the step context and runs the whole DAG, compensating on failure.

Each step is an `async fn(&self, ...) -> Result<T, E>`. Its parameters are injected from the saga context with markers the macro reads and strips:

- `#[input]` (the whole input) or `#[input("field")]` (one field of it);

- `#[from_step("id")]` — the `Ok` value an earlier step published;
- `#[variable("key")]` — a saga-scoped context variable;
- `#[ctx]` — the `StepContext` blackboard itself.

A step's `Ok(T)` is serialised and made available to later steps via `#[from_step]`; an `Err(E)` (where `E: std::error::Error + Send + Sync`) triggers compensation in reverse order. `#[saga_step]` accepts `id` (required), `depends_on = ["..."]`, `compensate = "method"`, and the per-step recovery knobs `retry`, `backoff_ms`, `timeout_ms`, and `jitter`. `#[saga(...)]` accepts a `name`, a `crate` facade override, and a compensation `policy`:

- `best_effort` (the engine default) — log and continue compensating the remaining steps even if one compensation fails.
- `stop_on_error` — abort rollback at the first compensation failure and surface a `SagaError::Compensation` wrapping the original.
- plus `retry_with_backoff`, `circuit_breaker`, `best_effort_parallel`, and `grouped_parallel` for larger fan-outs.

SPRING PARITY

This is the same shape as Java's `@Saga` / `@SagaStep` and pyfly's saga decorators — a step method, a compensation named by string, a `depends_on` order — but lowered onto Rust's type system, so a step that returns the wrong type or names a compensation that does not exist is a *compile error*, not a runtime surprise.

NOTE

the macro lowers onto a lower-level programmatic builder. If you ever need to construct a saga dynamically (steps known only at run time), the same engine exposes `Saga::new(name).step(Step::with_context(id, action).with_context_compensation(undo))`. The declarative `#[saga]` form is the way you'll write them in practice; the builder is the seam it expands onto.

Lumen's transfer saga

Lumen's transfer is a two-step saga: `debit` the source, then `credit` the destination. The debit's compensation refunds the source; the credit is the last leg, so it needs no compensation — a failure there rolls back only the debit. The whole thing lives in `src/transfer.rs`, and every leg drives the same `Ledger` the CQRS handlers use.

First, the wire types — the request body and the result `POST /api/v1/transfers` returns:

Rust

```

use serde::{Deserialize, Serialize};

/// `POST /api/v1/transfers` command – move `amount` (cents) from `from` to `to`.
#[derive(Debug, Clone, Default, PartialEq, Eq, Serialize, Deserialize)]
#[serde(default)]
pub struct TransferRequest {
    pub from: String,
    pub to: String,
    /// The amount to move, in minor units (cents); must be `> 0`.
    pub amount: i64,
}

/// The result of a completed (or compensated) transfer.
#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize)]
pub struct TransferResult {
    /// `"completed"` when both legs succeeded – the lowercase `SagaStatus`.
    pub status: String,
    pub from: String,
    pub to: String,
    pub amount: i64,
    #[serde(rename = "stepsExecuted")]
    pub steps_executed: Vec<String>,
    #[serde(rename = "stepsRolledBack")]
    pub steps_rolled_back: Vec<String>,
}

```

The result echoes the `SagaStatus` as a lowercase string plus the two step lists, so the API tells the caller *exactly* what the engine did — which steps ran and which were rolled back. A transfer also has a typed error that distinguishes a malformed request from a clean, compensated business failure:

Rust

```

/// The typed error a transfer surfaces to its caller.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum TransferError {
    /// The request was malformed (same wallet, non-positive amount).
    Invalid(String),
    /// The transfer failed and was rolled back; the inner string is the
    /// failing leg's domain error (e.g. `insufficient funds`).
    Compensated(String),
}

impl std::fmt::Display for TransferError {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        match self {
            TransferError::Invalid(detail) => f.write_str(detail),
            TransferError::Compensated(detail) => write!(f, "transfer rolled
back: {detail}"),
        }
    }
}

impl std::error::Error for TransferError {}

```

One dependency, even here. `TransferError` hand-writes `Display` + `std::error::Error` instead of pulling in `thiserror` — the same discipline the rest of Lumen keeps, so the whole framework and every typed error still arrive through the single `firefly` facade.

Declaring the saga

The saga is an `impl` block on a tiny struct that holds the `Ledger`. Each leg is an annotated method that calls the ledger directly and returns a typed `Result<(), DomainError>`. There is no closure capture, no `Mutex` to smuggle the cause out of an erased error channel, no builder call — the macro reads the attributes and generates all of it:

Rust

```

use std::sync::Arc;

use firefly::orchestration::SagaError;

use crate::domain::DomainError;
use crate::ledger::Ledger;
use crate::money::Money;

/// The money-transfer saga, declared with `#[firefly::saga]`: each leg is an
/// annotated method driving the `Ledger`. The macro generates
/// `TransferSaga::run` (used by `run_transfer`) and `TransferSaga::saga`.
struct TransferSaga {
    ledger: Ledger,
}

#[firefly::saga(name = "money-transfer")]
impl TransferSaga {
    /// Debit the source wallet (a real `MoneyWithdrawn` event). Rolled back by
/// `refund_debit` when a later leg fails.
    #[saga_step(id = "debit", compensate = "refund_debit")]
    async fn debit(&self, #[input] req: TransferRequest) -> Result<(),
DomainError> {
        self.ledger.withdraw(&req.from, Money::cents(req.amount)).await?;
        Ok(())
    }

    /// Compensation for `debit`: a refund is a normal deposit, so it raises a
/// real `MoneyDeposited` event on the source stream.
    async fn refund_debit(&self, #[input] req: TransferRequest) -> Result<(),
DomainError> {
        self.ledger.deposit(&req.from, Money::cents(req.amount)).await?;
        Ok(())
    }

    /// Credit the destination (a real `MoneyDeposited` event). The last leg, so
/// a failure here rolls back only the debit.
    #[saga_step(id = "credit", depends_on = ["debit"])]
    async fn credit(&self, #[input] req: TransferRequest) -> Result<(),

```

```
DomainError> {
    self.ledger.deposit(&req.to, Money::cents(req.amount)).await?;
    Ok(())
}
}
```

How it reads. `debit` is the first step (`id = "debit"`), and it names its undo with `compensate = "refund_debit"`. `refund_debit` carries no `#[saga_step]` marker — it is a plain method referenced by name, and the macro includes it in the generated saga only because `debit` points at it. `credit` declares `depends_on = ["debit"]`, so the engine runs it strictly after the debit; it has no compensation because it is the last leg, so the only thing to undo on its failure is the debit, which the engine handles automatically.

Each leg takes `#[input] req: TransferRequest`, so the macro deserialises the saga's input (the request) into that parameter on every step and compensation. Because the methods return `DomainError` directly, the typed failure cause is preserved all the way through the engine — no shared `Mutex` needed.

Running the saga

`run_transfer` validates the request, constructs the saga value behind an `Arc`, and calls the generated `run`. On success it reads the `Outcome`; on failure it unwraps the failing leg's typed `DomainError` out of the engine's `SagaError::Step` so the API can answer `insufficient funds` verbatim:

Rust

```

/// Validates and runs a money transfer as a declarative saga, returning the
/// terminal `TransferResult`.
pub async fn run_transfer(
    ledger: &Ledger,
    req: &TransferRequest,
) -> Result<TransferResult, TransferError> {
    if req.amount <= 0 {
        return Err(TransferError::Invalid("amount must be > 0".into()));
    }
    if req.from == req.to {
        return Err(TransferError::Invalid("from and to must differ".into()));
    }

    let saga = Arc::new(TransferSaga {
        ledger: ledger.clone(),
    });
    match saga.run(req.clone()).await {
        Ok(outcome) => Ok(TransferResult {
            status: outcome.status.to_string(),
            from: req.from.clone(),
            to: req.to.clone(),
            amount: req.amount,
            steps_executed: outcome.steps_executed,
            steps_rolled_back: outcome.steps_rolled,
        }),
        Err(failure) => {
            // Surface the failing leg's typed domain error (e.g. "insufficient
            // funds"), unwrapped from the saga's generic step error.
            let detail = match failure.error() {
                SagaError::Step { source, .. } => source.to_string(),
                other => other.to_string(),
            };
            Err(TransferError::Compensated(detail))
        }
    }
}

```


What the generated `run` does for you. `saga.run(req.clone())` is the macro's `TransferSaga::run(self: Arc<Self>, input)`: it serialises `req` into a fresh `StepContext`, builds the saga (`debit` → `credit`, with the debit's compensation attached), and runs the DAG. On the happy path it returns an `Outcome` whose `status` is `Completed`, `steps_executed` lists the legs that ran, and `steps_rolled` is empty. On failure it returns a `SagaFailure`: its `outcome()` is fully populated (status `Compensated`, with `steps_rolled` naming the compensations that ran), and its `error()` is a `SagaError`. We match `SagaError::Step { source, .. }` to recover the leg's `DomainError` message — that is how `POST /api/v1/transfers` answers `insufficient funds` instead of an opaque "step \"credit\" failed".

***Design note.** Nothing here is reflection or runtime scanning. `#[saga]` expands at compile time into the exact `Saga::new(...).step(...)` calls you would otherwise write by hand, threaded through the `firefly` facade's `__rt` contract so a one-dependency service compiles it without ever naming `firefly-orchestration`. A step that returns the wrong type, or a `compensate` that points at a method that doesn't exist, fails the build with a message pointing at the offending line.*

The endpoint

The controller method in `src/web.rs` is thin: drive the saga, then translate the typed outcome into the HTTP contract. A clean rollback is a *business* failure, so it surfaces as a `422` problem carrying the cause — not a `500`:

Rust

```

/// `POST /api/v1/transfers` – run a money transfer as a saga.
#[post("/transfers")]
async fn transfer(
    State(api): State<WalletApi>,
    Json(body): Json<TransferRequest>,
) -> WebResult<Json<TransferResult>> {
    let result = run_transfer(&api.ledger, &body)
        .await
        .map_err(|e| match e {
            TransferError::Invalid(detail) =>
                WebError::from(FireflyError::validation(detail)),
            TransferError::Compensated(detail) => {
                WebError::from(FireflyError::validation(detail))
            }
        })?;
    // A transfer touches both wallets' views; invalidate the family.
    api.query_cache.invalidate_type::<GetWallet>();
    Ok(Json(result))
}

```

Note the `invalidate_type::<GetWallet>()` at the end: a transfer changed two balances, so the cached `GetWallet` views must be dropped to keep a read after the write honest. That cache and its invalidation are the subject of [Caching](#); the transfer is just one more mutation that has to play by its rules.

What the saga does on each path

The tests in `src/transfer.rs` exercise all three paths, and they are the best documentation of the behavior. The happy path moves funds and rolls back nothing:

Rust

```

let result = run_transfer(
    &ledger,
    &TransferRequest { from: src.id.clone(), to: dst.id.clone(), amount: 300 },
)
.await
.unwrap();

assert_eq!(result.status, "completed");
assert_eq!(result.steps_executed, ["debit", "credit"]);
assert!(result.steps_rolled_back.is_empty());
assert_eq!(balance(&ledger, &src.id).await, 700);
assert_eq!(balance(&ledger, &dst.id).await, 300);

```

The **overdraft** path short-circuits at the debit — the source never has the funds, so the withdraw fails *before* anything applies. There is nothing to compensate, and both balances are left intact:

Rust

```

let err = run_transfer(
    &ledger,
    &TransferRequest { from: src.id.clone(), to: dst.id.clone(), amount: 500 },
)
.await
.unwrap_err();

assert_eq!(err, TransferError::Compensated("insufficient funds".into()));
assert_eq!(balance(&ledger, &src.id).await, 100); // untouched
assert_eq!(balance(&ledger, &dst.id).await, 0);   // untouched

```

The **credit-failure** path is where compensation earns its keep. The debit applied, then the credit failed (the destination does not exist), so the engine runs the debit's compensation — a refund deposit. The source's net balance is restored, and the stream records *both* the debit and its refund, an audit trail of exactly what happened:

Rust

```

let err = run_transfer(
    &ledger,
    &TransferRequest { from: src.id.clone(), to: "wlt_missing".into(), amount:
400 },
)
.await
.unwrap_err();
assert!(matches!(err, TransferError::Compensated(_)));

// open(1000) - withdraw(400) + refund(400) = 1000, with 3 events on the stream.
let src_events = ledger.load_events(&src.id).await.unwrap();
assert_eq!(Wallet::rehydrate(&src.id, &src_events).view().balance, 1_000);
assert_eq!(src_events.len(), 3); // open + withdraw + refund-deposit

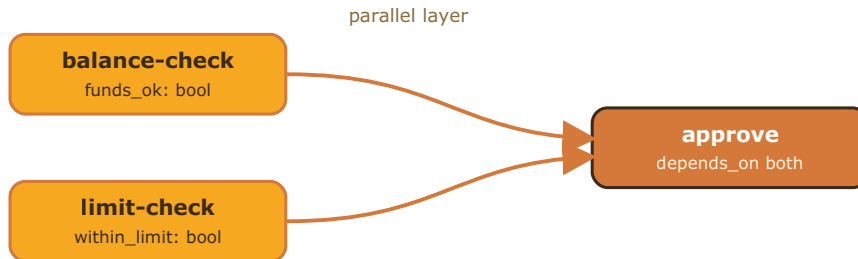
```

***Sagas are eventually consistent.** A saga does not give you serializability. Between the moment the source is debited and the moment the credit commits (or the refund runs), another request could read the source and see a balance lower than it will ultimately be. That is the trade-off for operating across independent aggregates without a distributed lock: consistency in the end — all legs committed, or all compensated — not at every instant.*

Workflow – a DAG with parallel layers and conditions

When a process has *independent* steps that should run in parallel, reach for a **Workflow**: a DAG of nodes with **depends_on** declarations. Independent nodes run concurrently within a topological layer, and a node that declares dependencies runs only after they complete. You declare it with **#[firefly::workflow]** and mark each node with **#[workflow_step]** — the same parameter injection as a saga.

`#[workflow_step]` accepts `id` (required), `depends_on = ["..."]`, `compensate = "method"`, `when = "expr"` (a skip condition — the node is skipped when the predicate is false), and `fire_and_forget` (schedule the node without blocking the layer). The macro generates `Workflow::workflow(self: Arc<Self>)` and `run(self, input) -> Result<(), WorkflowError>`.



The compliance workflow: `balance-check` and `limit-check` have no dependency on each other, so they run in the same layer; `approve` waits for both and consumes their verdicts.

Lumen's compliance workflow

A large transfer should be gated behind compliance checks *before* the money moves. Lumen's `src/compliance.rs` runs two independent checks in parallel and then an approval gate that consumes both. `balance-check` and `limit-check` have no dependency on each other, so the engine runs them in the same topological layer; `approve` declares `depends_on` on both and reads their boolean verdicts through `#[from_step(...)]`:

Rust

```

use std::sync::Arc;

use firefly::orchestration::WorkflowError;

use crate::domain::Wallet;
use crate::ledger::Ledger;
use crate::transfer::TransferRequest;

/// The per-transfer ceiling, in minor units (cents).
pub const MAX_TRANSFER_CENTS: i64 = 1_000_000; // 10,000.00

/// The compliance workflow: each node drives the `Ledger` or a policy input.
struct ComplianceCheck {
    ledger: Ledger,
    max_cents: i64,
}

#[firefly::workflow(name = "transfer-compliance")]
impl ComplianceCheck {
    /// Does the source wallet hold enough to cover the transfer? Reads the real
    /// source aggregate. Errors if the source does not exist.
    #[workflow_step(id = "balance-check")]
    async fn balance_check(&self, #[input] req: TransferRequest) ->
    Result<bool, ComplianceError> {
        let events = self
            .ledger
            .load_events(&req.from)
            .await
            .map_err(|e| ComplianceError::NotFound(e.to_string()))?;
        if events.is_empty() {
            return Err(ComplianceError::NotFound(req.from.clone()));
        }
        let balance = Wallet::rehydrate(&req.from, &events).view().balance;
        Ok(balance >= req.amount)
    }

    /// Is the amount within the per-transfer ceiling? Independent of the
    /// balance check, so it runs in the same parallel layer.

```

```

#[workflow_step(id = "limit-check")]
async fn limit_check(&self, #[input] req: TransferRequest) -> Result<bool,
ComplianceError> {
    Ok(req.amount <= self.max_cents)
}

/// The decision node: runs only after both checks (`depends_on`) and
/// consumes their boolean verdicts via `#[from_step]`.
#[workflow_step(id = "approve", depends_on = ["balance-check", "limit-
check"])]
async fn approve(
    &self,
    #[from_step("balance-check")] funds_ok: bool,
    #[from_step("limit-check")] within_limit: bool,
) -> Result<(), ComplianceError> {
    if !funds_ok {
        return Err(ComplianceError::Rejected("insufficient funds".into()));
    }
    if !within_limit {
        return Err(ComplianceError::Rejected(format!(
            "amount exceeds the {} cent per-transfer ceiling",
            self.max_cents
        )));
    }
    Ok(())
}
}

```

This is the payoff of the injection model. `balance_check` returns `Ok(true)` or `Ok(false)`; the engine serialises that `bool` under the node id `balance-check`. `approve` declares `#[from_step("balance-check")] funds_ok: bool`, and the macro deserialises the stored value back into that parameter — typed both ends, with no manual context plumbing. `balance-check` reads the *real* source aggregate from the `Ledger`; only the per-transfer ceiling is a new policy input.

`run_compliance` builds the workflow behind an `Arc` and calls the generated `run`. `Ok(())` means approved; an `Err` is recovered into a typed `ComplianceError`. The workflow engine surfaces a node failure as `WorkflowError::Node { source, .. }`, where the boxed `source` can be downcast back to the original error:

Rust

/// Runs the compliance workflow for `req`. `Ok()` means the transfer is approved (both checks passed); `Err` carries the typed reason it was rejected.

```
pub async fn run_compliance(
    ledger: &Ledger,
    req: &TransferRequest,
) -> Result<(), ComplianceError> {
    let check = Arc::new(ComplianceCheck {
        ledger: ledger.clone(),
        max_cents: MAX_TRANSFER_CENTS,
    });
    match check.run(req.clone()).await {
        Ok(()) => Ok(()),
        Err(failure) => Err(compliance_cause(failure)),
    }
}
```

/// Recovers a typed `ComplianceError` from the failing node's error.

```
fn compliance_cause(failure: WorkflowError) -> ComplianceError {
    let detail = match &failure {
        WorkflowError::Node { source, .. } => {
            if let Some(err) = source.downcast_ref:::<ComplianceError>() {
                return err.clone();
            }
            source.to_string()
        }
        other => other.to_string(),
    };
    if detail.contains("not found") {
        ComplianceError::NotFound(detail)
    } else {
        ComplianceError::Rejected(detail)
    }
}
```

The endpoint in `src/web.rs` is a read-only pre-check that never moves funds — `200 OK` with the decision when approved, `404` when the source wallet is unknown, and `422` carrying the reason when a compliance check rejects:

Rust

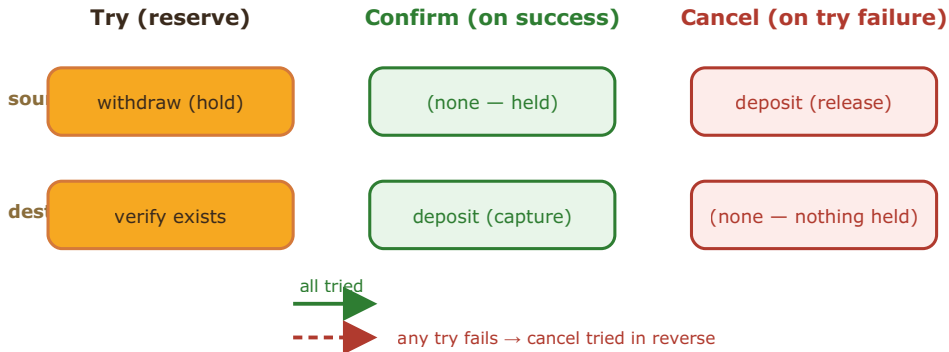
```
/// `POST /api/v1/transfers/compliance` – gate a transfer through the parallel
/// compliance workflow (balance + limit checks → approve).
#[post("/transfers/compliance")]
async fn transfer_compliance(
    State(api): State<WalletApi>,
    Json(body): Json<TransferRequest>,
) -> WebResult<Json<serde_json::Value>> {
    run_compliance(&api.ledger, &body).await.map_err(|e| match e {
        // An unknown source wallet is a 404 (like GET /wallets/:id); a
        // failed check is a 422.
        ComplianceError::NotFound(detail) =>
        WebError::from(FireflyError::not_found(detail)),
        ComplianceError::Rejected(detail) =>
        WebError::from(FireflyError::validation(detail)),
    })?;
    Ok(Json(serde_json::json!({
        "decision": "approved",
        "from": body.from,
        "to": body.to,
        "amount": body.amount,
    }))))
}
```

Where Lumen would grow this. The experience-tier starter, `firefly-starter-experience`, builds on exactly this workflow engine with signal-driven steps that park until an external caller delivers a named signal, then resume from where they left off. We return to that tier in [HTTP Clients](#).

TCC – reserve all, then confirm all or cancel all

Tcc runs a two-phase protocol: **Try** every participant (reserve resources), then **Confirm** all on success; on any Try failure, **Cancel** the participants already tried, in reverse order. Where a saga applies each leg immediately and *undoes* a committed leg on failure, TCC reserves first and only commits once every reservation succeeded — so a failed reservation is cancelled, never compensated after the fact.

You declare it with `#[firefly::tcc]` and mark each *try* method with `#[participant(name, confirm, cancel)]`. The confirm and cancel methods are plain `async fn` referenced by name. A participant's try result is published under its name, so confirm and cancel can read it via `#[from_step("<name>")]`. `#[participant]` accepts `name` and `confirm` (required), plus `cancel`, `retry`, `backoff_ms`, and `timeout_ms`. The macro generates `Tcc::tcc(self: Arc<Self>)` and `run(self, input) -> Result<(), TccError>`.



The two-phase transfer: Try holds on the source and verifies the destination; Confirm captures on the destination; a failed Try cancels by releasing the source hold.

Lumen's two-phase transfer

Lumen's `src/tcc_transfer.rs` models the transfer as a reserve-then-capture. The source's try *holds* the funds by debiting now; its confirm is a no-op (the debit already captured), and its cancel releases the hold with a refund. The destination's try *verifies* it exists (nothing committed yet, so no cancel); its confirm captures by crediting:

Rust

```

use std::sync::Arc;

use firefly::orchestration::TccError;

use crate::domain::DomainError;
use crate::ledger::Ledger;
use crate::money::Money;
use crate::transfer::{TransferError, TransferRequest};

/// The two-phase transfer coordinator: each participant drives the `Ledger`.
struct TwoPhaseTransfer {
    ledger: Ledger,
}

#[firefly::tcc(name = "transfer-2pc")]
impl TwoPhaseTransfer {
    /// Source **try**: hold the funds by debiting now (a real
    `MoneyWithdrawn`).
    #[participant(name = "source", confirm = "capture_source", cancel =
    "release_source")]
    async fn hold_source(&self, #[input] req: TransferRequest) -> Result<(),
    DomainError> {
        self.ledger
            .withdraw(&req.from, Money::cents(req.amount))
            .await?;
        Ok(())
    }

    /// Source **confirm**: the debit on try already captured the funds.
    async fn capture_source(&self) -> Result<(), DomainError> {
        Ok(())
    }

    /// Source **cancel**: release the hold by refunding it (a real
    `MoneyDeposited`).
    async fn release_source(&self, #[input] req: TransferRequest) -> Result<(),
    DomainError> {
        self.ledger
            .deposit(&req.from, Money::cents(req.amount))
            .await?;
    }
}

```

```

    Ok(())
}

/// Destination **try**: pre-authorize by verifying the destination exists;
/// nothing is committed yet, so there is no cancel.
#[participant(name = "dest", confirm = "capture_dest")]
async fn hold_dest(&self, #[input] req: TransferRequest) -> Result<(),
DomainError> {
    let events = self.ledger.load_events(&req.to).await?;
    if events.is_empty() {
        return Err(DomainError::NotFound(req.to.clone()));
    }
    Ok(())
}

/// Destination **confirm**: capture by crediting the destination.
async fn capture_dest(&self, #[input] req: TransferRequest) -> Result<(),
DomainError> {
    self.ledger.deposit(&req.to, Money::cents(req.amount)).await?;
    Ok(())
}
}

```

The `source` participant names all three phases — `confirm = "capture_source"`, `cancel = "release_source"` — while `dest` omits `cancel` because its try holds nothing. The `capture_source` confirm takes only `&self`: a participant method with no injected parameters is valid, and a no-op confirm is exactly the right shape when the try already captured.

`run_tcc_transfer` builds the coordinator behind an `Arc` and runs it. On success both sides captured (`status: "confirmed"`); on any reservation failure the tried participants are cancelled and the failing phase's cause is rendered out of `TccError`:

Rust

```
/// Validates and runs a two-phase transfer. On success both sides captured
/// (`status: "confirmed"`); on any reservation failure the tried participants
/// are cancelled (the source hold released) and this returns
/// `TransferError::Compensated` with the cause.
```

```
pub async fn run_tcc_transfer(
    ledger: &Ledger,
    req: &TransferRequest,
) -> Result<TccTransferResult, TransferError> {
    if req.amount <= 0 {
        return Err(TransferError::Invalid("amount must be > 0".into()));
    }
    if req.from == req.to {
        return Err(TransferError::Invalid("from and to must differ".into()));
    }
    let tcc = Arc::new(TwoPhaseTransfer {
        ledger: ledger.clone(),
    });
    match tcc.run(req.clone()).await {
        Ok(()) => Ok(TccTransferResult {
            status: "confirmed".into(),
            from: req.from.clone(),
            to: req.to.clone(),
            amount: req.amount,
        }),
        Err(err) => Err(TransferError::Compensated(tcc_cause(err))),
    }
}
```

```
/// Renders the failing phase's cause for the caller.
```

```
fn tcc_cause(err: TccError) -> String {
    match err {
        TccError::Try { source, .. } => source.to_string(),
        TccError::Confirm(errors) => errors
            .into_iter()
            .map(|e| e.to_string())
            .collect::<Vec<_>>()
            .join("; "),
    }
}
```

```

    }
}

```

`TccError::Try { source, .. }` carries the reservation that failed (e.g. an overdrawn source or a missing destination); `TccError::Confirm(errors)` collects the confirm-phase failures if a capture somehow fails after every reservation succeeded. The endpoint mirrors the saga's: `200 OK` with the confirmed result, or `422` when a reservation failed and the source hold was released.

The tests pin the two-phase semantics. A transfer to a missing destination *holds then releases* the source, leaving it untouched:

Rust

```

let err = run_tcc_transfer(
    &ledger,
    &TransferRequest { from: src.id.clone(), to: "wlt_missing".into(), amount:
400 },
)
.await
.unwrap_err();
assert!(matches!(err, TransferError::Compensated(_)));
// Source try held the funds, then the dest try failed → source cancel
// released them: the hold + its release net to the original balance.
assert_eq!(balance(&ledger, &src.id).await, 1_000);

```

TCC vs. Saga. Lumen ships both framings of the same transfer so you can compare them. The saga applies each leg locally and refunds the debit if the credit fails — simplest when an undo is itself a clean local action. The TCC reserves on both sides, then commits or releases together — better when a participant can cheaply hold a reservation and you want all-or-nothing semantics with no window where one side has committed and the other has not.

Cancellation

All three engines respect a `CancellationToken` for cooperative cancellation. The engines depend only on `futures`, so any executor (Tokio included) drives them. When you need it, the lower-level builder API exposes `run_cancellable(&token)`; cancel the token from a timeout or a shutdown signal to drain the run.

Choosing an engine

Need	Engine
Dependency-ordered process, undo on failure	<code>Saga</code>
Parallel branches that join	<code>Workflow</code>
Reserve-then-commit across resources	<code>Tcc</code>

Lumen's money-transfer is a `Saga` (debit → credit, with a debit refund). Its pre-flight compliance gate is a `Workflow` (balance + limit checks in parallel, then approve). And the same transfer reframed as reserve-then-capture is a `Tcc`. All three are declared the same way — an annotated `impl` block — and all three are mounted on the web surface.

What changed in Lumen

- Lumen now declares its orchestrations with **macros**, not hand-built values. The transfer is a `#[firefly::saga(name = "money-transfer")] impl` whose `debit` step names `compensate = "refund_debit"` and whose `credit` step declares `depends_on = ["debit"]`. The macro generates `TransferSaga::saga` and `TransferSaga::run`, and `run_transfer` just calls `saga.run(req.clone())`.

- A new **compliance workflow** in `src/compliance.rs`: `#[firefly::workflow(name = "transfer-compliance")]` runs `balance-check` and `limit-check` in one parallel layer, then `approve` (which `depends_on` both) consumes their `bool` verdicts through `#[from_step(...)]`.
- A new **two-phase TCC transfer** in `src/tcc_transfer.rs`: `#[firefly::tcc(name = "transfer-2pc")]` with a `source` participant (`confirm = "capture_source"`, `cancel = "release_source"`) and a `dest` participant (`confirm = "capture_dest"`, no `cancel`) — reserve all, then confirm all or cancel the tried ones.
- All three are mounted on the web surface in `src/web.rs`: `POST /api/v1/transfers` (saga), `POST /api/v1/transfers/compliance` (workflow), and `POST /api/v1/transfers/2pc` (TCC) — each rendering a clean rollback as a `422` business problem and invalidating the `GetWallet` cache when it moves funds.
- Because each leg returns its typed `DomainError` / `ComplianceError`, the failing cause is preserved through the engine and recovered from `SagaError::Step`, `WorkflowError::Node`, and `TccError::Try` — no `Mutex` smuggling, no opaque boxed strings.
- The behaviors — happy path, overdraft short-circuit, credit-failure refund (three events on the source stream), parallel rejection, and a released TCC hold — are all pinned by tests, so the prose can never drift from the code.

Exercises

1. Add a **notify** step to the saga. Append a third `#[saga_step(id = "notify", depends_on = ["credit"])]` method to `TransferSaga` that "sends a receipt" (return `Ok(())` for now). Assert that on the happy path `steps_executed == ["debit", "credit", "notify"]`, and that when the credit fails the notify step never runs and only the debit is rolled back.

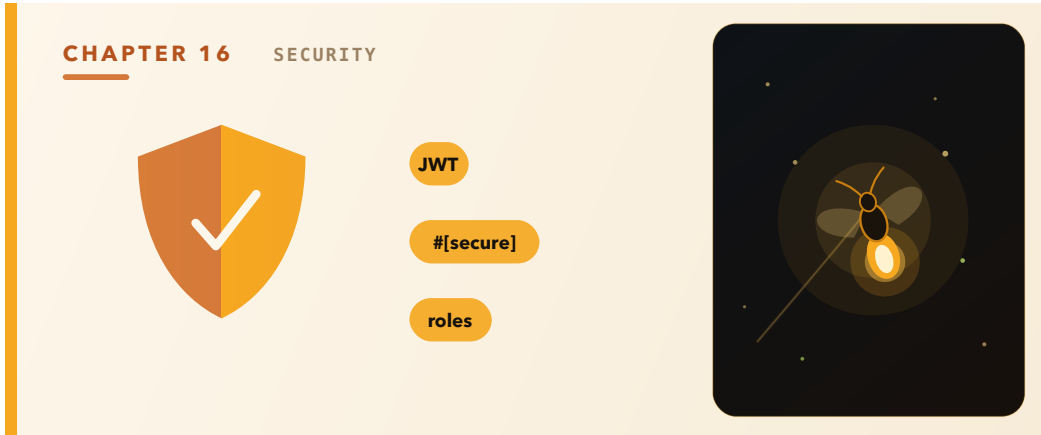
2. **Make the debit retry.** Give the `debit` step `#[saga_step(id = "debit", compensate = "refund_debit", retry = 2, backoff_ms = 50)]`. Drive a flaky `Ledger` that fails the first withdraw and succeeds the second, and assert the transfer still completes — proving the per-step retry recovers a transient failure before compensation is ever considered.
3. **Add a KYC node to the workflow.** Add a third independent `#[workflow_step(id = "kyc-check")]` to `ComplianceCheck` that returns a `bool`, and make `approve` `depends_on` all three, reading the new verdict via `#[from_step("kyc-check")]`. Assert that `kyc-check` runs in the same parallel layer as the existing checks and that a failed KYC rejects the transfer.
4. **Confirm the TCC is all-or-nothing.** Write a test that runs `run_tcc_transfer` with an overdrawn source and asserts neither balance moved — the source try aborts before holding anything, so there is nothing to cancel. Contrast it with the missing-destination test, where the source is held and then released.

To call the external services these engines coordinate — a Payments processor, an FX provider — you need an HTTP client. Continue to [HTTP Clients](#).



PART V

Secure, Observe & Ship



CHAPTER 16

Security, Sessions & Identity

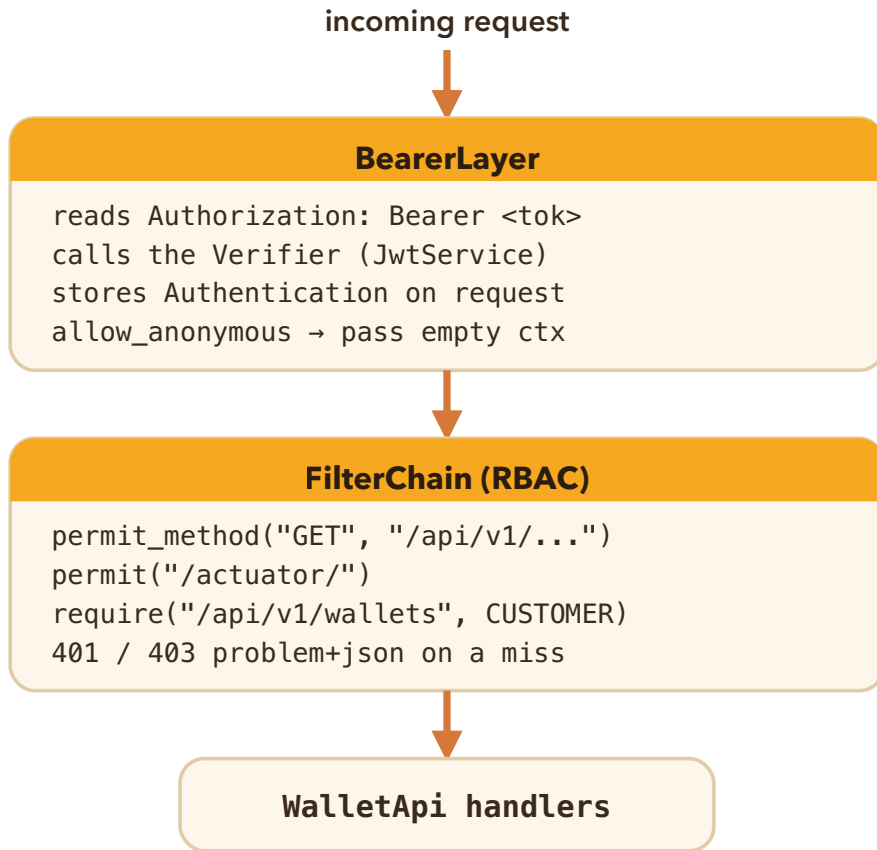
In Chapter 13 you saw how Lumen *would* call an external payments or FX provider. Lumen itself, though, is still wide open: any caller can open a wallet, deposit, withdraw, or move money between wallets. Before Part V can ship Lumen to production, you have to close that door.

By the end of this chapter Lumen will **authenticate** every request with a signed JWT, **authorize** the mutating routes with a path-based RBAC filter chain, and leave the public reads and the management surface open. The whole thing is built on `firefly-security`, reached through the one `firefly` facade — no new dependency, no hand-rolled crypto, and (true to Lumen's promise) not even a `thiserror` derive.

What's in the box. `firefly-security` is a complete resource-server stack: a `BearerLayer` + `Verifier` for token validation, a URL-based authorization `FilterChain`, method guards, JWKS validation, an OAuth2 client + authorization server, a role hierarchy, CSRF, and

a bcrypt encoder. Auth failures render as RFC 9457 `application/problem+json` on a 401/403 — a stable, standards-based wire contract that off-the-shelf clients and gateways understand.

The mental model



The security request pipeline. Every request passes the `BearerLayer` (authentication) and the RBAC `FilterChain` (authorization) before reaching the `WalletApi` handlers.

`firefly-security` carries far more than Lumen uses — JWKS, OAuth2 (client + authorization server), `RoleHierarchy`, method-level `guards`, `CsrfLayer`, `BcryptPasswordEncoder`. We tour those at the end; first, the four pieces Lumen actually wires.

Minting and verifying tokens – JwtService

Lumen is a stateless API. Sessions would need sticky routing or a shared store on every replica; a signed JWT lets each request carry its own credential, so the service scales horizontally with no shared state. The framework's `JwtService` both **mints** the demo tokens and **verifies** the incoming ones, using a symmetric HS256 key — which is exactly what makes Lumen runnable and testable with no external IdP.

Here is the whole of Lumen's token surface, from `src/security.rs`:

Rust

```

use firefly::security::{
    BearerConfig, BearerLayer, FilterChain, JwtService, SecurityError,
    Verifier, VerifierFn,
};
use serde_json::json;

/// The demo signing key. A real service reads this from configuration / a
/// secret store; it is inlined here so the sample is runnable as-is.
pub const DEMO_SIGNING_KEY: &[u8] = b"lumen-demo-signing-key-change-me";

/// The role every mutating wallet command requires.
pub const CUSTOMER_ROLE: &str = "CUSTOMER";

/// The shared HS256 service that both signs the demo tokens and verifies
/// incoming bearer tokens.
fn jwt_service() -> JwtService {
    JwtService::new(DEMO_SIGNING_KEY)
}

/// Mints a signed HS256 access token for `subject` with `roles`, valid for the
/// service's default lifetime (one hour).
pub fn mint_token(subject: &str, roles: &[&str]) -> String {
    jwt_service()
        .encode(json!({ "sub": subject, "roles": roles }))
        .expect("mint_token: HS256 encode")
}

```

`JwtService::new(secret)` builds an HS256 service. `encode` signs a JSON payload and — this is the load-bearing detail — injects an `exp` claim when the payload has none, defaulting to one hour out. Every token Lumen issues has a bounded lifetime; a token with no `exp` is rejected at `decode` time. The `mint_token` helper is what the HTTP tests call to obtain a credential the verifier will accept.

The Verifier and the Authentication

A **Verifier** is the resource-server port: validate a token, return an **Authentication** (the principal, username, roles, and the raw claims). **VerifierFn** adapts a plain async closure into one. Lumen's verifier delegates straight to

JwtService::to_authentication, which maps **sub** → principal and **roles** → roles, then re-wraps any failure as a **SecurityError::Verification**:

Rust

```
/// Builds the resource-server Verifier: validates the token's HS256
/// signature + expiry, then maps `sub` → principal and `roles` → roles onto an
/// Authentication. A bad signature / expired token surfaces as a
/// SecurityError::Verification, which the BearerLayer renders as a
/// `401 application/problem+json`.
pub fn build_verifier() -> impl Verifier {
    VerifierFn(|token: String| async move {
        jwt_service()
            .to_authentication(&token)
            .map_err(|e: SecurityError| SecurityError::verification(format!(
                "invalid token: {e}")))
    })
}
```

Authentication carries the fields a handler or filter inspects:

Field	Type	From the claim
principal	String	sub
username	String	sub (or a friendlier claim)
roles	Vec<String>	roles
claims	HashMap<String, serde_json::Value>	every decoded claim

Its helpers — `has_role(r)`, `has_any_role(&[.])`, `has_authority(a)` (matches a role or a fine-grained permission/scope), and `Authentication::anonymous()` — cover the common checks. Lumen's unit test asserts the round-trip directly:

Rust

```
#[tokio::test]
async fn mint_then_verify_roundtrips_claims() {
    let token = mint_token("u-alice", &[CUSTOMER_ROLE]);
    let auth: Authentication = build_verifier().verify(&token).await.unwrap();
    assert_eq!(auth.principal, "u-alice");
    assert!(auth.has_role(CUSTOMER_ROLE));
}
```

A tampered token (`"not.a.jwt"`) or one signed with the wrong key is rejected with `SecurityError::Verification` — the two negative tests in `security.rs` prove it.

***Production swap.** Move to a real identity provider by replacing `build_verifier` with `firefly::security::JwksVerifier`, pointed at your IdP's JWKS URI (RS256, `kid` cache, `iss` / `aud` validation, `exp` required). The `Verifier` port is identical, so `security_layers` — and every handler — is untouched. That is the "swap the adapter, keep the code" promise applied to identity.*

URL authorization – the FilterChain

`JwtService` answers *who is this caller?*; the `FilterChain` answers *may they do this?* It matches request paths against rules in declaration order — first match wins — and renders a 401 (no/invalid credential) or 403 (authenticated but under-privileged) as an RFC 9457 problem. Lumen composes the bearer layer and the chain in one place:

Rust

```

/// Builds the BearerLayer + FilterChain that protect the service.
///
/// | Route | Rule |
/// |-----|-----|
/// | `GET /api/v1/wallets/:id` | permit (public read) |
/// | `GET /actuator/*` | permit (management) |
/// | `POST /api/v1/wallets` | require `CUSTOMER` |
/// | `POST /api/v1/wallets/:id/deposit` / `withdraw` | require `CUSTOMER` |
/// | `POST /api/v1/transfers` | require `CUSTOMER` |
pub fn security_layers() -> (BearerLayer, FilterChain) {
    // `allow_anonymous` lets an unauthenticated request reach the chain; the
    // chain (not the bearer layer) then decides – a 401 on a `require` route
    // without a valid token, a pass on a permitted route.
    let bearer =
        BearerLayer::new(BearerConfig::new(build_verifier()).allow_anonymous(true));
    let chain = FilterChain::new()
        .permit_method("GET", "/api/v1/wallets")
        .permit("/actuator/")
        .require("/api/v1/wallets", &[CUSTOMER_ROLE])
        .require("/api/v1/transfers", &[CUSTOMER_ROLE])
        .any_request_permit();
    (bearer, chain)
}

```

Two design choices are worth dwelling on:

- **allow_anonymous(true) on the bearer layer.** With it set, a request with no `Authorization` header is *not* rejected at the bearer layer — it reaches the chain carrying an anonymous `Authentication`. That keeps a single decision-maker: the `FilterChain` decides every route. A public `GET` passes; a `require` route with no valid token becomes a 401. Without `allow_anonymous`, the bearer layer would reject anonymous traffic before the chain could permit the public reads.
- **Order matters.** `permit_method("GET", "/api/v1/wallets")` and `permit("/actuator/")` come *first*, so the public reads and the management surface are

decided before the broad `require("/api/v1/wallets", ...)` could catch them. `any_request_permit()` re-opens the unmatched tail.

⚠ WARNING

Once any rule is declared, a `FilterChain` is **fail-closed**: a request matching no rule is rejected with 403 (deny-by-default). Re-open the unmatched tail explicitly with `any_request_permit()` / `any_request_authenticated()` / `any_request_deny()`. A chain with *no* rules is a no-op and passes everything (never a surprise blanket lockout).

Layering it onto the router

The chain is attached to the `WebStack` at build time, and the bearer layer is applied around the whole router so the chain sees a populated `Authentication`. This is the relevant slice of `LumenApp::router` and `build_app` in `src/web.rs`:

Rust

```
// in build_app():
let (_bearer, chain) = crate::security::security_layers();
let web = WebStack::new(firefly::starter_web::CoreConfig {
    app_name: APP_NAME.into(),
    app_version: VERSION.into(),
    ..Default::default()
})
.with_security(chain);

// in LumenApp::router():
let (bearer, _chain) = crate::security::security_layers();
// The WebStack carries the FilterChain (set in build_app); layer the
// bearer auth on the outside so the chain sees a populated Authentication.
self.web.apply_middleware(routes).layer(bearer)
```

`WebStack::with_security(chain)` stores the chain; `WebStack::apply_middleware` layers it *inside* the inherited correlation / security-headers / CORS edge, so even a 401 response carries those headers and a correlation id. The bearer layer goes on the outside (axum runs the last-added layer first): authenticate, then authorize.

Method security

The `FilterChain` guards *routes*. But authorization is often a property of a *service method* — a domain operation that several handlers, a scheduled job, and a CQRS handler all call. Pushing the check down to the method means it holds no matter how the operation is reached, and the route table stays about routes.

Firefly does this with two attribute macros and an **ambient security context**. The macros declare the rule; the context carries the caller's `Authentication` through the call stack so the method never has to thread an argument or touch the `Request`.

`#[firefly::pre_authorize(...)]`

`#[firefly::pre_authorize(...)]` guards a function *before* its body runs. It attaches to any function returning `Result<T, E>` where `E: From<firefly_security::SecurityError>`, and reads the ambient `Authentication` (below) to decide. The rules:

Rule	Passes when
<code>(empty)</code> / <code>authenticated</code>	a caller is in scope (the default)
<code>role = "ADMIN"</code>	the caller has role <code>ADMIN</code>
<code>any_role = ["A", "B"]</code>	the caller has <i>any</i> of the listed roles
<code>authority = "wallet:write"</code>	the caller holds that authority (role or scope)
<code>any_authority = ["a", "b"]</code>	the caller holds <i>any</i> of the listed authorities

On denial the macro returns early with `Err(..)`: an `Unauthenticated` `SecurityError` when no caller is in scope, a `Forbidden` one when a caller is present but the authorities don't match.

Rust

```
use firefly_security::SecurityError;

/// Only a CUSTOMER may withdraw. The check runs before any balance logic.
#[firefly::pre_authorize(role = "CUSTOMER")]
pub async fn withdraw(wallet: WalletId, amount: Money) -> Result<Wallet,
WalletError>
where
    WalletError: From<SecurityError>,
{
    // ... domain logic; reached only for an authenticated CUSTOMER ...
}

/// A coarse "must be logged in" gate – the empty form is `authenticated`.
#[firefly::pre_authorize]
pub fn current_balance(wallet: WalletId) -> Result<Money, WalletError> {
    // ...
}

/// A fine-grained scope check rather than a role.
#[firefly::pre_authorize(authority = "wallet:approve")]
pub async fn approve(wallet: WalletId) -> Result<(), WalletError> {
    // ...
}
```

`#[firefly::post_authorize(<bool expr>)]`

Sometimes you can only decide *after* you have the value — "you may read this wallet only if you own it." `#[firefly::post_authorize(...)]` attaches to an `async fn` returning `Result<T, E>` and evaluates a boolean expression once the body has produced its `Ok(T)`. The expression sees two bindings:

- `result` — a `&T`, the value the function is about to return (the *return object*).
- `auth` — a `&Authentication`, the ambient caller.

If the expression is `false` the value is **discarded** and the call resolves to a `Forbidden` error instead:

Rust

```
/// A caller may fetch a wallet only if they own it.
#[firefly::post_authorize(result.owner == auth.principal)]
pub async fn get_wallet(id: WalletId) -> Result<Wallet, WalletError> {
    repo().load(id).await // produces Ok(Wallet); the rule then vets the owner
}
```

The ambient context — `firefly_security`

Both macros read an ambient `Authentication` rather than an argument. That scope is managed by a small set of functions in `firefly_security` — the security context that travels with the task:

Rust

```

use firefly_security::{
    with_authentication_scope, current_authentication, check_access,
    AccessRule, Authentication, SecurityError,
};

// Run `fut` with `auth` installed as the ambient caller for its whole duration.
let wallet = with_authentication_scope(auth, async {
    withdraw(id, amount).await // #[pre_authorize] inside sees `auth`
}).await?;

// Read the current caller anywhere downstream (None if unauthenticated).
let who: Option<Authentication> = current_authentication();

// Imperative check when a macro doesn't fit – returns the Authentication on
// success, a SecurityError on failure.
let auth: Authentication = check_access(&AccessRule::Role("CUSTOMER"))?;

```

`AccessRule` is the runtime form of the macro rules: `AccessRule::Authenticated`, `Role(&str)`, `AnyRole(&[&str])`, `Authority(&str)`, and `AnyAuthority(&[&str])`.

The payoff is that `BearerLayer` installs the scope for you. On every request — both the verified path *and* the anonymous (`allow_anonymous`) path — the bearer layer wraps the downstream call in `with_authentication_scope`, so a service method decorated with `#[pre_authorize]` works correctly even though it never sees the `Request`. URL rules and method rules then compose: the `FilterChain` is your coarse perimeter, the method macros are your defense in depth.

Wiring security from configuration

Lumen inlines its signing key and builds the bearer layer by hand because that makes the sample runnable as-is. A deployed service instead reads its security posture from configuration, and `firefly_security` binds it directly — no DI container, no framework callback. The properties live under `firefly.security.*` and bind through `serde`:

Rust

```
use firefly_security::{
    SecurityProperties, JwtProperties, BearerProperties,
    verifier_from_config, bearer_layer_from_config,
};
```

TOML

```
# firefly.security.* - JWKS resource-server example
[firefly.security.jwt]
jwk_set_uri = "https://idp.example.com/.well-known/jwks.json"
issuer_uri  = "https://idp.example.com/"
audience   = "lumen"
algorithm   = "RS256"

[firefly.security.bearer]
header_name = "Authorization"
allow_anonymous = true
```

The structs mirror that shape:

Rust

```
pub struct SecurityProperties {
    pub jwt: JwtProperties,
    pub bearer: BearerProperties,
}

// Both structs derive `Default, Deserialize` with `#[serde(default)]`, so a
// missing field falls back to its zero value (empty `String`, `0`).
pub struct JwtProperties {
    pub jwk_set_uri: String,
    pub issuer_uri: String,
    pub audience: String,
    pub secret: String,
    pub algorithm: String,
    pub expiration_seconds: u64,
}

pub struct BearerProperties {
    pub header_name: String,
    pub allow_anonymous: bool,
}
```

Two builder functions turn bound properties into ready components:

Rust

```
use std::sync::Arc;
use firefly_security::{Verifier, BearerLayer, SecurityError};

// Pick a verifier by what configuration provides – JWKS first, then HMAC,
// then nothing.
let verifier: Option<Arc<dyn Verifier>> = verifier_from_config(&props.jwt)?;

// The fully-assembled bearer layer (header name + anonymous policy applied),
// or None when no verifier is configured.
let bearer: Option<BearerLayer> = bearer_layer_from_config(&props)?;
```

`verifier_from_config(&JwtProperties)` resolves the verifier by precedence: a non-empty `jwk_set_uri` builds a JWKS (RS256) resource-server verifier; otherwise a non-empty `secret` builds an HMAC (HS256/384/512) verifier; otherwise it returns `None`. `bearer_layer_from_config(&SecurityProperties)` builds the verifier the same way and, if there is one, wraps it in a `BearerLayer` with the configured header name and anonymous policy already applied — the same layer `security_layers` builds by hand, sourced from config instead. Switching Lumen from the demo HMAC key to a production IdP becomes a configuration change, with no edit to `security.rs`.

Proving it end to end

The HTTP suite (`tests/http.rs`) drives the fully-wired router with `tower::oneshot` and asserts the security behavior directly. A mutation with no token is a 401 problem; a malformed body on an authenticated request is a 422 problem:

Rust

```
#[tokio::test]
async fn missing_token_is_401_problem_on_mutations() {
    let res = build_router()
        .await
        .oneshot(post(
            "/api/v1/wallets",
            serde_json::json!({ "owner": "mallory", "openingBalance": 10 }),
            false, // no Authorization header
        ))
        .await
        .unwrap();
    assert_eq!(res.status(), StatusCode::UNAUTHORIZED);
    assert!(content_type(&res).contains("application/problem+json"));
}
```

The test helper builds the `Authorization` header from `mint_token`, so an authenticated request is just `post(path, body, true)`:

Rust

```
fn bearer() -> String {
    format!("Bearer {}", mint_token("u-alice", &[CUSTOMER_ROLE]))
}
```

Every other test in the suite — open, get, deposit/withdraw, transfer — runs as the minted `CUSTOMER`, so authentication is exercised on the happy path too.

The rest of firefly-security (Lumen's growth room)

Lumen uses the symmetric-key fast path. The same crate carries the production surface you reach for as a real wallet service matures:

- **JWKS verification.** `JwksVerifier` is a drop-in `Verifier` for RS256 tokens from an external IdP (Keycloak, Auth0, Cognito): `kid` cache, `iss` / `aud` checks, `exp` required, and the same `sub` / `roles` / `permissions` claim mapping.
- **Method guards.** For per-handler checks, the `guards` module composes typed predicates: `has_role("CUSTOMER").or(has_authority("wallet:approve"))`, then `guard.authorize(Some(&auth))?` — 401 with no principal, 403 if the predicate is false. For a declarative spelling, see [Method security](#) above.
- **Role hierarchy.** `RoleHierarchy::from_string("ADMIN > CUSTOMER")` parses the spec; attach it with `chain.with_role_hierarchy(..)` so granting `ADMIN` implies `CUSTOMER`.
- **Pattern rules.** Alongside the prefix rules Lumen uses, the chain offers an fnmatch-style glob DSL — `permit_pattern("/public/**")`, `require_pattern("/api/admin/**", &["ADMIN"])`, `require_authority("/api/reports/**", &["reports:read"])`, `authenticated("/api/**")`.
- **Sessions.** For browser flows where logout must mean logout, the `firefly-session` crate adds a `SessionLayer` over a `SessionStore`

(`MemorySessionStore` for dev, a Redis-backed store for scale). A handler pulls the request's `Session` with the `SessionExt` extractor and calls `session.rotate_id()` after login (session-fixation defense), `session.set_attribute("user_id", id)`, and `session.invalidate()` on logout.

- **OAuth2.** The `oauth2` module covers both sides: `ClientRegistration` + `OAuth2LoginHandler` for the authorization-code login flow (state + nonce + PKCE S256, OIDC id-token validation), and an `AuthorizationServer` that issues tokens for `client_credentials` / `refresh_token`.
- **CSRF & passwords.** `CsrfLayer` implements the double-submit-cookie pattern for cookie-session flows; `BcryptPasswordEncoder` (default work factor 12) hashes credentials. The `$2b$` hashes interchange with the `firefly-idp-internal-db` adapter and every other port.

Rust

```
use firefly_security::{BcryptPasswordEncoder, PasswordEncoder};

let enc = BcryptPasswordEncoder::new(); // work factor 12 (the default)
let hash = enc.hash("s3cret").unwrap();
assert!(enc.verify("s3cret", &hash).unwrap());
assert!(!enc.verify("wrong", &hash).unwrap());
```

What changed in Lumen

This chapter closed Lumen's open front door without adding a dependency or a line of business logic to the handlers:

- A single HS256 `JwtService` (`src/security.rs`) both mints the demo tokens and verifies incoming ones; `encode` auto-stamps a one-hour `exp`, and a token without `exp` is rejected at decode time.

- `build_verifier` turns the service into a `Verifier` via `VerifierFn`, mapping `sub` → principal and `roles` → roles onto an `Authentication`, with a bad token surfacing as `SecurityError::Verification` → a 401 problem.
- `security_layers` composes a `BearerLayer` (with `allow_anonymous(true)`) and a path-ordered RBAC `FilterChain`: public `GET /api/v1/wallets/:id`, public `/actuator/*`, and `CUSTOMER`-only `POST /api/v1/wallets`, `/deposit`, `/withdraw`, and `/transfers`.
- `WebStack::with_security` + `apply_middleware` attach the chain inside the correlation/headers edge; `LumenApp::router` layers the bearer auth on the outside.
- **Method security** pushes authorization onto domain operations:
 - `#[firefly::pre_authorize(role = "CUSTOMER")]` guards before the body runs,
 - `#[firefly::post_authorize(result.owner == auth.principal)]` vets the return object, and the `BearerLayer` installs the ambient context
 - (`with_authentication_scope` / `current_authentication` / `check_access`) so a service method enforces the rule without ever seeing the `Request`.
- **Config-driven wiring** lets a deployed service read its posture from
 - `firefly.security.*: verifier_from_config` picks JWKS (RS256), HMAC, or none by precedence, and `bearer_layer_from_config` hands back the ready-to-mount `BearerLayer` — moving from the demo key to a production IdP with no code edit.
- The HTTP suite proves the contract: an unauthenticated mutation is a 401 problem, the happy paths run as a minted `CUSTOMER`, and the unit tests round-trip and reject tokens.

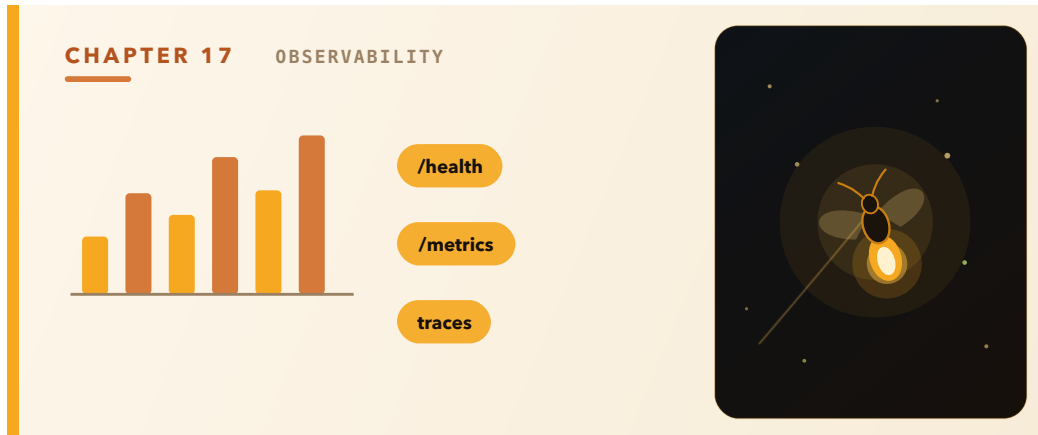
Exercises

1. Add an ADMIN-only route. Give Lumen a hypothetical `GET /api/v1/wallets` collection list and protect it with `require_pattern` so only an `ADMIN` may list

every wallet, while `CUSTOMER` keeps access to the single-wallet read. Mint an `ADMIN` token in a test and assert a `CUSTOMER` token gets a 403.

2. **Role hierarchy.** Introduce a `SUPER` role that implies `CUSTOMER`. Build a `RoleHierarchy` from `"SUPER > CUSTOMER"`, attach it with `chain.with_role_hierarchy(..)`, mint a `SUPER`-only token, and assert it passes the `require("/api/v1/wallets", &["CUSTOMER"])` rule.
3. **Swap in JWKS.** Sketch a `build_verifier_jwks()` that returns a `JwksVerifier::new("https://idp.example.com/.well-known/jwks.json")` and confirm (by reading the `Verifier` trait) that `security_layers` needs no other change. Why does the rest of Lumen not care which verifier it got?
4. **Expiry.** Lower the token lifetime with `JwtService::new(KEY).expiration_seconds(1)`, mint a token, wait two seconds, and assert the verifier now returns `SecurityError::Verification`.

A secure service is only trustworthy if you can *see* what it is doing. The next chapter gives Lumen eyes and ears — structured logs, health, metrics, and the admin dashboard. Continue to [Observability](#).



CHAPTER 17

Observability & Health

In Chapter 14 you locked Lumen's mutating routes behind a JWT and a role filter. The service is now safe — but it is still a black box. When a deposit is slow in production you need to know *where* the time went; when the broker degrades you want a dashboard that turns red before your pager does; when an auditor asks why a transfer was rejected you need a structured log line with the context to reconstruct the decision.

By the end of this chapter Lumen will expose an **actuator admin surface** on a separate port — health, info, metrics, loggers, scheduled tasks — feed it an **info contributor** describing the build, emit **structured logs** with a correlation id on every request, and (as the next step it grows into) surface all of it in the embedded **admin dashboard** with its fifteen views, including the new **Beans** view. Lumen is **observable by default**: `WebStack::new` already wired the logging layer, the health composite, the metric registry, and the request-metrics middleware. This chapter turns those defaults into an admin surface and explains what each piece reports.

An **/actuator** management surface. Firefly exposes a JSON management surface under **/actuator/*** — health, info, metrics, env, loggers, scheduled tasks — with structured, correlation-enriched logs and an embedded admin dashboard. The logger endpoint reports **configuredLevel** / **effectiveLevel**, the conventional shape standard metrics and management tooling expects, so off-the-shelf scrapers and dashboards work unchanged.

The admin surface in main.rs

Lumen serves the public API on **127.0.0.1:8080** and the actuator on **127.0.0.1:8081** — a separate listener, so **/actuator/*** never leaks onto the public network. The **WebStack** **Deref**s to its **Core**, so **actuator_router** is reachable directly. This is the relevant slice of **src/main.rs**:

Rust

```
use firefly::starter_core::InfoContributor;

let api = app.router();
let contributor: InfoContributor = Box::new(|| {
    let mut info = serde_json::Map::new();
    info.insert(
        "sample".into(),
        serde_json::json!({ "name": APP_NAME, "store": "in-memory", "eventBus":
"in-memory" }),
    );
    info
});
let admin = app.web.actuator_router(vec![contributor]);
```

actuator_router(info_contributors) returns an axum **Router** carrying the full management surface. Lumen mounts it on the admin listener in its lifecycle wiring (the production chapter covers that end to end); for now, the two routers are served on two ports.

The actuator endpoints

`actuator_router` exposes the management endpoints below. Lumen's reach the admin port at `http://127.0.0.1:8081/actuator/*`:

Endpoint	Returns
<code>/actuator/health</code>	the composite rollup (+ liveness/readiness probes)
<code>/actuator/info</code>	app metadata + your info contributors
<code>/actuator/metrics</code>	the registered meter listing
<code>/actuator/metrics/:name</code>	one meter's detail
<code>/actuator/prometheus</code>	the Prometheus text-exposition scrape target
<code>/actuator/env</code>	masked, origin-attributed property sources
<code>/actuator/scheduledtasks</code>	scheduled-task descriptors
<code>/actuator/version</code>	the running version
<code>/actuator/loggers[:name]</code>	runtime log-level control
<code>/actuator/threaddump</code>	a thread/task dump
<code>/actuator/httpexchanges</code>	recent HTTP exchanges (when wired)
<code>/actuator/caches[:name]</code>	cache listing + eviction (when wired)
<code>/actuator/refresh</code>	reload config (the <code>ReloadableConfig</code> hook)

The info contributor

An `InfoContributor` is a boxed closure returning a `serde_json::Map`; each one adds a section to `/actuator/info`. Lumen's reports its store and event-bus kind, so an operator hitting `/actuator/info` sees that this instance is running the in-memory infrastructure:

JSONC

```
// GET /actuator/info
{
  "app": { "name": "lumen", "version": "26.6.5" },
  "sample": { "name": "lumen", "store": "in-memory", "eventBus": "in-memory" }
}
```

The `app` block is filled from the `CoreConfig.app_name` / `app_version` Lumen set in Chapter 3; the `sample` block is the contributor above.

Health

A health `Indicator` is an async probe returning a `HealthResult` (status + message + details); a `Composite` aggregates them into the canonical rollup — `DOWN` if any indicator is `DOWN`, else `DEGRADED` if any is `DEGRADED`, else `UP`. The `Core` already carries a `HealthComposite`; bridge an observability indicator onto it with `core.add_observability_indicator(..)`. A real Lumen deployment would add a broker-liveness indicator and a store probe:

Rust

```
use firefly_observability::{HealthResult, IndicatorFn};

// Wire a probe onto the WebStack's composite (it Derefs to Core):
app.web.add_observability_indicator(IndicatorFn::new("event-bus", || async {
    HealthResult::up() // a real probe would ping the broker
}));
```

`/actuator/health/liveness` and `/actuator/health/readiness` are the sub-paths your orchestrator's probes hit — separate so an in-flight migration that fails readiness need not trigger a liveness restart.

Request metrics – already on

Request metrics are auto-instrumented **on by default** — at the `Core` layer (so even a bare `Core` emits them) and through `WebStack::new`, which fills in a `RequestMetricsConfig` if you left one unset. For every request the middleware records the labeled `http_server_requests_seconds` timer plus a companion `_max` gauge, tagged `method` / templated `uri` (the axum matched path, so `/api/v1/wallets/:id` not the concrete id) / `status` / `outcome` / `exception`, and bridges them onto the actuator `MetricRegistry`. A clean request carries `exception="None"`. So the moment Lumen booted in Chapter 6 it was already emitting per-route latency; this chapter just exposes it at `/actuator/metrics`.

NOTE

To turn the auto-instrumentation off, set `CoreConfig { disable_request_metrics: true, .. }`; to tune the rolling-max window or path exclusions, supply a `request_metrics: Some(RequestMetricsConfig { .. })`.

The dot-case meter names map straight to Prometheus

(`http_server_requests_seconds`), so pointing a Prometheus `scrape_config` at `/actuator/prometheus` lights up Grafana with no extra code.

Beyond the request timer, you record your own meters on the same registry. Pull it off the `Core` with `metric_registry()`, then increment a counter or set a gauge — both surface at `/actuator/metrics` and `/actuator/prometheus` immediately:

Rust

```
let metrics = app.web.metric_registry();

// A domain counter, bumped each time the transfer saga completes.
metrics.counter("lumen_transfers_total").inc(); // or .add(1) for an explicit
count

// A gauge sampling a live value (e.g. wallets currently held in the read
model).
metrics.gauge("lumen_wallets_active").set(wallet_count as f64);
```

Structured logging and correlation

WebStack installs a **tracing** layer that formats every event as one structured line and enriches it with the request's correlation id (set by the correlation middleware, on by default). Lumen calls **init_logging** once at startup, best-effort so a test harness that already owns the global subscriber does not panic:

Rust

```
let app = build_app().await;
// Best-effort: a test harness may already own the global subscriber.
let _ = app.web.init_logging();
```

After that, plain **tracing** macros produce enriched lines; fields recorded on an enclosing span merge into each event. The field names (**time**, **level**, **msg**, **service**, **correlationId**) follow a stable, documented schema, so one log pipeline parses every Firefly service consistently.

Because the correlation id lives in a task-local scope, it flows automatically into every log line, every event Lumen's ledger publishes (`Event::new` stamps it), and every outbound client call (the W3C `traceparent` is propagated). A request that opens a wallet, publishes `WalletOpened`, and projects it into the read model stitches together under one id with no manual threading.

Correlation flows automatically. `init_logging` installs a structured `tracing` subscriber; the task-local correlation id is attached to every log line in place of manual thread-local plumbing, so a request stitches together across logs, events, and outbound calls with no field threading.

Configuring logging

Logging is configured the way you configure everything else — from the one main config file. Bind the `firefly.logging.*` section into a `LogConfig` with `firefly_observability::log_config_from_properties(props, base)`:

YAML

```
firefly:
  logging:
    format: json                # json | text (logfmt) | console
    level:                      # one level map (like logging.level.<logger>)
      root: info               # root level
      firefly_web: warn        # per-logger levels
      app::ledger: trace
    file:
      name: lumen.log          # enable the rolling file appender
      max-size: 10MB
      max-history: 7
```

Per-logger levels, the output format, and the rolling file appender all come from config; an external logging file can be folded in with `apply_external_config`. And every level can be changed **without a restart** through `POST /actuator/loggers/{name}` — the actuator's runtime logger control.

Tracing / OpenTelemetry

`firefly-observability` exposes the building blocks that compose with the `tracing` ecosystem and propagates W3C trace-context (`traceparent` / `tracestate`) on the HTTP edges and outbound client calls. The OpenTelemetry SDK wiring — exporters, sampling, resource attributes — is left to your `main.rs`, where you add your preferred OTel `tracing` layer alongside the correlation layer. Lumen ships without an exporter (it is teaching code with no external collector), but the trace-context propagation is already on the edges.

When you do want spans flowing to a collector, build an OTLP tracer and add `tracing-opentelemetry`'s layer to the subscriber Firefly installed — the correlation layer keeps working alongside it:

Rust

```

use opentelemetry_otlp::WithExportConfig;
use tracing_subscriber::prelude::*;

// Build an OTLP pipeline pointing at your collector.
let tracer = opentelemetry_otlp::new_pipeline()
    .tracing()
    .with_exporter(opentelemetry_otlp::new_exporter().tonic().with_endpoint("http://
otel-collector:4317"))
    .install_batch(opentelemetry_sdk::runtime::Tokio)?;

// Register the OTel layer alongside Firefly's structured-log + correlation
layers.
tracing_subscriber::registry()
    .with(tracing_opentelemetry::layer().with_tracer(tracer))
    .init();

```

The `traceparent` headers Firefly already propagates become the parent/child edges between spans, so a request that fans out to an outbound call appears as a single distributed trace in your backend.

The admin dashboard

The actuator surface is JSON for machines. `firefly-admin` mounts a single-page admin dashboard — vendored, no `npm` build — that ties health, metrics, loggers, beans, mappings, caches, CQRS handlers, sagas, traces, and a live log tail into one pane of glass with Server-Sent-Event streams. It is the next surface Lumen grows into; you enable the facade's `admin` feature and wire it from the `Core` accessors.

`mount(AdminConfig, AdminDeps)` returns the dashboard router. `AdminDeps::new` takes the required collaborators; the rest are optional fields you fill in with struct-update syntax. A Lumen wiring drawing on its `Core` looks like:

Rust

```

use std::sync::Arc;
use firefly::admin::{mount, AdminConfig, AdminDeps, LogBuffer, TraceBuffer};

let traces = Arc::new(TraceBuffer::new());
let logs = LogBuffer::new();

// `AdminDeps::new` takes the required collaborators; the optional fields that
// back the remaining views are set afterwards with struct-update syntax.
let deps = AdminDeps {
    scheduler: Some(app.web.scheduler()), // → Scheduled Tasks view
    bus: Some(app.web.cqrs_bus()), // → CQRS view
    container: Some(container), // → Beans view
    ..AdminDeps::new(
        APP_NAME,
        VERSION,
        app.web.health_composite(), // Arc<HealthComposite>
        app.web.metric_registry(), // Arc<MetricRegistry>
        Arc::clone(&traces),
        logs,
    )
};

let dashboard = mount(AdminConfig::default(), deps);

```

The dashboard auto-discovers what those collaborators expose and renders it in **fifteen built-in views**, grouped in the sidebar:

Section	Views
Dashboard	Overview, Health
Application	Beans, Environment, Configuration, Loggers
Monitoring	Metrics, Scheduled Tasks, HTTP Traces, Log Viewer
Infrastructure	Mappings, Caches, CQRS, Transactions
Fleet	Instances (server mode)

Each view is backed by a `/admin/api/*` JSON endpoint; the SSE streams (`/admin/api/sse/{health,metrics,traces,logfile,beans,runtime,server}`) push updates without your code polling. Admin and actuator paths are excluded from trace capture so they never pollute the trace panel.

The Beans view

The newest view is **Beans** — the dashboard's window onto the dependency-injection container. When you set `deps.container = Some(container)`, the dashboard serves:

Endpoint	Returns
<code>GET /admin/api/beans</code>	every registered bean with its stereotype and scope
<code>GET /admin/api/beans/graph</code>	the dependency graph between beans
<code>GET /admin/api/beans/:name</code>	one bean's detail (type, scope, dependencies)
<code>GET /admin/api/sse/beans</code>	a live snapshot at each refresh interval

The Overview view also rolls up a `beans` block (`{ total, stereotypes }`) and a `wiring` block (live CQRS-handler and scheduled-task counts) drawn from the same container, so the landing page shows how much the service is wired without opening the full Beans view. Because Lumen wires its composition root explicitly rather than

scanning a container, the Beans view is sparse for Lumen itself — but the moment you adopt `#[derive(Component)] + firefly::scan` (Chapter 4), the graph fills in. Without a container the endpoints degrade gracefully to an empty `{ total: 0 }` block.

***Beans and server mode.** The Beans view is the dashboard's window onto the DI container. `firefly-admin` also runs in server mode (`AdminServerConfig`): instances self-register through an `AdminClient`, and the server aggregates a fleet of services into the Instances view. The dashboard is a vanilla-JS SPA driven entirely by the `/admin/api` JSON + SSE endpoints — no frontend build step.*

Custom views

To add your own sidebar view, implement the `AdminView` trait and push it onto `AdminDeps::views`; the dashboard lists it under `/admin/api/views[/:id]`. A Lumen "Treasury" view surfaces the total custody balance across all wallets, queried from the read model:

Rust

```

use firefly::admin::AdminView;

struct TreasuryView {
    read_model: Arc<WalletReadModel>,
}

#[async_trait::async_trait]
impl AdminView for TreasuryView {
    fn view_id(&self) -> &str { "treasury" }
    fn display_name(&self) -> &str { "Treasury" }
    fn icon(&self) -> &str { "wallet" }

    // Backs GET /admin/api/views/treasury (keyed by view_id).
    async fn data(&self) -> serde_json::Value {
        let total: i64 = self.read_model.all().iter().map(|w| w.balance).sum();
        serde_json::json!({ "custodyTotal": total, "wallets":
self.read_model.len() })
    }
}

// Register it before mounting:
let mut deps = AdminDeps::new(/* ... */);
deps.views.push(Arc::new(TreasuryView { read_model: Arc::clone(&read_model) }));

```

What changed in Lumen

This chapter turned Lumen's always-on observability defaults into a working admin surface:

- `main.rs` builds an `InfoContributor` describing the in-memory store and event bus, and serves `app.web.actuator_router(vec![contributor])` on the admin port — `/actuator/health`, `/info`, `/metrics`, `/loggers`, `/scheduledtasks`, and the rest.

- **init_logging** (best-effort, so the test harness can own the subscriber) switches on structured, correlation-enriched logging; the correlation id flows into every log line, published event, and outbound call automatically.
- The **request-metrics** middleware — on since `WebStack::new` — records `http_server_requests_seconds` per templated route, now exposed at `/actuator/metrics` and `/actuator/prometheus`.
- The **admin dashboard** (the `firefly-admin` step Lumen grows into) ties it all together in fifteen views, including the new **Beans** view backed by the DI container, with live SSE streams.

Exercises

1. **Reach the actuator.** Run `cargo run --bin lumen`, then `curl http://127.0.0.1:8081/actuator/info` and confirm the `sample` block reports the in-memory store. Hit `/actuator/health` and `/actuator/metrics`.
2. **Add a health indicator.** Wire a `IndicatorFn::new("read-model", ..)` onto the composite with `add_observability_indicator` that returns `UP` when the read model holds at least one wallet view, and watch it appear under `/actuator/health`.
3. **A Lumen metric.** Record a counter — e.g. `lumen_transfers_total` — on the `metric_registry()` each time the transfer saga completes, and verify it appears at `/actuator/metrics`. (Recall the housekeeping heartbeat in Chapter 16 keeps an `AtomicU64` you could surface the same way.)
4. **Mount the dashboard.** Enable the facade's `admin` feature, mount the dashboard with the `container` field set, and open `/admin` — note how the Beans view is sparse for Lumen's explicit root, then convert one collaborator to `#[derive(Component)]` and watch it appear.

With Lumen observable, the next chapter adds background work and the path to outbound notifications. Continue to [Scheduling & Notifications](#).

**CHAPTER 18**

Caching & Resilience

By the end of this chapter, you will understand exactly how Lumen's `GET /api/v1/wallets/:id` serves a wallet view from a 30-second cache — and why every deposit, withdrawal, and transfer *invalidates* that cache so a read after a write never lies. The cache itself was switched on back in `CQRS` with one annotation and one middleware; this chapter opens it up: the unified cache port behind it, the backends you can swap in, and the `firefly-resilience` decorators that protect the slow call a cache miss falls through to.

`firefly-cache` exposes a single port — `Adapter` — and ships in-process, no-op, and fallback implementations plus a typed memoization wrapper. Code against the port and select the backend (in-memory, Redis, Postgres) at wiring time. All of it reaches Lumen through the one `firefly` facade, as `firefly::cache` and `firefly::resilience`.

The cache and resilience model. `Adapter` is Firefly's single cache port; `Typed<T>::get_or_set` is the read-through memoization primitive; the query-side `#[firefly(cache_ttl = "30s")]` is the declarative cache TTL Lumen uses. The resilience decorators — circuit breaker, rate limiter, bulkhead, timeout — compose into a `Chain` around any async call.

Lumen's read-side cache

Lumen's `GetWallet` query carries its caching policy as a declaration sitting right next to the type — no cache code in the handler:

Rust

```
/// `GET /api/v1/wallets/:id` query. `#[firefly(cache_ttl = "30s")]` is
/// reflected
/// on the generated `Message::cache_ttl`, so a `QueryCache` memoises reads for
/// 30 seconds.
#[derive(Debug, Clone, Default, Serialize, Deserialize, Query)]
#[firefly(cache_ttl = "30s")]
pub struct GetWallet {
    pub id: String,
}
```

The `#[derive(Query)]` macro reads that attribute and emits a `cache_ttl()` on the generated `Message` impl — a fact the test pins so it can never silently disappear:

Rust

```
#[test]
fn get_wallet_carries_cache_ttl() {
    assert!(GetWallet::default().cache_ttl().is_some());
}
```


The TTL is inert until something *honors* it. That something is the `QueryCache` middleware, installed on the bus in Lumen's composition root (`build_app` in `src/web.rs`):

Rust

```
use firefly::cqrs::QueryCache;

// Read-side caching on the bus (honours GetWallet's 30s cache_ttl). The
// handle is kept so a mutation can invalidate it for read-after-write.
let query_cache = QueryCache::new();
bus.use_middleware(query_cache.middleware());
// Validation middleware enforces the `#[firefly(validate)]` checks.
bus.use_middleware(firefly::cqrs::ValidationMiddleware::new());
```

`QueryCache::new()` builds the cache; `query_cache.middleware()` returns the bus middleware that consults it. When a `GetWallet` flows through the bus, the middleware checks the cache: a hit returns the memoized `WalletView` without ever reaching the handler; a miss runs the handler, stores the result under the query's key for 30 seconds, and returns it. The `query_cache` handle is kept on `LumenApp` precisely so the write side can reach back into it.

Why the handle, not just the middleware? The middleware reads and fills the cache; only a holder of the handle can invalidate it. Lumen keeps `query_cache` on `LumenApp` and passes a clone into the controller state, so the mutating handlers can drop stale entries the moment they change a balance.

Read-after-write invalidation

A 30-second TTL is great for a read-heavy view and a disaster for correctness if you never invalidate. Deposit `$2.50`, then read the balance within 30 seconds, and a naive cache would happily serve the *old* number. Lumen avoids that by invalidating the whole `GetWallet` family after every mutation. From the controller in `src/web.rs`:

Rust

```
#[post("/wallets/:id/deposit")]
async fn deposit(
    State(api): State<WalletApi>,
    Path(id): Path<String>,
    Json(body): Json<AmountBody>,
) -> WebResult<Json<WalletView>> {
    let cmd = Deposit { wallet_id: id, amount: body.amount };
    let view: WalletView = api.bus.send(cmd).await.map_err(cqrs_to_web)?;
    api.query_cache.invalidate_type::<GetWallet>();
    Ok(Json(view))
}
```

`invalidate_type::<GetWallet>()` drops every cached `GetWallet` entry, so the next read re-runs the handler and reflects the write. The withdraw handler does the same, and so does the transfer endpoint from [Sagas](#) — a transfer changes *two* balances, so it must invalidate the family too:

Rust

```
// In the transfer handler – a transfer touches both wallets' views.
api.query_cache.invalidate_type::<GetWallet>();
```

The end-to-end HTTP test proves the loop closes: deposit then withdraw, then read back, and the cached view reflects both writes rather than a stale balance.

Rust

```
// after a deposit(+250) and a withdraw(-50) on an opening balance of 100:
let view: WalletView = body_json(res).await;
assert_eq!(view.balance, 300); // read-after-write is honest
assert_eq!(view.version, 3);
```

Read-through plus explicit evict. The cache declaration lives on the message (`#[firefly(cache_ttl)]`) and the eviction is an explicit `invalidate_type` at the write boundary — read-after-write stays honest because the writer drops the family the moment it changes a balance. The backing store is the same swappable `Adapter` port every other consumer uses.

The cache port

Everything above runs on `MemoryAdapter` by default, but the `QueryCache` — like every other consumer (the session store) — depends on the abstract `Adapter` port, never on a concrete client. That is what lets you move Lumen's cache to Redis without touching a handler. The port:

Rust

```
#[async_trait]
pub trait Adapter: Send + Sync {
    async fn get(&self, key: &str) -> Result<Vec<u8>, CacheError>;
    async fn set(&self, key: &str, value: &[u8], ttl: Option<Duration>) ->
    Result<(), CacheError>;
    async fn delete(&self, key: &str) -> Result<(), CacheError>;
    async fn clear(&self) -> Result<(), CacheError>;
    async fn set_if_absent(&self, key: &str, value: &[u8], ttl:
    Option<Duration>) -> Result<bool, CacheError>;
    async fn exists(&self, key: &str) -> Result<bool, CacheError>;
    async fn delete_prefix(&self, prefix: &str) -> Result<u64, CacheError>;
    async fn health_check(&self) -> Result<(), CacheError>;
}
```

A miss is `CacheError::NotFound`; `ttl: None` (or zero) means no expiry.

Implementation	Backing	Use
<code>MemoryAdapter</code>	<code>HashMap</code> + <code>RwLock</code>	in-process, TTL-aware, the default
<code>NoOpAdapter</code>	none	tests / disabled-cache configs
<code>FallbackAdapter { primary, secondary }</code>	composite	primary-then-secondary, writes to both
<code>RedisAdapter (firefly-cache-redis)</code>	Redis (RESP)	distributed cache

Typed memoization

Lumen's `QueryCache` keys and serializes for you, but when you cache something *outside* the query bus — say, a wallet's risk score fetched from an external service — the `Typed<T>` wrapper is the primitive. It serializes values as `serde_json` bytes (wire-compatible with the other ports) and gives you `get_or_set`: consult the cache, call the loader on a miss, persist, and return the value. A caching error never masks a successful loader result:

Rust

```

use std::sync::Arc;
use std::time::Duration;
use firefly::cache::{MemoryAdapter, Typed};

#[derive(serde::Serialize, serde::Deserialize)]
struct WalletView { id: String }

#[tokio::main(flavor = "current_thread")]
async fn main() -> Result<(), firefly::cache::CacheError> {
    let cache = Arc::new(MemoryAdapter::new());
    let typed: Typed<WalletView> = Typed::new(cache);

    let view = typed
        .get_or_set("wallet:wlt_alice", Some(Duration::from_secs(60)), || async
    {
        // loaded from the ledger / read model on a miss
        Ok(WalletView { id: "wlt_alice".into() })
    })
    .await?;
    assert_eq!(view.id, "wlt_alice");
    Ok(())
}

```

Choosing and composing backends

`Core::new` (and therefore `WebStack`, which Lumen builds on) uses `MemoryAdapter` by default; pass an `Arc<dyn cache::Adapter>` in `CoreConfig.cache` to swap it. For high availability, compose Redis with an in-process fallback so a Redis blip degrades to local caching instead of failing:

Rust

```

use std::sync::Arc;
use firefly::cache::{FallbackAdapter, MemoryAdapter};

// Tries Redis first; on a transport error or miss, falls through to memory
// and writes to both.
let cache = FallbackAdapter::new(redis_adapter, Arc::new(MemoryAdapter::new()));

```

Because everything downstream depends on the port, swapping the backend changes one constructor — Lumen's handlers, the `QueryCache`, and the session store are untouched. A single-process Lumen keeps the default in-memory cache; a multi-node deployment swaps in `RedisAdapter` so a `GetWallet` cached on one node is seen on the next, and so an `invalidate_type` on any node clears the shared entry.

The in-memory → real-infra swap. This mirrors the event store and broker swap you have already seen: develop and test against the in-memory adapter, wire the distributed backend in production via `CoreConfig`. The teaching baseline stays a no-infra `cargo run`; the production path is one line of wiring.

Resilience decorators

`firefly-resilience` guards the calls a cache miss falls through to — and any outbound call, like the Payments settlement from `HTTP Clients`. Four primitives, composable into a `Chain`:

Decorator	Guards against	Error on trip
<code>CircuitBreaker</code>	cascading failure of a slow / failing dep	<code>ResilienceError::CircuitOpen</code>
<code>RateLimiter</code>	outbound rate overrun (token bucket)	<code>ResilienceError::RateLimited</code>
<code>Bulkhead</code>	resource exhaustion from runaway concurrency	<code>ResilienceError::BulkheadFull</code>
<code>Timeout</code>	stuck calls	<code>ResilienceError::Timeout</code>

A `Chain` composes them into a single guarded call:

Rust

```
use std::{sync::Arc, time::Duration};
use firefly::resilience::{Bulkhead, Chain, CircuitBreaker, CircuitConfig, Timeout};

# async fn ex() -> Result<(), firefly::resilience::ResilienceError> {
let breaker = Arc::new(CircuitBreaker::new(CircuitConfig::default()));

let guarded = Chain::new()
    .with(Timeout::new(Duration::from_secs(2))) // per-call deadline
    (outermost)
    .with(breaker) // open the circuit on
    repeated failures
    .with(Bulkhead::new(20)); // cap concurrent in-flight
    calls

guarded.execute(|| async {
    // the protected operation – a cache loader, an upstream call, ...
    Ok(())
}).await?;
# Ok(())
# }
```

Each decorator also stands alone:

Rust

```
use std::time::Duration;
use firefly::resilience::{Bulkhead, CircuitBreaker, CircuitConfig, RateLimiter,
Timeout};

let cb = CircuitBreaker::new(CircuitConfig::default());
let _ = cb.execute(|| async { settle().await }).await;

let rl = RateLimiter::new(100.0, 200);           // 100 rps, burst 200
let _ = rl.execute(|| async { call().await }).await;

let bh = Bulkhead::new(20);
let _ = bh.try_execute(|| async { call().await }).await; // non-blocking;
BulkheadFull if full

let to = Timeout::new(Duration::from_secs(2));
let _ = to.execute(|| async { slow_call().await }).await;
```

Decorator ordering. `Chain` composes the four decorators in call-wrap order — timeout outermost, breaker, bulkhead innermost — so a deadline bounds the whole guarded call while the breaker and bulkhead protect the inner operation. `CircuitBreaker::execute` returns the operation's value (so a guarded read still hands you the `WalletView`), while `Chain::execute` is for guarded operations whose value you discard.

A complete read path

The pieces fit together as a cache-aside read protected by a circuit breaker — exactly the shape a multi-node Lumen would use to serve a wallet view from Redis, repairing from the read model (or the event stream) on a miss while a breaker protects that repair:

Rust

```

use std::sync::Arc;
use std::time::Duration;
use firefly::cache::{MemoryAdapter, Typed};
use firefly::resilience::{CircuitBreaker, CircuitConfig};

let typed: Typed<WalletView> = Typed::new(Arc::new(MemoryAdapter::new()));
let breaker = CircuitBreaker::new(CircuitConfig::default());

let view = typed
    .get_or_set("wallet:wlt_alice", Some(Duration::from_secs(30)), || async {
        // the loader is what the circuit protects: the read model / event
        store.
            breaker.execute(|| async { load_wallet_view("wlt_alice").await }).await
            .map_err(|e| firefly::cache::CacheError::Backend(e.to_string()))
    })
    .await?;

```

You now have a fast, resilient read path — the same one Lumen's declarative `#[firefly(cache_ttl = "30s")]` gives you for free on the query bus.

What changed in Lumen

- We opened up the read-side cache Lumen has used since CQRS: `#[firefly(cache_ttl = "30s")]` on `GetWallet` is honored by the `QueryCache` middleware that `build_app` installs on the bus with `bus.use_middleware(query_cache.middleware())`.
- We saw why the `query_cache` handle lives on `LumenApp`: so every mutating handler — deposit, withdraw, **and** transfer — can call `invalidate_type::<GetWallet>()` and keep read-after-write honest within the 30-second TTL.
- We traced the cache down to its `Adapter` port, the swappable backends (`MemoryAdapter` default → `RedisAdapter` for a multi-node deployment via

`CoreConfig.cache`), the `Typed<T>::get_or_set` memoization primitive, and the `FallbackAdapter` for Redis-with-local-fallback.

- We covered the `firefly-resilience` decorators (`CircuitBreaker`, `RateLimiter`, `Bulkhead`, `Timeout`) and the `Chain` that composes them — the guard that protects both a cache loader and the outbound Payments call from [HTTP Clients](#).

Exercises

1. **Prove the TTL.** Write a test that opens a wallet, reads it (priming the cache), then deposits *directly through the ledger* (bypassing the controller, so no invalidation runs) and reads again within 30 seconds. Assert you still see the *old* balance — demonstrating the TTL is real — then call `query_cache.invalidate_type::<GetWallet>()` and assert the read now reflects the deposit.
2. **Swap in a fallback adapter.** Build a `FallbackAdapter` whose primary always errors on `get/set` and whose secondary is a `MemoryAdapter`. Wire it into `CoreConfig.cache`, run the deposit/withdraw/read flow, and assert correctness is unaffected — the cache degrades to the in-process layer instead of failing.
3. **Guard a loader with a `Chain`.** Wrap a deliberately slow loader in a `Chain::new().with(Timeout::new(...))` and assert that a loader exceeding the deadline surfaces `ResilienceError::Timeout` rather than hanging. Then add a `CircuitBreaker` to the chain, trip it with repeated failures, and assert the next call fails fast with `ResilienceError::CircuitOpen`.

The remaining chapters fold Lumen back together through the declarative-macro lens, then cover shipping it. Continue to [Declarative Services with Macros](#).



CHAPTER 19

Scheduling, Notifications & Webhooks

A real wallet service does more than answer HTTP. It sweeps abandoned wallets overnight, recomputes interest, retries stuck transfers, and — the part a customer notices — emails a daily statement. Two framework concerns cover that back-office work: running code on a timer (`firefly-scheduling`) and delivering messages through swappable providers (`firefly-notifications`), with outbound webhooks (`firefly-callbacks`) and inbound webhooks (`firefly-webhooks`) rounding out the integration story.

By the end of this chapter Lumen will run a **scheduled housekeeping heartbeat**, declared with `#[scheduled]` and started from `main.rs`, and you will know exactly where a daily-statement notification, an outbound balance-changed webhook, and an inbound payment-provider callback would hang off it. Lumen keeps the heartbeat deliberately tiny — it records that it ran — so the macro is shown wired end to end without dragging in a provider SDK.

The back-office concerns. `#[scheduled]` runs code on a timer (cron, fixed rate, fixed delay). `firefly-notifications` delivers messages through a `Channel` + `Dispatcher` abstraction; `firefly-callbacks` pushes signed outbound webhooks; `firefly-webhooks` validates and ingests inbound ones. Each swaps a provider for a one-line registration, never a code change.

The scheduled heartbeat

Lumen's `src/housekeeping.rs` is the whole feature. A zero-argument `async fn` carries `#[scheduled(...)]`; the macro generates a `schedule_<fn>(scheduler)` registration helper. Here is the file, end to end:

Rust

```

use std::sync::atomic::{AtomicU64, Ordering};
use firefly::prelude::*;

/// The number of times the heartbeat has run – observable from a test (and, in
/// a real service, a counter you would surface on `/actuator/metrics`).
static HEARTBEAT_TICKS: AtomicU64 = AtomicU64::new(0);

/// A periodic housekeeping heartbeat. `#[scheduled(fixed_rate = "60s")]`
/// generates `schedule_ledger_heartbeat(scheduler)`; the framework calls this
/// on every tick after the initial delay.
#[scheduled(fixed_rate = "60s", initial_delay = "5s")]
pub async fn ledger_heartbeat() -> Result<(), std::io::Error> {
    HEARTBEAT_TICKS.fetch_add(1, Ordering::Relaxed);
    Ok(())
}

/// Registers the heartbeat on a fresh scheduler and returns it – `main()`
/// starts it; tests assert it registered.
pub fn build_scheduler() -> std::sync::Arc<Scheduler> {
    let scheduler = std::sync::Arc::new(Scheduler::new());
    schedule_ledger_heartbeat(&scheduler);
    scheduler
}

/// How many heartbeat ticks have run so far.
pub fn heartbeat_ticks() -> u64 {
    HEARTBEAT_TICKS.load(Ordering::Relaxed)
}

```

Three things are happening:

- The macro generates the wiring. `#[scheduled(fixed_rate = "60s", initial_delay = "5s")]` on `ledger_heartbeat` emits a `schedule_ledger_heartbeat(&scheduler)` free function. You write the work; the macro writes the trigger + registration. `Scheduler` comes from `firefly::prelude::*`, so the one facade import covers it.

- `build_scheduler` is the composition root for timers. It makes a fresh `Scheduler`, registers the heartbeat, and hands it back. `main.rs` owns when it starts.
- The heartbeat is observable. It bumps an `AtomicU64`, which a test reads — and which, in a real deployment, you would surface as a counter on `/actuator/metrics` (Chapter 15).

`main.rs` starts the scheduler on a background task, because `Scheduler::start` runs until the scheduler is stopped:

Rust

```
// in main():
let scheduler = build_scheduler();
tokio::spawn(async move { scheduler.start().await });
```

`Scheduler::start` runs each task on its own tokio task with panic recovery, and `stop()` shuts down gracefully (in-flight runs finish first). The scheduled tasks also surface on `/actuator/scheduledtasks`.

What the tests assert

`housekeeping.rs`'s test module proves both halves — the task registered, and it ticks when called:

Rust

```

#[test]
fn scheduled_task_registers() {
    let scheduler = build_scheduler();
    let names: Vec<String> = scheduler.tasks().into_iter().map(|t|
t.name).collect();
    assert!(names.contains(&"ledger_heartbeat".to_string()));
}

#[tokio::test]
async fn heartbeat_runs() {
    let before = heartbeat_ticks();
    ledger_heartbeat().await.unwrap();
    assert_eq!(heartbeat_ticks(), before + 1);
}

```

`scheduler.tasks()` returns a `Vec<TaskDescriptor>` (each has a `name`), so a test can introspect the schedule without waiting for a tick — and the dashboard's Scheduled Tasks view (Chapter 15) reads the same list.

The trigger menu

`#[scheduled]` covers the everyday case; the underlying `Scheduler` exposes all four trigger kinds directly, which is what a real Lumen statement run would use:

Rust

```

use std::{sync::Arc, time::Duration};
use firefly::prelude::Scheduler;

let s = Arc::new(Scheduler::new());

// Cron: a 5-field expression (or the 6-field form with leading seconds).
s.cron("daily-statements", "0 2 * * *", || async { Ok(()) }).unwrap();

// FixedRate: fire every period from a fixed anchor (slips on a slow run).
s.fixed_rate("metrics-emit", Duration::from_secs(30), || async { Ok(()) });

// FixedDelay: fire `delay` after the previous run finished.
s.fixed_delay("sweep-abandoned", Duration::from_secs(300), || async { Ok(()) });

```

Trigger	Behavior
<code>CronTrigger</code>	fires when the local wall clock matches the expression
<code>ZonedCronTrigger</code>	fires per the expression in an IANA time zone
<code>FixedRateTrigger</code>	fires every period from a fixed start anchor (slips)
<code>FixedDelayTrigger</code>	fires the delay after the previous run finished

The cron grammar is the canonical 5-field `minute hour day-of-month month day-of-week`, plus an optional 6-field form with leading seconds, the `?` placeholder, and the `@daily` / `@hourly` / `@weekly` macros. For a time zone, build a `ZonedCronTrigger`:

Rust

```

use firefly::scheduling::{parse_cron, ZonedCronTrigger};

// Lumen would run statements at 9am in the customer's region.
// 1-5 is Monday through Friday (the dow domain is 0=Sunday .. 6=Saturday).
let expr = parse_cron("0 9 * * 1-5").unwrap();
let trigger = ZonedCronTrigger::in_zone(expr, "America/New_York").unwrap();

```

Cron grammar. Firefly's cron parser accepts the canonical 5-field expression, an optional 6-field leading-seconds form, the `?` placeholder, and the `@daily` / `@hourly` / `@weekly` macros. `#[scheduled]` generates the registration helper (`schedule_<fn>`) so you write only the work function.

Notifications – where the daily statement hangs

The heartbeat is the hook for outbound messaging. `firefly-notifications` gives you a channel-agnostic `Notification` envelope, a `Channel` transport trait, and a `Dispatcher` that routes a message to the channel registered for its `Kind`. The default `MemoryChannel` records every message (ideal for tests); a real provider adapter slots in behind the same trait. A Lumen statement run, in sketch:

Rust

```

use std::sync::Arc;
use firefly_notifications::{Dispatcher, Kind, MemoryChannel, Notification};

let dispatcher = Dispatcher::new();
dispatcher.register(Arc::new(MemoryChannel::new(Kind::EMAIL)));

dispatcher
    .dispatch(Notification {
        channel: Kind::EMAIL,
        to: "alice@example.com".into(),
        subject: "Your Lumen statement".into(),
        body: "Closing balance: $42.00".into(),
        ..Notification::default()
    })
    .await
    .unwrap();

```

The `Dispatcher` routes by `Kind` (`Kind::EMAIL` / `Kind::SMS` / `Kind::PUSH`), or a custom `Kind::new("...")`; a message for an unregistered kind is a `NotificationError::NoChannel`. For production, register a real channel in place of `MemoryChannel` — same trait, real delivery:

Crate	Channel	Backing
<code>firefly-notifications-smtp</code>	email	<code>lettre</code> (real MIME, STARTTLS)
<code>firefly-notifications-twilio</code>	SMS	Twilio
<code>firefly-notifications-firebase</code>	push	Firebase Cloud Messaging
<code>firefly-notifications-sendgrid</code>	email	SendGrid
<code>firefly-notifications-resend</code>	email	Resend

Because every channel is an `Arc<dyn Channel>`, the heavy provider SDKs stay out of a service that does not select that channel: code against the `Channel` port, register the concrete adapter at wiring time. So Lumen's daily statement is "on the heartbeat tick, build a `Notification` per wallet and `dispatch` it" — the provider is a wiring decision, not a rewrite.

Outbound webhooks – telling other systems a balance changed

When another system needs to *react* to a Lumen event — a fraud monitor that wants every large deposit, say — you push it an outbound webhook with `firefly-callbacks`. Services register `Target` s; the `HmacDispatcher` signs each payload with HMAC-SHA256, retries with exponential backoff, and records every `Attempt` to a pluggable `Store` for audit:

Rust

```
use std::sync::Arc;
use firefly_callbacks::{CallbackEvent, DispatcherConfig, HmacDispatcher,
MemoryStore};

let store = Arc::new(MemoryStore::new());
let dispatcher = HmacDispatcher::new(store, DispatcherConfig::default()); // 3
attempts, 200ms, doubling

// On a large deposit, Lumen would publish a CallbackEvent; the dispatcher signs
// and POSTs it to every registered Target with a stable HMAC-SHA256 signature.
```

Each delivery carries `X-Firefly-Event`, `X-Firefly-Event-Id`, `X-Firefly-Timestamp`, and an `X-Firefly-Signature: sha256=<hmac-hex>` keyed on the target's secret, so any receiver that knows the shared secret verifies Lumen's deliveries with a standard HMAC check.

Inbound webhooks – receiving a provider callback

The mirror image is `firefly-webhooks`: when Lumen's external payment provider calls *back* (a charge settled, a payout failed), the inbound pipeline validates the signature, deduplicates, and dispatches to a processor. The signature validators ship for the common providers:

Validator	Header	Algorithm
<code>HmacValidator</code>	<code>X-Signature</code> (default)	HMAC-SHA256 hex (optional <code>sha256=</code> prefix)
<code>StripeValidator</code>	<code>Stripe-Signature</code>	<code>t=<unix>,v1=<hmac-hex></code> , 5-minute tolerance
<code>GitHubValidator</code>	<code>X-Hub-Signature-256</code>	HMAC-SHA256 hex
<code>TwilioValidator</code>	<code>X-Twilio-Signature</code>	HMAC-SHA1 base64 of URL + sorted form fields

Rust

```
use std::sync::Arc;
use firefly_webhooks::{web, MemoryDlq, Pipeline, StripeValidator};

let pipeline = Arc::new(Pipeline::new(Arc::new(MemoryDlq::new())));
pipeline.register_validator(StripeValidator::new(b"whsec_test"));
let app: axum::Router = web::router(pipeline); // mount under /api/webhooks/...
```

You test that receiver with the `firefly-testkit` signers — `sign_stripe`, `sign_github`, `sign_twilio`, `sign_hmac` — which produce header values byte-identical to what the validators expect, so a signed test request validates exactly as a real provider's would (Chapter 18).

What changed in Lumen

This chapter gave Lumen its first background work and mapped its integration surface:

- `src/housekeeping.rs` declares the `ledger_heartbeat` with `#[scheduled(fixed_rate = "60s", initial_delay = "5s")]`; the macro generates `schedule_ledger_heartbeat(&scheduler)`, and `build_scheduler` registers it on a fresh `Scheduler`.
- `src/main.rs` starts the scheduler on a background tokio task (`(tokio::spawn(async move { scheduler.start().await }));`); the tasks surface on `/actuator/scheduledtasks` and in the dashboard's Scheduled Tasks view.
- The heartbeat's `AtomicU64` is the seam where a real **daily-statement notification** (`firefly-notifications`), an **outbound balance-changed webhook** (`firefly-callbacks`), and an **inbound provider callback** (`firefly-webhooks`) would attach — each a registration, not a rewrite.

Exercises

1. **Cron the statement run.** Replace the `fixed_rate` heartbeat with a `#[scheduled(cron = "0 2 * * *")]` task (or register `s.cron("statements", "0 2 * * *", ..)` directly) and assert it appears in `scheduler.tasks()` by name.
2. **Dispatch on the tick.** Inside `ledger_heartbeat`, build a `Dispatcher` with a `MemoryChannel::new(Kind::EMAIL)`, dispatch a one-line statement, and assert (via `MemoryChannel::messages`) that the message was recorded.
3. **Sign and receive.** Stand up a `Pipeline` with a `StripeValidator`, mount `web::router(..)`, and use `firefly_testkit::sign_stripe` to drive a signed request through it with a `TestClient` — assert it is accepted, then tamper with the body and assert it is rejected.

4. **Audit the outbound.** Wire an `HmacDispatcher` over a `MemoryStore`, dispatch a `CallbackEvent` to a `Target`, and read the recorded `Attempt` rows from the store to confirm the retry policy fired.

Lumen now does background work and knows where its messaging hangs. The next chapter deepens the read-side cache the CQRS layer introduced. Continue to [Caching](#).



CHAPTER 20

Declarative Services with Macros

Lumen is finished. Over twenty chapters it grew from an empty scaffold into a secure, observable, event-sourced CQRS service with a transfer saga and a streaming endpoint — and it depends on exactly **one** Firefly crate. This capstone re-reads the whole service through a single lens: the **declarative macros**. By the end of this chapter you will be able to point at every `#[derive(...)]` and `#[...]` in `samples/lumen` and say precisely what wiring it collapsed into a declaration next to the code. That is the thesis the running crate proves: *one facade + macros = the framework, with the boilerplate gone*.

***Compile-time, not reflective.** Firefly's declarative layer generates wiring at compile time with `proc-macro`s — there is no startup scanning cost and no reflective surprises. A declaration sits next to the code it describes, and the macro emits the `impl`s, routers, and helper functions you would otherwise hand-write. If you have used a batteries-included framework before, the shape will feel familiar; the difference is that the glue is generated and checked by the compiler rather than discovered at startup.*

One dependency, one prelude

Every chapter began the same way. Lumen's `Cargo.toml` lists one Firefly crate:

TOML

[dependencies]

```
firefly = { workspace = true } # the whole framework + every macro
axum    = { workspace = true } # you author the controller handlers
serde   = { workspace = true } # your messages + event payloads
serde_json = { workspace = true }
tokio    = { workspace = true }
uuid     = { workspace = true }
chrono   = { workspace = true }
async-trait = { workspace = true }
```

and every module opens with one glob:

Rust

```
use firefly::prelude::*;
```

That glob brings the whole framework into scope — `Bus`, `Container`, `Scheduler`, `Saga / Step`, `Application / ShutdownHandle`, `Core / CoreConfig`, `WebResult / WebError`, `FireflyError / FireflyResult`, `Mono / Flux` — and every macro. There are also per-crate aliases (`firefly::cqrs::Bus`, `firefly::eventsourcing::EventStore`, `firefly::security::JwtService`, ...) for the types you name explicitly. `axum` and `serde` are the only two ecosystem crates Lumen writes against directly.

Staying lean

The default `firefly` build pulls only the framework's lean *port* crates — no heavy third-party drivers. Lumen needs none, so its build is minimal. Heavy adapters are opt-in cargo features (the swap path every chapter pointed at):

Feature	Pulls in
<code>data-sqlx</code>	relational repository adapter (Postgres / MySQL / SQLite)
<code>data-mongodb</code>	document repository adapter (MongoDB)
<code>eda-kafka</code> / <code>eda-rabbitmq</code> / <code>eda-redis</code> / <code>eda-postgres</code>	event-broker transports
<code>cache-redis</code> / <code>cache-postgres</code>	cache backends
<code>admin</code>	the admin dashboard
<code>full</code>	all of the above

The macros Lumen uses

Lumen exercises the full declarative set. Here is the catalogue, each mapped to the exact Lumen file it lands in:

Macro	Lumen file	Generates
<code>#[derive(Command)] /</code> <code>#[derive(Query)]</code>	<code>commands.rs</code>	the <code>Message</code> impl (<code>#[firefly(validate)]</code> , <code>#[firefly(cache_ttl = "...")]</code>)
<code>#[command_handler] /</code> <code>#[query_handler]</code>	<code>commands.rs</code>	a <code>register_<fn>(bus)</code> helper
<code>#[derive(DomainEvent)]</code>	<code>domain.rs</code>	<code>EVENT_TYPE</code> + <code>to_domain_event</code>
<code>#[derive(AggregateRoot)]</code>	<code>domain.rs</code>	<code>AGGREGATE_TYPE</code> + <code>aggregate()</code> / <code>aggregate_mut()</code>
<code>#[event_listener(topic =</code> <code>"...")]</code>	<code>ledger.rs</code>	a <code>subscribe_<fn>(broker)</code> helper
<code>#[rest_controller] +</code> <code>#[get/post]</code>	<code>web.rs</code>	a <code>routes(state) -> axum::Router</code>
<code>#[scheduled(fixed_rate =</code> <code>"...")]</code>	<code>housekeeping.rs</code>	a <code>schedule_<fn>(scheduler)</code> helper
<code>#[firefly::saga] +</code> <code>#[saga_step]</code>	<code>transfer.rs</code>	<code>TransferSaga::saga()</code> + a <code>run</code> -style step graph with compensation
<code>#[firefly::workflow] +</code> <code>#[workflow_step]</code>	<code>compliance.rs</code>	a workflow <code>run</code> over the DAG of steps
<code>#[firefly::tcc] +</code> <code>#[participant]</code>	<code>tcc_transfer.rs</code>	a TCC <code>run</code> driving each participant's try / confirm / cancel

Lumen also exercises the declarative orchestration macros above (`#[firefly::saga]` / `#[firefly::workflow]` / `#[firefly::tcc]`) in its own source. Several more macros round out the declarative set; Lumen does not exercise these in its own source — it is event-sourced (so the relational `#[firefly::repository]` /

`#[firefly::transactional]` never appear) and handles the remaining cross-cutting concerns by other means — but each is a first-class part of the framework, shown here as a focused standalone example:

Macro	Purpose	Generates
<code>#[derive(Builder)]</code>	a fluent constructor with required/defaulted fields	<code>T::builder()</code> → fluent setters → <code>build() -> Result<T, String></code>
<code>#[derive(Mapper)]</code>	compile-time struct-to-struct conversion	one compile-time <code>From<Source></code> per <code>#[firefly(from = "...")]</code>
<code>#[firefly::repository]</code>	derived-query and custom-query method bodies	method bodies on a <code>SqlxReactiveRepository</code> impl from method names or <code>#[query(...)]</code>
<code>#[firefly::transactional]</code>	a declared transaction boundary	a commit-on- <code>Ok</code> / rollback-on- <code>Err</code> boundary around an <code>async fn</code> body
<code>#[firefly::pre_authorize]</code> / <code>#[firefly::post_authorize]</code>	method-level access control	an access check before the body, or a <code>returnObject</code> check after it
<code>#[derive(Validate)] (+ <code>Valid<T></code>)</code>	JSR-380 bean validation	<code>impl Validate</code> running the field <code>#[validate(email/url/not_empty/length/range/pattern/custom)]</code> checks; the <code>Valid<T></code> web extractor rejects a constraint failure with 422
<code>#[cacheable]</code> / <code>#[cache_put]</code> / <code>#[cache_evict]</code>	declarative caching	a read-through / write-through / evict body around the process-registered cache adapter

Macro	Purpose	Generates
<code>#[async_method]</code>	fire-and-forget async	rewrites an <code>async fn(self: Arc<Self>, ...) -> R</code> into a non-async <code>fn ... -> TaskHandle<R></code> that spawns the body on the registered executor
<code>#[application_event_listener]</code> / <code>#[transactional_event_listener]</code>	in-process events	an <code>@EventListener</code> / <code>@TransactionalEventListener</code> discovered via <code>inventory</code> and fired by <code>publish_event</code> (the latter bound to a transaction commit phase)
<code>#[aspect]</code> (+ <code>#[before]</code> / <code>#[after]</code> / <code>#[around]</code> / ...)	aspect-oriented advice	<code>impl firefly_aop::Aspect</code> + an <code>inventory</code> registration; advice runs around the explicit <code>advised(...)</code> weave point

The DI stereotype derives (`#[derive(Component/Service/Repository/Configuration/AutoConfiguration/Controller)]`, `#[bean]`, `#[autowired]`, `register_all!`) and `#[derive(ConfigProperties)]` round out the set. `#[derive(AutoConfiguration)]` is the auto-config holder whose `#[bean]` s back off behind a `condition_on_missing_bean`, so an application can override any default by declaring its own bean of the same type; `Container::scan()` auto-registers every `#[bean]` method, and `Container::scan_packages(...)` restricts discovery to the named module paths. Lumen wires its collaborators explicitly rather than through the container (chapter 4), so the [DI deep-dive](#) is where you saw those at work.

CQRS – messages and handlers (`commands.rs`)

`#[derive(Command)]` / `#[derive(Query)]` generate the `Message` impl. `#[firefly(validate)]` makes an empty / zero field fail validation before the handler runs; `#[firefly(cache_ttl = "...")]` feeds the query cache. These are verbatim Lumen:

Rust

```
#[derive(Debug, Clone, Default, Serialize, Deserialize, Command)]
#[serde(default)]
pub struct OpenWallet {
    #[firefly(validate)]
    pub owner: String,
    #[serde(rename = "openingBalance")]
    pub opening_balance: i64,
}

#[derive(Debug, Clone, Default, Serialize, Deserialize, Query)]
#[firefly(cache_ttl = "30s")]
pub struct GetWallet {
    pub id: String,
}
```

`#[command_handler]` / `#[query_handler]` mark a free `async fn` and generate a `register_<fn>(bus)` helper that installs it on the bus:

Rust

```
#[command_handler]
pub async fn open_wallet(cmd: OpenWallet) -> Result<WalletView, CqrsError> { /*
... */ }

#[query_handler]
pub async fn get_wallet(q: GetWallet) -> Result<WalletView, CqrsError> { /* ...
*/ }

// generated: register_open_wallet(&bus); register_get_wallet(&bus);
```

Lumen calls each generated helper in one `register(&bus)` fn. Because free fns cannot capture wiring state, the resolved `Ledger` + `ReadModel` are published once through a `bind` / `OnceLock` and reached from the handlers — the pattern chapter 9 introduced.

What it generates. `#[derive(Command)]` / `#[derive(Query)]` emit the `Message` impl; `#[command_handler]` / `#[query_handler]` emit a typed `register_<fn>(bus)` helper that installs the handler. `#[firefly(cache_ttl)]` exposes a TTL the query cache reads. The `Bus` is the command/query gateway every dispatch flows through.

Event sourcing – domain events and the aggregate (`domain.rs`)

`#[derive(DomainEvent)]` stamps each payload with a stable `EVENT_TYPE` discriminator (its struct name) and a `to_domain_event` conversion; `#[derive(AggregateRoot)]` finds the embedded `AggregateRoot` field and generates `Wallet::AGGREGATE_TYPE` plus `aggregate()` / `aggregate_mut()`:

Rust

```
#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize, DomainEvent)]
pub struct WalletOpened {
    pub wallet_id: String,
    pub owner: String,
    pub opening_balance: i64,
}

#[derive(Debug, Clone, AggregateRoot)]
#[firefly(aggregate_type = "Wallet")]
pub struct Wallet {
    pub root: AggregateRoot, // the framework root – uncommitted-event buffer
    + version
    pub owner: String,
    pub balance: Money,
    pub opened: bool,
}
```

The only event-sourcing wiring Lumen writes by hand is the `apply` fold; the discriminators and the wire conversion are generated.

What it generates. `#[derive(DomainEvent)]` emits a stable `EVENT_TYPE` discriminator plus a `to_domain_event` conversion; `#[derive(AggregateRoot)]` emits `AGGREGATE_TYPE` and the `aggregate()` / `aggregate_mut()` accessors over the embedded root. The discriminator pins each event's identity in a stable, versioned JSON wire format, so persisted streams stay readable as the schema evolves.

Messaging – the projection listener (`ledger.rs`)

`#[event_listener(topic = "...")]` marks a free `async fn(Event) -> FireflyResult<()>` and generates a `subscribe_<fn>(broker)` helper that subscribes it to the topic. Lumen's read-model projection is one declaration:

Rust

```
#[event_listener(topic = "wallets.events")]
pub async fn project_wallet_event(ev: Event) -> FireflyResult<()> {
    // reload the wallet's stream, fold to a WalletView, upsert – idempotent.
    Ok(())
}
// generated: subscribe_project_wallet_event(broker)
```

The composition root calls `ledger::subscribe_project_wallet_event(broker)` after binding, closing the CQRS loop.

What it generates. `#[event_listener(topic = "...")]` emits a `subscribe_<fn>(broker)` helper that subscribes the function to the topic on whatever broker transport is wired in. You write only the handler body; the subscription wiring is generated.

Web – the controller (`web.rs`)

`#[rest_controller(path = "...")]` turns an `impl` block into a generated `WalletApi::routes(state) -> axum::Router`. Each method carries one verb mapping and uses ordinary axum extractors, returning `WebResult<T>` so a handler error renders as RFC 9457 `application/problem+json`:

Rust

```
#[rest_controller(path = "/api/v1")]
impl WalletApi {
    #[post("/wallets")]
    async fn open(State(api): State<WalletApi>, Json(body): Json<OpenWallet>)
        -> WebResult<(axum::http::StatusCode, Json<WalletView>)> { /* dispatch
via api.bus */ }

    #[get("/wallets/:id")]
    async fn get(State(api): State<WalletApi>, Path(id): Path<String>)
        -> WebResult<Json<WalletView>> { /* ... */ }
    // ... deposit / withdraw / transfer
}
// generated: WalletApi::routes(state) -> axum::Router
```

The macro also submits each route into a link-time table, so the OpenAPI generator and the actuator `/mappings` endpoint can enumerate Lumen's routes without re-parsing the source.

What it generates. `#[rest_controller]` + `#[get]` / `#[post]` emit a `WalletApi::routes(state) -> axum::Router` plus a link-time mapping-table entry per route. `WebResult<T>` renders any handler error as an `application/problem+json` body (RFC 9457), so error shaping is uniform across every endpoint without per-handler code.

Scheduling – the heartbeat (`housekeeping.rs`)

`#[scheduled(...)]` generates a `schedule_<fn>(scheduler)` helper that registers a zero-argument `async fn` on a `Scheduler`:

Rust

```
#[scheduled(fixed_rate = "60s", initial_delay = "5s")]
pub async fn ledger_heartbeat() -> Result<(), std::io::Error> { /* ... */ }
// generated: schedule_ledger_heartbeat(&scheduler)
```

What it generates. `#[scheduled(...)]` emits a `schedule_<fn>(scheduler)` helper that registers a zero-argument `async fn` on a `Scheduler`. Use `fixed_rate = "60s"` for a fixed cadence (with an optional `initial_delay`), or `cron = "..."` for a cron expression.

Construction – the fluent builder (`#[derive(Builder)]`)

Rust's stdlib derives already cover value-object boilerplate — debug formatting, cloning, structural equality, defaults — with `#[derive(Debug, Clone, PartialEq, Default)]` over `pub` fields. The one ergonomic gap they leave is a *fluent builder* — and that is what `#[derive(Builder)]` fills. It generates `T::builder()` returning a `TBuilder` with one setter per field and a `build() -> Result<T, String>`. By default every field is **required**: `build` returns an `Err` naming the first unset field.

`#[builder(default)]` falls back to `Default::default()`, `#[builder(default = "expr")]` to a custom expression, and `#[builder(into)]` makes the setter accept `impl Into<FieldTy>`:

Rust

```

#[derive(Debug, Clone, Default, Serialize, Deserialize, Command, Builder)]
#[serde(default)]
pub struct OpenWallet {
    #[firefly(validate)]
    #[builder(into)] // accept &str, String, ...
    pub owner: String,
    #[serde(rename = "openingBalance")]
    #[builder(default)] // unset → 0
    pub opening_balance: i64,
}

// A build-style constructor, errors surfaced as a String:
let cmd = OpenWallet::builder()
    .owner("ada") // impl Into<String>
    .opening_balance(10_000)
    .build()?; // Result<OpenWallet, String>

```

A required field left unset surfaces as a `build()` error, not a compile error — the tradeoff a fluent builder makes against a hand-written typed constructor, where the compiler enforces arity. Reach for `#[derive(Builder)]` when a struct has many optional/defaulted fields; keep a plain literal when every field is required and present.

What it generates. `#[derive(Builder)]` emits `T::builder()` returning a `TBuilder` with one setter per field and a `build() -> Result<T, String>` that errors on the first unset required field. `#[builder(default)]` falls back to `Default::default()`, `#[builder(default = "expr")]` to a custom expression, and `#[builder(into)]` makes a setter accept `impl Into<FieldTy>`. Returning a `Result` keeps missing-field handling on the normal `?` path rather than a panic.

Conversion – the compile-time mapper (`#[derive(Mapper)]`)

`#[derive(Mapper)]` generates a compile-time, type-checked `From<Source>` that maps a source struct to a target struct field-by-field. One `#[firefly(from = "Source")]` produces one `From` impl, and the attribute is **repeatable** to map from several sources. Per-field attributes adjust the mapping: `#[firefly(rename = "src")]` reads a differently named source field, `#[firefly(into)]` applies `.into()` to the source value, `#[firefly(with = "fn")]` runs a conversion function, and `#[firefly(default)]` / `#[firefly(default_expr = "expr")]` fill a target field with no source read:

Rust

```
// A read-model view assembled from the domain aggregate.
#[derive(Debug, Clone, Serialize, Deserialize, Mapper)]
#[firefly(from = "Wallet")]
pub struct WalletView {
    #[firefly(rename = "root", with = "aggregate_id")] // read src.root, run
    aggregate_id(..)
    pub id: String,
    pub owner:
String, // same name on both ends: a plain move
    #[firefly(with =
"Money::cents_value")] // src.balance: Money -> i64 via a conversion fn
    pub balance: i64,
    #[firefly(default)] // version is set by the
    projector, not the fold
    pub version: i64,
}
// generates: impl From<Wallet> for WalletView { fn from(src: Wallet) -> Self
{ ... } }

let view: WalletView = wallet.into();
```

Because the generated code is a plain `From` impl, every field is checked by the compiler and there is no runtime cost — that compile-time guarantee is the whole point. Contrast this with the runtime `firefly_data::Mapper`, which converts via two serde passes and is checked at runtime: use the runtime mapper as a dynamic fallback when the source type is not known at compile time (e.g. mapping arbitrary JSON), and prefer `#[derive(Mapper)]` whenever both ends are concrete types — which, in Lumen's projection, they are.

What it generates. `#[derive(Mapper)]` emits one `impl From<Source> for Target` per `#[firefly(from = "Source")]` (the attribute is repeatable). Per-field attributes shape each move: `#[firefly(rename = "src")]` reads a differently named source field, `#[firefly(into)]` applies `.into()`, `#[firefly(with = "fn")]` runs a conversion function, and `#[firefly(default)]` / `#[firefly(default_expr = "expr")]` fill a target field with no source read. The runtime `firefly_data::Mapper` is the dynamic, serde-based fallback for when the source type isn't known until runtime.

Persistence – derived queries and transactions (relational)

These two macros sit on the relational persistence path. Lumen's read model is an in-memory projection over an event stream (chapter 7), so it uses neither — but in a relational service they are the everyday tools, so here is each in brief, with the [persistence chapter](#) as the full reference.

`#[firefly::repository]` turns a `find_by_...` / `count_by_...` / `exists_by_...` / `delete_by_...` method name into a working query body. Applied to an `impl` block, it parses each method name into a derived query, marshals the typed arguments, and delegates to the tested runtime engine — so you declare a typed method and get a working, compiler-checked implementation. The runtime method is chosen from the return type (`Vec<T>` / `Option<T>` → find, `i64` → count, `bool` → exists, `u64` →

delete), and the type exposes its backing `SqlxReactiveRepository` through an accessor (default `self.repository()`, overridable with `#[repository(repo = "...")]`):

Rust

```
#[firefly::repository]
impl AccountRepo {
    async fn find_by_status(&self, status: &str) -> Result<Vec<Account>,
DataError> { unimplemented!() }
    async fn find_by_iban(&self, iban: &str) -> Result<Option<Account>,
DataError> { unimplemented!() }
    async fn count_by_owner(&self, owner: &str) -> Result<i64,
DataError> { unimplemented!() }
    async fn exists_by_email(&self, email: &str) -> Result<bool,
DataError> { unimplemented!() }
}
```

The placeholder `unimplemented!()` bodies are discarded and replaced by the generated delegations.

Paged derived queries. Give a `find_by_...` method a trailing `Pageable` argument (and a `Result<Vec<T>, DataError>` return) and the generated body appends the page's sort and window to the query, delegating to the runtime `SqlxReactiveRepository::find_by_derived_paged`:

Rust

```
#[firefly::repository]
impl AccountRepo {
    async fn find_by_owner(&self, owner: &str, page: Pageable)
        -> Result<Vec<Account>, DataError> { unimplemented!() }
}

// Build the page (1-based index) with sort + window:
let page = Pageable::of(1, 20, RequestSort::of([Order::desc("id")]));
let rows = repo.find_by_owner("ada", page).await?;
```

Custom queries with `#[query(...)]`. When a name-derived query isn't enough, annotate a stub method with `#[query(...)]` and write the statement directly. Native SQL binds each `:name` placeholder to the argument named `name`; the return type selects the operation exactly as for derived queries — `Vec<T>` / `Option<T>` is a list, `i64` a count, `bool` an existence check, and `u64` a modifying statement (INSERT/UPDATE/DELETE, returning affected rows):

Rust

```
#[firefly::repository]
impl AccountRepo {
    #[query("SELECT id, owner FROM accounts WHERE status = :status ORDER BY id DESC")]
    async fn active_by_status(&self, status: &str) -> Result<Vec<Account>,
DataError> { unimplemented!() }

    #[query("UPDATE accounts SET status = :status WHERE id = :id")]
    async fn set_status(&self, id: &str, status: &str) -> Result<u64,
DataError> { unimplemented!() }
}
```

`#[query(sql = "...")]` is the explicit spelling of the native form, and `#[query(jpql = "...", entity = "Account")]` writes the statement against entity names. Each lowers to the matching runtime call — `query_list` / `query_count` / `query_exists` / `query_execute`.

`#[firefly::transactional]` wraps an `async fn`'s body in a transaction governed by the registered `TransactionManager` — commit on `Ok`, roll back on `Err` — so the boundary becomes a declaration. The function must be `async`, must return a `Result<T, E>`, and its error type must implement `From<firefly_transactional::TxError>` so begin/commit failures surface through the normal `?` path. Bare or with options (`propagation`, `isolation`, `read_only`, `timeout_ms`):

Rust

```
#[firefly::transactional]
async fn open_account(repo: &AccountRepo, acct: Account) -> Result<(),
DataError> {
    repo.insert(&acct).await?;    // committed together on Ok,
    repo.insert_audit(&acct).await?; // rolled back together on Err
    Ok(())
}

#[firefly::transactional(propagation = "requires_new", isolation =
"serializable", read_only = false, timeout_ms = 5000)]
async fn reconcile(repo: &LedgerRepo) -> Result<(), DataError> { /* ... */ }
```

These two are the relational counterpart to how Lumen achieves consistency *without* a transaction manager: it appends events to the `EventStore` under optimistic concurrency and projects them, rather than mutating rows inside a `#[transactional]` boundary (chapter 11). Same goal — atomic, consistent writes — reached by two different architectures.

What it generates. `#[firefly::repository]` replaces each stub body with a delegation to `SqlxReactiveRepository`: name-derived methods to `find_by_derived` / `find_by_derived_paged`, and `#[query(...)]` methods to `query_list` / `query_count` / `query_exists` / `query_execute`, picking the call from the return type. `#[firefly::transactional]` wraps the body in a begin/commit/rollback boundary on the registered `TransactionManager`; `propagation` / `isolation` / `read_only` / `timeout_ms` tune that boundary.

Method security – `#[pre_authorize]` / `#[post_authorize]`

Two macros enforce access control at the method boundary, reading the caller's identity from the ambient security context rather than from a `Request`. The full treatment is in the [security chapter](#); here is the catalogue entry.

`#[firefly::pre_authorize(...)]` runs an access check **before** the function body.

Apply it to a `fn` returning `Result<T, E>` whose error type implements

`From<firefly_security::SecurityError>`, so a denial surfaces through the normal `?` path. The rule forms are:

Rust

```

#[firefly::pre_authorize]                                // `authenticated` – any
caller in scope
async fn whoami() -> Result<Profile, AppError> { /* ... */ }

#[firefly::pre_authorize(role = "ADMIN")]                // a single role
async fn close_books(&self) -> Result<(), AppError> { /* ... */ }

#[firefly::pre_authorize(any_role = ["TELLER", "ADMIN"])]
async fn open_account(&self, req: OpenAccount) -> Result<Account, AppError>
{ /* ... */ }

#[firefly::pre_authorize(authority = "wallet:write")]    // a single fine-grained
authority
async fn deposit(&self, id: &str, cents: i64) -> Result<(), AppError> { /* ...
*/ }

#[firefly::pre_authorize(any_authority = ["wallet:write", "wallet:admin"])]
async fn withdraw(&self, id: &str, cents: i64) -> Result<(), AppError> { /* ...
*/ }

```

When no caller is in scope the body is skipped and the macro returns

`Err(SecurityError::Unauthenticated.into())`; when a caller is present but lacks the required role/authority it returns `Err(SecurityError::Forbidden.into())`.

`#[firefly::post_authorize(<bool expr>)]` runs **after** an `async fn` returns `Result<T, E>` and gates the value on a boolean expression. The expression sees `result` (a `&T` to the returned value — the `returnObject`) and `auth` (a `&Authentication`); if it evaluates to `false` the value is discarded and the call returns `Forbidden`:

Rust

```
// Only return the wallet if the caller owns it.
#[firefly::post_authorize(result.owner == auth.subject())]
async fn get_wallet(&self, id: &str) -> Result<WalletView, AppError> { /* ... */ }
```

Both macros read the caller from the ambient context in `firefly_security`: `with_authentication_scope(auth, fut).await` runs a future with an `Authentication` in scope, `current_authentication() -> Option<Authentication>` reads it, and `check_access(&AccessRule) -> Result<Authentication, SecurityError>` is the runtime check the macros expand to, over `AccessRule:: {Authenticated, Role, AnyRole, Authority, AnyAuthority}`. Because `BearerLayer` scopes the authentication for the whole downstream call (on both the anonymous and verified paths), these checks work on a service method that never sees the `Request` — the macro reads from scope, not from a handler argument.

What it generates. `#[pre_authorize]` emits a `check_access(&AccessRule)?` against the ambient context before your body, with an empty attribute defaulting to `AccessRule::Authenticated`. `#[post_authorize]` evaluates its boolean over `result` and `auth` after the body and converts `false` into a `Forbidden` error. Both rely on `SecurityError: From` for the function's error type, so denials travel the normal `Result` path.

How it works: the `__rt` contract

A `proc-macro` crate cannot re-export runtime types, so macro-generated code references every runtime type through the facade's hidden `__rt` contract path — e.g. `::firefly::__rt::firefly_cqrs::Bus`. That is why Lumen, depending only on `firefly` (plus the `axum` / `serde` it writes against anyway), compiles whatever a macro

expands to without listing the underlying `firefly-*` crates. You never write `__rt` yourself. If you rename or shim the facade, pass `#[firefly(crate = "my_firefly")]` to any macro to override the leading segment.

Rust has no reflective autowiring, so the declarative layer removes the *mechanical* boilerplate, not the explicitness: Lumen still calls `register(&bus)`, still calls `subscribe_project_wallet_event(broker)`, still hands `WalletApi::routes(state)` to the web stack. The macros generate the `impl`s, the routers, and the helper functions — the wiring you would otherwise hand-write.

The whole crate, declaratively

Read top to bottom, the macros tell Lumen's story:

Text

```
money.rs      (no macros – a pure value object; the no-thiserror promise)
domain.rs     #[derive(DomainEvent)] x3   #[derive(AggregateRoot)]
ledger.rs     #[event_listener(topic = "wallets.events")]
commands.rs   #[derive(Command)] x3      #[derive(Query)]
              #[command_handler] x3      #[query_handler]
transfer.rs   #[firefly::saga] + #[saga_step] x2
compliance.rs #[firefly::workflow] + #[workflow_step] x3
tcc_transfer.rs #[firefly::tcc] + #[participant] x2
security.rs   (JwtService / BearerLayer / FilterChain – runtime APIs)
web.rs        #[rest_controller] + #[get] / #[post] x7
housekeeping.rs #[scheduled(fixed_rate = "60s", initial_delay = "5s")]
```

Note what is *not* a macro: the security filter chain is built with a runtime builder (`FilterChain::new().require(...)`), because its shape is data, not a fixed declaration — and Lumen keeps it explicit so the control flow stays visible. The saga, workflow, and TCC are now declarative macros (`#[firefly::saga]`,

`#[firefly::workflow]`, `#[firefly::tcc]`); only the filter chain remains a runtime builder. Declarative where it collapses boilerplate, explicit where the graph is the point: that balance is the whole design.

Verifying the crate

Everything above compiles and is tested. From the workspace root:

Shell

```
cargo build -p firefly-sample-lumen
cargo test -p firefly-sample-lumen
# 42 unit + 12 HTTP + 1 doctest
cargo test -p firefly-sample-lumen --features streaming # + 3 streaming tests
cargo clippy -p firefly-sample-lumen --all-targets -- -D warnings
```

The HTTP tests drive the macro-generated `WalletApi::routes(...)` router in-process through `tower::ServiceExt::oneshot` (no socket bound), proving the generated routes, the CQRS handlers, validation (422), the not-found path (404), the auth boundary (401), the transfer saga (happy + compensation), and the projection convergence all work end to end — every prose listing in this book is a slice of that running crate.

What changed in Lumen

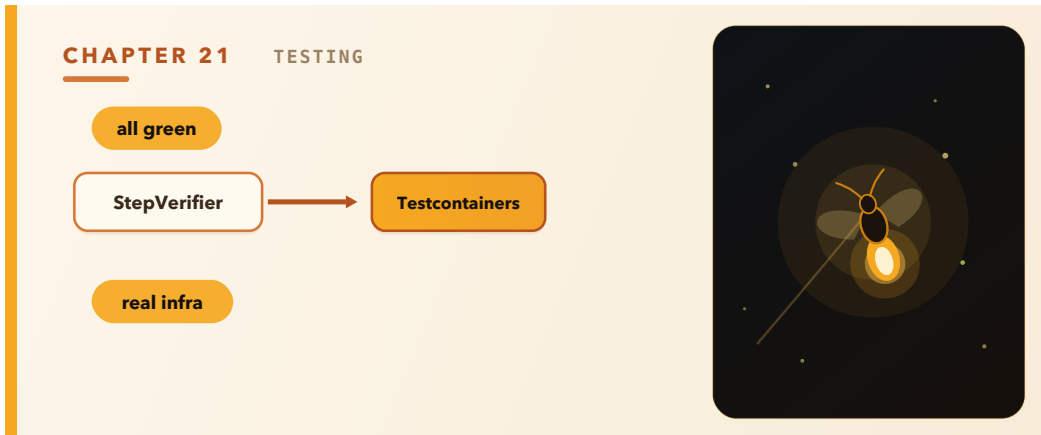
Nothing — this chapter is the retrospective, not a new feature. Re-read as a catalogue, Lumen's macros are: three `#[derive(DomainEvent)]` + one `#[derive(AggregateRoot)]` (`domain.rs`); one `#[event_listener]` (`ledger.rs`); three `#[derive(Command)]` + one `#[derive(Query)]` + three `#[command_handler]` + one `#[query_handler]` (`commands.rs`); the declarative orchestration set — `#[firefly::saga]` + `#[saga_step]` (`transfer.rs`), `#[firefly::workflow]` + `#[workflow_step]` (`compliance.rs`), and `#[firefly::tcc]` + `#[participant]`

(`tcc_transfer.rs`); one `#[rest_controller]` with seven verb methods (`web.rs`); and one `#[scheduled]` (`housekeeping.rs`). Each replaced a chunk of hand-written wiring with a declaration next to the code — and all of it arrived through one dependency and one prelude glob.

Exercises

1. **Trace one macro end to end.** Pick `#[derive(Query)]` on `GetWallet`. Find where its generated `cache_ttl()` is read (the `QueryCache` middleware in `web.rs`) and the test that asserts it (`get_wallet_carries_cache_ttl` in `commands.rs`). Change the TTL to `"5s"` and re-run the tests.
2. **Add a verb.** Add a `#[get("/wallets/:id/events")]`-style read method to the `#[rest_controller]` impl (non-streaming: return the event list as JSON) and confirm `WalletApi::routes` picks it up with no other change.
3. **Add a scheduled task.** Write a second `#[scheduled(cron = "0 0 * * *")]` function in `housekeeping.rs`, register it in `build_scheduler`, and assert it appears in `scheduler.tasks()`.
4. **Count the wiring you didn't write.** For each macro in the catalogue, name the helper or impl it generated (`register_*`, `subscribe_*`, `schedule_*`, `routes`, `EVENT_TYPE`, `AGGREGATE_TYPE`). That list is the boilerplate the declarative layer wrote for you.

That is Lumen, complete and declarative. The appendices that follow are reference: a [module index](#) and a [glossary](#).



CHAPTER 21

Testing Firefly Applications

Every chapter so far has shown Lumen's listings *and* the tests that keep them honest — that is the whole point of the book: the prose is verified against a crate that compiles and passes its suite. This chapter steps back and looks at the test strategy as a whole, the way you would design it for your own service.

By the end of this chapter you will know how Lumen tests at three levels — pure unit tests with no I/O, in-process HTTP tests that drive the *real* router through `tower::oneshot`, and the `firefly-testkit` helpers (`TestClient`, `Slice`, `assert_event_published`) that make all of it terse — and how the same crate scales up to real-infrastructure integration tests when you need a live Postgres or Kafka. Lumen's gate is **42 unit tests + 12 HTTP tests + 1 doctest**, all hermetic; the streaming feature adds 3 more.

***Design note.** Firefly's testing surface is built as three deliberate tiers that map onto how a service actually layers: plain `#[tokio::test]` unit tests with no I/O, in-process slice and HTTP tests that drive the real router without binding a socket, and env-gated integration tests*

against live infrastructure. `TestClient` is Firefly's in-process HTTP client, `Slice` is a focused dependency-injection container for a single test, and `assert_event_published` is the event-emission assertion — one terse helper per tier. The split will feel familiar if you've used a batteries-included framework, but each piece is a native Firefly API designed around the in-memory-first stack the rest of the book builds on.

The in-process testing model

Lumen's default stack is entirely in-memory — `MemoryEventStore`, `InMemoryBroker`, a `Mutex<HashMap>` read model — so almost every test runs as a plain `#[tokio::test]` with **no socket and no external service**. The HTTP tests do not bind a port; they hand a `Request` to the router and `await` the `Response`. That is fast, deterministic, and CI-friendly.

One caveat is worth stating up front, because Lumen's tests are built around it. Lumen's free-function command handlers and its read-model projection publish their collaborators through process-global `OnceLock`s (the declarative-macro pattern from Chapter 9). So the first `build_router()` in a test binary wires the shared ledger that every later test in that binary then drives. The tests cope by giving each wallet a fresh owner and a server-assigned id, so there is no cross-test interference even though the ledger is shared.

Unit tests with no infrastructure

The domain and the value object are pure, so their tests need nothing. Lumen's `money.rs` and `domain.rs` assert invariants directly — exact-cents arithmetic, positive amounts, sufficient funds, owner required. The CQRS layer is just as direct: dispatch a command through a real `Bus` and assert the result. This is the heart of `commands.rs`'s test module:

Rust

```

#[tokio::test]
async fn handlers_dispatch_through_the_bus() {
    let ledger = Ledger::new(
        Arc::new(MemoryEventStore::new()),
        Arc::new(InMemoryBroker::new()),
    );
    bind(ledger, Arc::new(ReadModel::default()));
    let bus = Bus::new();
    // Validation middleware enforces the `#[firefly(validate)]` checks.
    bus.use_middleware(firefly::cqrs::ValidationMiddleware::new());
    register(&bus);

    let opened: WalletView = bus
        .send(OpenWallet { owner: "alice".into(), opening_balance: 100 })
        .await
        .unwrap();
    assert_eq!(opened.balance, 100);

    let after: WalletView = bus
        .send(Deposit { wallet_id: opened.id.clone(), amount: 50 })
        .await
        .unwrap();
    assert_eq!(after.balance, 150);
}

```

Validation is tested without ever touching HTTP — call `.validate()` on the command directly, because `#[derive(Command)]` generated it from the `#[firefly(validate)]` fields:

Rust

```
#[test]
fn open_wallet_validates_owner() {
    assert!(OpenWallet::default().validate().is_err()); // empty owner
    assert!(OpenWallet { owner: "alice".into(), opening_balance:
0 }.validate().is_ok());
}
```

Security (`security.rs`), the saga (`transfer.rs`), and the scheduled task (`housekeeping.rs`) each carry their own `#[cfg(test)] mod tests` in the same spirit — mint-then-verify a token, run the saga happy path and the compensation path, register the heartbeat and assert it ticks.

In-process HTTP tests with `tower::oneshot`

The end-to-end suite lives in `tests/http.rs` and drives the **fully-wired** `build_router()` — the real `#[rest_controller]` routes, the CQRS handlers, the event-sourced ledger, the read-model projection, the transfer saga, *and* the JWT/RBAC enforcement from Chapter 14. No mocks: every layer is the production layer, just over in-memory infrastructure.

The pattern is `Router` + `tower::ServiceExt::oneshot`. Lumen wraps it in two tiny helpers so each test reads as one HTTP round-trip:

Rust

```

use http_body_util::BodyExt;
use tower::ServiceExt;

fn post(path: &str, body: serde_json::Value, auth: bool) -> Request<Body> {
    let mut b = Request::post(path).header("content-type", "application/json");
    if auth {
        b = b.header("authorization", bearer()); // Bearer <minted CUSTOMER
token>
    }
    b.body(Body::from(serde_json::to_vec(&body).unwrap())).unwrap()
}

async fn body_json<T: serde::de::DeserializeOwned>(res: Response) -> T {
    let bytes = res.into_body().collect().await.unwrap().to_bytes();
    serde_json::from_slice(&bytes).unwrap()
}

```

A test then opens a wallet and asserts the projected read came back through CQRS:

Rust

```

#[tokio::test]
async fn open_then_get_round_trips_through_cqrs() {
    let opened = open_wallet("alice", 1_000).await; // POST /api/v1/wallets,
    asserts 201
    assert_eq!(opened.balance, 1_000);

    // GET dispatches the #[query_handler]; the projection has already folded
    // the WalletOpened event into the read model.
    let res = build_router()
        .await
        .oneshot(get(&format!("/api/v1/wallets/{}", opened.id)))
        .await
        .unwrap();
    assert_eq!(res.status(), StatusCode::OK);
    let fetched: WalletView = body_json(res).await;
    assert_eq!(fetched.balance, 1_000);
}

```

The same file proves the saga

(`transfer_saga_happy_path_moves_funds_between_wallets`), the compensation path (`transfer_saga_overdraft_compensates_and_is_422`), and the problem-rendering for the three failure modes — a missing token is a 401, an empty owner is a 422, an unknown id is a 404 — each asserting the `application/problem+json` content type. That single suite is the proof that the whole stack composes.

The testkit: `TestClient`, `Slice`, event assertions

`firefly-testkit` packages the boilerplate above into reusable helpers. Lumen's own tests use the raw `tower::oneshot` form to show the mechanism with no magic, but in your service the testkit makes the same tests far shorter. Three pieces matter most.

TestClient – an in-process HTTP client (feature `web`)

`TestClient::new(router)` wraps any axum `Router` and gives you `get` / `post` / `put` / `patch` / `delete` plus a fluent assertion API on the `TestResponse`. The `open_then_get` test above, rewritten with `TestClient`:

Rust

```
use firefly_testkit::TestClient;

#[tokio::test]
async fn open_then_get_with_testclient() {
    let client = TestClient::new(build_router().await);

    let created = client
        .post("/api/v1/wallets", &serde_json::json!({ "owner": "alice",
"openingBalance": 1000 }))
        .await
        .assert_status(201);
    let id = created.json_path("$.id").unwrap();

    client
        .get(&format!("/api/v1/wallets/{}", id.as_str().unwrap()))
        .await
        .assert_status(200)
        .assert_json_path("$.balance", 1000);
}
```

`assert_status`, `assert_success`, `assert_header`, `assert_body_contains`, `assert_json_eq`, `assert_json_path` / `assert_json_path_exists` / `assert_json_path_absent`, and `json::<T>()` / `json_path("$.field")` cover the common assertions; the path grammar is the single-result JSONPath subset (a leading `$`, dotted or bracketed member access, array indexing). Each assertion returns `&Self` so they chain. (Blocking variants — `post_blocking`, `get_blocking`, ... — exist for non-async test contexts.)

Slice – a focused DI container for a test

`Slice` builds a minimal `firefly-container` for a slice test: register only the collaborators the unit under test needs, then resolve them. It gives you a focused dependency-injection container — the wiring for the unit under test without standing up the full application context:

Rust

```
use firefly_testkit::Slice;
use firefly_container::{Container, ContainerError, Scope};

let slice = Slice::new()
    .instance(ReadModel::default()) // a ready instance (a
    "mock_bean")
    .register::<MyService, _>(Scope::Singleton, |c: &Container| {
        Ok(MyService::new()) // a factory; resolve
        deps from `c`
    })
    .build();

let read_model: std::sync::Arc<ReadModel> = slice.get();
```

`register` / `register_named` take a factory `|c: &Container| -> Result<T, ContainerError>` (it may resolve its own dependencies from `c`); `instance` / `instance_named` install a ready value (the override/mock path); `bind` coerces a concrete type to a trait object; and `eager` forces construction at build time. `build()` returns a `BuiltSlice` you resolve from with `get::<T>()` / `get_named::<T>(name)`.

Asserting emitted events

`SpyBroker` records what a handler published; `assert_event_published(&spy, "Type")` asserts an event of that type was recorded and returns it (the `_with` variant also checks the payload, parsed as a JSON object, contains the given key/value pairs (a

subset match); `assert_no_events_published` asserts none). `must_encode` / `must_decode` are panic-on-failure JSON helpers. A Lumen-flavored example — proving an open emits a `WalletOpened` :

Rust

```
use firefly_testkit::{assert_event_published, must_encode, SpyBroker};

#[test]
fn open_emits_wallet_opened() {
    let spy = SpyBroker::new();
    // The ledger would publish through the broker; here we record the envelope
    // the projection would consume.
    spy.record("wallets.events", "WalletOpened",
               &must_encode(&serde_json::json!({ "id": "wlt_1", "owner":
"alice" })));

    let event = assert_event_published(&spy, "WalletOpened");
    assert_eq!(event.topic, "wallets.events");
}
```

Webhook signers

When Lumen grows an inbound webhook (Chapter 16), the testkit's HMAC signers — `sign_hmac`, `sign_stripe`, `sign_github`, `sign_twilio` — produce header values byte-identical to what each `firefly-webhooks` validator expects, so a signed test request validates exactly as a real provider's would:

Rust

```
use firefly_testkit::sign_stripe;

let sig = sign_stripe(b"whsec_test", br#"{"type":"charge.succeeded"}"#,
1_700_000_000);
// Attach `sig` as the `Stripe-Signature` header on a TestClient POST.
```

Testing reactive pipelines

The streaming endpoint (Chapter 20) builds a `Flux`. A reactive pipeline is tested by driving it to a terminal — `block()`, `collect_list()`, `count()` — and asserting the resolved value. This is `firefly-reactive`'s way to verify a stream end to end; it will feel familiar if you've worked with a reactive-streams library, but it is plain async Rust assertions over a resolved `Flux`:

Rust

```
use firefly_reactive::Flux;

#[tokio::test]
async fn pipeline_filters_and_maps() {
    let out = Flux::range(1, 5)
        .filter(|x| x % 2 == 1)
        .map(|x| x * 10)
        .collect_list()
        .block()
        .await
        .unwrap()
        .unwrap();
    assert_eq!(out, vec![10, 30, 50]);
}
```

Lumen's streaming tests (`tests/streaming.rs` , gated behind the `streaming` feature) take the HTTP route instead: open a wallet, deposit, then `GET /events` and assert two NDJSON lines (`WalletOpened` + `MoneyDeposited`) by default, `text/event-stream` with `?format=sse` , and a 404 for an unknown wallet.

Real-infrastructure integration tests

Lumen runs hermetically, but the production adapters need real services. The workspace ships a `docker-compose.yml` with Postgres, Redis, RabbitMQ, a Kafka-compatible Redpanda, Keycloak, S3/Blob emulators, and an SMTP capture. The convention throughout the adapter crates is: a test reads a connection URL from the environment and **skips when it is unset**, so the default `cargo test` stays green on a bare machine while CI flips the full suite on:

Rust

```
#[tokio::test]
#[ignore = "requires postgres (DATABASE_URL)"]
async fn postgres_event_store_round_trips() {
    let Ok(url) = std::env::var("DATABASE_URL") else { return }; // skip on a
    bare machine
    // ... drive the Postgres-backed EventStore against the live database
}
```

Shell

```
docker compose up -d                # start the backing services
DATABASE_URL=postgres://firefly:firefly@localhost:5432/firefly \
REDIS_URL=redis://localhost:6379/0 \
  cargo test --workspace -- --ignored # run the env-gated suite
docker compose down
```

Running Lumen's suite

From the workspace root (with `export PATH="/opt/homebrew/bin:$PATH"`):

Shell

```

cargo build -p firefly-sample-lumen
cargo test -p firefly-sample-lumen           # 42 unit + 12 HTTP +
1 doctest
cargo test -p firefly-sample-lumen --features streaming # + 3 streaming tests
cargo clippy -p firefly-sample-lumen --all-targets -- -D warnings
cargo fmt -p firefly-sample-lumen -- --check

```

If a snippet in any chapter drifts from the file, this gate fails — which is how the book stays honest.

What changed in Lumen

Nothing in `src/` — this chapter is the retrospective on the test code that grew alongside every feature:

- Unit tests per module: `money` and `domain` invariants, `commands` validation + bus dispatch, `security` mint/verify/reject, `transfer` happy + compensation, `housekeeping` registration + tick.
- The `tests/http.rs` end-to-end suite drives the real `build_router()` with `tower::oneshot`, covering open → get → deposit/withdraw → transfer (happy + compensated) → projection convergence → 401/422/404 problems.
- `tests/streaming.rs` (feature-gated) exercises the NDJSON / SSE endpoint.
- The `firefly-testkit` helpers — `TestClient`, `Slice`, `assert_event_published`, the HMAC signers — are the terse path to the same coverage in your own service.

Exercises

1. Rewrite a test with `TestClient`. Take `deposit_and_withdraw_update_the_balance` from `tests/http.rs` and rewrite it

- using `TestClient` + `assert_json_path`. Add the `Authorization` header via `TestClient::request` so the mutation is authenticated.
2. A `Slice` test for the read model. Use `Slice` to register a `ReadModel::default()` instance, project a `WalletOpened` into it by hand, and assert `find` returns the view — all without the bus or the router.
 3. Event assertion on the ledger. Wire a `SpyBroker` into a `Ledger` in a test, commit a deposit, and use `assert_event_published_with(&spy, "MoneyDeposited", &serde_json::json!({ "amount": 50 })))` to prove the payload's `amount` field equals 50.
 4. A skipping integration test. Write an `#[ignore]` d test that reads `DATABASE_URL`, returns early when unset, and otherwise opens a wallet against a Postgres-backed event store. Confirm it skips with a plain `cargo test` and runs with the variable set.

With the stack proven at every level, the remaining chapters cover the CLI and shipping Lumen to production. Continue to [The CLI](#).

CHAPTER 22 THE CLI

```
$ firefly new  
$ firefly run  
$ firefly migrate █
```



CHAPTER 22

The firefly CLI

So far you have built Lumen by hand — a file at a time, `cargo build` after each chapter. By the end of this chapter you will know the other way to start a service like Lumen: the `firefly` developer CLI scaffolds a project, generates the same artifacts the earlier chapters wrote by hand, runs the binary with profiles and overrides, manages migrations, exports an OpenAPI document, and introspects a running Lumen over its actuator surface — all from one binary built for a compiled Cargo workspace.

One binary, the whole lifecycle. `firefly` scaffolds a project, generates code artifacts, runs the binary with profiles and overrides, stamps build-info, manages migrations, exports OpenAPI, and introspects a running service — the everyday developer loop in a single command-line tool (`new` / `run` / `generate` / `db` / `doctor` / `sbom` / `license`).

Installing

Shell

```
cargo install --path crates/cli  # installs the `firefly` binary
firefly --help                  # prints the banner + every command
firefly --version                # 26.6.5
```

Command overview

Command	Purpose
<code>firefly new <name></code>	scaffold a new firefly-rust project
<code>firefly generate <kind> <name></code> (alias <code>g</code>)	generate a code artifact
<code>firefly run</code>	<code>cargo run</code> with profile / override flags
<code>firefly build <info\ image></code>	stamp <code>build-info.json</code> / build an OCI image
<code>firefly info</code>	framework + environment information
<code>firefly doctor</code>	toolchain checks (rustc, cargo, git, ...)
<code>firefly db <init\ migrate\ upgrade\ status></code>	migration management
<code>firefly openapi --format json\ yaml [-o file]</code>	export an OpenAPI 3.1 document
<code>firefly actuator <endpoint> --url <base></code>	query a running app's <code>/actuator/*</code>
<code>firefly routes\ env\ health\ metrics --url <base></code>	remote introspection of a running app
<code>firefly beans\ conditions --url <base></code>	DI / auto-config report of a running app
<code>firefly completion <shell></code>	print a shell-completion script
<code>firefly sbom [--json]</code>	software bill of materials from <code>Cargo.lock</code>
<code>firefly license</code>	framework + dependency license report

Scaffolding a project

`firefly new` generates a workspace-less Cargo crate with a `src/` tree, a `firefly.yaml`, a `.gitignore`, a `README.md`, a `Dockerfile`, and `tests/` — roughly the shape Lumen started from in chapter 2:

Shell

```
firefly new lumen --archetype web-api --features web,data,cqrs --git
firefly new my-lib --archetype library --dep-path ../../crates # local dev
deps
firefly new --list # archetypes +
features
firefly new svc --dry-run # plan without
writing
```

Archetypes: `core`, `web-api`, `web`, `hexagonal`, `library`, `cli`. The generated `firefly-*` dependencies are git / path / version configurable (`--dep-path` / `--dep-version`, defaulting to the canonical GitHub repo). `--git` initializes a repository with an initial commit; `--force` overwrites an existing target directory.

Archetypes scaffold a working service. `firefly new --archetype web-api` stamps the entry point, a controller, and the dependency set so the first `cargo run` boots. `--features` selects the opt-in adapters the [facade chapter](#) lists (`web`, `data`, `cqrs`, `eda`, `cache`, `security`, ...).

Generating artifacts

`firefly generate` writes a code artifact into the current project, detecting the package, archetype, and feature flags from `Cargo.toml` + `firefly.yaml`. These are the same pieces you wrote by hand for Lumen — a command + handler, an aggregate, a saga:

Shell

```

firefly generate command OpenWallet      # command + handler in src/cqrs/
firefly generate query  GetWallet         # query  + handler
firefly generate aggregate Wallet         # a struct embedding
firefly_eventsourcing::AggregateRoot
firefly generate saga    MoneyTransfer --dry-run
firefly generate migration AddWallets     # V###__add_wallets.sql in migrations/
firefly g handler Deposit                 # `g` is the alias

```

Artifact kinds: `handler`, `route`, `entity`, `repository`, `dto`, `aggregate`, `command`, `query`, `saga`, `migration`. Names are accepted in any case and converted as needed; `--force` overwrites, `--dry-run` plans without writing.

Running the app

`firefly run` is a thin wrapper over `cargo run` that maps profile and configuration flags to the `FIREFLY_*` environment variables the framework reads at startup, then exec's Cargo:

Shell

```

firefly run                                # cargo run
firefly run -p dev -p test                 # FIREFLY_PROFILES_ACTIVE=dev,test
firefly run -D server.port=9090            # FIREFLY_SERVER_PORT=9090
firefly run --env LUMEN_ADDR=0.0.0.0:8080  # a raw env var for the process
firefly run --debug                        # FIREFLY_LOGGING_LEVEL_ROOT=DEBUG
firefly run --release --bin lumen          # cargo run --release --bin lumen
firefly run --dry-run                      # print the resolved env + cargo
command

```


Flags become **FIREFLY_*** env vars. `firefly run -p dev -D server.port=9090` maps to `FIREFLY_PROFILES_ACTIVE=dev` and `FIREFLY_SERVER_PORT=9090`, then exec's `cargo run`. The resolution order is CLI flag → env → config file. A Firefly service is a single compiled binary, so there is no live-reload or worker-process selection — you rebuild and rerun.

For Lumen specifically, recall that `main.rs` reads `LUMEN_ADDR` / `LUMEN_ADMIN_ADDR` directly, so the equivalent of the two-port bind is:

Shell

```
firefly run --bin lumen \
  --env LUMEN_ADDR=127.0.0.1:8080 \
  --env LUMEN_ADMIN_ADDR=127.0.0.1:8081
```

Building for release

Plain compilation is `cargo build`; the `build` group adds the two artifacts a release pipeline needs:

Shell

```
firefly build info                                # write build-info.json (git SHA + UTC
time)
firefly build info -o target/build-info.json
firefly build image -t lumen:1.0.0
# OCI image via Cloud Native Buildpacks (`pack`)
firefly build image --builder docker              # or a plain Dockerfile build
```

`build info` writes the `build-info.json` that `/actuator/info` surfaces — the admin-port `info` endpoint chapter 15 wired for Lumen reads it when present, so the git SHA and build time show up next to the `InfoContributor` block.

Database migrations

Lumen runs over an in-memory event store, so it ships no SQL migrations — but the moment you swap in the Postgres event store from chapter 20, `firefly db` manages the schema. It wraps the `firefly-migrations` forward-only runner:

Shell

```
firefly db init                                # migrations/ + starter
V001__init.sql
firefly db migrate -m "create wallets"         # writes V002__create_wallets.sql
firefly db upgrade --url sqlite://app.db       # apply pending migrations
firefly db status --url sqlite://app.db        # show applied + pending
```

The database URL resolves from `--url`, then `$DATABASE_URL`, then `firefly.datasource.url` in `firefly.yaml`, defaulting to `sqlite://firefly.db`.

Forward-only migrations. `firefly db` drives Firefly's own forward-only migration runner. Because the runner is forward-only, `firefly db downgrade` is unsupported — write a corrective migration instead.

NOTE

The CLI migration backend is **SQLite** via `rusqlite`; a `postgres://` / `mysql://` URL returns a clear "not wired into the CLI" error. For another driver, adapt the `firefly_migrations::Database` port and call `run` directly from your build, rather than through the CLI.

Exporting OpenAPI

Shell

```
firefly openapi                                # OpenAPI 3.1 JSON to stdout
firefly openapi --format yaml -o openapi.yaml
```

The document metadata (`info.title` / `info.version` / `description`) is read from `firefly.yaml` then `Cargo.toml`. A compiled binary cannot boot an arbitrary app to enumerate live routes, so the CLI emits a metadata-stamped **skeleton**; to emit Lumen's real routes, build them with `firefly_openapi::Builder` (which reads the `#[rest_controller]` route table) and serve them with `Builder::router()`.

Introspecting a running app

These commands query a *running* Lumen over HTTP — a compiled binary has no offline DI context to boot, so `--url` is required. Point them at Lumen's admin port (the actuator surface from chapter 15):

Shell

```
firefly health --url http://localhost:8081 # -> /actuator/health
firefly env    --url http://localhost:8081 # -> /actuator/env
firefly routes --url http://localhost:8081 # -> /actuator/mappings
firefly metrics requests --url http://localhost:8081
firefly actuator info    --url http://localhost:8081 --json
firefly actuator metrics requests --url http://localhost:8081 --json
firefly beans           --url http://localhost:8081 # the DI container's bean table
firefly conditions      --url http://localhost:8081 # the auto-configuration report
```

Introspecting a running service. `firefly beans` renders the *DI container's bean table*, `firefly conditions` the conditional-bean evaluation report, and `firefly routes` the route table — all read over HTTP from a running service's actuator port (`/actuator/beans` , `/actuator/conditions` , and `/actuator/mappings`).

Diagnosing, completing, and auditing

Shell

```
firefly info           # framework version + which optional adapters are
built
firefly doctor         # checks rustc, cargo, git, clippy, rustfmt, docker
firefly completion zsh # > ~/.zfunc/_firefly (bash | zsh | fish |
powershell)
firefly sbom           # a software bill of materials from Cargo.lock
firefly sbom --json    # machine-readable, for a compliance pipeline
firefly license        # the framework + dependency license report
```

`firefly doctor` is the first thing to run on a fresh machine: it reports your `rustc` / `cargo` versions and whether `git` , `clippy` , `rustfmt` , and `docker` are on the `PATH` , ending with "All required checks passed!" or a list of what to fix.

Running through cargo

If you have not installed the binary, drive the CLI through cargo from a framework checkout:

Shell

```
make cli ARGS="doctor"
make cli ARGS="new orders --archetype web-api"
cargo run -p firefly-cli --bin firefly -- info
```

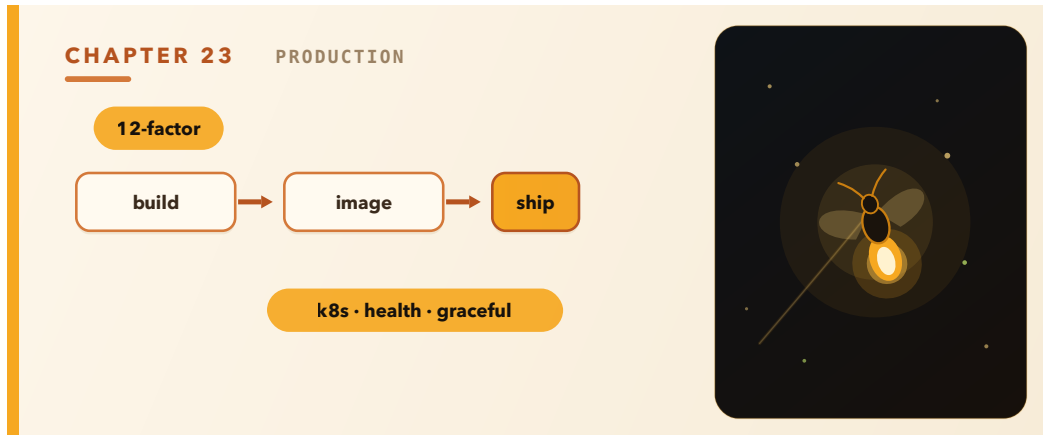
What changed in Lumen

Nothing in `samples/lumen` itself — this chapter is operational. But you saw the CLI path to every artifact Lumen grew by hand: `firefly new --archetype web-api` scaffolds the chapter-2 skeleton, `firefly generate command/query/aggregate/saga` writes the CQRS, DDD, and orchestration pieces, `firefly run --bin lumen` launches it with `LUMEN_ADDR` overrides, and `firefly health/routes/beans --url :8081` introspects the actuator surface from chapter 15. The CLI never invents APIs — every command maps to a framework crate (`firefly-migrations`, `firefly-openapi`, the actuator endpoints) you have already met.

Exercises

1. **Scaffold a Lumen twin.** Run `firefly new lumen2 --archetype web-api --features web,cqrs --dry-run`, then without `--dry-run`. Compare the generated `src/` tree to Lumen's, and `cargo build` it.
2. **Generate the CQRS pieces.** In the scaffolded project, run `firefly generate command OpenWallet` and `firefly generate query GetWallet`. Inspect the generated handlers and note how they match the `#[command_handler]` / `#[query_handler]` shape from chapter 9.
3. **Run with a profile and an override.** Start the app with `firefly run -p dev -D server.port=9090 --dry-run` and read the resolved `FIREFLY_*` environment it would export. Then drop `--dry-run` and confirm the port.
4. **Introspect the real Lumen.** `cargo run --bin lumen`, then in another shell run `firefly health --url http://localhost:8081`, `firefly routes --url http://localhost:8081`, and `firefly beans --url http://localhost:8081`. Match the route table against the endpoint table in `web.rs`.

With a project scaffolded, generated, run, and introspected, the next chapter takes Lumen all the way to production. Continue to [Production & Deployment](#).

**CHAPTER 23**

Extending Firefly & Going to Production

Lumen has grown from a bare scaffold into a secure, observable, event-sourced CQRS service with a saga and a scheduled task. This last chapter of the build arc wires the **process entry point** — the thing that boots Lumen and keeps it running reliably — and turns on the optional **reactive streaming endpoint**. It also covers everything between "it works on my machine" and a service in production: graceful shutdown, the management split, configuration, packaging, and the swap from in-memory infrastructure to real Postgres + Kafka.

By the end of this chapter you will have read Lumen's `main.rs` end to end: a banner, two servers (public API + actuator) wired through the lifecycle `Application`, graceful SIGINT/SIGTERM shutdown, and the streaming endpoint (`GET /api/v1/wallets/:id/events` → NDJSON / SSE) behind the `streaming` feature.

***The lifecycle Application.** `firefly-lifecycle`'s `Application` is the process supervisor: it traps SIGINT/SIGTERM, gives each server task its own drain signal, runs lifecycle hooks, and exits cleanly once in-flight work drains. Running the actuator on a second listener keeps the management surface off the public network. A streaming endpoint returns a `Flux<T>` as NDJSON or SSE.*

The lifecycle and graceful shutdown

`firefly-lifecycle`'s `Application` traps SIGINT/SIGTERM, gives each server task its own drain signal, and grants a drain budget before exiting.

`Core::new_application()` (reachable through the `WebStack`'s `Deref`) builds one named after the app. This is the spine of Lumen's `src/main.rs`:

Rust

```

let api_addr = std::env::var("LUMEN_ADDR").unwrap_or_else(|_|
    DEFAULT_ADDR.to_owned());
let admin_addr =
    std::env::var("LUMEN_ADMIN_ADDR").unwrap_or_else(|_|
    DEFAULT_ADMIN_ADDR.to_owned());

let application = app
    .web
    .new_application()
    .on_server("api", move |shutdown| async move {
        let listener = tokio::net::TcpListener::bind(&api_addr).await?;
        axum::serve(listener, api)
            .with_graceful_shutdown(shutdown.wait())
            .await?;
        Ok(())
    })
    .on_server("admin", move |shutdown| async move {
        let listener = tokio::net::TcpListener::bind(&admin_addr).await?;
        axum::serve(listener, admin)
            .with_graceful_shutdown(shutdown.wait())
            .await?;
        Ok(())
    });

if let Err(err) = application.run().await {
    if !err.is_cancelled() {
        eprintln!("application failed: {err}");
        std::process::exit(1);
    }
}

```

The key moves:

- **Two servers, two drains.** `on_server("api", ..)` and `on_server("admin", ..)` register the public API on `:8080` and the actuator on `:8081`. Each closure receives its own `shutdown` handle; passing `shutdown.wait()` to

`axum::serve(...).with_graceful_shutdown` drains that listener when the signal arrives. Multiple servers are allowed, each on its own task.

- **`run()` blocks until a signal.** It returns when SIGINT/SIGTERM is received and the drain completes. A clean shutdown surfaces as a *cancelled* error — `err.is_cancelled()` — which Lumen treats as success; any other error exits non-zero. (`on_start` / `on_stop` hooks and `app.shutdown_handle()` for programmatic shutdown round out the API; Lumen needs neither.)
- **Env-overridable binds.** `LUMEN_ADDR` and `LUMEN_ADMIN_ADDR` override the defaults, so a container reads its ports from the environment.

The full boot sequence

Read top to bottom, `main()` does exactly six things — build, log, assemble the two routers, start the scheduler, print the banner, run:

Rust

```

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let app = build_app().await;
    // Best-effort: a test harness may already own the global subscriber.
    let _ = app.web.init_logging();

    let api = app.router();
    let contributor: InfoContributor = Box::new(|| {
        let mut info = serde_json::Map::new();
        info.insert(
            "sample".into(),
            serde_json::json!({ "name": APP_NAME, "store": "in-memory",
"eventBus": "in-memory" }),
        );
        info
    });
    let admin = app.web.actuator_router(vec![contributor]);

    // Register and start the scheduled housekeeping task on a background task.
    let scheduler = build_scheduler();
    tokio::spawn(async move { scheduler.start().await });

    app.web.print_banner();
    println!("::: {APP_NAME} :: digital-wallet & ledger (v{VERSION})");

    // ... the lifecycle Application from the previous section ...
}

```

`build_app()` is the composition root from Chapter 4 — it wires the `WebStack`, the CQRS bus, the event-sourced ledger, the read model, the projection, and the security chain over in-memory infrastructure. `app.router()` (Chapter 14) adds the JWT bearer + RBAC layers. `print_banner()` emits the ASCII Firefly banner and the version tagline. Everything else you have already seen, chapter by chapter — `main.rs` just assembles it.

Teaching code runs with no dependencies. Lumen's binary boots with an in-memory event store and broker, so `cargo run --bin lumen` needs nothing external. The tests drive `build_router()` in-process rather than this binary.

The reactive streaming endpoint (feature `streaming`)

Lumen's last endpoint streams a wallet's event history. It is feature-gated so the teaching baseline stays lean — `Cargo.toml` declares it, and it needs nothing beyond the facade (`firefly::reactive::Flux` + `firefly::web::{NdJson, Sse}`):

TOML

```
[features]
default = []
streaming = []
```

The handler lives on a separate, feature-gated sub-router merged into `LumenApp::router`, so the macro-generated `routes()` never references a method that is compiled out. It loads the wallet's persisted events, maps them to the view shape, wraps them in a `Flux`, and returns NDJSON by default or SSE when `?format=sse` is passed:

Rust

```

async fn stream_events(
    State(api): State<WalletApi>,
    Path(id): Path<String>,
    axum::extract::Query(params): axum::extract::Query<StreamParams>,
) -> Response {
    use crate::domain::WalletEvent;
    use firefly::reactive::Flux;
    use firefly::web::{NdJson, Sse};

    // `load_events` returns `Err(NotFound)` for an absent wallet, so the 404 is
    // decided before the streaming response head is committed.
    let events = match api.ledger.load_events(&id).await {
        Ok(events) => events,
        Err(e) => return WebError::from(domain_to_web(e)).into_response(),
    };

    let items: Vec<WalletEvent> =
events.iter().map(WalletEvent::from_domain).collect();
    let flux = Flux::just(items);
    if params.format.as_deref() == Some("sse") {
        Sse(flux).into_response()
    } else {
        NdJson(flux).into_response()
    }
}

```

`NdJson(flux)` renders one JSON document per line (`application/x-ndjson`);
`Sse(flux)` renders Server-Sent Events (`text/event-stream`). The 404 for an unknown wallet is resolved *before* the response head is committed, because once a streaming body starts you can no longer change the status. `tests/streaming.rs` (run with `--features streaming`) proves all three behaviors:

Rust

```

#[tokio::test]
async fn events_stream_as_ndjson_by_default() {
    let id = open_with_deposit().await;           // two events: opened +
    deposited
    let res = build_router()
        .await
        .oneshot(Request::get(format!("/api/v1/wallets/{id}/
events"))).body(Body::empty()).unwrap()
        .await
        .unwrap();
    assert_eq!(res.status(), StatusCode::OK);
    // content-type contains "ndjson"; the body is two JSON lines:
    // one "WalletOpened", one "MoneyDeposited".
}

```

Streaming responses. Returning a `Flux<T>` as `application/x-ndjson` or `text/event-stream` streams element-by-element with backpressure. `Flux::just` materializes a known `Vec`; a production stream would use `Flux::from_stream` over a live subscription so the body is produced lazily rather than buffered.

The management split in production

Always serve the actuator on a different listener from the public API so `/actuator/*` is reachable by your orchestrator but never on the public network — exactly the `:8080` / `:8081` split Lumen uses. Firewall the admin port. The health sub-paths feed your orchestrator's probes:

Probe	Endpoint
liveness	<code>/actuator/health/liveness</code>
readiness	<code>/actuator/health/readiness</code>
overall	<code>/actuator/health</code>

Production hardening middleware

Lumen's `WebStack::new` already turns on CORS, OWASP security headers, request metrics, and the access log (the web-tier batteries). The remaining production middleware is opt-in through `CoreConfig`, weaving in at the correct filter order:

Knob	Adds
<code>cors</code>	CORS preflight + simple-request decoration
<code>security_headers</code>	OWASP response headers (<code>nosniff</code> , <code>DENY</code> , HSTS, ...)
<code>csrf</code>	double-submit-cookie CSRF (for browser flows)
<code>request_log</code>	one structured access-log event per request
<code>request_metrics</code>	<code>http_server_requests_seconds</code> + <code>_max</code> (actuator)
<code>http_exchanges</code>	recent-exchange recorder + <code>/actuator/httpexchanges</code>
<code>loggers</code>	<code>/actuator/loggers</code> runtime log-level control

The effective chain (outermost → innermost) is CORS → Problem → SecurityHeaders → Correlation → Metrics → HttpExchanges → RequestLog → CSRF → Idempotency → your router. Idempotency stays innermost so a replayed request still passes every outer concern.

Configuration in production

Bind configuration from layered sources with environment overrides on top, so a container reads its settings from the environment (Chapter 3). For Lumen, the two bind addresses are already env-driven:

Shell

```
FIREFLY_PROFILE=prod \  
LUMEN_ADDR=0.0.0.0:8080 \  
LUMEN_ADMIN_ADDR=0.0.0.0:8081 \  
./lumen
```

`FIREFLY_*` variables beat the YAML files, secrets are masked in `/actuator/env`, and `${...}` placeholders resolve env-then-config-then-default. The JWT signing key (Chapter 14) is the obvious thing to inject this way rather than inline.

From in-memory to real infrastructure

This is the payoff of the whole architecture: Lumen swaps its in-memory defaults for real backends at the **composition root**, and nothing downstream changes. `build_app` constructs a `MemoryEventStore` and reads the `WebStack`'s in-memory broker; production replaces those two lines:

Rust

```
// build_app() today:
let store: Arc<dyn firefly::eventsourcing::EventStore> =
Arc::new(MemoryEventStore::new());
let broker = Arc::clone(&web.broker);

// production: a Postgres-backed event store and a Kafka broker, same ports.
// let store: Arc<dyn EventStore> = Arc::new(postgres_event_store); // firefly-
// eda-postgres
// let broker = Arc::new(kafka_broker); // firefly-
// eda-kafka
```

The **Ledger**, the projection, the CQRS handlers, the saga, and every test are written against the **EventStore** and **Broker** *ports* — so the wallet domain never learns it moved from a **HashMap** to Postgres + Kafka. That is "swap the adapter, keep the code," applied to the storage and messaging tiers.

Container packaging

A typical multi-stage build for Lumen:

Dockerfile

```
FROM rust:1.88 AS build
WORKDIR /app
COPY . .
RUN cargo build --release -p firefly-sample-lumen

FROM debian:bookworm-slim
COPY --from=build /app/target/release/lumen /usr/local/bin/lumen
EXPOSE 8080 8081
ENTRYPOINT ["/usr/local/bin/lumen"]
```


Because the lifecycle `Application` traps `SIGTERM` and drains, the container stops cleanly when the orchestrator sends a termination signal — no `--init` shim or signal-forwarding wrapper required.

A deployment checklist

- [] Actuator (`:8081`) on a separate, firewalled port from the API (`:8080`).
- [] Liveness/readiness probes pointed at `/actuator/health/{liveness,readiness}`.
- [] `security_headers`, `cors`, and (for browser flows) `csrf` on.
- [] `request_log` + `request_metrics` on; logs shipped as JSON, metrics scraped from `/actuator/prometheus`.
- [] Correlation propagation verified end-to-end across services.
- [] JWT signing key injected from the environment / a secret store, not inline.
- [] In-memory event store + broker swapped for Postgres + Kafka at `build_app`.
- [] Graceful-shutdown drain budget tuned for the slowest in-flight transfer.
- [] The verification gate green: `cargo test -p firefly-sample-lumen` and `--features streaming`, plus `clippy -D warnings` and `fmt --check`.

What changed in Lumen

This chapter wired the entry point and the optional stream — the last code the arc adds:

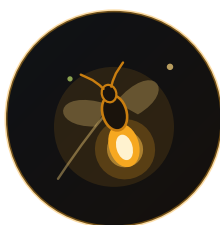
- `src/main.rs` boots the whole service: `build_app` → `init_logging` → assemble the API router and the actuator router (with the info contributor) → start the scheduler on a background task → `print_banner` → run the lifecycle `Application` with two `on_server` listeners and graceful `SIGINT`/`SIGTERM` drain. A clean shutdown is a *cancelled* error and exits zero.

- `Cargo.toml` declares the `streaming` feature; `src/web.rs` adds the feature-gated `streaming_router` / `stream_events` handler that returns a `Flux<WalletEvent>` as NDJSON (default) or SSE (`?format=sse`), with the 404 resolved before the response head.
- `tests/streaming.rs` proves the NDJSON default, the SSE switch, and the 404, all behind the `streaming` feature.
- The chapter showed the one-line `in-memory` → `Postgres` + `Kafka` swap at `build_app`, proving the port-and-adapter design end to end.

Exercises

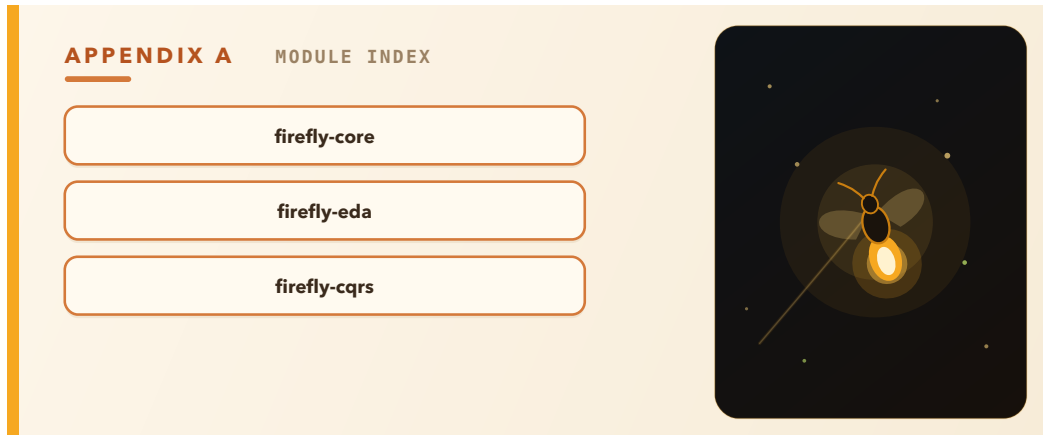
1. **Run and drain.** `cargo run --bin lumen`, open a wallet, then Ctrl-C and watch the graceful drain. Confirm the process exits zero.
2. **Override the ports.** Start Lumen with `LUMEN_ADDR=0.0.0.0:9000`
`LUMEN_ADMIN_ADDR=0.0.0.0:9001 cargo run --bin lumen` and confirm the API and actuator move.
3. **Stream the history.** Build with `--features streaming`, open a wallet and deposit, then `curl http://127.0.0.1:8080/api/v1/wallets/<id>/events` (NDJSON) and `...?format=sse` (SSE). Compare the two content types.
4. **Sketch the Postgres swap.** Write the two lines in `build_app` that would replace `MemoryEventStore` with a Postgres-backed `EventStore`, and explain in one sentence why `Ledger`, the projection, and the tests need no change.

That completes the guided tour of Lumen. The remaining chapters revisit the declarative macros as a capstone and provide reference material: the [Module Index](#) and the [Glossary](#).



Appendices





APPENDIX A

Crate & Module Index

The Firefly Rust workspace ships **79 members** — 74 framework crates under `crates/`, plus the cross-crate integration suite and four samples: the book's running example `lumen`, alongside `orders`, `reactive-banking`, and `macro-quickstart`. Each crate carries its own `README.md` with its full public surface, design rationale, and a runnable quick-start.

The canonical, always-current catalogue is `MODULES.md` in the repository root. This appendix reproduces it tier-by-tier as a navigable index.

Front door

Crate	What it provides
<code>firefly</code>	The one-dependency facade: <code>use firefly::prelude::*;</code> , per-crate aliases, the <code>__rt</code> macro contract, feature-gated adapters
<code>firefly-macros</code>	Declarative macros: <code>#[derive(Command/Query/Component/DomainEvent/AggregateRoot)]</code> , <code>#[command_handler]</code> / <code>#[query_handler]</code> , <code>#[scheduled]</code> , <code>#[rest_controller]</code> + verbs, <code>#[event_listener]</code>

Foundational

Crate	What it provides
<code>firefly-kernel</code>	RFC 7807 <code>ProblemDetail</code> , <code>FireflyResult<T></code> , <code>Clock</code> , the <code>FireflyError</code> hierarchy, task-local correlation/request/tenant scopes, the <code>ddd</code> kit (<code>Entity</code> / <code>Specification</code> / domain events)
<code>firefly-reactive</code>	The <code>Mono<T></code> / <code>Flux<T></code> reactive core — the keystone of every reactive surface in Firefly
<code>firefly-utils</code>	Try helpers, retry with exponential backoff, slug, AES-256-GCM, templates
<code>firefly-validators</code>	IBAN, BIC, Luhn, currency, phone, password, VAT, national/tax IDs
<code>firefly-web</code>	Problem renderer, correlation, idempotency, PII masking, CORS, CSRF, security headers, server metrics, the reactive <code>MonoJson</code> / <code>NdJson</code> / <code>Sse</code> responders, TLS bootstrap
<code>firefly-config</code>	Typed YAML+env+flag binding, profiles, <code>\${...}</code> placeholders, runtime reload, masked property sources, <code>ApplicationEventBus</code> , config-server client
<code>firefly-i18n</code>	Locale-aware message <code>Bundle</code> + Accept-Language picker
<code>firefly-session</code>	Server-side HTTP sessions + <code>SessionStore</code> + <code>SessionLayer</code>

Platform

Crate	What it provides
<code>firefly-cache</code>	<code>Adapter</code> port + Memory/NoOp/Fallback + typed <code>Typed<T></code> memoisation
<code>firefly-observability</code>	<code>tracing</code> + correlation enrichment, health composite, startup banner, W3C trace-context, metrics
<code>firefly-data</code>	Storage-agnostic ports: Filter DSL + <code>Specification</code> , <code>SqlDialect</code> (pg/mysql/sqlite) + <code>Specification::to_mongo()</code> , <code>Page<T></code> , <code>Repository<T, K></code> , <code>ReactiveCrudRepository</code> , <code>PostgresReactiveRepository</code> , auditing + soft-delete, derived queries, paging
<code>firefly-cqrs</code>	Command/query <code>Bus</code> with validation/cache/authorization middleware + reactive <code>send_mono</code> / <code>query_mono</code>
<code>firefly-eda</code>	<code>Event</code> envelope, <code>Publisher</code> / <code>Subscriber</code> / <code>Broker</code> ports, <code>InMemoryBroker</code> , glob topics, consumer groups, retry/DLQ, reactive <code>Flux</code> subscriptions
<code>firefly-eventsourcing</code>	Aggregate roots, event store, snapshots, projections, global stream, transactional outbox, multi-tenancy
<code>firefly-orchestration</code>	<code>Saga</code> , <code>Workflow</code> (DAG), <code>Tcc</code> — compensation, per-step retry, <code>StepContext</code>
<code>firefly-rule-engine</code>	YAML DSL → AST → evaluator with <code>between</code> / null / <code>regex</code> , <code>EvaluationMode</code> , validator + <code>ActionHandler</code>
<code>firefly-plugins</code>	Lifecycle SPI + composite registry
<code>firefly-lifecycle</code>	<code>Application::run()</code> orchestrator with signal trap + drain
<code>firefly-actuator</code>	<code>/actuator/{health,info,metrics,env,tasks,version}</code> + probes, loggers, httpexchanges, threaddump, refresh
<code>firefly-scheduling</code>	Cron + FixedRate + FixedDelay <code>Scheduler</code> with zones

Crate	What it provides
<code>firefly-resilience</code>	<code>CircuitBreaker</code> , <code>RateLimiter</code> , <code>Bulkhead</code> , <code>Timeout</code> , composable <code>Chain</code>
<code>firefly-security</code>	<code>BearerLayer</code> , <code>RBAC</code> , <code>FilterChain</code> , <code>JwksVerifier</code> , <code>oauth2</code> , <code>RoleHierarchy</code> , <code>CsrfLayer</code> , <code>BcryptPasswordEncoder</code>
<code>firefly-migrations</code>	Versioned forward-only SQL migrations over a <code>Database</code> port
<code>firefly-openapi</code>	OpenAPI 3.1 generator + Swagger-UI shim
<code>firefly-sse</code>	Server-Sent Events writer with heartbeat + Last-Event-Id
<code>firefly-transactional</code>	<code>with_tx(ctx, db, f)</code> over pluggable <code>Database</code> / <code>Transaction</code> ports
<code>firefly-testkit</code>	HMAC signers, <code>SpyBroker</code> , JSON test helpers
<code>firefly-aop</code>	Aspect-oriented advice — <code>Pointcut</code> , <code>JoinPoint</code> , <code>Aspect</code> , <code>intercept</code>
<code>firefly-shell</code>	Interactive CLI framework + <code>CommandLineRunner</code> / <code>ApplicationRunner</code> startup hooks
<code>firefly-websocket</code>	WebSocket server over axum + topic <code>BroadcastHub</code>

Adapters

Crate	Port → backend
<code>firefly-client</code>	REST <code>RestClient</code> + reactive <code>WebClient</code> + SOAP/gRPC/GraphQL/WS
<code>firefly-config-server</code>	Centralized configuration REST endpoint for distributed services
<code>firefly-idp</code> + <code>idp-internal-db</code> / <code>idp-keycloak</code> / <code>idp-azure-ad</code> / <code>idp-aws-cognito</code>	Identity providers
<code>firefly-ecm</code> + <code>ecm-storage-aws</code> / <code>ecm-storage-azure</code> / <code>ecm-esignature-*</code>	Content management + e-signature
<code>firefly-notifications</code> + <code>notifications-smtp</code> / <code>-twilio</code> / <code>-firebase</code> / <code>-sendgrid</code> / <code>-resend</code>	Notification channels
<code>firefly-callbacks</code>	Outbound webhook subsystem (HMAC dispatcher + audit + admin)
<code>firefly-webhooks</code>	Inbound ingestion (Stripe / GitHub / Twilio / generic HMAC + DLQ)
<code>firefly-data-sqlx</code>	<code>firefly_data</code> ports → relational (Postgres / MySQL / SQLite over <code>sqlx</code>)
<code>firefly-data-mongodb</code>	<code>firefly_data</code> ports → document store (MongoDB)
<code>firefly-cache-redis</code> / <code>firefly-cache-postgres</code>	<code>cache::Adapter</code> → Redis (RESP) / Postgres key-value table
<code>firefly-eda-kafka</code> / <code>-rabbitmq</code> / <code>-postgres</code> / <code>-redis</code>	<code>eda::Broker</code> → Kafka / RabbitMQ / Postgres outbox / Redis Streams
<code>firefly-session-redis</code> / <code>firefly-session-postgres</code> / <code>firefly-session-mongodb</code>	distributed <code>SessionRegistry</code> → Redis sorted set / Postgres table / MongoDB collection

Starters

Crate	What it bundles
<code>firefly-starter-core</code>	web + cache + observability + eda + cqrs + actuator + lifecycle + scheduling
<code>firefly-starter-application</code>	starter-core + plugins registry
<code>firefly-starter-domain</code>	starter-core + in-memory event-sourcing stores
<code>firefly-starter-data</code>	starter-core (you supply the DB)
<code>firefly-starter-web</code>	<code>WebStack</code> — <code>Core</code> + CORS + security headers + request metrics + access log
<code>firefly-starter-experience</code>	<code>ExperienceStack</code> (alias <code>Bff</code>) — <code>WebStack</code> + <code>DomainClients</code> (the <code>ClientFactory</code>) + <code>SignalService</code> gates + Redis-capable <code>WorkflowState</code> + <code>WorkflowQueryService</code> + <code>ChildWorkflowService</code> — the experience (BFF) tier
<code>firefly-backoffice</code>	starter-application + back-office context middleware

DI / Operations / Tooling

Crate	What it provides
<code>firefly-container</code>	Opt-in <code>TypeId</code> -keyed DI container (service locator)
<code>firefly-admin</code>	Embedded management dashboard + JSON API + SSE streams
<code>firefly-cli</code>	The <code>firefly</code> developer binary (<code>new</code> / <code>generate</code> / <code>db</code> / <code>openapi</code> / <code>actuator</code> / <code>doctor</code> / <code>completion</code> / <code>sbom</code> / <code>license</code>)

For per-crate detail, open the crate's `README.md` in the [repository](#), or read the `MODULES.md` catalogue.

Glossary

Definitions of the terms and types that recur throughout this book, in the precise sense Firefly uses them.

Actuator

The management surface (`firefly-actuator`) exposing the endpoints `/actuator/health` , `/actuator/info` , `/actuator/metrics` , `/actuator/env` , `/actuator/tasks` , `/actuator/version` , `/actuator/loggers` , `/actuator/httpexchanges` , and `/actuator/refresh` . Mounted by `core.actuator_router(..)` , typically on a separate, firewalled port.

Adapter

A concrete implementation of a **port**. Selected at wiring time as an `Arc<dyn Port>` so heavy SDK dependencies stay out of services that do not use them. Examples:

`RedisAdapter` (a `cache::Adapter`), `KafkaBroker` (an `eda::Broker`).

Aggregate root

The consistency boundary in DDD. The non-event-sourced variant holds a `PendingEvents<E>` buffer (`firefly_kernel::ddd`); the event-sourced variant is an `AggregateRoot` whose state is rebuilt by replaying its `DomainEvent` s (`firefly-eventsourcing`).

Autowired

A component field the DI container resolves and injects by type (`#[autowired]` , `firefly-container`). The field type sets the shape: `Arc<T>` (required), `Option<Arc<T>>` (optional), `Vec<Arc<T>>` (all implementations), `Provider<T>` (deferred). Resolves and injects a component field by type.

Backpressure

A slow consumer throttling a fast producer. Firefly's reactive `Flux` streams (NDJSON/SSE responders, `WebClient::body_to_flux`, Postgres row streams) honour backpressure end-to-end so a large stream never lands fully in memory.

Bean

Any value the DI `Container` builds, wires, and owns. Declared with a `stereotype` derive (`#[derive(Component/Service/Repository/Configuration/ Controller)]`) or produced by a `#[bean]` factory method on a `#[derive(Configuration)]` holder. Keyed by `TypeId`; resolvable with `resolve::<T>()`.

BFF (Backend-for-Frontend)

See Experience tier.

Bus

The CQRS command/query dispatcher (`firefly_cqrs::Bus`). Handlers register by input type; dispatch is keyed on `std::any::TypeId`. `send` / `query` are async; `send_mono` / `query_mono` are the reactive twins.

Compensation

The undo step a `saga` runs in reverse order when a later step fails (`Step::with_compensation`). In Lumen's transfer saga, the debit's compensation is a refund deposit on the source wallet.

CompensationPolicy

How a saga/workflow rolls back on failure: `BestEffort` (continue compensating even if one compensation fails) or `StopOnError` (abort at the first compensation failure).

Component scanning

Link-time bean discovery (`Container::scan()` / `firefly::scan`): every non-generic stereotype derive submits an `inventory` thunk, and `scan` collects them across the crate graph, applies **conditions** and **profiles**, and registers the survivors. Discovery is link-time, not reflective. Generic beans use the `register_all!` fallback.

Conditional bean

A bean registered only when the environment matches — `#[firefly(profile = "...", condition_on_property = "k=v", condition_on_bean = "T", condition_on_missing_bean = "T", condition_on_class = "label", condition_on_single_candidate = "T")]`. Evaluated by `scan` in two passes (config/profile facts first, registry-dependent checks second).

Container

The opt-in, `TypeId`-keyed DI service locator (`firefly-container`): registers beans, resolves by type or name, supports scopes, trait-object bindings, `primary` / `order` disambiguation, deferred `Provider<T>`, lifecycle hooks, and component scanning. Distinct from **Core** (the wired infrastructure bundle).

Core

The wired infrastructure bundle returned by `Core::new(CoreConfig)` (`firefly-starter-core`): cache, CQRS bus, event broker, health composite, metrics, scheduler, logging, and the middleware chain.

Correlation id

A per-request identifier carried in the `X-Correlation-Id` header and a kernel task-local scope. It auto-enriches every log line, every published event, and every outbound client call so a request stitches together across services.

DomainEvent

The event-sourced, versioned, wire-formatted event in `firefly-eventsourcing` (distinct from the transient `TransientDomainEvent` in `firefly_kernel::ddd`). Its JSON uses a stable, versioned wire format.

Event (EDA)

The envelope every `firefly-eda` event flows through — `id`, `type`, `source`, `topic`, `correlationId`, `time`, `headers`, `payload`, `key`. Constructed with `Event::new`; the envelope follows a stable JSON schema.

Experience tier (BFF)

The top service tier (`firefly-starter-experience`, `ExperienceStack` / `Bff`): a Backend-for-Frontend that composes several **domain** SDKs into journey-specific, atomic REST endpoints. It owns no database and calls only domain services (`channel` → `experience` → `domain` → `core`). Built from `DomainClients` (the `ClientFactory`), `SignalService` gates, Redis-capable `WorkflowState`, and `WorkflowQueryService`.

FireflyError

The framework's error type (`firefly_kernel::FireflyError`). It renders as an RFC 7807 `application/problem+json` response, and it is the fixed error channel of the reactive `Mono` / `Flux` (their terminal `Err` signal).

FilterChain

The path-based authorization matcher in `firefly-security` (`permit` / `require` / glob `permit_pattern` / `require_pattern`). Fail-closed once any rule is declared: an undeclared path is denied by default.

Flux

A reactive publisher of $0..N$ values plus a terminal completion-or-error (`firefly_reactive::Flux`).

Idempotency

The replay behaviour applied to `POST` / `PUT` / `PATCH` requests carrying an `Idempotency-Key` header. A repeat replays the stored response (`Idempotent-Replay: true`); reuse with a different body is a 409.

Mono

A reactive publisher of *at most one* value plus a terminal error (`firefly_reactive::Mono`). An empty `Mono` is `Ok(None)` .

NDJSON

Newline-delimited JSON (`application/x-ndjson`) — one compact JSON document per line. The `NdJson(Flux<T>)` responder streams it with backpressure.

Outbox (transactional)

A pattern (`TransactionalOutbox` , `firefly-eda-postgres`) that writes events in the same transaction as the state change and delivers them to consumers afterward, giving at-least-once delivery without a separate broker.

Port

An object-safe `async_trait` trait defining an integration point — `cache::Adapter` , `eda::Broker` , `notifications::Channel` , `idp::Adapter` . Code depends on the port; an `adapter` implements it.

Primary

The disambiguator (`#[firefly(primary)]`) that picks one bean when several implementations are bound to the same port. Resolving with no primary among multiple candidates is a `NoUniqueBean` error naming every candidate.

Problem (RFC 7807 / 9457)

The `application/problem+json` error envelope (`type` , `title` , `status` , `detail`) that every Firefly service renders for errors and panics, following the RFC 9457 standard. RFC 9457 obsoletes and is wire-compatible with RFC 7807; the book uses both numbers interchangeably.

Profile

A named environment (`prod` , `dev` , `test`) that gates conditional beans (`#[firefly(profile = "expr")]`). The expression grammar supports `&` , `|` , `!` , comma-as-OR, and parentheses. Active profiles live on the `ApplicationContext` / `ConditionContext` .

Projection

A read-side handler that builds a read model from events (`firefly_eventsourcing::Projection`), driven per-aggregate (`replay`) or over the global stream (`drive_once` / `replay_all`).

Qualifier

A name used to select a specific bean when several share a type (`#[firefly(qualifier = "replica")]`) → `resolve_named`).

Reactive

The `Mono` / `Flux` programming model (`firefly-reactive`) and everything built on it — reactive endpoints, repositories, the `WebClient` , reactive EDA/CQRS. If you have used a reactive-streams library, the publisher model will feel familiar.

Saga

A sequential distributed-transaction engine (`firefly_orchestration::Saga`) with reverse-order compensation on failure. See also `Workflow` (DAG) and `Tcc` .

Scheduler

The task runner (`firefly_scheduling::Scheduler`) owning Cron, FixedRate, and FixedDelay triggers, each on its own tokio task with panic recovery.

Scope

A bean's lifecycle (`#[firefly(scope = "...")]`): `singleton` (one cached instance, the default), `transient` (fresh per resolve), `request`, or `session` (both driven by a `ScopeHandler`).

Signal

An external event that satisfies a parked workflow gate in the `experience tier` (`SignalService::deliver` / `Node::wait_for_signal`). Delivery is buffered, so a signal that arrives before the gate parks is not lost.

SSE (Server-Sent Events)

A one-way streaming protocol (`text/event-stream`). The `Sse(Flux<T>)` responder and `firefly-sse`'s `SseWriter` emit it; `WebClient::body_to_flux` decodes it.

Specification

A composable business-rule predicate (`firefly_kernel::ddd::Specification<T>`) combined with `.and()`, `.or()`, `.not()`. Any `Fn(&T) -> bool` is one.

Starter

A crate that bundles a sensible default stack so a service depends on one crate. `firefly-starter-core` is the common starting point; `firefly-starter-domain` and `firefly-starter-experience` add the domain and BFF tiers.

Stereotype

The architectural-role label a DI bean carries (`component` , `service` , `repository` , `configuration` , `controller` , `bean`), set by which derive declared it. Functionally equivalent; the differences are the documented intent and the grouping shown in the admin `/beans` view.

TCC (Try-Confirm-Cancel)

A two-phase distributed-transaction engine (`firefly_orchestration::Tcc`): Try all participants, Confirm all on success, Cancel the tried participants on any Try failure.

Value object

A domain type defined entirely by its attributes (no identity) and **immutable**: every operation returns a new value. Lumen's `Money` is the canonical example — exact integer-cent arithmetic, closed under `add` / `subtract` . The DDD counterpart of an **aggregate root** (which has identity).

Verifier

The async port (`firefly_security::Verifier`) that validates a bearer token and returns an `Authentication` . `JwksVerifier` , IDP adapters, and `VerifierFn` closures all satisfy it.

WebClient

The reactive HTTP client (`firefly_client::WebClient`) whose terminal operators return `Mono` / `Flux` (`body_to_mono` , `body_to_flux` , `exchange`).

Workflow

A DAG distributed-transaction engine (`firefly_orchestration::Workflow`): independent nodes run concurrently within a wave, with reverse-order compensation under a configurable `CompensationPolicy` . In the **experience tier**, a node may park on a **signal gate**.

WorkflowState

Redis-capable persisted journey state in the **experience tier**

(`firefly_starter_experience::WorkflowState`): round-trips a workflow run's `StepContext` snapshot through the cache `Adapter` , keyed by correlation id, so a parked journey survives a client disconnect (`save` / `load` / `delete`).